
Kiến trúc SOA & Microservices

Từ lý thuyết đến thực tiễn với case study KBLab

Phiên bản 1.0.0

Đặng Ngọc Hùng

Kiến trúc SOA & Microservices

Từ lý thuyết đến thực tiễn với case study KBLab

Phiên bản 1.0.0

Lời nói đầu

Cuốn sách này ra đời từ một nhu cầu thực tế: giúp sinh viên, lập trình viên và kỹ sư phần mềm Việt Nam tiếp cận kiến trúc microservices một cách **có hệ thống, thực tiễn, và dễ hiểu**.

Tại sao viết sách này?

Thị trường đã có nhiều sách có giá trị về microservices — *Microservices Patterns* của Chris Richardson, *Building Microservices* của Sam Newman, *Designing Data-Intensive Applications* của Martin Kleppmann. Tuy nhiên, hầu hết đều viết bằng tiếng Anh và hướng đến developer đã có kinh nghiệm.

Cuốn sách này khác ở ba điểm:

1. **Tiếng Việt** — thuật ngữ kỹ thuật giữ nguyên tiếng Anh khi cần thiết, nhưng phần giải thích và phân tích được biên soạn trực tiếp bằng tiếng Việt. Người học Việt Nam cần tài liệu có ngữ cảnh phù hợp, không phải bản dịch máy.
2. **Case study xuyên suốt** — thay vì ví dụ rời rạc, toàn bộ 12 chương sử dụng chung KBLab, một hệ thống LMS (Learning Management System) thực tế. Mỗi chương phân tích *hiện trạng* → *gap* → *migration path* để nội dung không chỉ dừng ở lý thuyết.
3. **Problem-first** — mỗi pattern, mỗi concept bắt đầu bằng “bài toán”: *vấn đề gì?* → *tại sao monolith không giải quyết được?* → *microservices giải quyết thế nào?* Đây là cách tiếp cận phù hợp với người học cần hiểu trade-off trước khi áp dụng kỹ thuật.

Độc giả mục tiêu

- **Sinh viên CNTT** năm 3-4 muốn hiểu kiến trúc phần mềm hiện đại
- **Developer ở mức junior/mid-level** đang làm việc với monolith và muốn hiểu microservices
- **Kỹ sư phần mềm** cần tài liệu tham chiếu có hệ thống về patterns và practices
- **Trưởng nhóm kỹ thuật** cần đánh giá khi nào nên/không nên dùng microservices

Yêu cầu kiến thức nền:

- Lập trình Java cơ bản (Spring Boot là lợi thế)
- Hiểu REST API, SQL, HTTP
- Biết dùng Git, Docker (ở mức cơ bản)

Cách sử dụng sách

Đọc tuần tự: Sách thiết kế theo logic tiến trình — từ khái niệm (Phần I) → giao tiếp & dữ liệu (Phần II) → hạ tầng & vận hành (Phần III). Mỗi chương xây trên kiến thức chương trước.

Tra cứu: Phụ lục C (Pattern Catalog) liệt kê 40+ patterns với chương tham chiếu để tra cứu nhanh khi cần.

Case study: Theo dõi KBLab xuyên suốt sách để thấy một kiến trúc thực tế tiến hóa từ monolith sang microservices.

Quy ước trong sách

Ký hiệu	Ý nghĩa
 Nguyên tắc	Triết lý hoặc best practice quan trọng
 Phân tích gap	So sánh KBLab hiện trạng vs best practice
 Sai lầm thường gặp	Anti-patterns và cách phòng tránh
 Tip	Mẹo thực hành
[2a, Ch.3]	Tham chiếu đến sách (mã trong Đọc thêm)

Lời cảm ơn

Cuốn sách này được hình thành từ quá trình giảng dạy, phát triển hệ thống KBLab, và tổng hợp kinh nghiệm thực tiễn về kiến trúc phần mềm, SOA và microservices. Tác giả trân trọng cảm ơn các cá nhân và tập thể đã đóng góp vào hệ thống, cung cấp phản hồi từ quá trình sử dụng, và giúp các ví dụ trong sách có nền tảng thực tế.

KBLab là hệ thống học tập và thực hành lập trình được sử dụng làm case study xuyên suốt cuốn sách. Trong dự án này, tác giả giữ vai trò Product Owner, Principal Architect, Technical Lead và Core Developer: định hướng sản phẩm, quyết định kiến trúc tổng thể, thiết kế domain/case study, tham gia phát triển lõi, và điều phối triển khai/vận hành qua nhiều giai đoạn.

Tác giả cảm ơn Nguyễn Thành Phước (D21) vì các đóng góp trong giai đoạn đầu cho Judge/SQL Lab; Nguyễn Văn Đức và Nguyễn Trung Hiếu (D21) vì các đóng góp cho Assignment và module thi trắc nghiệm; Trần Tất Quốc Huy (D17) vì các đóng góp trong triển khai DevOps và hỗ trợ vận hành; Vương Đức Trọng và Lê Đức Huy vì các đóng góp cho DevOps Lab.

Tác giả cũng cảm ơn các thế hệ sinh viên từ D17 đến nay đã sử dụng KBLab trong quá trình học tập, thực hiện bài tập, kiểm thử và phản hồi. Các phản hồi từ môi trường sử dụng thực tế giúp hệ thống được cải tiến liên tục, đồng thời cung cấp nhiều tình huống kỹ thuật có giá trị cho nội dung sách.

Tác giả cảm ơn đội ngũ phát triển và vận hành KBLab đã duy trì hệ thống qua nhiều giai đoạn thay đổi yêu cầu, mở rộng chức năng và cải tiến hạ tầng. Những kinh nghiệm từ quá trình phát triển này là nguồn dữ liệu thực tế quan trọng cho các thảo luận về phân rã dịch vụ, giao tiếp liên dịch vụ, dữ liệu phân tán, triển khai, quan sát hệ thống và vận hành.

Tác giả trân trọng ghi nhận ảnh hưởng chuyên môn từ các tác giả và cộng đồng kỹ thuật về kiến trúc phần mềm, trong đó có Chris Richardson, Sam Newman, Martin Kleppmann, Thomas Erl và Martin Fowler. Các công trình và bài viết của họ là nguồn tham khảo quan trọng khi hệ thống hóa nội dung về SOA, microservices, distributed systems và software architecture cho bối cảnh đào tạo tại Việt Nam.

Mọi thiếu sót trong nội dung, diễn giải và lựa chọn ví dụ thuộc trách nhiệm của tác giả.

Giới thiệu

Bức tranh tổng thể: Monolith → SOA → Microservices

Kiến trúc phần mềm đã trải qua một hành trình tiến hóa kéo dài hơn hai thập kỷ:

Monolith (trước 2010) — Toàn bộ ứng dụng trong một codebase, deploy thành một artifact duy nhất. Đơn giản, hiệu quả cho đội nhỏ, nhưng gặp giới hạn khi hệ thống phát triển: deploy chậm, scale khó, technology lock-in.

SOA (2000-2015) — Service-Oriented Architecture tách hệ thống thành các services giao tiếp qua Enterprise Service Bus (ESB). Giải quyết bài toán integration giữa hệ thống lớn, nhưng ESB trở thành bottleneck và single point of failure.

Microservices (2014-nay) — Tiến hóa từ SOA với nguyên tắc “smart endpoints, dumb pipes”: services nhỏ, tự trị, deploy độc lập. Phổ biến nhờ Netflix, Amazon, Uber chứng minh hiệu quả ở quy mô lớn.

Vị trí của sách

Cuốn sách này **không** dạy bạn xây dựng microservices từ con số không. Thay vào đó, nó giúp bạn **hiểu** microservices: tại sao cần, khi nào dùng, trade-off là gì, và chuyển đổi từ monolith như thế nào.

Sử dụng KBLab, một hệ thống LMS thực tế, làm case study xuyên suốt, mỗi chương phân tích: *bài toán thực tế → pattern giải quyết → áp dụng cho KBLab → migration path*.

Tổng quan 12 chương

Phần I: Nền tảng (Ch.1-3)

- **Ch.1:** Tổng quan SOA & Microservices — từ lý thuyết đến thực tế (Netflix, Amazon, Uber)
- **Ch.2:** Domain-Driven Design — Bounded Contexts, Context Map, và tổ chức đội phát triển
- **Ch.3:** Thiết kế API — REST, versioning, schema evolution, documentation

Phần II: Giao tiếp & Dữ liệu (Ch.4-7)

- **Ch.4:** Giao tiếp đồng bộ — REST, gRPC, OpenFeign, và Resilience Patterns
- **Ch.5:** Giao tiếp bất đồng bộ — Kafka, Event-driven, Event Storming
- **Ch.6:** Saga Pattern — distributed transactions, orchestration vs choreography
- **Ch.7:** Quản lý dữ liệu — database-per-service, CQRS, Event Sourcing

Phần III: Hạ tầng & Vận hành (Ch.8-12)

- **Ch.8:** API Gateway — routing, cross-cutting concerns, Spring Cloud Gateway
- **Ch.9:** Bảo mật — JWT, OAuth2, RBAC
- **Ch.10:** Chuyển đổi thực tế — Strangler Fig, database decomposition, migration roadmap
- **Ch.11:** Observability — logging, metrics, tracing, alerting
- **Ch.12:** Deployment & DevOps — Docker, CI/CD, deployment strategies, IaC

Mục lục

Lời nói đầu	i
Tại sao viết sách này?	i
Độc giả mục tiêu	i
Cách sử dụng sách	i
Quy ước trong sách	ii
Lời cảm ơn	ii
Giới thiệu	ii
Bức tranh tổng thể: Monolith → SOA → Microservices	iii
Vị trí của sách	iii
Tổng quan 12 chương	iii
Phần I: Nền tảng (Ch.1-3)	iii
Phần II: Giao tiếp & Dữ liệu (Ch.4-7)	iii
Phần III: Hạ tầng & Vận hành (Ch.8-12)	iii
1. Tổng quan SOA & Microservices	25
1.1. Bạn sẽ học được gì	25
1.2. Kiến trúc phần mềm theo thời gian	25
1.2.1. Thời kỳ Monolith	25
1.2.2. Sự ra đời của SOA	26
1.2.3. Từ SOA đến Microservices	26
1.3. SOA là gì? Nguyên lý cốt lõi	27
1.3.1. Bốn đặc trưng cốt lõi	27
1.3.2. Tám nguyên lý hướng dịch vụ	27
1.3.3. Bốn kiểu kiến trúc SOA	28
1.4. Microservices là gì? So sánh với SOA	29
1.4.1. Định nghĩa và đặc trưng	29
1.4.2. Scale Cube — Ba chiều mở rộng	29
1.4.3. So sánh SOA truyền thống vs Microservices	31
1.5. Microservices trong thực tế — Bài học từ các công ty công nghệ	31
1.5.1. Netflix — Khi downtime là không thể chấp nhận	31
1.5.2. Amazon — API Mandate và “two-pizza teams”	32
1.5.3. Uber — Thực tế của hybrid approach	32
1.5.4. Bài học tổng hợp	32
1.5.5. Fast Flow Success Triangle — DevOps + Team Topologies + Architecture	33

1.5.6. Đo lường Fast Flow — DORA Metrics	34
1.5.7. Quality Attribute Scenarios — Chỉ định yêu cầu kiến trúc	34
1.6. Modular Monolith — Lựa chọn hợp lệ	35
1.6.1. Monolith không phải là xấu	35
1.6.2. Khi nào Modular Monolith phù hợp?	36
1.6.3. Con đường từ Monolith đến Microservices	36
1.7. Khi nào nên — và không nên — dùng Microservices	36
1.7.1. Quyết định kiến trúc không chỉ là kỹ thuật	36
1.7.2. Decision Framework	37
1.7.3. Phân tích lợi ích và chi phí	37
1.8. Case Study: Hệ thống KBLab LMS	38
1.8.1. Bài toán nghiệp vụ	38
1.8.2. Kiến trúc tổng quan	40
1.8.3. Các bài toán kỹ thuật chính	40
1.8.4. Những quyết định chúng ta muốn làm lại — Cautionary Tale	42
1.9. Tổng kết	44
1.10. Đọc thêm	45
2. Phân tích hướng Domain — DDD & Bounded Contexts	45
2.1. Bạn sẽ học được gì	46
2.2. Conway's Law — Tổ chức quyết định kiến trúc	46
2.2.1. Quy luật Conway	46
2.2.2. Inverse Conway Maneuver	46
2.2.3. Team Topologies — Bốn kiểu team	47
2.2.4. Industry Case Study: Spotify Squad Model	48
2.2.5. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service	49
2.3. DDD cơ bản — Ngôn ngữ chung và các building blocks	51
2.3.1. Ubiquitous Language — Ngôn ngữ chung	51
2.3.2. Các building blocks	51
2.3.3. DDD cho developer: nghĩ bằng domain, không bằng database	52
2.4. Strategic DDD — Bounded Context & Context Map	53
2.4.1. Bounded Context — Ranh giới ngôn ngữ	53
2.4.2. Context Map — Bản đồ quan hệ giữa các context	54
2.5. Xác định Bounded Context từ requirements	56
2.5.1. Các heuristic thực tế	56
2.5.2. Event Storming — Phương pháp khám phá domain	56
2.5.3. Phân rã hướng dịch vụ theo Erl — Cách tiếp cận step-by-step	57
2.6. Case Study: Bounded Contexts trong KBLab	59

2.6.1. Phân tích domain	59
2.6.2. Năm bounded contexts	60
2.6.3. Từ bounded context đến service	61
2.7. Shared Kernel Pattern — Chia sẻ hay không chia sẻ?	61
2.7.1. Vấn đề thực tế	61
2.7.2. Phân tích: cái gì nên shared, cái gì không?	62
2.7.3. Hướng cải thiện	62
2.8. Assemblage Process — Quy trình thiết kế service architecture	64
2.9. Tổng kết	65
2.10. Đọc thêm	65
 3. Thiết kế Dịch vụ & API	 66
3.1. Bạn sẽ học được gì	66
3.2. REST API Design — Nguyên tắc cho Microservices	66
3.2.1. Tại sao API design quan trọng hơn trong microservices?	66
3.2.2. Richardson Maturity Model	66
3.2.3. Nguyên tắc thiết kế REST API	67
3.3. API Versioning & Schema Evolution	70
3.3.1. Tại sao cần versioning?	70
3.3.2. Ba chiến lược versioning	70
3.3.3. Schema Evolution — Thay đổi mà không breaking	71
3.4. OpenAPI & API Documentation	72
3.4.1. Tại sao documentation quan trọng?	72
3.4.2. OpenAPI Specification (Swagger)	72
3.5. DTO Pattern — Tách Internal Model và External Contract	73
3.5.1. Vấn đề: Entity $\neq$ API Response	73
3.5.2. DTO (Data Transfer Object)	74
3.5.3. Lợi ích thực tế	75
3.5.4. Hexagonal Architecture — Service như hệ thống cổng và adapter	75
3.6. Case Study: API Design trong KBLab — Bài học từ thực tế	76
3.6.1. Hiện trạng	76
3.6.2. Bài học rút ra	77
3.6.3. GraphQL — Khi REST không đủ linh hoạt	77
3.7. Tổng kết	80
3.8. Đọc thêm	81
 4. Giao tiếp Đồng bộ — REST, gRPC & Resilience	 83
4.1. Bạn sẽ học được gì	83
4.2. Giao tiếp đồng bộ — Khi nào nên, khi nào tránh?	83

4.2.1. Request-Response: mô hình quen thuộc	83
4.2.2. Ưu và nhược điểm	84
4.2.3. Design-time Coupling vs Runtime Coupling	84
4.2.4. Khi nào dùng sync?	85
4.2.5. Interaction Styles — Phân loại kiểu tương tác	85
4.2.6. REST vs gRPC — Hai lựa chọn cho sync	86
4.2.7. Resilience Metrics — Đo lường độ tin cậy	87
4.3. OpenFeign — Declarative REST Client	88
4.3.1. Vấn đề: gọi service khác không đơn giản như gọi hàm	88
4.3.2. OpenFeign: gọi REST như gọi hàm	88
4.3.3. Cách hoạt động	88
4.4. Service Discovery & Load Balancing	89
4.4.1. Vấn đề: URL cứng trong distributed system	89
4.4.2. Service Discovery patterns	89
4.4.3. Netflix Eureka trong KBLab	89
4.5. Resilience Patterns — Sống sót trong Distributed System	90
4.5.1. Tại sao cần resilience?	90
4.5.2. Bốn resilience patterns cốt lõi	91
4.6. Case Study: KBLab SQL Judge — Coordinator và DBMS Workers	94
4.6.1. Bài toán	94
4.6.2. Hiện trạng: Strategy Pattern + OpenFeign	94
4.6.3. Phân tích các vấn đề	95
4.6.4. Đề xuất migration	95
4.6.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice	96
4.7. Tổng kết	96
4.8. Đọc thêm	97
5. Giao tiếp Bất đồng bộ — Kafka, Events & Messaging	97
5.1. Bạn sẽ học được gì	97
5.2. Tại sao cần Async? — Giới hạn của giao tiếp đồng bộ	98
5.2.1. Vấn đề mà sync không giải quyết được	98
5.2.2. Messaging patterns	98
5.3. Message Broker — Hai trường phái thiết kế	99
5.3.1. Durable vs Ephemeral Brokers	99
5.3.2. RabbitMQ — Smart Routing & Flexible Messaging	100
5.3.3. So sánh toàn diện	102
5.3.4. Khi nào dùng gì?	102
5.4. Apache Kafka — Kiến trúc chi tiết	103

5.5. Producer/Consumer Pattern trong KBLab	104
5.5.1. Kafka Producer	104
5.5.2. Kafka Consumer	104
5.5.3. Flow hoàn chỉnh	105
5.6. Delivery Guarantees & Idempotency	105
5.6.1. Ba cấp độ đảm bảo	105
5.6.2. Idempotency — Thiết kế to chịu được duplicate	106
5.7. Event Schema Design	107
5.7.1. Cấu trúc một event	107
5.7.2. Bốn loại message	107
5.7.3. Schema evolution	108
5.7.4. Kafka vs DB Queue trong KBLab	109
5.8. WebSocket — Real-time Notifications	109
5.8.1. Bài toán	109
5.9. Case Study: Kafka Pipeline trong Contest Mode	110
5.9.1. Bài toán Contest	110
5.9.2. Kiến trúc 4-topic pipeline	110
5.9.3. Phân tích các vấn đề	111
5.9.4. Đề xuất migration	111
5.10. Tổng kết	112
5.11. Đọc thêm	113
6. Giao dịch Phân tán — Saga Pattern	113
6.1. Bạn sẽ học được gì	113
6.2. Vấn đề: Distributed Transactions	113
6.2.1. Khi một transaction span nhiều services	113
6.2.2. Tại sao 2PC không phù hợp?	114
6.3. Saga Pattern — Chuỗi Local Transactions + Compensation	115
6.3.1. Định nghĩa	115
6.3.2. KBLab Submit Saga	115
6.3.3. Ba loại transactions trong saga	116
6.4. Choreography vs Orchestration	117
6.4.1. Choreography — Phi tập trung, event-driven	117
6.4.2. Orchestration — Tập trung, commanding	117
6.4.3. Khi nào dùng gì?	118
6.5. Compensating Transactions & Isolation	119
6.5.1. Thiết kế compensation trong KBLab	119
6.5.2. Vấn đề isolation — Anomalies	119

6.5.3. Countermeasures	120
6.5.4. Tổng hợp: Anomaly → Countermeasure	122
6.6. Eventual Consistency — Chấp nhận và quản lý	122
6.6.1. Từ ACID đến BASE	122
6.6.2. Consistency window trong KBLab	122
6.6.3. Strategies cho UI và business	123
6.7. Case Study: Submit Flow trong KBLab — Implicit Saga	123
6.7.1. Hiện trạng: Implicit Saga (Choreography)	123
6.7.2. Phân tích các vấn đề	124
6.7.3. Đề xuất migration	124
6.7.4. Workflow học vụ cũng là saga	125
6.8. Tổng kết	126
6.9. Đọc thêm	127
7. Quản lý Dữ liệu trong Microservices	127
7.1. Bạn sẽ học được gì	127
7.2. Database-per-Service — Nguyên tắc Data Ownership	127
7.2.1. Vấn đề: shared database phá vỡ service independence	127
7.2.2. Database-per-service pattern	128
7.2.3. Industry Case Study: Uber — Domain-Oriented Microservice Architecture (DOMA)	130
7.2.4. CAP Theorem — Hiểu đúng giới hạn	130
7.2.5. Caching Strategies — Giảm load và latency	132
7.3. Chiến lược tách Database từ Monolith	134
7.3.1. Vấn đề: tách service dễ, tách data khó	134
7.3.2. Năm chiến lược tách database	134
7.3.3. Áp dụng cho KBLab	135
7.4. Data Duplication vs Coupling — Khi nào chấp nhận duplicate?	135
7.4.1. Vấn đề: cần data từ service khác	135
7.4.2. Ba chiến lược truy cập data xuyên service	135
7.4.3. Khi nào chấp nhận duplication?	138
7.5. CQRS — Command Query Responsibility Segregation	139
7.5.1. Từ CQS đến CQRS	139
7.5.2. Vấn đề: cùng model cho read và write không đủ	139
7.5.3. CQRS pattern	139
7.5.4. Khi nào cần CQRS?	140
7.5.5. Ví dụ: Leaderboard trong Contest Mode	141
7.6. Event Sourcing — Lưu Events thay vì State	142
7.6.1. Vấn đề: mất lịch sử khi chỉ lưu state	142

7.6.2. Event Sourcing pattern	142
7.6.3. Snapshot Pattern — Giải quyết Performance của Event Replay	143
7.6.4. Projection Rebuilds — Tạo lại Views từ Events	143
7.6.5. Event Schema Evolution — Versioning Events	144
7.6.6. Khi nào dùng Event Sourcing?	145
7.7. Case Study: Quản lý dữ liệu trong KBLab	145
7.7.1. Hiện trạng	145
7.7.2. Phân tích cross-service query patterns	146
7.7.3. Từ hiện trạng đến best practice	147
7.7.4. Đề xuất migration path	147
7.8. Tổng kết	148
7.9. Đọc thêm	149
8. API Gateway	151
8.1. Bạn sẽ học được gì	151
8.2. API Gateway Pattern — Tại sao cần?	151
8.2.1. Vấn đề: client giao tiếp trực tiếp với nhiều services	151
8.2.2. API Gateway pattern	152
8.2.3. API Gateway vs BFF (Backend for Frontend)	153
8.3. Spring Cloud Gateway — Reactive Gateway	154
8.3.1. Vấn đề: chọn gateway technology	154
8.3.2. Kiến trúc Spring Cloud Gateway	155
8.3.3. Dependency	155
8.4. Route Configuration với Eureka	155
8.4.1. Vấn đề: routes hardcoded hay dynamic?	155
8.4.2. LMS Gateway Route Configuration	156
8.4.3. Cách lb:// hoạt động	157
8.5. Cross-Cutting Concerns tại Gateway	157
8.5.1. Vấn đề: mỗi service tự xử lý authentication, CORS, logging	157
8.5.2. Authentication — JWT Validation tại Gateway	157
8.5.3. CORS — Cross-Origin Resource Sharing	159
8.5.4. Rate Limiting	159
8.5.5. Logging & Tracing	161
8.6. Case Study: Gateway trong KBLab	161
8.6.1. Kiến trúc tổng thể	161
8.6.2. Phân tích configuration	162
8.6.3. Vấn đề JWT Version Inconsistency	162
8.6.4. Đề xuất migration	162

8.7. Tổng kết	163
8.8. Đọc thêm	164
9. Bảo mật Microservices	164
9.1. Bạn sẽ học được gì	164
9.2. Security Challenges trong Microservices	165
9.2.1. Vấn đề: attack surface mở rộng	165
9.2.2. Nguyên tắc “Defense in Depth”	166
9.2.3. Advanced: mTLS, Secrets Management, OAuth2 Scopes	167
9.3. JWT — Cấu trúc và Cơ chế	168
9.3.1. Vấn đề: sessions không scale trong microservices	168
9.3.2. JWT (JSON Web Token)	168
9.3.3. HS256 vs RS256	169
9.3.4. JWT Flow trong KBLab	170
9.3.5. Token Refresh & Rotation — Vòng đời của JWT	170
9.4. Dual Validation Strategy	172
9.4.1. Vấn đề: validate ở đâu? Mỗi service hay chỉ gateway?	172
9.4.2. LMS approach: Dual Validation	172
9.5. OAuth2 — External Identity Provider	174
9.5.1. Vấn đề: quản lý credentials phức tạp	174
9.5.2. OAuth2 Authorization Code Flow	174
9.5.3. Multiple Authentication Methods	174
9.6. RBAC — Role-Based Access Control	174
9.6.1. Vấn đề: ai được làm gì?	174
9.6.2. RBAC Implementation trong KBLab	175
9.6.3. Role Hierarchy	175
9.6.4. API-driven Route Protection (Frontend)	176
9.7. Case Study: Kiến trúc bảo mật trong KBLab	176
9.7.1. Tổng quan	176
9.7.2. Phân tích tổng hợp	177
9.7.3. Đề xuất migration	177
9.7.4. Secret Management trong Production	178
9.8. Tổng kết	180
9.9. Đọc thêm	181
10. Chuyển đổi Thực tế — Từ Monolith đến Microservices	181
10.1. Bạn sẽ học được gì	181
10.2. Khi nào (KHÔNG) nên chuyển sang Microservices	182
10.2.1. Vấn đề: microservices không phải đích đến mặc định	182

10.2.2. Decision Matrix — Khi nào cân nhắc migration	182
10.2.3. KBLab: Evolutionary Architecture, không phải Big Bang	182
10.2.4. Bài học thực tế — Khi Migration đi sai hướng (Anti-patterns)	183
10.3. Strangler Fig Pattern — Migration Tăng dần	184
10.3.1. Vấn đề: “Big Bang” vs Tăng dần migration	184
10.3.2. Strangler Fig Pattern	184
10.3.3. API Gateway — “Cây đa” trong microservices	185
10.4. Tách Database — Thách thức Lớn nhất	186
10.4.1. Vấn đề: database coupling nguy hiểm hơn code coupling	186
10.4.2. chiến lược tách database	187
10.4.3. Dual-Write Pitfall và Outbox Pattern	188
10.5. Anti-Corruption Layer và Migration Patterns	190
10.5.1. Anti-Corruption Layer (ACL) — Bảo vệ ranh giới	190
10.5.2. Branch by Abstraction	191
10.5.3. Parallel Run — Verify trước khi switch	192
10.6. Migration Roadmap cho KBLab — Tổng hợp xuyên suốt	193
10.6.1. Tổng hợp Gap Analyses	193
10.6.2. Migration Roadmap — 4 Phases	195
10.6.3. Decision Matrix — Ưu tiên bằng Impact × Effort	197
10.7. Sai lầm thường gặp khi Migration	198
10.8. Tổng kết	198
10.9. Đọc thêm	199
11. Observability	199
11.1. Bạn sẽ học được gì	200
11.2. Ba trụ cột của Observability	200
11.2.1. Vấn đề: “hệ thống lỗi nhưng không biết lỗi ở đâu”	200
11.2.2. Ba trụ cột	201
11.2.3. Observability vs Monitoring	203
11.2.4. Observability Maturity Model	203
11.3. Logging — Từ Console đến Centralized Aggregation	204
11.3.1. Vấn đề: logs phân tán, không có context	204
11.3.2. Structured Logging — Từ text sang JSON	205
11.3.3. Centralized Log Aggregation	205
11.3.4. Correlation IDs — Kỹ thuật quan trọng nhất	206
11.4. Distributed Tracing — Theo dõi Request xuyên Services	207
11.4.1. Vấn đề: request chậm — chậm ở đâu?	207
11.4.2. Distributed Tracing — Tự động đo timing	208

11.4.3. Concepts: Trace, Span, Context Propagation	208
11.4.4. Sampling Strategies	208
11.4.5. Tracing trong Spring Boot 3.x	209
11.5. Metrics, SLI/SLO, và Alerting Strategy	209
11.5.1. Metrics — Đo lường hành vi theo thời gian	209
11.5.2. Hai framework đo lường: RED và USE	210
11.5.3. SLI/SLO/SLA — Từ metrics đến cam kết chất lượng	210
11.5.4. Alerting Strategy	211
11.6. Error Handling Strategy — Nhất quán xuyên Services	212
11.6.1. Vấn đề: mỗi service trả error format khác nhau	212
11.6.2. Nguyên tắc thiết kế Error Handling	212
11.6.3. Error Handling và Observability — Mối liên hệ	213
11.7. Health Checks và Service Discovery	214
11.7.1. Health Checks — Nền tảng cho tự động hóa	214
11.7.2. User Activity Tracking — Observability ở mức Business	214
11.7.3. Chaos Engineering — Kiểm tra Resilience chủ động	215
11.7.4. Industry Case Study: Netflix Simian Army	216
11.8. Case Study: Observability trong KBLab	217
11.8.1. Hiện trạng — Đánh giá theo Maturity Model	217
11.8.2. Phân tích theo business context	218
11.9. Tổng kết	220
11.10. Đọc thêm	221
12. Triển khai & DevOps	221
12.1. Bạn sẽ học được gì	222
12.2. Thách thức Triển khai Microservices	222
12.2.1. Vấn đề: từ “deploy một file WAR” đến “deploy N services đồng bộ”	222
12.2.2. Từ Manual đến Automation — DevOps Mindset	222
12.3. Containerization với Docker	223
12.3.1. Vấn đề: “Works on my machine”	223
12.3.2. Container — Lightweight Isolation	223
12.3.3. Deployment Patterns — Từ Language-Specific đến Serverless	224
12.3.4. Dockerfile — Định nghĩa Container Image	225
12.4. Docker Compose — Orchestration trên Một Máy	226
12.4.1. Vấn đề: chạy nhiều services + databases + Kafka bằng tay?	226
12.4.2. Docker Compose — Declarative Multi-Service	226
12.4.3. Docker Compose — Điểm mạnh và giới hạn	227
12.5. CI/CD Pipeline cho Microservices	227

12.5.1. Vấn đề: $N \text{ services} \times M \text{ stages} = \text{phức tạp}$	227
12.5.2. CI/CD Architecture cho Microservices	228
12.5.3. Mono-repo vs Poly-repo — Ảnh hưởng đến CI/CD	228
12.5.4. Pipeline Best Practices	229
12.6. Deployment Strategies	229
12.6.1. Vấn đề: deploy version mới mà không downtime	229
12.6.2. Ba strategy chính	229
12.6.3. So sánh	231
12.7. Infrastructure as Code (IaC)	231
12.7.1. Vấn đề: “Ai cấu hình server? Config ở đâu?”	231
12.7.2. IaC — Hạ tầng như Code	231
12.7.3. Docker Compose as IaC	231
12.7.4. Kubernetes — Container Orchestration cho Production	232
12.7.5. Serverless Deployment — Khi nào phù hợp?	235
12.7.6. Sidecar Pattern và Service Mesh	235
12.8. Case Study: Deployment Architecture của KBLab	236
12.8.1. Hiện trạng	236
12.8.2. Phân tích theo deployment maturity	238
12.8.3. Phân tích business context	238
12.9. Tổng kết	240
12.10. Đọc thêm	241
A. Phụ lục A: Bảng thuật ngữ	243
B. Phụ lục B: Công cụ & Tài nguyên	245
B.1. Frameworks & Libraries	245
B.2. Messaging & Data	246
B.3. Infrastructure & DevOps	246
B.4. Migration & Release Management	246
B.5. Observability	247
B.6. Security & Identity	247
B.7. Development Tools	247
C. Phụ lục C: Pattern Catalog	247
C.1. Communication Patterns	248
C.2. Resilience Patterns	248
C.3. Data Patterns	249
C.4. Infrastructure Patterns	249
C.5. Security Patterns	250
C.6. Observability Patterns	250

C.7. Deployment Patterns	251
C.8. Migration Patterns	251
C.9. DDD Patterns	251
D. Phụ lục D: Anti-pattern Catalog	252
D.1. Architecture & Design	253
D.2. API Design	254
D.3. Communication	255
D.4. Transactions & Data	256
D.5. Security & Gateway	258
D.6. Migration & Observability	259
D.7. Deployment & DevOps	261
E. Bài tập & Case Studies	261
E.1. Hướng dẫn sử dụng	261
E.2. Chương 1: Monolith → Microservices	262
E.2.1. 1-1  Khi nào KHÔNG nên Microservices? ●●	262
E.2.2. 1-2  Case Study: Shopify — Monolith tỷ đô ●●●	263
E.3. Chương 2: DDD & Bounded Contexts	263
E.3.1. 2-1  Event Storming: Thiết kế Hệ thống Đặt Xe ●●	263
E.3.2. 2-2  Shared Library vs API — Khi nào nên gì? ●●	263
E.4. Chương 3: Thiết kế API	264
E.4.1. 3-1  Case Study: Stripe API — Tại sao được coi là “gold standard”? ●●	264
E.4.2. 3-2  Debugging: API Breaking Change Incident ●●●	264
E.5. Chương 4: Giao tiếp Đồng bộ & Resilience	265
E.5.1. 4-1  Debugging: Cascading Failure — “Tại sao cả hệ thống chết?” ●●●	265
E.5.2. 4-2  REST vs gRPC vs GraphQL — Chọn gì cho scenario nào? ●●	266
E.6. Chương 5: Giao tiếp Bất đồng bộ	266
E.6.1. 5-1  Kafka vs RabbitMQ — Quyết định dựa trên gì? ●●	266
E.6.2. 5-2  Debugging: “Messages biến mất” — Dead Letter & Poison Pills ●●●	267
E.7. Chương 6: Saga Pattern	268
E.7.1. 6-1  Design: Uber Ride Saga — Choreography hay Orchestration? ●●●	268
E.7.2. 6-2  Trade-off: Choreography vs Orchestration ●●	268
E.8. Chương 7: Quản lý Dữ liệu	269
E.8.1. 7-1  CAP Theorem trong Thực tế — “Pick Two” có đúng không? ●●●	269
E.8.2. 7-2  Design: Database Migration Strategy cho KBLab ●●	270
E.9. Chương 8: API Gateway	271
E.9.1. 8-1  Case Study: Netflix Zuul → Spring Cloud Gateway → Custom Gateway ●●	271

E.9.2. 8-2  ADR: Rate Limiting Strategy ●●	271
E.10. Chương 9: Bảo mật	272
E.10.1. 9-1  Security Incident: JWT Token Theft ●●●	272
E.10.2. 9-2  OAuth2 + OIDC Flows — Chọn flow nào cho scenario nào? ●●	272
E.11. Chương 10: Chuyển đổi Thực tế	273
E.11.1. 10-1  Case Study: Amazon — Monolith → SOA → Microservices (2002–2020) ●●●	273
E.11.2. 10-2  Outbox Pattern vs Event Sourcing vs Change Data Capture ●●	274
E.12. Chương 11: Observability	274
E.12.1. 11-1  Debugging: Latency Spike Mystery ●●●	274
E.12.2. 11-2  Observability Stack: Build vs Buy vs Managed ●●	275
E.13. Chương 12: Triển khai & DevOps	275
E.13.1. 12-1  Docker Compose vs Kubernetes — At What Scale? ●●	275
E.13.2. 12-2  Design: CI/CD Pipeline cho Microservices ●●●	276
E.14. Capstone: System Design Interview — Thiết kế “Online Judge” ●●●	277
F. Tài liệu tham khảo	277
F.1. Sách tham khảo chính (theo chương)	278
F.1.1. Chương 1	278
F.1.2. Chương 10	278
F.1.3. Chương 2	278
F.1.4. Chương 3, 4, 6, 8	278
F.1.5. Chương 5	278
F.1.6. Chương 7	278
F.1.7. Chương 9	278
F.1.8. Chương 11	278
F.1.9. Chương 12	278
F.2. Tài liệu bổ sung	278
F.3. Bài báo & Tài nguyên trực tuyến	279

Danh mục hình ảnh

Hình 1.1: Tiến hóa kiến trúc phần mềm doanh nghiệp từ thập niên 1990 đến 2020	27
Hình 1.2: Bốn kiểu kiến trúc SOA — từ dịch vụ đơn lẻ đến quy mô doanh nghiệp	29
Hình 1.3: Scale Cube — ba chiều mở rộng hệ thống	30
Hình 1.4: Cây quyết định chuyển đổi kiến trúc — từ monolith đến microservices	32
Hình 1.5: Fast Flow Success Triangle — ba yếu tố cần thiết cho fast flow	33
Hình 1.6: Con đường từ Patchwork Monolith đến Microservices	36
Hình 1.7: Decision Framework — ba câu hỏi trước khi chọn microservices	37
Hình 1.8: Kiến trúc tổng quan hệ thống KBLab LMS	40
Hình 2.1: Conway’s Law (thụ động) vs Inverse Conway Maneuver (chủ động)	47
Hình 2.2: Mô hình Squad/Tribe/Chapter/Guild của Spotify	48
Hình 2.3: Các building blocks trong DDD — Aggregate, Entity, Value Object, Repository .	52
Hình 2.4: Cùng thuật ngữ “Submission” mang ý nghĩa khác nhau trong ba bounded context	54
Hình 2.5: Context Map của KBLab — quan hệ giữa các bounded contexts	55
Hình 2.6: Event Storming — luồng từ Command đến Domain Event	56
Hình 2.6b: Áp dụng Erl’s Service-Oriented Analysis cho KBLab — từ automation candidates đến service layers	58
Hình 2.7: Năm bounded contexts chính trong KBLab	60
Hình 2.8: Hướng tách shared library — từ monolithic sang modular	63
Hình 3.1: Richardson Maturity Model — bốn cấp độ trưởng thành của REST API	67
Hình 3.2: Quy tắc schema evolution — thay đổi nào là breaking change	72
Hình 3.3: Pipeline từ code annotations đến Swagger UI và client SDK	73
Hình 3.4: DTO pattern — tách internal model và external contract	74
Hình 3.6b: Hexagonal Architecture cho Core Service trong KBLab	75
Hình 4.1: Giao tiếp đồng bộ — Core Service blocked trong khi chờ Judge response	83
Hình 4.2: Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing	88
Hình 4.3: Client-side discovery — service tự query Eureka và load balance	89

Hình 4.4: Server-side discovery — load balancer điều hướng request	89
Hình 4.5: Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng	90
Hình 4.6: Circuit Breaker state machine — Closed, Open, Half-Open	91
Hình 4.7: Bulkhead pattern — thread pool cách ly theo function	92
Hình 4.8: SQL Judge coordinator — dispatch SQL đến các worker <code>judge-*</code> theo DBMS	94
Hình 4.9: Lộ trình migration cho SQL Judge — từ resilience đến async	95
Hình 5.1: So sánh giao tiếp đồng bộ (coupled) và bất đồng bộ (decoupled)	98
Hình 5.2: Point-to-Point (Queue) vs Publish/Subscribe (Topic)	99
Hình 5.3: Ephemeral broker (message xóa sau ack) vs Durable broker (message vẫn còn)	100
Hình 5.4: Kiến trúc RabbitMQ — Exchange routing messages đến queues	101
Hình 5.5: Kiến trúc Kafka — Topic, Partitions và Consumer Group	103
Hình 5.6: Luồng hoàn chỉnh — từ submit qua Kafka đến kết quả qua WebSocket	105
Hình 5.7: Kiến trúc 4-topic pipeline chấm bài trong Contest mode	110
Hình 6.1: Two-Phase Commit — Coordinator điều phối prepare và commit	114
Hình 6.2: KBLab Submit Saga — chuỗi local transactions và compensating transactions	116
Hình 6.3: Choreography — các service tự phối hợp qua events	117
Hình 6.4: Orchestration — saga orchestrator điều phối toàn bộ flow	118
Hình 6.5: Lost Updates anomaly — hai saga ghi đè kết quả của nhau	120
Hình 6.6: Implicit Saga trong KBLab — choreography qua Kafka events	123
Hình 7.1: Ba hậu quả của shared database trong microservices	128
Hình 7.2: Shared Database (đọc/ghi chung) vs Database-per-Service (data qua API)	129
Hình 7.3: CAP Theorem — CP (chọn consistency) vs AP (chọn availability) khi partition	131
Hình 7.4: Ba caching patterns — Cache-Aside, Read-Through, Write-Behind	133
Hình 7.5: Năm chiến lược tách database — từ ít rủi ro đến nhiều rủi ro	134
Hình 7.7: Truy cập dữ liệu xuyên service bằng API Call	136
Hình 7.8: Truy cập dữ liệu xuyên service bằng Data Duplication qua Event Pipeline	137
Hình 7.9: API Composition pattern — Gateway hoặc BFF gọi nhiều services và tổng hợp	138
Hình 7.10: CQRS pattern — tách command model (write) và query model (read)	140

Hình 7.11: CQRS cho Leaderboard — write side publish events, read side denormalized .	141
Hình 7.12: Event Sourcing — lưu chuỗi events thay vì chỉ state hiện tại	142
Hình 7.12b: Snapshot pattern — replay từ snapshot thay vì từ đầu	143
Hình 7.13: Projection Rebuilds — tạo views mới từ events mà không cần migration . . .	144
Hình 7.14: Kiến trúc dữ liệu hiện tại của KBLab — shared database là gap chính	146
Hình 8.1: Không có Gateway — client phải biết địa chỉ của từng service	151
Hình 8.2: API Gateway — single entry point route đến từng service	152
Hình 8.3: API Gateway (một gateway chung) vs BFF (gateway riêng cho từng client) . .	153
Hình 8.4: Kiến trúc Spring Cloud Gateway — Predicates, Filters, Routes	155
Hình 8.5: Luồng <code>lb://</code> — Eureka lookup + load balance + route	157
Hình 8.6: Luồng JWT validation tại Gateway — claims propagation qua trusted headers .	159
Hình 8.7: Kiến trúc tổng thể KBLab — Gateway là single entry point	161
Hình 9.1: Monolith (1 điểm bảo vệ) vs Microservices (N điểm bảo vệ)	165
Hình 9.2: Defense in Depth — bảo vệ nhiều lớp từ network đến data	166
Hình 9.3: Cấu trúc JWT — Header, Payload, Signature	168
Hình 9.4: JWT Flow trong KBLab — login, token generation, và subsequent requests . .	170
Hình 9.5b: Token Refresh & Rotation flow — access token mới + refresh token mới . . .	171
Hình 9.5: Dual Validation — full validation (Auth) vs claims-only (internal services) . .	173
Hình 9.6: Role Hierarchy — ADMIN inherits LECTURER inherits STUDENT	175
Hình 9.7: Kiến trúc bảo mật tổng thể của KBLab	176
Hình 9.8: Vault Dynamic Secrets — credentials tạm thời, tự động xoay vòng	179
Hình 10.1: Strangler Fig Pattern — dần thay thế monolith qua 3 giai đoạn	185
Hình 10.2: API Gateway làm seam cho Strangler Fig — client không biết route nào đến monolith, route nào đến service mới	185
Hình 10.3: Shared Database (coupled) vs Database-per-Service (decoupled)	186
Hình 10.4: Thứ tự thực hiện 5 chiến lược tách database	188
Hình 10.5: Dual-Write Pitfall — DB thành công nhưng event thất bại	189
Hình 10.6: Outbox Pattern — event ghi cùng transaction với business data	189
Hình 10.7: Anti-Corruption Layer — lớp dịch giữa service mới và legacy	191
Hình 10.8: Branch by Abstraction — migration an toàn qua feature flag	191

Hình 10.9: Parallel Run — chạy song song old và new logic, so sánh output	192
Hình 10.10: Migration Roadmap 4 phases — Quick Wins → Observability → Resilience → Decomposition	195
Hình 10.11: Decision Matrix — ưu tiên theo Impact × Effort	197
Hình 11.1: Request flow qua 5+ services — lỗi xảy ra ở đâu?	201
Hình 11.2: Ba trụ cột của Observability — Logs, Metrics, Traces	202
Hình 11.3: Logs phân tán (manual debug) vs Logs tập trung — Search một nơi	205
Hình 11.5: Truyền Correlation ID xuyên suốt request, kể cả qua Kafka	207
Hình 11.6: Gantt chart của một Trace — hiển thị timing cho từng Span	208
Hình 11.7: Mối quan hệ giữa SLI, SLO và SLA	211
Hình 11.8: User Activity Tracking pipeline qua Kafka	215
Hình 11.9: Đánh giá hiện trạng Observability của KBLab	217
Hình 12.1: Triển khai thủ công (Manual) vs Tự động (Automated Pipeline)	223
Hình 12.2: So sánh kiến trúc Virtual Machines và Containers	224
Hình 12.3: CI/CD Pipeline chuẩn cho Microservices	228
Hình 12.4: Quá trình diễn ra Rolling Update	230
Hình 12.5: Kiến trúc Blue/Green Deployment	230
Hình 12.6: Kiến trúc Kubernetes — Control Plane và Worker Nodes	233
Hình 12.7: Sidecar Pattern — Proxy xử lý infrastructure concerns (mTLS, tracing)	236
Hình 12.8: Deployment Architecture KBLab ở mức khái quát	237

Danh mục bảng biểu

Bảng 1.1: Các biểu hiện khi monolith đạt đến giới hạn	26
Bảng 1.2: Tám nguyên lý hướng dịch vụ — SOA truyền thống vs Microservices	28
Bảng 1.3: So sánh SOA truyền thống và Microservices	31
Bảng 1.3b: Fast Flow Architecture Qualities	33
Bảng 1.4: DORA Metrics — So sánh Elite vs Low Performers	34
Bảng 1.4b: Patchwork Monolith vs Modular Monolith	35
Bảng 1.5: Lợi ích của kiến trúc microservices	37
Bảng 1.6: Chi phí và thách thức của microservices	38
Bảng 1.7: Ánh xạ bài toán kỹ thuật KBLab LMS theo chương	41
Bảng 1.8: Năm quyết định kiến trúc KBLab muốn làm lại	43
Bảng 2.1: Bốn kiểu team theo Team Topologies	47
Bảng 2.2: Cấu trúc tổ chức Spotify — ý nghĩa cho microservices	48
Bảng 2.3: Mười lực Dark Energy / Dark Matter	50
Bảng 2.3b: Database-first vs Domain-first	53
Bảng 2.4: Các kiểu quan hệ giữa bounded contexts	55
Bảng 2.7: Ba loại service theo Erl	57
Bảng 2.8: Hai phương pháp phân tách service — Erl Service-Oriented Analysis vs DDD .	59
Bảng 2.9: Ánh xạ bounded context → service trong KBLab	61
Bảng 2.10: Phân tích shared library — cái gì nên shared, cái gì không	62
Bảng 3.1: HTTP methods và ngữ nghĩa	67
Bảng 3.2: HTTP response codes thường dùng trong microservices	68
Bảng 3.3: Hai chiến lược pagination	68
Bảng 3.4: Các fields trong standardized error response (RFC 7807-inspired)	69
Bảng 3.5: Idempotency theo HTTP method	70
Bảng 3.6: Ba chiến lược API versioning	70
Bảng 3.6b: Decision criteria — chọn versioning strategy nào?	71

Bảng 3.7: Lợi ích của DTO pattern	75
Bảng 3.8: Phân tích API design trong KBLab — các điểm cần cải thiện	77
Bảng 3.9: Over-fetching và Under-fetching trong REST — GraphQL giải quyết	78
Bảng 3.10: REST vs GraphQL — so sánh chi tiết	78
Bảng 3.11: Khi nào chọn REST, khi nào chọn GraphQL	79
Bảng 4.1: Ưu và nhược điểm của giao tiếp đồng bộ	84
Bảng 4.1b: Hai loại coupling trong microservices	84
Bảng 4.2: Interaction styles trong microservices theo Richardson [2a, Ch.3]	85
Bảng 4.3: REST vs gRPC — so sánh chi tiết	86
Bảng 4.4: Resilience metrics cốt lõi	87
Bảng 4.5: Availability “nines” — downtime cho phép theo target	87
Bảng 4.6: Thành phần Eureka trong KBLab	90
Bảng 4.7: Bốn resilience patterns cốt lõi	91
Bảng 4.8: Giá trị recommend cho từng environment	93
Bảng 4.9: Phân tích vấn đề SQL Judge coordinator/worker	95
Bảng 5.1: Giao tiếp đồng bộ vs bất đồng bộ — giải pháp cho ba vấn đề	98
Bảng 5.2: Các concepts cốt lõi của RabbitMQ	101
Bảng 5.3: Apache Kafka vs RabbitMQ — so sánh toàn diện	102
Bảng 5.4: Khi nào chọn Kafka, khi nào chọn RabbitMQ	102
Bảng 5.5: Các concepts cốt lõi của Apache Kafka	103
Bảng 5.6: Ba cấp độ delivery guarantee	106
Bảng 5.7: Cấu trúc event — metadata và payload	107
Bảng 5.8: Bốn loại message trong event-driven architecture	108
Bảng 5.8b: Chọn Kafka hay DB queue theo use case	109
Bảng 5.9: Ba cách đưa kết quả đến client	109
Bảng 5.10: Chi tiết các Kafka topics trong pipeline	111
Bảng 5.11: Phân tích vấn đề Kafka pipeline trong KBLab	111
Bảng 6.1: Tại sao 2PC không phù hợp với microservices	115
Bảng 6.2: Ba loại transactions trong saga — áp dụng cho KBLab Submit Saga	116

Bảng 6.3: Ưu và nhược điểm của Choreography	117
Bảng 6.4: Ưu và nhược điểm của Orchestration	118
Bảng 6.5: Choreography vs Orchestration — khi nào dùng gì	118
Bảng 6.6: Thiết kế compensation trong KBLab	119
Bảng 6.7: Anomaly → Countermeasure — áp dụng cho KBLab	122
Bảng 6.8: ACID (Monolith) vs BASE (Microservices)	122
Bảng 6.9: Strategies cho UI khi chấp nhận eventual consistency	123
Bảng 6.10: Phân tích vấn đề Submit Flow trong KBLab	124
Bảng 6.11: Ví dụ workflow học vụ dưới góc nhìn Saga	125
Bảng 7.1: Ba cấp độ tách database	129
Bảng 7.2: Uber — hành trình từ microservices spaghetti đến DOMA	130
Bảng 7.3: CP vs AP cho từng service trong KBLab	132
Bảng 7.4: So sánh ba caching patterns	133
Bảng 7.5: Chiến lược cache invalidation	133
Bảng 7.6: Migration path tách database cho KBLab	135
Bảng 7.7: Ưu và nhược điểm của API Call pattern	136
Bảng 7.8: Ưu và nhược điểm của Data Duplication pattern	137
Bảng 7.9: Khi nào chấp nhận data duplication	138
Bảng 7.10: Khi nào cần CQRS	140
Bảng 7.11: Ưu và nhược điểm của Event Sourcing	143
Bảng 7.12: Chiến lược snapshot	143
Bảng 7.13: Event Store — các công cụ phổ biến	144
Bảng 7.14: Khi nào dùng Event Sourcing	145
Bảng 7.15: Tổng hợp vấn đề data management trong KBLab và chiến lược migration . .	147
Bảng 8.1: Vấn đề khi client gọi trực tiếp microservices	152
Bảng 8.2: API Gateway vs BFF — khi nào dùng gì	153
Bảng 8.3: Kịch bản chọn Single Gateway vs BFF	154
Bảng 8.4: Spring Cloud Gateway vs Netflix Zuul	154
Bảng 8.5: Ba khái niệm cốt lõi của Spring Cloud Gateway	155

Bảng 8.6: Chiến lược rate limiting	160
Bảng 8.7: Phân tích configuration Gateway trong KBLab	162
Bảng 9.1: Thách thức bảo mật trong microservices	166
Bảng 9.2: So sánh không có mTLS và có mTLS	167
Bảng 9.3: Các cấp độ lưu trữ secrets	167
Bảng 9.4: Ba phần của JWT — nội dung và ví dụ	169
Bảng 9.5: HS256 (symmetric) vs RS256 (asymmetric)	169
Bảng 9.5b: Chiến lược token expiry	172
Bảng 9.6: Ba loại validation trong KBLab	173
Bảng 9.7: Các phương thức xác thực trong KBLab	174
Bảng 9.8: RBAC — vai trò và permissions trong KBLab	175
Bảng 9.9: Phân tích bảo mật toàn diện KBLab	177
Bảng 9.10: Chiến lược Secret Management — từ cơ bản đến enterprise	178
Bảng 10.1: Decision Matrix — khi nào cân nhắc migration	182
Bảng 10.1b: Anti-patterns khi chuyển đổi Microservices (minh họa với KBLab)	184
Bảng 10.2: Big Bang vs Tăng dần migration	184
Bảng 10.3: Ba implementation strategies cho Strangler Fig	186
Bảng 10.4: 5 chiến lược tách database — từ góc migration	187
Bảng 10.5: Hai cách implement Outbox Pattern	190
Bảng 10.6: Tổng hợp Gap Analyses xuyên suốt Ch.1–12	194
Bảng 10.7: Giai đoạn 1 — Quick Wins	195
Bảng 10.8: Giai đoạn 2 — Observability Foundation	196
Bảng 10.9: Giai đoạn 3 — Resilience & CI/CD	196
Bảng 10.10: Giai đoạn 4 — Database Decomposition	197
Bảng 11.1: Vai trò của ba trụ cột Observability	202
Bảng 11.2: Observability vs Monitoring	203
Bảng 11.3: Observability Maturity Model	204
Bảng 11.4: So sánh hai stack ELK và PLG	206
Bảng 11.5: Các khái niệm cơ bản trong Distributed Tracing	208

Bảng 11.6: Sampling strategies cho Tracing	209
Bảng 11.7: Bốn loại metrics cần thu thập	210
Bảng 11.8: Framework RED và USE	210
Bảng 11.9: Sự khác biệt giữa SLI, SLO và SLA	210
Bảng 11.10: SLO đề xuất cho KBLab	211
Bảng 11.11: Chiến lược Alerting theo mức độ nghiêm trọng	212
Bảng 11.12: Taxonomy hệ thống mã lỗi (Error Code)	213
Bảng 11.13: Ba loại Health Checks	214
Bảng 11.14: Ứng dụng của User Activity Tracking	215
Bảng 11.15: Phân loại Failure Injection trong Chaos Engineering	216
Bảng 11.16: Chaos experiments cho KBLab	216
Bảng 11.17: Netflix Simian Army	216
Bảng 11.18: Mức độ trưởng thành observability phân theo component	218
Bảng 11.19: Hai chế độ hoạt động chính của LMS	218
Bảng 12.1: Thách thức triển khai Monolith vs Microservices	222
Bảng 12.2: Ba nguyên tắc DevOps nền tảng	222
Bảng 12.3: So sánh chi tiết VM và Container	224
Bảng 12.4: Năm mô hình triển khai (Deployment Patterns)	225
Bảng 12.5: Điểm mạnh và giới hạn của Docker Compose	227
Bảng 12.6: Mono-repo vs Poly-repo	228
Bảng 12.7: Các best practices thiết kế CI/CD Pipeline	229
Bảng 12.8: So sánh các Deployment Strategies	231
Bảng 12.9: Các công cụ Infrastructure as Code phổ biến	231
Bảng 12.10: Các mức độ trưởng thành của IaC	232
Bảng 12.11: 6 Concepts cốt lõi trong Kubernetes	233
Bảng 12.12: So sánh chi tiết Docker Compose và Kubernetes	234
Bảng 12.13: Container (Docker/K8s) vs Serverless	235
Bảng 12.14: So sánh không dùng và dùng Service Mesh	236
Bảng 12.15: Phân tích mức độ trưởng thành triển khai của KBLab	238

Bảng 12.16: Deployment windows cho LMS	238
---	-----

PHẦN I

Nền tảng

Nền tảng kiến trúc — Hiểu rõ trước khi xây dựng

Phần đầu tiên thiết lập nền tảng tư duy kiến trúc phần mềm hướng dịch vụ. Chúng ta sẽ đi từ bức tranh toàn cảnh của SOA và Microservices, đến phương pháp phân tích hệ thống bằng Domain-Driven Design, và cuối cùng là nghệ thuật thiết kế dịch vụ và hợp đồng API. Sau phần này, bạn sẽ có khả năng nhìn một hệ thống monolith và nhận ra các đường ranh giới tự nhiên để tách thành các dịch vụ độc lập.

- Chương 1** Tổng quan SOA & Microservices
- Chương 2** Phân tích hướng Domain — DDD & Bounded Contexts
- Chương 3** Thiết kế Dịch vụ & API

1. Tổng quan SOA & Microservices

“Microservices are not the goal; sustainable fast flow of change is.” — Chris Richardson, *Microservices Patterns*, 2nd Edition

1.1. Bạn sẽ học được gì

- Hiểu bối cảnh lịch sử dẫn đến kiến trúc hướng dịch vụ (SOA) và microservices
- Nắm được các nguyên lý cốt lõi của SOA và sự kế thừa trong microservices
- Phân biệt rõ SOA truyền thống, microservices, và modular monolith
- Tìm hiểu cách các công ty công nghệ lớn đã buộc phải chuyển đổi — và bài học rút ra
- Đánh giá khi nào nên — và khi nào *không* nên — áp dụng microservices
- Làm quen với case study của sách: KBLab — một hệ thống LMS cho trường đại học

1.2. Kiến trúc phần mềm theo thời gian

Lịch sử phát triển phần mềm doanh nghiệp là câu chuyện về những lần vượt qua giới hạn: mỗi khi một hệ thống đạt đến quy mô mà kiến trúc hiện tại không còn đáp ứng được, cộng đồng lại tìm ra cách tiếp cận mới. Nhìn lại hành trình này giúp chúng ta hiểu *tại sao* microservices tồn tại, thay vì chỉ biết *nó là gì*.

1.2.1. Thời kỳ Monolith

Trong nhiều thập kỷ, hầu hết phần mềm được xây dựng theo kiến trúc **monolithic** (nguyên khối) — toàn bộ logic nghiệp vụ, giao diện người dùng, và truy cập dữ liệu nằm trong một ứng dụng duy nhất, triển khai như một đơn vị.

Kiến trúc monolith mang lại nhiều ưu điểm không thể phủ nhận. Một codebase duy nhất giúp phát triển đơn giản: mở IDE, chạy ứng dụng, debug với breakpoint — mọi thứ liền mạch. Deploy chỉ cần một artifact, một lệnh. Hiệu năng tốt vì gọi hàm cục bộ (in-process) luôn nhanh hơn gọi qua mạng nhiều bậc. Và quan trọng nhất, transactions trên database thống nhất đảm bảo data consistency một cách tự nhiên.

Tuy nhiên, khi hệ thống phát triển — thêm tính năng, thêm thành viên, thêm khách hàng — monolith bắt đầu bộc lộ những giới hạn nghiêm trọng. Thomas Erl gọi đây là kiến trúc dạng silo (*silo-based application architecture*) — một trạng thái mà mỗi ứng dụng trở thành “ốc đảo” với logic trùng lặp, khó tích hợp.

Các biểu hiện cụ thể:

Bảng 1.1: Các biểu hiện khi monolith đạt đến giới hạn

Biểu hiện	Mô tả	Hệ quả
Coordination bottleneck	Mỗi thay đổi nhỏ đòi hỏi phối hợp nhiều team	Tốc độ phát triển giảm dần theo kích thước team
Deployment coupling	Một tính năng mới = deploy lại toàn bộ	Release cycles kéo dài, rủi ro cao
Technology lock-in	Toàn ứng dụng dùng chung stack	Không thể thử nghiệm công nghệ mới cho một phần
Scaling inflexible	Scale = nhân bản toàn bộ ứng dụng	Chi phí cao, lãng phí tài nguyên
Cognitive overload	Codebase quá lớn, ranh giới module mờ nhạt	Developer mới mất tuần/tháng để hiểu hệ thống

Khi monolith đạt đến trạng thái mà không ai hiểu toàn bộ, mỗi thay đổi đều tiềm ẩn “side effect” không lường trước, và chu kỳ release kéo dài từ tuần sang tháng — đó chính là trạng thái mà Chris Richardson gọi là “**monolithic hell**”.

1.2.2. Sự ra đời của SOA

Đầu thập niên 2000, ngành công nghiệp phần mềm nhận ra rằng vấn đề không nằm ở quy mô ứng dụng, mà ở *cách tổ chức* logic bên trong. *Service-Oriented Architecture* (SOA — kiến trúc hướng dịch vụ) ra đời với ý tưởng cốt lõi: **phân rã hệ thống thành các dịch vụ có thể tái sử dụng**, giao tiếp qua giao thức chuẩn.

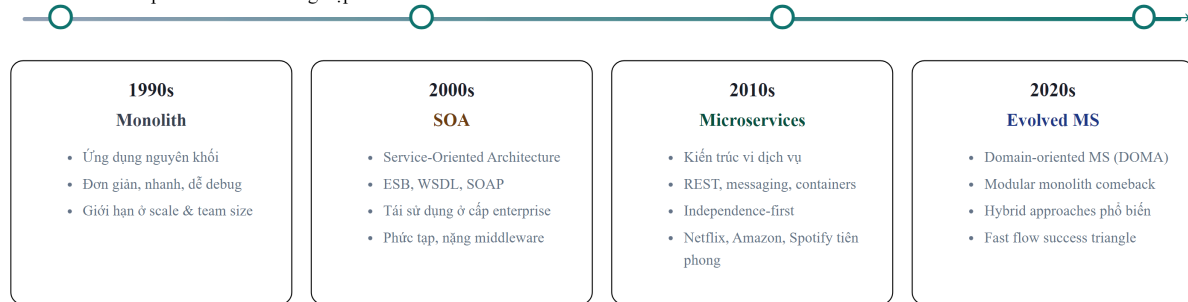
SOA hướng tới ba mục tiêu lớn: tăng khả năng tái sử dụng logic nghiệp vụ trên toàn doanh nghiệp, giảm tổng lượng code trùng lặp giữa các hệ thống, và cải thiện khả năng tích hợp giữa các nền tảng khác nhau.

Tuy nhiên, SOA truyền thống thường đi kèm với Enterprise Service Bus (ESB) — một middleware trung tâm điều phối mọi giao tiếp. ESB nhanh chóng trở thành điểm nghẽn: phức tạp, nặng nề, và tạo ra một dạng coupling mới — *coupling vào middleware*. Đây là bài học mà microservices rút ra để thiết kế theo triết lý ngược: “smart endpoints, dumb pipes”.

1.2.3. Từ SOA đến Microservices

Khoảng năm 2014, James Lewis và Martin Fowler đăng bài viết “Microservices” trên martinfowler.com, phổ biến rộng rãi thuật ngữ **microservices** để mô tả một phong cách kiến trúc đã tiến hóa từ SOA nhưng với triết lý khác biệt (thuật ngữ này được sử dụng trước đó tại các workshop phần mềm từ ~2011). Microservices không phải SOA phiên bản nhỏ — đây là sự chuyển dịch căn bản từ **tái sử dụng** (reuse) sang **tính độc lập** (independence) như ưu tiên số một.

Tiến hóa kiến trúc phần mềm doanh nghiệp



Hình 1.1: Tiến hóa kiến trúc phần mềm doanh nghiệp từ thập niên 1990 đến 2020

1.3. SOA là gì? Nguyên lý cốt lõi

Dù microservices đã trở thành xu hướng chính, việc hiểu SOA vẫn quan trọng vì hai lý do: thứ nhất, nhiều doanh nghiệp lớn vẫn vận hành hệ thống SOA; thứ hai, các nguyên lý hướng dịch vụ của SOA là **nền tảng** mà microservices kế thừa và phát triển.

1.3.1. Bốn đặc trưng cốt lõi

Thomas Erl xác định SOA qua bốn đặc trưng không thể thiếu:

Business-Driven — Dịch vụ được thiết kế từ quy trình nghiệp vụ, không phải từ bảng cơ sở dữ liệu hay module kỹ thuật. Một dịch vụ “Xử lý Đơn hàng” phản ánh quy trình kinh doanh, không phải bảng *orders* trong database. Nguyên tắc này vẫn đúng 100% trong microservices — và trở thành nền tảng cho Domain-Driven Design (DDD) mà chúng ta sẽ khám phá trong Chương 2.

Vendor-Neutral — Dịch vụ giao tiếp qua giao thức chuẩn (HTTP, messaging), không phụ thuộc vào công nghệ cụ thể. Đây là điều kiện tiên quyết cho **technology diversity** — một trong những lợi thế lớn nhất của microservices.

Enterprise-Centric — SOA tư duy ở cấp toàn tổ chức: dịch vụ không phục vụ riêng một ứng dụng mà phục vụ toàn doanh nghiệp. Microservices thu hẹp phạm vi này vào từng *bounded context* — nhưng vẫn giữ tư duy dịch vụ vượt ra ngoài một team duy nhất.

Composition-Centric — Dịch vụ được thiết kế để tổ hợp (compose) thành workflow phức tạp. Trong microservices, điều này thể hiện qua *Saga pattern* và *API Gateway* — các chủ đề của Chương 6 và 8.

1.3.2. Tám nguyên lý nền tảng

Erl đúc kết 8 nguyên lý nền tảng cho mọi dịch vụ. **Bảng 1.2** cho thấy microservices kế thừa và chuyển hóa chúng như thế nào:

Bảng 1.2: Tám nguyên lý hướng dịch vụ — SOA truyền thống vs Microservices

#	Nguyên lý	Trong SOA truyền thống	Trong Microservices
1	Standardized Contract	WSDL, XML Schema	OpenAPI/Swagger, Protocol Buffers
2	Loose Coupling	ESB giảm coupling	Ưu tiên số 1: database-per-service, async messaging
3	Abstraction	Interface ẩn implementation	API là contract duy nhất, code là black box
4	Reusability	Mục tiêu chính — tái sử dụng ở cấp enterprise	Giảm ưu tiên — <i>independence</i> quan trọng hơn <i>reuse</i>
5	Autonomy	Hạn chế bởi shared infrastructure	Mục tiêu chính — own database, own lifecycle
6	Statelessness	Stateless design	RESTful by default, state externalized (Redis, DB)
7	Discoverability	UDDI registry	Service Discovery (Eureka, Consul, K8s DNS)
8	Composability	ESB orchestration	Saga, API Gateway, event choreography

Điểm mấu chốt: microservices không phủ nhận SOA — mà **chọn lọc và đẩy mạnh** những nguyên lý phù hợp (Loose Coupling, Autonomy) và giảm nhẹ những nguyên lý gây coupling (Reusability ở cấp enterprise, centralized Discoverability).

1.3.3. Bốn kiểu kiến trúc SOA

SOA tồn tại ở nhiều cấp độ, từ nhỏ đến lớn:

Kiến trúc phân cấp dịch vụ — SOA theo Erl



Hình 1.2: Bốn kiểu kiến trúc SOA — từ dịch vụ đơn lẻ đến quy mô doanh nghiệp

Hầu hết dự án microservices thực tế hoạt động ở tầng **Service Architecture** và **Service Composition Architecture**. Sự khác biệt đáng chú ý: SOA truyền thống cố gắng quản lý một “Service Inventory” thống nhất cho toàn doanh nghiệp — trong khi microservices chấp nhận sự phân tán và trao quyền cho từng team.

1.4. Microservices là gì? So sánh với SOA

1.4.1. Định nghĩa và đặc trưng

Microservices là phong cách kiến trúc trong đó ứng dụng được cấu thành từ **tập hợp các dịch vụ nhỏ, tự trị**. Mỗi dịch vụ thỏa mãn bốn điều kiện:

1. **Chạy trong tiến trình riêng** — deploy và restart độc lập
2. **Sở hữu dữ liệu riêng** — database-per-service, không chia sẻ bảng
3. **Giao tiếp qua giao thức nhẹ** — HTTP/REST, gRPC, hoặc messaging (Kafka, RabbitMQ)
4. **Có thể deploy độc lập** — thay đổi service A không yêu cầu redeploy service B

Đặc biệt, điều kiện thứ tư (*independent deployability*) chính là thước đo quan trọng nhất. Nếu không thể deploy một service mà không phải deploy các service khác — đó chưa phải microservices thực sự.

1.4.2. Scale Cube — Ba chiều mở rộng

Để hiểu microservices trong bối cảnh scalability, chúng ta sử dụng mô hình **Scale Cube** — chia mở rộng hệ thống theo ba chiều:

The Scale Cube — 3 trục mở rộng hệ thống

X — Horizontal Cloning

Nhân bản instance để tăng throughput

Instance 1

Instance 2

Instance N

Load Balancer phân phối request giữa các bản sao giống hệt nhau.

Y — Functional Decomposition

Chia tách theo Microservices

Auth

Order

Payment

Mỗi service đảm nhận một domain riêng, deploy độc lập.

Z — Data Partitioning

Phân mảnh Database / Sharding

Users A–M

Users N–Z

Mỗi shard phục vụ một tập con dữ liệu theo key (vd: user ID range).

Khi nào dùng trục nào?

X-axis
Traffic tăng cao → scale nhanh bằng load balancer
Y-axis
Domain rõ ràng → tách service, team độc lập
Z-axis
Dataset lớn → sharding DB theo tenant/region

Kết hợp cả 3 trục cho hệ thống phân tán quy mô lớn

Hình 1.3: Scale Cube — ba chiều mở rộng hệ thống

- **Trục X** — Nhân bản: chạy nhiều instance giống nhau phía sau load balancer. Đơn giản nhất, giới hạn bởi database bottleneck
- **Trục Y** — Phân rã chức năng: tách ứng dụng thành các service theo domain. **Đây là microservices**
- **Trục Z** — Phân mảnh dữ liệu: chia data theo tiêu chí (region, user ID). Thường kết hợp với microservices

Scaling truyền thống (thêm server, tăng RAM) chỉ mở rộng trên trục X. Khi trục X đạt giới hạn — database trở thành bottleneck, deployment trở nên rủi ro — đó là lúc cần xem xét trục Y.

1.4.3. So sánh SOA truyền thống vs Microservices

Bảng 1.3: So sánh SOA truyền thống và Microservices

Khía cạnh	SOA truyền thống	Microservices
Phạm vi	Toàn doanh nghiệp	Một ứng dụng hoặc bounded context
Kích thước dịch vụ	Lớn, tái sử dụng tối đa	Nhỏ, focused, single responsibility
Giao tiếp	ESB — middleware trung tâm	“Smart endpoints, dumb pipes” — REST/messaging
Dữ liệu	Thường shared database	Database-per-service
Governance	Centralized, formal	Decentralized, team-owned
Triết lý thiết kế	Reuse-first	Independence-first
Deploy	Thường cùng deploy	Independent deployment
Team model	Chuyên môn theo layer (DB team, UI team)	Cross-functional, own the service

Cần lưu ý rằng SOA, microservices, và event-driven architecture (EDA) không phải các lựa chọn loại trừ nhau. Chúng là các lớp của một cách tiếp cận đang tiến hóa: SOA cung cấp nguyên lý nền tảng, microservices hiện thực hóa chúng ở mức granularity cao hơn, và EDA bổ sung khả năng giao tiếp bất đồng bộ cho các hệ thống phân tán phức tạp. Chúng ta sẽ khám phá EDA chi tiết trong Chương 5.

1.5. Microservices trong thực tế — Bài học từ các công ty công nghệ

Lý thuyết về microservices chỉ thực sự trở nên thuyết phục khi nhìn vào *tại sao* các tổ chức lớn buộc phải chuyển đổi. Không phải vì microservices “thời thượng” — mà vì họ đối mặt với **những giới hạn mà kiến trúc truyền thống và cách mở rộng theo trục X không thể vượt qua**.

1.5.1. Netflix — Khi downtime là không thể chấp nhận

Năm 2008, Netflix gặp sự cố corruption database nghiêm trọng, gây gián đoạn dịch vụ. Sự cố này là chất xúc tác cho quyết định chuyển toàn bộ hạ tầng lên cloud (AWS) và tái cấu trúc sang microservices. Theo Netflix Technology Blog, quá trình migration hoàn thành đầu năm 2016, khi hệ thống vận hành hàng trăm microservices phục vụ hơn 100 triệu người dùng (con số tại thời điểm 2016).

Vấn đề cốt lõi không phải là code xấu — mà là **yêu cầu nghiêm ngặt về uptime và scale**. Với lượng người dùng tăng exponential, một monolith duy nhất trở thành single point of failure. Vertical scaling (nâng cấp server) đạt giới hạn vật lý và chi phí. Netflix cần hai thứ mà monolith không thể cung cấp: khả năng scale từng thành phần riêng biệt, và **fault isolation** — lỗi ở hệ thống recommendation không được phép ảnh hưởng đến streaming.

Netflix đóng góp ngược lại cho cộng đồng bằng các công cụ mã nguồn mở: **Eureka** (service discovery), **Zuul** (API gateway), **Hystrix** (circuit breaker) — nhiều trong số đó trở thành nền tảng của Spring Cloud mà chúng ta sẽ sử dụng trong case study.

1.5.2. Amazon — API Mandate và “two-pizza teams”

Khoảng 2001–2002, Amazon đối mặt với tình trạng tightly coupled monolith — frontend và backend quấn chặt vào nhau đến mức *bất kỳ thay đổi nào, dù nhỏ, đều ảnh hưởng toàn hệ thống*. Theo câu chuyện nổi tiếng được Steve Yegge kể lại trong “Google Platforms Rant” (2011), Jeff Bezos ra “**API mandate**” — một chỉ thị nội bộ: tất cả team phải giao tiếp với nhau *chỉ* qua service interfaces, không có ngoại lệ.

 Lưu ý — Lưu ý nguồn

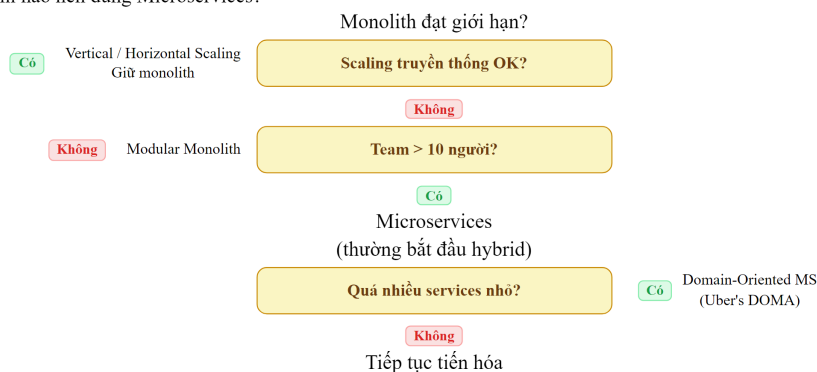
1.5.3. Uber — Thực tế của hybrid approach

Uber bắt đầu như một monolith Python đơn giản, phát triển chóng mặt, và khoảng 2014–2016 chuyển sang microservices. Tuy nhiên, câu chuyện Uber đáng chú ý ở phần *hậu chuyển đổi*: đến khoảng 2020, với hơn 2.000 microservices, họ nhận ra mình đang rơi vào “*microservice madness*” — quá nhiều service nhỏ tạo ra mạng lưới phụ thuộc phức tạp, khó debug và vận hành.

Uber phản ứng bằng cách giới thiệu **DOMA** (*Domain-Oriented Microservice Architecture*) — nhóm các microservices liên quan thành “domain” lớn hơn, giảm bề mặt tương tác. Kiến trúc cuối cùng của Uber là **hybrid** — không phải microservices thuần túy, cũng không quay về monolith.

1.5.4. Bài học tổng hợp

Cây quyết định — Khi nào nên dùng Microservices?



Hình 1.4: Cây quyết định chuyển đổi kiến trúc — từ monolith đến microservices

Ba pattern chung nổi lên:

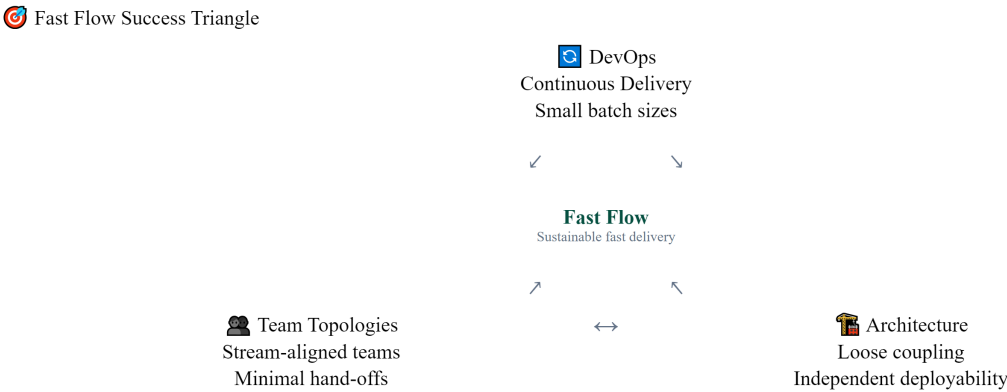
- Chuyển đổi vì bị buộc, không vì mong muốn.** Mỗi công ty trên đều chuyển sang microservices khi cách mở rộng truyền thống thất bại trước yêu cầu nghiêm ngặt — uptime 99.99%, hàng triệu requests/giây, hàng trăm developers cần deploy độc lập.
- Team structure thay đổi trước (hoặc cùng) kiến trúc.** Amazon chia two-pizza teams trước khi tách service. Netflix tạo “full cycle developers” — mỗi developer chịu trách nhiệm từ code đến production. Kiến trúc phần mềm phản ánh cấu trúc tổ chức (Conway’s Law) — chúng ta sẽ phân tích sâu trong Chương 2.
- Hybrid là trạng thái phổ biến nhất.** Rất ít tổ chức đạt “pure microservices”. Hầu hết — kể cả Netflix và Uber — vận hành ở trạng thái kết hợp, và đây hoàn toàn hợp lý.

! Nguyên tắc — Fast Flow

“The primary motivation for using the microservice architecture is to enable fast flow — the continuous delivery of a stream of small changes with rapid feedback on each one.”
— Chris Richardson, *Microservices Patterns*, 2nd Ed.

1.5.5. Fast Flow Success Triangle — DevOps + Team Topologies + Architecture

Richardson trong phiên bản thứ hai (2025) giới thiệu một framework quan trọng: **Fast Flow Success Triangle** — mô tả ba yếu tố cần thiết để đạt được fast flow:



Ba trụ cột phối hợp cùng nhau để đạt được luồng phân phối nhanh bền vững

Hình 1.5: Fast Flow Success Triangle — ba yếu tố cần thiết cho fast flow

DevOps (Quy trình) — Không phải tên team hay job title, mà là tập hợp nguyên tắc và practices để deliver phần mềm nhanh và tin cậy. Hai nguyên tắc quan trọng nhất: (1) *giảm batch size* — commit và push thay đổi nhỏ, lý tưởng ít nhất 1 lần/ngày; (2) *giảm hand-offs giữa các team* — mỗi hand-off tạo ra delay và mất thông tin.

Team Topologies (Tổ chức) — Cấu trúc tổ chức theo bốn kiểu team (stream-aligned, platform, enabling, complicated subsystem) với ba kiểu tương tác (X-aaS, collaboration, facilitating). Stream-aligned teams làm phần lớn công việc, các team khác hỗ trợ. Chi tiết ở §2.1.

Architecture (Kiến trúc) — Mục đích của kiến trúc, ít nhất từ góc độ fast flow, là **enable** DevOps và Team Topologies. Richardson trong [2b, Ch.5] xác định ba **architecture qualities** cần thiết:

Bảng 1.3b: Fast Flow Architecture Qualities

Quality	Định nghĩa	Ví dụ KBLab
Deployability	Mỗi service deploy độc lập, pipeline riêng, không cần coordinate với service khác	Judge Service deploy mà Core Service không cần biết
Testability	Test service isolated, không cần spin up toàn bộ hệ thống	Test Judge logic mà không cần Gateway, Auth, Kafka
Developability	Team thay đổi code dễ dàng, minimal conflicts, clear ownership	Team A sở hữu Judge, Team B sở hữu Assignment — không block nhau

Ba qualities này là *điều kiện cần* cho fast flow — thiếu bất kỳ quality nào sẽ tạo bottleneck. KBLab hiện tại vi phạm cả ba: shared library buộc rebuild tất cả (deployability ❌), shared database buộc integration test toàn bộ (testability ❌), và god classes 1000+ dòng tạo merge conflicts liên tục (developability ❌).

💡 Tip — Ba chân của tam giác

Thiếu bất kỳ chân nào, tam giác sụp đổ. DevOps tốt + Team tốt nhưng kiến trúc monolith coupling chặt → fast flow không thể đạt được. Ngược lại, microservices architecture tốt nhưng team vẫn tổ chức theo layer (DB team, UI team) → lợi ích bị triệt tiêu bởi coordination overhead.

1.5.6. Đo lường Fast Flow — DORA Metrics

Làm sao biết mình đã đạt fast flow? Nghiên cứu của Forsgren, Humble và Kim trong *Accelerate* (2018) định nghĩa bốn chỉ số DORA (DevOps Research and Assessment):

Bảng 1.4: DORA Metrics — So sánh Elite vs Low Performers

Metric	Elite Performers	Low Performers	Chênh lệch
Deployment Frequency	Nhiều lần/ngày	1 lần/1-6 tháng	182×
Lead Time (commit → production)	< 1 giờ	1-6 tháng	—
Change Failure Rate	< 5%	40-60%	8× tốt hơn
Recovery Time (MTTR)	< 1 giờ	1 tuần - 1 tháng	—

Nguồn: *State of DevOps 2024 report*

Điều đáng chú ý: **deploy thường xuyên hơn lại TIN CẬY hơn**. Elite performers deploy 182 lần nhiều hơn nhưng tỷ lệ lỗi thấp hơn 8 lần. Thay đổi nhỏ để test, dễ debug, dễ rollback — nguyên lý “move fast and *don’t* break things.”

🔍 Phân tích — DORA Metrics cho LMS

KBLab hiện tại:

- **Deployment Frequency:** Manual, khi cần (không scheduled) → Low Performer
- **Lead Time:** Chưa đo, nhưng thiếu CI/CD tự động → ước tính ngày-tuần
- **Change Failure Rate:** Không tracking → unknown
- **Recovery Time:** Manual restart → phút-giờ (acceptable cho academic)

Migration path: Implement CI/CD pipeline (Chương 12) là bước đầu tiên để cải thiện DORA metrics. Mục tiêu: đạt Medium Performer (deploy hàng tuần, lead time < 1 tuần).

1.5.7. Quality Attribute Scenarios — Chỉ định yêu cầu kiến trúc

DORA metrics đo lường kết quả cuối cùng, nhưng làm sao chỉ định kiến trúc phải hỗ trợ kết quả đó ngay từ khâu thiết kế? Richardson trong [2b, Ch.5] khuyên dùng **Quality Attribute Scenarios** — kỹ thuật do Software Engineering Institute (SEI) phát triển, cho phép viết “spec” cho các thuộc tính phi chức năng một cách đo lường được.

Thay vì nói chung chung “hệ thống phải dễ deploy”, một scenario định nghĩa rõ ràng theo format: **Source** → **Stimulus** → **Artifact** → **Environment** → **Response** → **Response Measure**

Ví dụ — deployability scenario cho LMS:

- **Source:** Developer (stream-aligned team)
- **Stimulus:** Commit code thay đổi logic chấm điểm SQL
- **Artifact:** Core Service
- **Environment:** Deployment pipeline
- **Response:** Pipeline tự động chạy CI, build, test, và deploy version mới lên production — không cần sự can thiệp của team khác
- **Response Measure:** Lead time < 40 phút

Ví dụ — scalability scenario cho LMS:

- **Source:** 200 sinh viên thi cùng lúc (contest mode)
- **Stimulus:** Spike 200 submissions trong 5 phút
- **Artifact:** Judge Service cluster
- **Response:** Tất cả submissions được xử lý mà không có request bị reject
- **Response Measure:** p99 latency < 10 giây, 0% message loss

Bằng cách viết ra các scenarios này, chúng ta có một dạng “unit test cho kiến trúc” — bất kỳ quyết định tách hay gộp service nào (Chương 2) đều phải đối chiếu lại: “*Quyết định này có giúp chúng ta đạt scenario hay không?*”

1.6. Modular Monolith — Lựa chọn hợp lệ

Trước khi nghĩ đến microservices, chúng ta cần dành thời gian nghiêm túc cho một lựa chọn thường bị bỏ qua: **Modular Monolith**. Đây không phải bước lùi — đây có thể là *bước đi thông minh nhất* tùy vào ngữ cảnh.

1.6.1. Monolith không phải là xấu

Không phải mọi monolith đều giống nhau. Sự khác biệt nằm ở **tổ chức nội bộ**:

Bảng 1.4b: Patchwork Monolith vs Modular Monolith

Loại	Đặc điểm	Triển vọng
Patchwork Monolith	Code chấp vá, mọi class phụ thuộc lẫn nhau, ranh giới mờ nhạt, “big ball of mud”	Cần refactor nghiêm túc — cả internal và external boundaries
Modular Monolith	Module rõ ràng, giao tiếp qua interface, mỗi module có thể có schema riêng trong cùng DB	Deploy cùng nhau nhưng phát triển tương đối độc lập, có thể tách dần khi cần

 Nguyên tắc — Monolith ≠ Bad

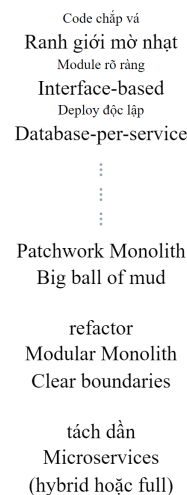
Monolith phản ánh kiến trúc đóng gói, không phản ánh chất lượng. Một modular monolith được thiết kế tốt hoàn toàn là lựa chọn kiến trúc hợp lệ — đặc biệt khi chi phí vận hành microservices vượt quá lợi ích mà nó mang lại.

1.6.2. Khi nào Modular Monolith phù hợp?

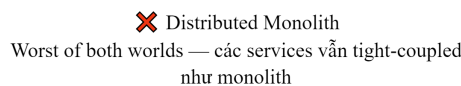
- **Team nhỏ (< 10 developers)** — Overhead vận hành microservices (CI/CD per service, monitoring, distributed debugging) có thể lớn hơn lợi ích
- **Domain chưa rõ ràng** — Chia service quá sớm khi chưa hiểu domain rõ ràng dẫn đến chia sai. Merge lại các service rất đau — đau hơn nhiều so với refactor module trong monolith
- **Startup/MVP giai đoạn** — Cần tốc độ phát triển, đưa sản phẩm ra thị trường nhanh
- **Không cần polyglot** — Nếu toàn team dùng cùng ngôn ngữ/framework, lợi ích technology diversity không tồn tại

1.6.3. Con đường từ Monolith đến Microservices

Con đường từ Monolith đến Microservices



△ Anti-Pattern: Tách trực tiếp không qua modular hóa



Bắt buộc đi qua Modular Monolith trước khi tách thành Microservices

Hình 1.6: Con đường từ Patchwork Monolith đến Microservices

Martin Fowler đề xuất chiến lược “**Monolith First**”: bắt đầu với monolith, hiểu rõ domain qua vài iteration, rồi mới tách microservices nếu thực sự cần. Lịch sử cho thấy hầu hết hệ thống microservices thành công — Netflix, Amazon, Uber — đều **bắt đầu từ monolith**, không phải microservices ngay từ đầu.

Điều nguy hiểm nhất là cố tách một patchwork monolith trực tiếp thành microservices mà không trải qua bước modular hóa. Kết quả thường là **distributed monolith** — mang toàn bộ nhược điểm của cả monolith lẫn distributed system, không có ưu điểm nào.

1.7. Khi nào nên — và không nên — dùng Microservices

1.7.1. Quyết định kiến trúc không chỉ là kỹ thuật

Điều quan trọng nhất cần nhấn mạnh trước khi đi vào chi tiết: **quyết định sử dụng microservices không phải thuần túy kỹ thuật**. Trong thực tế, các yếu tố phi kỹ thuật thường có ảnh hưởng lớn hơn cả yếu tố kỹ thuật:

- **Ngân sách** — Microservices đắt hơn để vận hành (infrastructure, monitoring, DevOps team)

- **Lịch trình** — Xây dựng microservices từ đầu chậm hơn monolith cho cùng feature set
- **Quy mô team** — Team < 5 người thường chịu overhead quá lớn
- **Kỹ năng hiện có** — Distributed systems đòi hỏi kinh nghiệm khác biệt
- **Văn hóa tổ chức** — Team silos + microservices = thảm họa

Ngay cả Netflix khi bắt đầu năm 2008, hay Uber năm 2014, kiến trúc hybrid ban đầu là phù hợp nhất cho bối cảnh lúc đó — không phải microservices thuần túy.

Trong cuốn sách này, chúng ta tập trung phân tích **khía cạnh kỹ thuật** — nhưng luôn ghi nhớ rằng đây chỉ là một chiều trong quyết định kiến trúc.

1.7.2. Decision Framework

Ba câu hỏi cần trả lời trước khi chọn microservices:

Decision Framework — 3 câu hỏi trước khi áp dụng Microservices



Hình 1.7: Decision Framework — ba câu hỏi trước khi chọn microservices

1.7.3. Phân tích lợi ích và chi phí

Bảng 1.5: Lợi ích của kiến trúc microservices

Lợi ích	Giải thích
Independent deployability	Deploy một service mà không ảnh hưởng các service khác — giảm rủi ro, tăng tần suất release
Team autonomy	Mỗi team sở hữu toàn bộ lifecycle — from code to production
Technology diversity	Chọn công nghệ phù hợp nhất cho từng bài toán cụ thể
Selective scalability	Scale riêng những service cần tài nguyên — tiết kiệm chi phí
Fault isolation	Lỗi ở một service không crash toàn bộ hệ thống

Bảng 1.6: Chi phí và thách thức của microservices

Chi phí	Giải thích
Complexity tax	Distributed systems phức tạp fundamentally: network latency, partial failures, eventual consistency — những thứ không tồn tại trong monolith
Operational overhead	CI/CD pipeline, monitoring, logging, tracing, service discovery <i>cho mỗi service</i>
Data consistency	Không có transactions xuyên service → phải dùng Saga pattern, eventual consistency
Debugging difficulty	Một request đi qua 5 services → cần distributed tracing (Zipkin, Jaeger)
Integration testing	Test toàn hệ thống trở nên khó khăn hơn bậc nhiều so với monolith

⚠ Lưu ý — Distributed Monolith

Nếu chia hệ thống thành nhiều “service” nhưng chúng vẫn phải deploy cùng nhau, chia sẻ database, và thay đổi đồng loạt — kết quả là một **distributed monolith**: mang toàn bộ nhược điểm của monolith VÀ distributed system, không có ưu điểm nào. Đây là kết quả phổ biến nhất khi tách service mà chưa modular hóa.

1.8. Case Study: Hệ thống KBLab LMS

1.8.1. Bài toán nghiệp vụ

Xuyên suốt cuốn sách này, chúng ta sử dụng một case study thực tế: **KBLab** — nền tảng học tập trực tuyến (LMS) chuyên biệt cho các môn tin học thực hành, được phát triển và vận hành tại một trường đại học kỹ thuật.

Vấn đề cần giải quyết. Tại trường đại học, các môn học liên quan đến Cơ sở dữ liệu, Lập trình mạng và Quản trị hệ thống/DevOps là bắt buộc cho hàng nghìn sinh viên mỗi năm. Giảng viên không thể thực thi và kiểm tra hàng trăm bài SQL, bài lập trình mạng hoặc bài thực hành Kubernetes mỗi tuần bằng tay. Sinh viên cần phản hồi nhanh để sửa lỗi ngay; giảng viên cần môi trường thi công bằng; hệ thống cần cô lập dữ liệu và môi trường thực hành để bài làm của sinh viên này không ảnh hưởng sinh viên khác.

Ba nhóm người dùng chính:

Nhóm	Vai trò	Quy mô
Sinh viên	Luyện tập SQL, tham gia thi, nộp bài tập, làm đồ án	Hàng nghìn / học kỳ
Giảng viên	Soạn đề, tổ chức thi/kiểm tra, chấm điểm, theo dõi tiến độ	Hàng chục
Quản trị viên	Quản lý tài khoản, cấu hình hệ thống, giám sát vận hành	Vài người

Bốn trụ cột nghiệp vụ — mỗi trụ cột đặt ra yêu cầu kiến trúc khác nhau:

- **SQL Lab:** Sinh viên viết SQL trên trình soạn thảo web, submit, và nhận kết quả trong Practice Mode hoặc Contest Mode. Practice Mode ưu tiên latency thấp; Contest Mode ưu tiên throughput, fairness, message queue và WebSocket realtime.
- **Network Lab:** Sinh viên viết Java client kết nối đến judge server qua nhiều nhóm giao thức: TCP, UDP, RMI, SOAP, REST và gRPC. Đây là nơi KBLab minh họa rất rõ technology diversity: không phải mọi service đều là Spring Boot REST API.
- **Course Management:** Giảng viên tạo khóa học, bài tập cá nhân/nhóm, MCQ, điểm danh, grade scheme, đồ án tốt nghiệp và tích hợp GitHub Classroom. Yêu cầu chính: data integrity, workflow nhiều bước, cross-service query.
- **DevOps Lab:** Sinh viên thực hành Docker/Kubernetes trong lab environment cô lập, truy cập qua terminal web, được chấm bằng validator script. Phần này dùng Go, k3s và Sysbox ở mức hạ tầng khái quát — đủ để phân tích kiến trúc, không cần public chi tiết vận hành cụ thể.

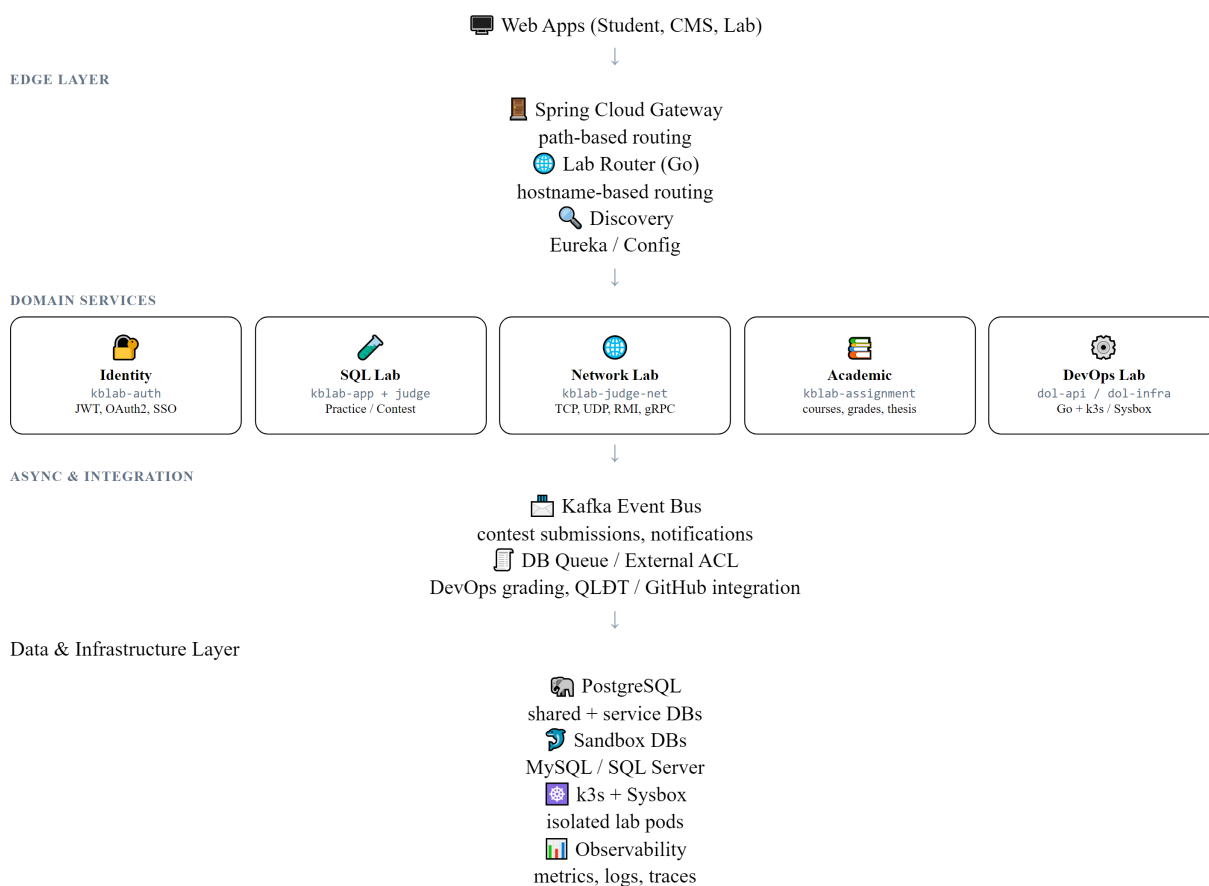
Ràng buộc thực tế — đây là điều làm cho case study có giá trị giảng dạy:

- **Team nhỏ:** 2–3 developers, không có dedicated DevOps hay QA team
- **Hạ tầng hạn chế:** hạ tầng nhỏ, LMS chính chạy Docker Compose; DevOps Lab dùng k3s/Sysbox cho bài toán lab isolation, không public sơ đồ máy chủ/domain cụ thể
- **Phát triển nhanh dưới áp lực:** Hệ thống phải phục vụ sinh viên ngay, nhiều quyết định kiến trúc được chọn vì *nhANH*, không phải vì *đúng*
- **Hệ thống đang chạy production:** Mọi cải thiện phải tăng dần — không thể dừng hoạt động để refactor toàn bộ

Điểm quan trọng: KBLab hiện tại **không phải là best practice** — và đó chính là lý do nó có giá trị làm case study. Giống như hầu hết dự án thực tế, developer hiếm khi xây dựng hệ thống hoàn hảo từ đầu. Cuốn sách sẽ sử dụng KBLab như **bài toán migration thực tế**: ở mỗi chương, chúng ta phân tích hệ thống đang ở đâu, vấn đề kỹ thuật là gì, best practice yêu cầu gì, và đề xuất lộ trình cải thiện.

1.8.2. Kiến trúc tổng quan

Kiến trúc tổng quan — KBLab LMS
FRONTEND



Hình 1.8: Kiến trúc tổng quan hệ thống KBLab LMS

1.8.3. Các bài toán kỹ thuật chính

Mỗi chương sách sẽ quay lại KBLab ở góc nhìn khác nhau. **Bảng 1.7** ánh xạ các bài toán kỹ thuật — từ hiện trạng đến hướng migration:

Bảng 1.7: Ảnh xạ bài toán kỹ thuật KBLab LMS theo chương

Bài toán kỹ thuật	Hiện trạng	Vấn đề cần giải quyết	Chương
Domain decomposition	5 bounded contexts chính + supporting services	Shared library và shared DB vẫn tạo coupling	Ch.2
API design	REST controllers + Network Lab đa giao thức	Naming/versioning chưa thống nhất; contract nhiều protocol	Ch.3
Sync communication	OpenFeign, SQL Judge coordinator + judge-* workers, external integrations	Cần resilience + Anti-Corruption Layer rõ ràng	Ch.4
Async communication	Kafka cho SQL Contest; DB queue cho DevOps Lab	Cần idempotency, retry, dead letter và workload-based choice	Ch.5
Distributed data	Shared database giữa Core và Assignment	Vi phạm database-per-service, coupling cao	Ch.6, Ch.7
API Gateway	Spring Cloud Gateway + Go reverse proxy cho lab routing	Path-based vs hostname-based gateway trade-off	Ch.8
Security	JWT HS256, OAuth2, SSO cross-platform	Secret management, token storage, defense-in-depth	Ch.9
Migration strategy	10 năm tiến hóa, legacy QLĐT, GitHub Classroom	Strangler Fig, ACL, roadmap theo rủi ro	Ch.10
Observability	Actuator/ Prometheus một phần, correlation ID đang mở rộng	Cần centralized logs, tracing, metrics end-to-end	Ch.11
Deployment	Docker Compose cho LMS chính; k3s/ Sysbox cho DevOps Lab	CI/CD, rollback, resource isolation, không over-engineer	Ch.12

① Phân tích — Shared database

KBLab sử dụng shared database giữa Core Service và Assignment Service — đây là **anti-pattern** theo nguyên tắc database-per-service [2a, Ch.2]. Hậu quả: thay đổi schema ảnh hưởng cả hai service, không thể scale/deploy độc lập, và hai team (nếu có) phải coordinate mọi migration. Lộ trình cải thiện: tách schema → API-based data access → database-per-service. Chi tiết sẽ phân tích tại Chương 6–7.

1.8.4. Những quyết định chúng ta muốn làm lại — Cautionary Tale

Richardson trong phiên bản 2 [2b, Ch.2] dành nguyên một chương kể câu chuyện FTGO migration thất bại — cho thấy *những sai lầm* trước khi dạy pattern đúng. KBLab cũng có những bài học tương tự — năm quyết định mà team sẽ chọn khác nếu làm lại:

Bảng 1.8: Năm quyết định kiến trúc KBLab muốn làm lại

#	Quyết định	Tại sao chọn lúc đó	Hậu quả thực tế	Anti-pattern tương ứng
①	Shared database giữa Core và Assignment	Nhanh hơn, không cần API giữa hai service	Thay đổi bảng <code>classroom</code> ở Core phá vỡ query ở Assignment; deploy cùng nhau bắt buộc	Richardson [2b]: <i>Data Services anti-pattern</i>
②	Shared library chứa quá nhiều thứ (entity, security, SQL parser)	Tái sử dụng code, giảm duplication	Mỗi thay đổi nhỏ buộc tất cả service rebuild + redeploy → mất independent deployability	Richardson [2b]: <i>Lock-step deployment</i>
③	JWT HS256 (symmetric key chia sẻ giữa services)	Đơn giản, setup 5 phút	Tất cả service biết secret key → bất kỳ service nào bị compromise đều có thể tạo JWT giả	Chi tiết tại Ch.9
④	Không có idempotency cho submit endpoint	“Ai submit trùng đầu?”	Contest mode: client retry do mạng chập → duplicate submission → điểm sai	Chi tiết tại Ch.5, Ch.6
⑤	docker logs thay cho observability	“Team nhỏ, đọc log trực tiếp nhanh hơn”	Contest mode: 200 submissions, Judge chậm, không biết bottleneck ở đâu → debug mất hàng giờ	Chi tiết tại Ch.11

Mỗi quyết định trên đều *hợp lý tại thời điểm đó* — team nhỏ, cần ship nhanh, ưu tiên tính năng hơn architecture. Nhưng khi hệ thống mở rộng (Contest Mode, Assignment, Thesis Management, Network Lab đa giao thức, DevOps Lab bằng Go), chi phí kỹ thuật (technical debt) tích lũy nhanh hơn tốc độ phát triển tính năng. Đây là bài học mà Richardson gọi là “*penny wise and pound foolish*” — tiết kiệm ở bước nhỏ, trả giá ở bước lớn.

⚠ Nguyên tắc — Imperfect Systems Are the Best Teachers

Nếu hệ thống hoàn hảo từ đầu, không có gì để học. Giá trị sư phạm của KBLab nằm ở chỗ: mỗi anti-pattern là cơ hội để hiểu *tại sao* best practice tồn tại — không phải vì sách giáo khoa nói vậy, mà vì *hậu quả thực tế* khi không tuân theo.

⚠ Lưu ý —

1. **Áp dụng microservices vì “trending”** — Nhiều team chọn microservices vì thấy Netflix, Amazon dùng, mà không tự hỏi: “Vấn đề cụ thể nào của chúng ta mà microservices giải quyết?” Hậu quả: thêm complexity mà không có lợi ích. *Phòng tránh*: luôn bắt đầu từ vấn đề (Decision Framework §1.7), không từ giải pháp.
2. **Nhảy thẳng từ monolith sang microservices** — Tách patchwork monolith trực tiếp, bỏ qua bước modular hóa. Hậu quả: distributed monolith — tệ hơn cả hai thế giới. *Phòng tránh*: refactor ranh giới module rõ ràng *trước*, rồi mới tách service (§1.5).
3. **Tách service trước khi tổ chức team** — Chia 10 microservices nhưng vẫn 1 team duy nhất, deploy cùng nhau, review code chéo nhau. Hậu quả: overhead vận hành mà không có autonomy thực sự (Conway’s Law). *Phòng tránh*: tổ chức team phù hợp *cùng lúc* hoặc *trước* khi tách service (chi tiết ở Ch.2).

🌐 Trực quan hóa tương tác (Interactive Demo)

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/monolith-vs-microservices.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **So sánh Monolith và Microservices**.

1.9. Tổng kết

Chương này đã phác họa bức tranh toàn cảnh về sự tiến hóa kiến trúc phần mềm — từ monolith nguyên khối, qua SOA với tham vọng tái sử dụng ở cấp doanh nghiệp, đến microservices với triết lý đặt tính độc lập lên hàng đầu.

Chúng ta đã thấy rằng **mỗi bước tiến hóa đều giải quyết giới hạn của bước trước, nhưng đồng thời tạo ra thách thức mới**. SOA giải phóng khỏi silo nhưng tạo ra coupling vào middleware. Microservices trao quyền tự trị cho từng service nhưng đổi lại bằng complexity tax của distributed systems. Không có kiến trúc nào là “tốt nhất” — chỉ có kiến trúc phù hợp nhất cho ngữ cảnh.

Từ bài học của Netflix, Amazon, và Uber, chúng ta học được rằng **chuyển đổi sang microservices là phản ứng trước giới hạn thực tế, không phải theo đuổi xu hướng**. Và trạng thái hybrid — nơi hầu hết doanh nghiệp đang vận hành — là hoàn toàn hợp lý.

Modular monolith không phải bước lùi mà là **bước đệm thông minh**: hiểu rõ domain trước khi chia tách. Quyết định kiến trúc cũng không chỉ là kỹ thuật — ngân sách, team, lịch trình, và văn hóa tổ chức đều ảnh hưởng ngang bằng.

Cuối cùng, chúng ta đã giới thiệu case study KBLab — hệ thống LMS thực tế với bốn trụ cột (SQL Lab, Network Lab, Course Management, DevOps Lab), những ràng buộc rất quen thuộc

với hầu hết dự án (team nhỏ, hạ tầng hạn chế, cần ship nhanh), và *năm quyết định kiến trúc team muốn làm lại* — bài học cautionary tale tương tự FTGO của Richardson. Ở Chương 2, chúng ta sẽ đi sâu vào **Domain-Driven Design** — phương pháp để xác định ranh giới dịch vụ, và bắt đầu phân tích domain của KBLab để tìm ra các bounded context tự nhiên.

1.10. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *SOA: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.3–4: Service-Oriented Principles, Understanding SOA
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.1: Escaping Monolithic Hell, Scale Cube
3. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.1: Fast Flow Success Triangle, DORA Metrics; Ch.5: Architectural Requirements for Fast Flow; Ch.6: Modular Monolith
4. [4b] Sam Newman, *Monolith to Microservices* — Ch.1: Just Enough Microservices, Should You Migrate?
5. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.1: SOA vs Microservices vs EDA

Sách bổ trợ:

6. Nicole Forsgren, Jez Humble, Gene Kim, *Accelerate* (2018) — DORA Metrics, nghiên cứu định lượng về software delivery performance
7. Gene Kim et al., *The DevOps Handbook*, 2nd Ed. (2021) — Three Ways of DevOps, deployment pipeline design

Nguồn trực tuyến:

- [W1] James Lewis & Martin Fowler, “Microservices” (2014) — martinfowler.com/articles/microservices.html
- [W2] Martin Fowler, “MonolithFirst” (2015) — martinfowler.com/bliki/MonolithFirst.html
- [W3] Netflix Technology Blog, “Completing the Netflix Cloud Migration” (2016)
- [W4] Uber Engineering, “Domain-Oriented Microservice Architecture” (2020)
- [W5] Steve Yegge, “Stevey’s Google Platforms Rant” (2011) — nguồn câu chuyện Bezos API Mandate
- DORA, “State of DevOps Report 2024” — dora.dev/research

2. Phân tích hướng Domain — DDD & Bounded Contexts

“Use the model as the backbone of a language. Commit the team to exercising that language relentlessly in all communication within the team and in the code.” — Eric Evans, Domain-Driven Design

2.1. Bạn sẽ học được gì

- Hiểu Conway's Law và tác động của cấu trúc tổ chức lên kiến trúc phần mềm
- Nắm vững các khái niệm DDD cốt lõi: Entity, Value Object, Aggregate, Repository
- Áp dụng Strategic DDD: Bounded Context, Context Map, Ubiquitous Language
- Thực hành xác định ranh giới dịch vụ từ domain — kỹ năng quan trọng nhất khi thiết kế microservices
- So sánh hai phương pháp phân tách: DDD (heuristic-based) vs Erl Service-Oriented Analysis (step-by-step)
- Phân tích case study KBLab: 5 bounded contexts và quyết định Shared Kernel

2.2. Conway's Law — Tổ chức quyết định kiến trúc

Trước khi đi vào Domain-Driven Design, chúng ta cần hiểu một quy luật nền tảng ảnh hưởng đến mọi quyết định kiến trúc: **kiến trúc phần mềm phản ánh cấu trúc tổ chức phát triển nó.**

2.2.1. Quy luật Conway

Năm 1968, Melvin Conway quan sát rằng:

! Nguyên tắc — Conway's Law

“Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.”

— *Melvin Conway, 1968*

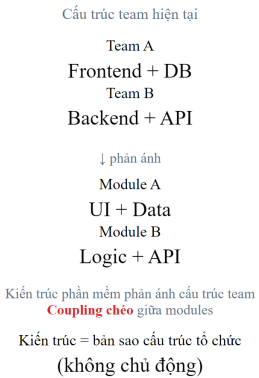
Nói cách khác: nếu tổ chức có 3 team, hệ thống sẽ có 3 service (hoặc 3 module lớn). Nếu 2 team giao tiếp nhiều, 2 service tương ứng sẽ coupling chặt. Đây không phải lý thuyết suông — nghiên cứu của MacCormack, Rusnak & Baldwin tại Harvard Business School (2008) đã cung cấp bằng chứng thực nghiệm hỗ trợ quy luật này trên các dự án mã nguồn mở.

Đối với developer và kiến trúc sư phần mềm, điều này có ý nghĩa thực tiễn rõ ràng: **không thể thiết kế microservices tốt nếu cấu trúc team không phù hợp.** Đây là lý do Amazon tái cấu trúc thành “two-pizza teams” *trước* khi tách monolith (đã thảo luận ở Chương 1).

2.2.2. Inverse Conway Maneuver

Thay vì chấp nhận thụ động, nhiều tổ chức chủ động tận dụng Conway's Law theo hướng ngược lại: **thiết kế cấu trúc team để đạt được kiến trúc phần mềm mong muốn.** Chiến lược này gọi là *Inverse Conway Maneuver*.

Conway’s Law vs Inverse Conway Maneuver
Conway’s Law (thụ động)



Inverse Conway (chủ động)



Conway’s Law: tổ chức tác động thụ động lên kiến trúc — Inverse Conway: chủ động thiết kế tổ chức phù hợp kiến trúc đích

Hình 2.1: Conway’s Law (thụ động) vs Inverse Conway Maneuver (chủ động)

Một team sở hữu một hoặc vài service liên quan, chịu trách nhiệm toàn bộ lifecycle — từ code đến production. Đây là mô hình “you build it, you run it” mà Amazon và Netflix đã chứng minh hiệu quả.

2.2.3. Team Topologies — Bốn kiểu team

Matthew Skelton và Manuel Pais đề xuất bốn kiểu team cơ bản cho tổ chức phần mềm hiện đại:

Bảng 2.1: Bốn kiểu team theo Team Topologies

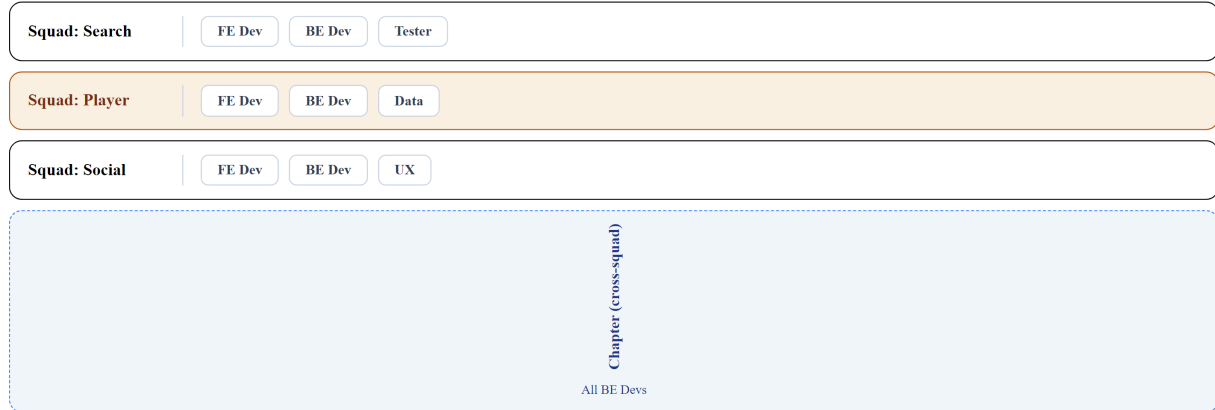
Kiểu team	Vai trò	Ví dụ trong KBLab
Stream-aligned	Phát triển theo luồng nghiệp vụ, sở hữu end-to-end	Team “Bài tập & Chấm thi”, Team “Quản lý người dùng”
Platform	Cung cấp nền tảng nội bộ giúp stream-aligned teams phát triển nhanh hơn	Team DevOps (CI/CD, Docker, monitoring)
Enabling	Hỗ trợ các team khác nâng cao kỹ năng	Team kiến trúc (DDD coaching, code review)
Complicated subsystem	Sở hữu thành phần phức tạp đòi hỏi chuyên môn sâu	Team “SQL Judge” (sandbox isolation, multi-DBMS)

Với KBLab — một dự án nhỏ (2–3 developers) — không thể áp dụng đầy đủ 4 kiểu team. Nhưng *tư duy* vẫn áp dụng: developer nào hiểu sâu domain nào sẽ tự nhiên sở hữu service tương ứng. Khi team mở rộng, cấu trúc này sẽ trở nên tường minh hơn.

2.2.4. Industry Case Study: Spotify Squad Model

Spotify (2012) là case study nổi tiếng nhất về tổ chức team cho microservices. Henrik Kniberg và Anders Ivarsson mô tả mô hình gồm bốn cấp:

Mô hình tổ chức Spotify — Tribe, Squad, Chapter, Guild
Tribe (40–150 người)



Guild (cross-tribe)

Ví dụ: Web Performance Guild

(Cộng đồng chia sẻ kiến thức — tự nguyện, không có hierarchy)

Squad = autonomous team; Chapter = kỹ năng đồng nhất; Guild = cộng đồng thực hành tự nguyện

Hình 2.2: Mô hình Squad/Tribe/Chapter/Guild của Spotify

Bảng 2.2: Cấu trúc tổ chức Spotify — ý nghĩa cho microservices

Cấu trúc	Mô tả	Ý nghĩa cho microservices
Squad (6-12 người)	Đơn vị tự trị, sở hữu tính năng end-to-end	= Stream-aligned team = sở hữu 1+ microservices
Tribe (max 150 — Dunbar's number)	Nhóm squads cùng mission	Alignment mà không mất autonomy
Chapter	Nhóm người cùng competency xuyên squads	Chia sẻ best practices (ví dụ: all backend devs)
Guild	Cộng đồng quan tâm xuyên tribes	Knowledge sharing (ví dụ: Web Performance Guild)

Bài học cho hệ thống nhỏ: Spotify có 2,000+ engineers khi áp dụng mô hình này — LMS chỉ có 2-3 (không cần squads/tribes). Nhưng nguyên tắc cốt lõi vẫn đúng: **mỗi service nên có owner rõ ràng**, và khi team mở rộng, tổ chức team theo bounded context (Ch.2) thay vì theo technical layer.

Nguồn: Henrik Kniberg, Anders Ivarsson, “Scaling Agile @ Spotify,” 2012. Kniberg cũng trình bày tại nhiều hội nghị (youtube.com). Lưu ý: Spotify đã evolve mô hình này đáng kể từ 2012 — mô hình gốc là điểm khởi đầu, không phải template cứng.

① Phân tích — Team nhỏ và Conway's Law

Trong LMS, cùng một developer phát triển cả Auth Service và Core Service. Điều này dẫn đến ranh giới giữa các service không rõ ràng — ví dụ: `spring.application.name` trùng nhau giữa Core và Assignment (cả hai đều là `app`), shared database vẫn được duy trì vì tiện lợi. Richardson trong [2a] cho thấy việc định nghĩa rõ service ownership từ đầu giúp tránh coupling ngầm. **Migration path**: tách rõ naming, định nghĩa API contract giữa services, và chuẩn bị cho việc mở rộng team.

2.2.5. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service

Richardson trong phiên bản thứ hai (2025) giới thiệu một framework phân tích trade-off cho việc service decomposition, mượn ẩn dụ từ vật lý thiên văn:

- **Dark Energy** — lực **đẩy** (repulsive): kéo các components ra xa nhau → nên tách thành service riêng
- **Dark Matter** — lực **hút** (attractive): kéo các components lại gần nhau → nên giữ cùng service

Bảng 2.3: Mười lực Dark Energy / Dark Matter

Lực	Loại	Mô tả	Ví dụ KBLab
Simple interactions	☉ Matter (hút)	Components tương tác phức tạp → giữ cùng service	ContestQuestion ↔ Contest luôn query chung → hợp lý ở cùng Core Service
Efficient interactions	☉ Matter (hút)	Cần latency thấp → in-process call nhanh hơn network	Submission → Score cập nhật cần nhanh → hiện cùng service
Prefer ACID	☉ Matter (hút)	Cần ACID transaction → cùng DB dễ hơn saga	Create contest + add questions = 1 transaction → cùng service
Minimize runtime coupling	☉ Matter (hút)	Giảm dependency lúc runtime → ít network calls	Judge Service xử lý độc lập (receive message → run → return)
Simple components	⚡ Energy (đẩy)	Mỗi service nhỏ, dễ hiểu	Core Service (24 files) vs nếu gộp tất cả (~50+ files)
Team autonomy	⚡ Energy (đẩy)	Team deploy độc lập, không coordination	Auth team vs Core team deploy riêng
Fast deployment pipeline	⚡ Energy (đẩy)	Service nhỏ → build + test nhanh	Auth Service build 30s vs monolith build 5 phút
Support multiple tech stacks	⚡ Energy (đẩy)	Service khác có thể dùng tech khác	Judge Service có thể viết bằng Go (performance)
Independent scalability	⚡ Energy (đẩy)	Scale riêng từng service theo nhu cầu	Judge Service cần scale khi contest, Auth thì không
Independent data ownership	⚡ Energy (đẩy)	Mỗi service own data riêng → encapsulation	Users ở Auth, Questions ở Core (lý tưởng)

Cách sử dụng: Khi phân vân “có nên tách component X thành service riêng?”, liệt kê các lực tác động. Nếu Dark Energy (đẩy) chiếm ưu thế → tách. Nếu Dark Matter (hút) mạnh hơn → giữ chung. Đây không phải công thức chính xác mà là **framework tư duy** giúp cấu trúc cuộc thảo luận.

① Phân tích — Dark Energy/Matter cho quyết định tách Core & Assignment

KBLab hiện có Core Service và Assignment Service dùng chung database. Phân tích:

- **Dark Matter (giữ chung)**: shared database (ACID dễ), `assignment_questions` reference `questions.id` (simple interactions), team rất nhỏ (2-3 người)
- **Dark Energy (tách)**: domain logic khác biệt (bài tập vs cuộc thi), data ownership rõ ràng, potential for independent deployment

Kết luận: Dark Matter hiện mạnh hơn (do team nhỏ, shared DB). Khi team grow > 5 người, cân bằng sẽ shift → lúc đó tách DB và service hợp lý hơn. Đây chính là nguyên tắc “*Don't decompose prematurely*” (Richardson [2b, Ch.7]).

2.3. DDD cơ bản — Ngôn ngữ chung và các building blocks

Domain-Driven Design (DDD) là phương pháp thiết kế phần mềm đặt **domain** (miền nghiệp vụ) làm trung tâm. Eric Evans giới thiệu DDD năm 2003 trong cuốn sách cùng tên, và nó trở thành nền tảng cho việc phân tách microservices.

Đối với developer, DDD trả lời câu hỏi quan trọng nhất khi thiết kế microservices: **tách ở đâu?** Không phải tách theo layer kỹ thuật (UI team, DB team, API team) mà tách theo **ranh giới nghiệp vụ** — nơi mà sự thay đổi trong một domain không ảnh hưởng đến domain khác.

2.3.1. Ubiquitous Language — Ngôn ngữ chung

Bước đầu tiên và quan trọng nhất trong DDD không phải vẽ diagram hay viết code — mà là **xây dựng ngôn ngữ chung** (*Ubiquitous Language*) giữa developer và domain expert.

Ubiquitous Language là tập hợp thuật ngữ mà *cả team* — developer, tester, product owner — sử dụng nhất quán trong mọi giao tiếp: email, meeting, code, tài liệu. Nếu domain expert nói “bài nộp” (*submission*), code phải có class `Submission`, database có bảng `submission`, API có endpoint `/submissions`. Không được có sự khác biệt.

Tại sao điều này quan trọng cho microservices? Vì khi hai team bắt đầu dùng cùng một thuật ngữ nhưng với **ý nghĩa khác nhau**, đó chính là tín hiệu rằng họ đang ở hai bounded context khác nhau — và service nên được tách.

2.3.2. Các building blocks

DDD định nghĩa một bộ building blocks để mô hình hóa domain:

DDD Building Blocks
 Repository
 (CRUD interface)
 Domain Service
 (cross-entity logic)
 Factory
 (tạo Aggregate)
 Aggregate

Aggregate Root
 (Entity chính — điểm vào duy nhất)

Entity
 (có identity)
 Value Object
 (không có identity)

↓
 Entity con

Ví dụ LMS:

Aggregate Root = Contest | **Entity** = ContestQuestion | **Value Object** = Score(85/100)

Repository = ContestRepository | **Domain Service** = SqlEvaluationService | **Factory** = ContestFactory.create(...)

Hình 2.3: Các building blocks trong DDD — Aggregate, Entity, Value Object, Repository

Entity — Đối tượng có danh tính (*identity*) duy nhất xuyên suốt vòng đời. Hai entity có cùng thuộc tính nhưng khác ID vẫn là hai entity khác nhau. Ví dụ trong KBLab: `Student(id=1, name="Nguyen Van A")` và `Student(id=2, name="Nguyen Van A")` là hai sinh viên khác nhau dù trùng tên.

Value Object — Đối tượng được xác định bởi *giá trị*, không có identity riêng. Hai value object cùng giá trị là tương đương. Ví dụ: `Score(value=85, max=100)` — không quan trọng “đây là điểm nào”, chỉ quan trọng giá trị 85/100.

Aggregate — Cụm entity và value object được quản lý như một đơn vị nhất quán (*consistency boundary*). Mỗi aggregate có một **Aggregate Root** — entity duy nhất mà bên ngoài có thể tham chiếu. Trong LMS, `Contest` có thể là aggregate root bao gồm `ContestQuestion`, `ContestParticipant` — không ai truy cập `ContestQuestion` mà không đi qua `Contest`.

💡 Tip — Aggregate = Transaction boundary

Trong microservices, aggregate chính là đơn vị transaction. Một transaction chỉ nên thao tác trên **một aggregate** duy nhất. Nếu cần transaction trên nhiều aggregate, đó là tín hiệu cần Saga pattern (Chương 6) — không phải distributed transaction.

Repository — Interface cung cấp ảo tưởng về collection cho aggregate. Developer tương tác với repository thay vì database trực tiếp. Trong Spring Boot: `@Repository` interface `ContestRepository` extends `JpaRepository<Contest, UUID>`.

Domain Service — Logic nghiệp vụ không thuộc về một entity hay value object cụ thể nào. Ví dụ: `SqlEvaluationService` so sánh kết quả SQL của sinh viên với đáp án — logic này không thuộc về `Submission` hay `Question` riêng lẻ.

2.3.3. DDD cho developer: nghĩ bằng domain, không bằng database

Sai lầm phổ biến nhất khi bắt đầu với DDD là **thiết kế entity từ bảng database**. DDD yêu cầu ngược lại: mô hình hóa domain trước, database sau.

Bảng 2.3b: Database-first vs Domain-first

Cách tiếp cận	Bắt đầu từ	Kết quả
Database-first ❌	Bảng, cột, foreign key	Entity = bảng, service = CRUD wrapper, ranh giới mờ nhạt
Domain-first ✅	Quy trình nghiệp vụ, ngôn ngữ domain	Entity phản ánh business concepts, ranh giới tự nhiên

Trong thực tế, nhiều dự án (kể cả LMS) bắt đầu database-first — và đó không phải là thảm họa. Nhưng khi cần tách microservices, bước đầu tiên phải là *ngẫm lại* mô hình từ góc nhìn domain.

2.4. Strategic DDD — Bounded Context & Context Map

Nếu tactical DDD (Entity, Aggregate) giúp mô hình hóa *bên trong* một service, thì strategic DDD giúp trả lời câu hỏi lớn hơn: **hệ thống cần bao nhiêu service, và ranh giới ở đâu?**

2.4.1. Bounded Context — Ranh giới ngôn ngữ

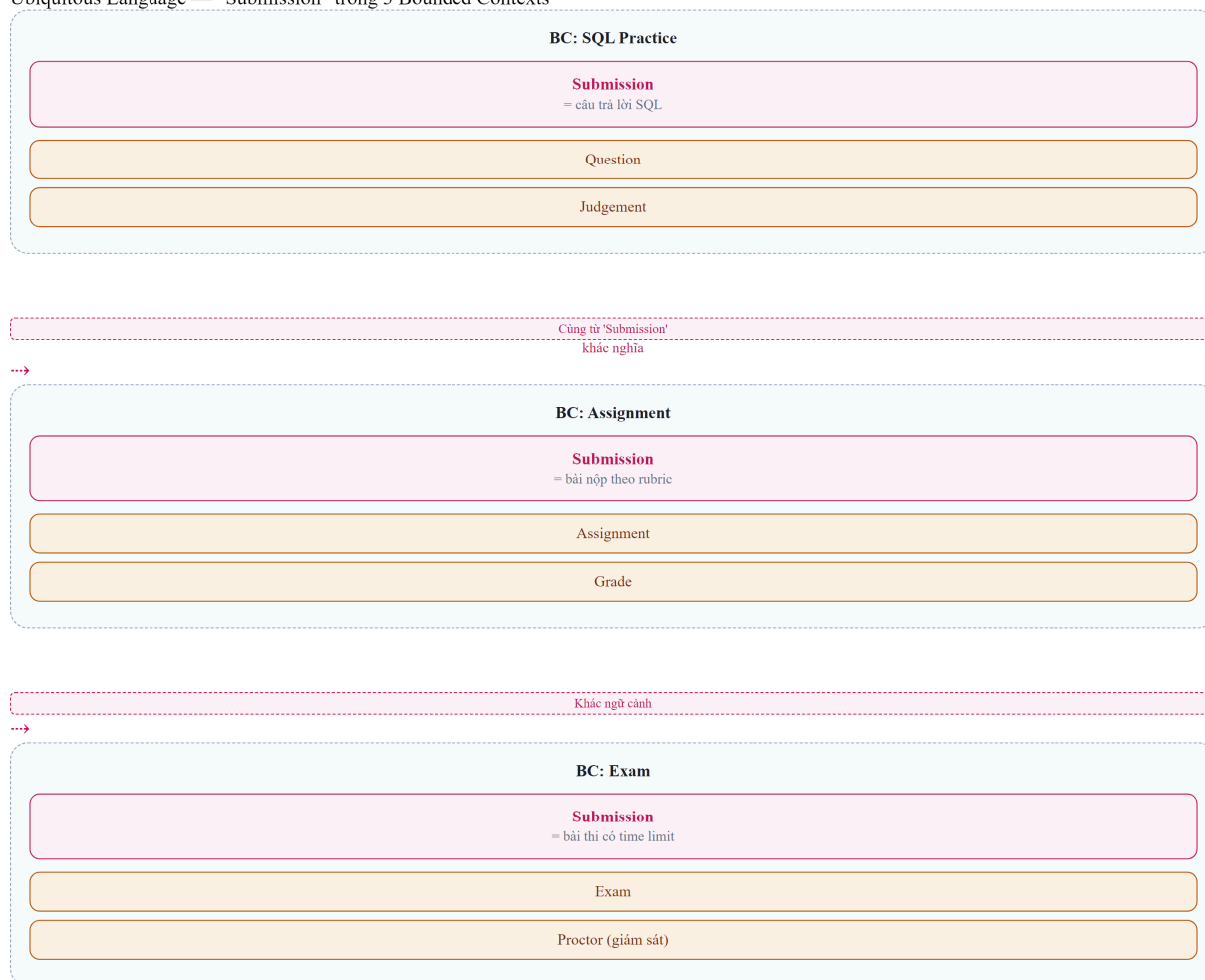
Bounded Context là khái niệm quan trọng nhất trong DDD cho microservices. Một bounded context là phạm vi trong đó một mô hình domain cụ thể có hiệu lực — và **ngôn ngữ có ý nghĩa nhất quán**.

Ví dụ thực tế: trong KBLab, thuật ngữ “bài nộp” (*submission*) có ý nghĩa khác nhau tùy ngữ cảnh:

- Trong context **Thực hành SQL**: submission = câu trả lời SQL của sinh viên, cần chấm đúng/sai
- Trong context **Bài tập**: submission = file/nội dung nộp bài, cần chấm điểm theo rubric
- Trong context **Thi**: submission = bài thi với giới hạn thời gian, cần kiểm tra gian lận

Cùng một từ, ba ý nghĩa khác nhau → ba bounded context tiềm năng. Đây là heuristic mạnh mẽ nhất để xác định ranh giới: **khi cùng một thuật ngữ mang ý nghĩa khác nhau, chúng ta đang ở ranh giới giữa hai context**.

Ubiquitous Language — "Submission" trong 3 Bounded Contexts



Cùng từ "Submission" nhưng mang ngữ nghĩa hoàn toàn khác trong mỗi Bounded Context — đây là lý do DDD đề cao Ubiquitous Language

Hình 2.4: Cùng thuật ngữ "Submission" mang ý nghĩa khác nhau trong ba bounded context

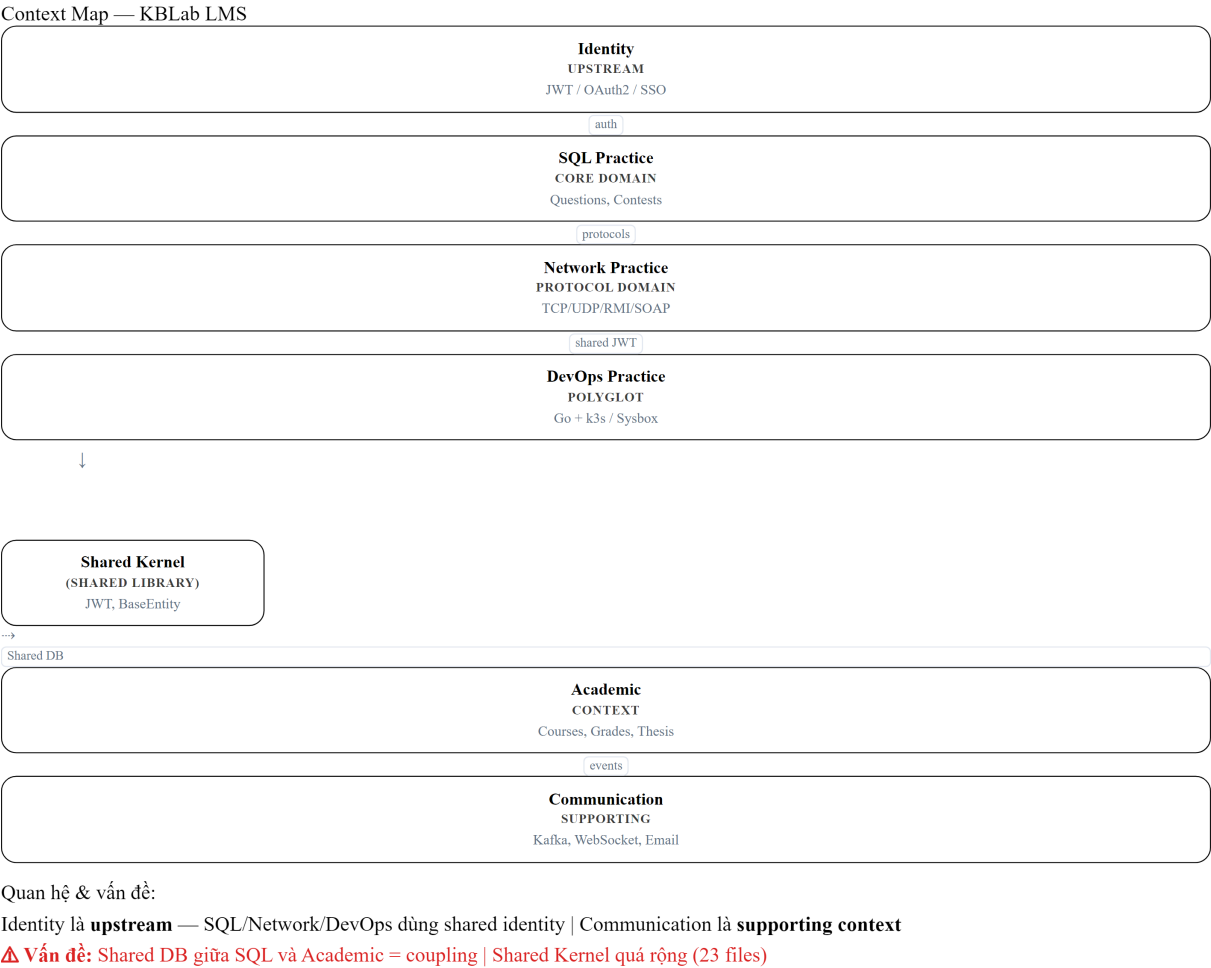
2.4.2. Context Map — Bản đồ quan hệ giữa các context

Khi đã xác định bounded contexts, bước tiếp theo là vẽ **Context Map** — sơ đồ quan hệ giữa chúng. Context Map không chỉ cho thấy context nào giao tiếp với nhau, mà còn cho thấy **kiểu quan hệ**.

Các kiểu quan hệ phổ biến:

Bảng 2.4: Các kiểu quan hệ giữa bounded contexts

Quan hệ	Mô tả	Ví dụ trong KBLab
Shared Kernel	Hai context chia sẻ một phần mô hình	Shared library (base entities, JWT, exceptions)
Customer-Supplier	Upstream context cung cấp, downstream tiêu thụ	Auth (upstream) → Core (downstream)
Conformist	Downstream chấp nhận hoàn toàn mô hình upstream	Core phải tuân theo model của Auth cho user data
Anti-Corruption Layer	Downstream dịch mô hình upstream thành mô hình riêng	Judge Service dịch Submission thành JudgeRequest nội bộ
Open Host Service	Context cung cấp API chuẩn cho nhiều consumer	Core API cung cấp question data cho Judge + Assignment
Published Language	Ngôn ngữ trao đổi được chuẩn hóa (JSON, Protobuf)	Kafka messages với schema chuẩn



Hình 2.5: Context Map của KBLab — quan hệ giữa các bounded contexts

2.5. Xác định Bounded Context từ requirements

2.5.1. Các heuristic thực tế

Làm thế nào để “tìm” bounded context trong một hệ thống? Không có công thức chính xác — đây là kỹ năng cần luyện tập. Tuy nhiên, có một số heuristic hữu ích:

1. Heuristic ngôn ngữ — Khi cùng một thuật ngữ mang ý nghĩa khác nhau cho các nhóm người khác nhau, đó là ranh giới context. Ngược lại, khi hai thuật ngữ khác nhau chỉ cùng một thứ (ví dụ: “ref-des” và “component instance” trong câu chuyện PCB của Eric Evans), chúng thuộc cùng context.

2. Heuristic thay đổi — Tự hỏi: “Khi requirement X thay đổi, code nào cần sửa?” Nếu thay đổi luôn ảnh hưởng đến cùng nhóm files, chúng thuộc cùng context. Nếu thay đổi lan sang nhóm khác, đó là tín hiệu coupling cần xem xét.

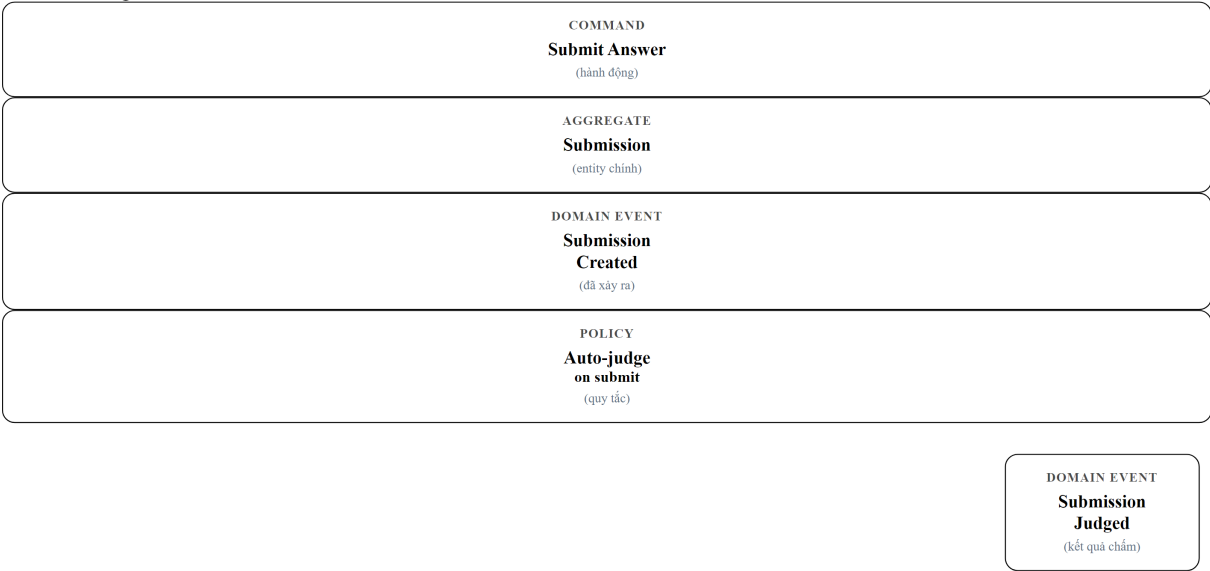
3. Heuristic team — Ai sẽ phát triển và bảo trì phần này? Nếu cùng team, chúng có thể cùng context. Nếu khác team, cần nhắc tách context (Conway’s Law).

4. Heuristic data — Dữ liệu nào “thuộc về” nhau? Entity nào luôn được truy vấn cùng nhau? Đó là aggregate trong cùng context. Entity nào chỉ được tham chiếu qua ID? Đó có thể thuộc context khác.

2.5.2. Event Storming — Phương pháp khám phá domain

Event Storming là workshop technique do Alberto Brandolini phát triển, giúp cả team cùng khám phá domain thông qua *domain events* — những sự kiện đã xảy ra trong hệ thống.

Event Storming Flow



Quy trình Event Storming:

- 1. Domain Events
Gì đã xảy ra?
- 2. Commands
Hành động nào gây ra event?
- 3. Aggregates
Nhóm events theo aggregate
- 4. Boundaries
Vẽ ranh giới Bounded Context

Hình 2.6: Event Storming — luồng từ Command đến Domain Event

Quy trình cơ bản:

1. **Liệt kê domain events** — post-it cam: “Submission Created”, “SQL Executed”, “Score Calculated”
2. **Tìm commands** — post-it xanh: “Submit Answer”, “Start Exam”, “Create Assignment”
3. **Nhóm events theo aggregate** — post-it vàng: events nào cùng xảy ra trên cùng entity?
4. **Vẽ ranh giới** — nhóm aggregates có liên quan chặt chẽ → bounded context

Kết quả Event Storming tự nhiên dẫn đến bounded contexts. Nhóm events/aggregates nào tương tác nhiều nội bộ nhưng ít tương tác với nhóm khác → đó là context.

2.5.3. Phân rã hướng dịch vụ theo Erl — Cách tiếp cận step-by-step

DDD và Event Storming là phương pháp *khám phá* — bắt đầu từ ngôn ngữ domain, dần dần tìm ra ranh giới. Cách tiếp cận này mạnh nhưng đòi hỏi kinh nghiệm: developer mới thường khó biết “đủ chưa” hoặc “đúng chưa”.

Thomas Erl trong *SOA: Analysis and Design for Services and Microservices* [1, Ch.8–10] đề xuất một phương pháp hỗ trợ: **Service-Oriented Analysis** — quy trình *prescriptive*, gồm các bước cụ thể từ yêu cầu nghiệp vụ đến danh mục dịch vụ. Đây là cách tiếp cận dễ dạy và dễ học hơn cho sinh viên, vì mỗi bước có input và output rõ ràng.

Năm bước phân rã dịch vụ theo Erl:

Bước 1: Xác định automation candidates — Phân tích quy trình nghiệp vụ (business process) và xác định những bước nào *nên* được tự động hóa bằng phần mềm. Erl gọi đây là *service candidates* — chưa phải services, mà là ứng viên.

Ví dụ KBLab: quy trình “Sinh viên nộp bài SQL” có các bước: (1) xác thực người dùng, (2) nhận câu trả lời, (3) thực thi SQL trên sandbox, (4) so sánh kết quả, (5) ghi điểm, (6) thông báo. Mỗi bước là một automation candidate.

Bước 2: Phân loại service layers — Erl định nghĩa ba loại service:

Bảng 2.7: Ba loại service theo Erl

Loại service	Vai trò	Ví dụ KBLab
Task Service	Điều phối quy trình nghiệp vụ (workflow), gọi các service khác	Submission orchestrator: nhận bài → gọi Judge → ghi điểm → thông báo
Entity Service	Quản lý dữ liệu nghiệp vụ (CRUD + business rules)	Question Service, User Service, Contest Service
Utility Service	Cung cấp chức năng kỹ thuật dùng chung	JWT validation, notification, logging

Phân loại này giúp tránh sai lầm phổ biến: tạo service chỉ là CRUD wrapper (entity service không có business logic) hoặc nhồi tất cả vào một “god service” (thiếu task/utility decomposition).

Bước 3: Tạo service inventory blueprint — Vẽ bản đồ toàn bộ services dự kiến, mỗi service với operations chính. Tương tự Context Map trong DDD, nhưng focused hơn vào *capabilities* (khả năng) thay vì *language boundaries* (ranh giới ngôn ngữ).

Bước 4: Xác định service boundaries + composition — Kiểm tra từng service candidate: nó có thể hoạt động **độc lập** không? Nếu service A *luôn* cần gọi service B trước khi xử lý bất

kỳ request nào → cân nhắc gộp. Nếu service A có thể hoạt động ngay cả khi B down → ranh giới tốt.

Bước 5: Verify qua 8 nguyên lý SOA — Mỗi service phải thỏa mãn 8 nguyên lý hướng dịch vụ đã thảo luận ở Bảng 1.2 (Chương 1): Standardized Contract, Loose Coupling, Abstraction, Reusability, Autonomy, Statelessness, Discoverability, Composability. Đây là “checklist kiểm tra chất lượng” cho service design.

2.5.3.1. Áp dụng Erl cho KBLab

Áp dụng 5 bước trên cho KBLab:

Erl's Service-Oriented Analysis — LMS
Bước 1: Automation Candidates

Xác thực người dùng
Quản lý câu hỏi SQL
Chăm bài tự động
Quản lý bài tập / khóa học
Thông báo kết quả

Bước 2: Service Layers

Task: Submission
Orchestrator
Entity: User
Entity: Question
Entity: Course
Utility: Auth
Utility: Notification
Complicated: Judge
Sandbox

Kết quả: Ánh xạ sang Bounded Contexts

= kblab-auth ← Utility Auth	Identity & Access
= kblab-app + judge ← Entity + Task	SQL Practice
= judge-network ← Protocol Subsystem	Network Practice
= kblab-assignment ← Entity Course	Academic
= dol-api ← Task + Infra	DevOps Practice

Hình 2.6b: Áp dụng Erl's Service-Oriented Analysis cho KBLab — từ automation candidates đến service layers

Kết quả: 5 nhóm dịch vụ — gần như trùng khớp với 5 bounded contexts từ phân tích DDD (§2.5): Identity & Access $\approx$ Utility Auth, SQL Practice $\approx$ Entity Question + Task Orchestrator, Network Practice $\approx$ Complicated Protocol Subsystem, Academic $\approx$ Entity Course, DevOps Practice $\approx$ Task Service + Infrastructure Automation.

2.5.3.2. So sánh: Erl vs DDD

Bảng 2.8: Hai phương pháp phân tách service — Erl Service-Oriented Analysis vs DDD

Tiêu chí	Erl Service-Oriented Analysis	DDD Bounded Context
Điểm bắt đầu	Quy trình nghiệp vụ (business process)	Ngôn ngữ domain (Ubiquitous Language)
Phong cách	Prescriptive — 5 bước rõ ràng, input/output mỗi bước	Explorative — heuristics, Event Storming workshop
Tiêu chí tách	Service layers (task/entity/utility) + 8 nguyên lý SOA	Ranh giới ngôn ngữ + aggregate consistency
Output	Service inventory blueprint	Context Map + Bounded Context diagram
Thế mạnh	Systematic, dễ dạy, phù hợp team mới	Sâu về domain, phát hiện ranh giới ẩn
Hạn chế	Có thể tạo ranh giới quá kỹ thuật (thiếu domain insight)	Cần kinh nghiệm, kết quả phụ thuộc người phân tích
Phù hợp khi	Team ít kinh nghiệm DDD, cần quy trình chuẩn	Team hiểu domain sâu, cần phát hiện complexity ẩn

Nguyên tắc — Hai phương pháp bổ trợ, không thay thế

Erl cho bạn *quy trình* (process) — đặc biệt hữu ích khi bạn chưa biết bắt đầu từ đâu. DDD cho bạn *insight* — phát hiện những ranh giới mà quy trình cơ học bỏ sót. Trong thực tế: dùng Erl để tạo bản phác thảo ban đầu (service inventory), sau đó dùng DDD (Event Storming, ngôn ngữ chung) để *tinh chỉnh* ranh giới. Richardson's Assemblage Process (§2.7) kết hợp cả hai hướng.

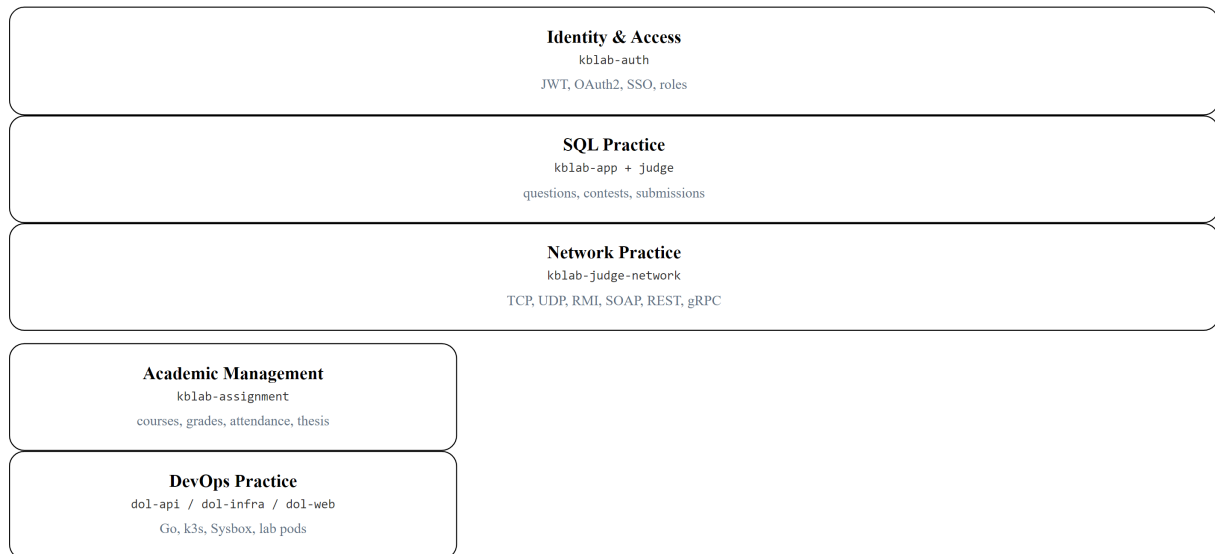
2.6. Case Study: Bounded Contexts trong KBLab

2.6.1. Phân tích domain

KBLab phục vụ các hoạt động giảng dạy, thực hành, đánh giá và vận hành lab thực hành. Từ góc nhìn DDD, chúng ta xác định 5 bounded contexts chính. Các service hỗ trợ như Gateway, Registry và Notification vẫn quan trọng, nhưng chúng là supporting/infrastructure services hơn là domain context độc lập.

5 Bounded Contexts — KBLab

Domain contexts (supporting services omitted for clarity)



Shared Kernel chỉ nên giữ cross-cutting concerns; domain logic phải quay về context sở hữu.

Hình 2.7: Năm bounded contexts chính trong KBLab

2.6.2. Năm bounded contexts

1. Identity & Access (Auth Service)

Domain rõ ràng nhất — quản lý danh tính người dùng, xác thực, phân quyền. Entities: *User*, *Role*, *Token*. Đây là context upstream — mọi context khác đều phụ thuộc vào kết quả authentication.

Trong KBLab, context này được tách thành service riêng sớm nhất (*kblab-auth*), vì ranh giới rất rõ: thay đổi cơ chế xác thực (tài khoản nội bộ, Google/Microsoft OAuth2, email verification, SSO từ hệ thống đào tạo) không ảnh hưởng logic bài tập hay chấm thi.

2. SQL Practice (*kblab-app* + *kblab-judge* — *Core Domain*)

Đây là **core domain** ban đầu — phần tạo ra giá trị chính cho hệ thống SQL judge. Entities: *Question*, *Submission*, *Contest*, *UserContest*, *Statistic*. Toàn bộ quy trình từ “sinh viên xem câu hỏi → viết SQL → nộp bài → nhận kết quả” nằm trong context này.

Đây là context lớn nhất và phức tạp nhất. Bên trong nó có kiến trúc chấm SQL gồm một service điều phối *judge* và nhiều processor *judge-** theo DBMS (*judge-mysql*, *judge-sqlserver*, ...). Đây mới là nơi cần phân tích kỹ về routing, resilience, idempotency và ownership giữa coordinator với worker; không nên nhầm với Network Practice.

3. Network Practice (*kblab-judge-network*)

Domain chuyên biệt cho bài thực hành lập trình mạng Java: TCP, UDP, RMI, SOAP, REST và gRPC. Khác với SQL Practice, sinh viên viết client trên IDE cá nhân và kết nối trực tiếp đến judge server. Context này minh họa ranh giới domain đến từ **protocol semantics**, không chỉ từ entity/database.

Network Practice được thiết kế khá độc lập với các services còn lại: nó phục vụ mục tiêu đánh giá giao tiếp qua mạng, có protocol contract riêng và không cần chia sẻ pipeline chấm SQL.

Trong sách, context này chủ yếu minh họa **technology diversity** và boundary theo protocol, không phải một “gap” kiến trúc cần sửa như SQL Judge.

4. Academic Management (Assignment Service)

Quản lý khóa học, bài tập dạng rubric, đăng ký môn, điểm số, điểm danh, MCQ daily, syllabus/CLO, GitHub Classroom và đồ án tốt nghiệp. Entities: Course, Assignment, Enrollment, Grade, AttendanceSession, Thesis.

Context này tách ra từ Core Service — nhưng *vẫn chia sẻ database (app_db)*. Đây là trade-off quan trọng mà chúng ta sẽ phân tích trong phần tiếp theo.

5. DevOps Practice (dol-api, dol-infra, dol-web)

Domain mới phục vụ bài thực hành Docker/Kubernetes: tạo lab environment cô lập, terminal web, validator script và progress tracking. Context này dùng Go, k3s và Sysbox, khác stack với LMS chính. Đây là ví dụ tốt cho polyglot microservices: dùng công nghệ khác khi domain và hạ tầng yêu cầu, nhưng vẫn chia sẻ identity qua JWT của KBLab.

2.6.3. Từ bounded context đến service

Bảng 2.9: Ánh xạ bounded context → service trong KBLab

Bounded Context	Service	Lý do tách
Identity & Access	kblab-auth	Ranh giới rõ ràng, thay đổi auth ≠ thay đổi logic học tập
SQL Practice	kblab-app, kblab-judge, sandbox DB services	Core domain, cần sandbox isolation và chấm tự động
Network Practice	kblab-judge-network	Protocol đa dạng, state/routing khác REST API thông thường
Academic Management	kblab-assignment	Domain học vụ, grade scheme, attendance, thesis
DevOps Practice	dol-api, dol-infra, dol-web	Domain lab infrastructure, Go/k3s/Sysbox, validator script

💡 Tip — Một bounded context ≠ một service

Bounded context là ranh giới logic. Một context có thể được implement bởi nhiều service (như SQL Practice = Core + Judge + sandbox DB services). Ngược lại, một repo cũng có thể sinh nhiều binary phục vụ cùng context (như DevOps Practice = API + router + grader trong một codebase Go) — miễn là ranh giới domain rõ ràng.

2.7. Shared Kernel Pattern — Chia sẻ hay không chia sẻ?

2.7.1. Vấn đề thực tế

Trong KBLab, tồn tại một **shared library** chứa 23 Java files: base entities, DTOs, JWT utilities, exception handling, validation utilities. Nhiều service Java phụ thuộc vào library này.

Đây là pattern **Shared Kernel** trong DDD: hai hoặc nhiều bounded contexts đồng ý chia sẻ một phần mô hình chung. Shared Kernel giảm duplication nhưng tạo coupling — mọi thay đổi trong shared library đều *có thể* ảnh hưởng tất cả service.

```
shared-library/
├─ common/      BaseMapper, BaseRequest, BaseResponse, BaseService
├─ config/      AuditConfig, CorsConfig, TimeZoneConfig
├─ enumerate/   AnswerStatus, ErrorCode, TypeQuestion
├─ exception/   GlobalExceptionHandler
├─ securityConfig/ JwtConfig, UserDetailsCustom
└─ utils/       CompareUtil, JwtUtil, SqlUtil, TableDependencyResolver
```

2.7.2. Phân tích: cái gì nên shared, cái gì không?

Bảng 2.10: Phân tích shared library — cái gì nên shared, cái gì không

Loại	Ví dụ	Nên shared?	Lý do
Cross-cutting concerns	JWT validation, exception handling, CORS config	✅ Có	Nhất quán hành vi xuyên service, ít thay đổi
Base abstractions	BaseEntity, BaseMapper, BaseResponse	⚠️ Cần nhắc	Tiện nhưng tạo coupling vào convention
Domain-specific logic	CompareUtil (SQL comparison), SqlUtil	❌ Không	Chỉ Judge context cần — không phải shared concern
Domain enums	TypeQuestion, AnswerStatus	❌ Không	Thuộc về SQL Practice context, không universal

2.7.3. Hướng cải thiện

Nếu team phát triển lớn hơn và cần giảm coupling, shared library có thể được tách thành:

1. **kblab-security** — JWT, auth config (mọi service cần)
2. **kblab-common** — Base entities, response format (convention)
3. **Domain-specific code** → chuyển vào service tương ứng (Judge, Core)

Hướng tách shared library

Hiện tại: 1 shared library

	shared-library
	(23 files — tất cả)
BaseEntity, BaseResponse	
JWT, Security Config	
CompareUtil, SqlUtil	
TypeQuestion, AnswerStatus	

Hướng cải thiện

	tách
	lms-security
	(JWT, auth)
	lms-common
	(base, response)
Domain code → service riêng	
CompareUtil → Judge	
SqlUtil → Core	
TypeQuestion → Core	

Kết quả: mỗi service chỉ depend những gì cần

Auth Service	deps: lms-security
Core Service	deps: lms-security, lms-common
Judge Service	deps: lms-security, domain/judge
Assignment Svc	deps: lms-security, lms-common

Hình 2.8: Hướng tách shared library — từ monolithic sang modular

⚠ Nguyên tắc — Shared Kernel Trade-off

Shared Kernel là *thỏa thuận có chủ đích*, không phải sự tiện lợi. Mỗi file trong shared library phải trả lời: “Nếu thay đổi file này, tất cả service *phải* thay đổi cùng nhau — và điều đó *hợp lý*.” Nếu câu trả lời là không, file đó không nên nằm trong shared kernel.

📖 Phân tích — Shared Kernel cần tách

Với 23 files trong shared library, KBLab đang gây coupling không cần thiết: domain-specific code như `CompareUtil` (so sánh SQL) nằm chung với cross-cutting concerns như JWT. Richardson trong [2a] khuyến nghị chỉ share những gì thực sự cross-cutting (event definitions, API contracts). **Migration path:** (1) tách domain code về service riêng, (2) giữ shared chỉ security + base response, (3) đưa domain enums (`TypeQuestion`, `AnswerStatus`) về context sở hữu.

⚠ Lưu ý —

1. **Thiết kế entity từ bảng database** — Nhìn ERD rồi tạo 1 entity = 1 bảng, 1 service = 1 nhóm bảng. Hậu quả: ranh giới service không phản ánh domain mà phản ánh schema — khi nghiệp vụ thay đổi, phải refactor cả service lẫn database. *Phòng tránh*: bắt đầu từ domain process (Event Storming §2.4), không từ schema.
2. **Chia quá nhỏ (nano-services)** — Tách mỗi entity thành 1 service riêng (UserService, RoleService, TokenService) khi chúng luôn thay đổi cùng nhau. Hậu quả: mỗi request cần gọi 3-4 services, latency tăng, debugging trở thành cơn ác mộng. *Phòng tránh*: tách theo bounded context, không theo entity. Nếu hai entity luôn thay đổi cùng nhau, chúng thuộc cùng service.
3. **Dùng Shared Kernel vì tiện, không vì thiết kế** — Cho tất cả code “chung” vào shared library mà không phân biệt cross-cutting concern và domain logic. Hậu quả: mỗi thay đổi shared lib ảnh hưởng tất cả services, deploy coupling ngàm. *Phòng tránh*: mỗi file trong shared kernel phải trả lời “tất cả service *phải* thay đổi cùng khi file này thay đổi?” (§2.6).

2.8. Assemblage Process — Quy trình thiết kế service architecture

Làm thế nào để tổng hợp tất cả các kỹ thuật trên — Bounded Context, Context Map, Dark Energy/Matter — thành một quy trình thiết kế mang tính thực hành? Richardson trong [2b, Ch.20] trình bày **Assemblage Process**: quy trình step-by-step để đi từ yêu cầu nghiệp vụ đến kiến trúc microservices cụ thể.

1. **Định nghĩa System Operations** — Liệt kê các thao tác chính mà hệ thống phải hỗ trợ (ví dụ: `submitAnswer()`, `createContest()`, `gradeSubmission()`).
2. **Xác định Subdomains** — Dùng Bounded Contexts (§2.3) để phân vùng nghiệp vụ: Identity, SQL Practice, Network Practice, Academic Management, DevOps Practice.
3. **Gán Operations → Subdomains** — Quyết định logic nghiệp vụ của mỗi operation thuộc subdomain nào.
4. **Áp dụng Dark Energy & Dark Matter** — Với mỗi cặp subdomains, phân tích 10 lực đã thảo luận ở §2.5: lực đẩy (Dark Energy) khuyến khích tách thành service độc lập; lực hút (Dark Matter) giữ các phần chặt chẽ lại cùng nhau.
5. **Thiết kế IPC** — Chọn mô hình giao tiếp giữa các service: đồng bộ REST/gRPC (Ch.4) hay bất đồng bộ Kafka (Ch.5).
6. **Lặp lại** — Quy trình không chạy một lần. Khi kiến thức domain sâu hơn, ranh giới service cần được đánh giá lại.

Assemblage process biến quá trình thiết kế kiến trúc — vốn mang tính nghệ thuật và chủ quan — thành chuỗi các quyết định kỹ thuật có thể giải thích được. Thay vì nói “tách Judge Service vì nó cảm thấy đúng”, chúng ta có thể lập luận: “*Judge Service được tách vì Dark Energy force #1 (simple components) và #3 (fast deployment pipeline) vượt trội so với Dark Matter force #2 (efficient interaction) trong context này.*”

 **Trực quan hóa tương tác (Interactive Demo)**

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/context-map.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Bản đồ Bounded Context (Context Map)**.

2.9. Tổng kết

Chương này đã trang bị cho chúng ta bộ công cụ tư duy để phân tích và phân tách hệ thống — từ quy luật vĩ mô (Conway’s Law) đến kỹ thuật vi mô (Entity, Aggregate, Repository), và quy trình tổng hợp (Assemblage Process).

Conway’s Law nhắc nhở rằng kiến trúc phần mềm không tồn tại trong chân không — nó phản ánh và bị ràng buộc bởi cấu trúc tổ chức. Inverse Conway Maneuver cho phép chúng ta chủ động thiết kế team để đạt kiến trúc mong muốn.

Chúng ta có hai phương pháp hỗ trợ để xác định ranh giới service. **DDD** cung cấp ngôn ngữ và heuristics — Bounded Context, Context Map, Event Storming — phù hợp khi team hiểu domain sâu. **Erl’s Service-Oriented Analysis** cung cấp quy trình step-by-step — từ automation candidates đến service layers — phù hợp khi team cần một framework có cấu trúc. Cả hai đều dẫn đến kết quả tương tự (như minh họa với KBLab: 5 service groups chính), nhưng từ góc nhìn khác nhau. Dark Energy/Dark Matter forces lượng hóa các yếu tố ảnh hưởng đến quyết định tách/gộp, và Assemblage Process (§2.7) tổng hợp tất cả thành quy trình thiết kế có hệ thống.

Phân tích KBLab cho thấy 5 bounded contexts rõ ràng, mỗi context tương ứng với một nhóm service hoặc binary. Shared Kernel — dù tiện lợi cho team nhỏ — cần được quản lý có chủ đích, phân biệt rõ giữa cross-cutting concerns (nên share) và domain logic (không nên share).

Ở Chương 3, chúng ta sẽ đi sâu vào **thiết kế API**: khi đã xác định được ranh giới service, câu hỏi tiếp theo là *các service giao tiếp với nhau như thế nào?* REST API design, contract-first approach, schema evolution, và documentation sẽ là trọng tâm.

2.10. Đọc thêm

Sách tham khảo chính:

- [1] Thomas Erl, *SOA: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.8–10: Service-Oriented Analysis, Service Layers (task/entity/utility), Service Inventory Blueprint
- [6] Eric Evans, *Domain-Driven Design* — Ch.1–5: Knowledge Crunching, Ubiquitous Language, Building Blocks; Ch.14: Bounded Context, Context Map
- [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.2: Decomposition Strategies
- [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Loose Coupling, Dark Energy/Dark Matter Forces; Ch.7: Microservice Architecture; Ch.20: Assemblage Process
- [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services; Ch.10: Conway’s Law
- [4b] Sam Newman, *Monolith to Microservices* — Ch.2: Planning a Migration, Decomposition by Domain
- [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.3: Service Boundaries, DDD/Bounded Context

Sách bổ trợ:

8. [9] Vaughn Vernon, *Implementing Domain-Driven Design* — Ch.2: Strategic Design with Bounded Contexts
9. [8] Matthew Skelton & Manuel Pais, *Team Topologies* — Four Team Types, Cognitive Load, Conway's Law
10. [10] Alberto Brandolini, *Introducing EventStorming* — Domain discovery workshops

Nguồn trực tuyến:

- MacCormack, Rusnak & Baldwin, “*Exploring the Duality between Product and Organizational Architectures*” (2008) — Harvard Business School Working Paper

3. Thiết kế Dịch vụ & API

“A well-designed API is a conversation between the provider and the consumer — clear, consistent, and forgiving.” — Adapted from REST API design best practices [1]

3.1. Bạn sẽ học được gì

- Nắm vững nguyên tắc thiết kế REST API cho microservices
- Hiểu các chiến lược API versioning và schema evolution
- Sử dụng OpenAPI/Swagger để tạo documentation tự động
- Áp dụng DTO pattern để tách biệt internal model và external contract
- Phân tích bài học thực tế từ API design trong KBLab

3.2. REST API Design — Nguyên tắc cho Microservices

3.2.1. Tại sao API design quan trọng hơn trong microservices?

Trong monolith, giao tiếp giữa các module là gọi hàm nội bộ (in-process call) — nếu interface thay đổi, compiler sẽ báo lỗi ngay. Trong microservices, giao tiếp qua network (HTTP, gRPC, messaging) — và **không có compiler nào bắt lỗi khi API thay đổi**. Một breaking change trong API có thể crash service khác mà không ai biết cho đến khi production gặp sự cố.

Do đó, API trong microservices cần được thiết kế **cẩn thận hơn, ổn định hơn**, và **có chiến lược evolution rõ ràng** — đây không phải optional, mà là điều kiện tiên quyết.

3.2.2. Richardson Maturity Model

Leonard Richardson đề xuất mô hình 4 cấp độ trưởng thành cho REST API [2a, Ch.3]:

Richardson Maturity Model



Mức độ trưởng thành REST tăng dần từ Level 0 (RPC thuần túy) đến Level 3 (HATEOAS). Hầu hết các API thực tế đạt Level 2.

Hình 3.1: Richardson Maturity Model — bốn cấp độ trưởng thành của REST API

Hầu hết microservices thực tế (bao gồm KBLab LMS) hoạt động ở **Level 2** — sử dụng đúng resources và HTTP verbs.

Level 3 (HATEOAS) — Khi nào cần? HATEOAS (*Hypermedia as the Engine of Application State*) yêu cầu response chứa links điều hướng — client không cần biết trước URL structure, chỉ follow links từ response. Ví dụ: `GET /questions/123` trả về `{..., "_links": {"submissions": "/questions/123/submissions", "edit": "/questions/123"}}`. Ưu điểm: API self-documenting, client không depend vào URL structure. Nhược điểm: tăng payload size, phức tạp response format, hầu hết frontend frameworks (React, Angular) không leverage HATEOAS. Trong thực tế, HATEOAS hiếm khi được áp dụng cho internal APIs [4a, Ch.4] — phù hợp hơn cho public APIs khi provider không kiểm soát consumer (GitHub API là ví dụ tiêu biểu).

3.2.3. Nguyên tắc thiết kế REST API

1. Resource-oriented URLs — URL đại diện cho resource (danh từ), không phải hành động (động từ):

- ✓ `GET /questions/{id}` → Lấy câu hỏi
- ✓ `POST /questions` → Tạo câu hỏi mới
- ✓ `GET /contests/{id}/questions` → Câu hỏi trong cuộc thi
- ✗ `GET /getQuestion?id=123` → Dùng verb, query string
- ✗ `POST /createQuestion` → URL chứa action

2. HTTP methods đúng ngữ nghĩa:

Bảng 3.1: HTTP methods và ngữ nghĩa

Method	Ý nghĩa	Idempotent?	Ví dụ
GET	Đọc resource	✓ Có	<code>GET /questions/123</code>
POST	Tạo resource mới	✗ Không	<code>POST /submissions</code>
PUT	Cập nhật toàn bộ	✓ Có	<code>PUT /questions/123</code>
PATCH	Cập nhật một phần	✗ Không	<code>PATCH /users/123</code>
DELETE	Xóa resource	✓ Có	<code>DELETE /questions/123</code>

3. Response codes có ý nghĩa:

Bảng 3.2: HTTP response codes thường dùng trong microservices

Code	Khi nào dùng	Ví dụ
200 OK	Request thành công	GET /questions/123 trả về question
201 Created	Tạo resource mới thành công	POST /questions trả về question vừa tạo
204 No Content	Thành công nhưng không có body	DELETE /questions/123
400 Bad Request	Input không hợp lệ	Thiếu field bắt buộc
401 Unauthorized	Chưa xác thực	Token hết hạn
403 Forbidden	Không có quyền	Student truy cập admin API
404 Not Found	Resource không tồn tại	GET /questions/99999
409 Conflict	Conflict trạng thái	Submission đã được chấm
500 Internal Server Error	Lỗi server	Database connection failed

4. Plural nouns, consistent naming — Luôn dùng số nhiều cho collection: /questions, không phải /question. Chọn một convention (kebab-case, camelCase, snake_case) và giữ nhất quán xuyên suốt.

5. Pagination, filtering, sorting — Mọi endpoint trả về collection phải hỗ trợ phân trang. Có hai chiến lược chính:

Bảng 3.3: Hai chiến lược pagination

Chiến lược	Request	Ưu điểm	Nhược điểm
Offset-based	?page=2&size=20	Đơn giản, jump to page	Performance giảm ở page lớn (DB phải skip N rows), kết quả không ổn định nếu data thêm/xóa giữa chừng
Cursor-based	?after=abc123&limit=20	Performance ổn định (O(1) với index), kết quả nhất quán	Không jump to page, phức tạp hơn

KBLab sử dụng **offset-based** (Spring Data Pageable) — hợp lý cho danh sách câu hỏi quy mô vừa. Với datasets lớn (submissions history, audit logs), cursor-based pagination hiệu quả hơn khi `page > 100`.

Response pagination nên bao gồm metadata để client biết có thêm data không:

Listing 3.1: Response pagination với metadata

```
GET /questions?page=0&size=20&sort=createdAt,desc&difficulty=HARD
```

```
// Response envelope
{
  "data": [...],
  "pagination": {
    "page": 0,
```

```
    "size": 20,  
    "totalElements": 157,  
    "totalPages": 8,  
    "hasNext": true  
  }  
}
```

6. Standardized Error Response — Error responses cần format nhất quán để client xử lý programmatically. Lấy cảm hứng từ RFC 7807 (Problem Details for HTTP APIs), một error response nên bao gồm:

Bảng 3.4: Các fields trong standardized error response (RFC 7807-inspired)

Field	Mô tả	Bắt buộc?
status	HTTP status code	✓
error	Mã lỗi machine-readable	✓
message	Mô tả human-readable	✓
timestamp	Thời điểm lỗi xảy ra	⚠ Nên có
path	Request path gây lỗi	⚠ Nên có
details	Chi tiết bổ sung (validation errors)	Optional

Listing 3.2: Standardized error response (RFC 7807-inspired)

```
// ✓ Standardized error response (RFC 7807-inspired)  
{  
  "status": 400,  
  "error": "VALIDATION_FAILED",  
  "message": "Input validation failed",  
  "timestamp": "2026-03-20T10:30:00Z",  
  "path": "/api/questions",  
  "details": [  
    {"field": "title", "message": "must not be blank"},  
    {"field": "difficulty", "message": "must be EASY, MEDIUM, or HARD"}  
  ]  
}
```

Mọi service trong hệ thống nên trả error cùng format — client chỉ cần viết **một** error handler. Trong Spring Boot, `@ControllerAdvice` + `GlobalExceptionHandler` giải quyết: mọi exception được catch tại một điểm duy nhất và biến thành response format chuẩn (xem thêm §3.5).

7. Idempotency — Một số operations cần đảm bảo **idempotent**: gọi nhiều lần cho cùng kết quả. GET, PUT, DELETE tự nhiên idempotent. POST thì không — nếu client gọi POST / submissions hai lần (do network retry), có thể tạo hai submissions.

Giải pháp: **Idempotency Key** — client gửi kèm header `Idempotency-Key: <UUID>`. Server kiểm tra key đã xử lý chưa: nếu rồi, trả kết quả cũ; nếu chưa, xử lý và lưu key. Stripe, PayPal đều dùng pattern này cho payment API — critical khi liên quan đến tài chính hoặc business operations quan trọng.

Bảng 3.5: Idempotency theo HTTP method

Method	Idempotent?	Giải pháp nếu không
GET	✓ Tự nhiên	—
PUT	✓ Tự nhiên	—
DELETE	✓ Tự nhiên	—
POST	✗ Không	Idempotency-Key header
PATCH	⚠ Tùy	Depends on implementation

! Nguyên tắc — Contract-First vs Code-First

Có hai cách tiếp cận thiết kế API: **Contract-first** (viết OpenAPI spec trước, generate code sau) và **Code-first** (viết code trước, generate spec sau). Contract-first tốt hơn cho API public hoặc cross-team. Code-first nhanh hơn cho API internal trong team nhỏ. KBLab sử dụng code-first (Spring Boot annotations → auto-generated docs) — hợp lý cho team nhỏ [1, Ch.7].

3.3. API Versioning & Schema Evolution

3.3.1. Tại sao cần versioning?

Trong monolith, thay đổi internal API chỉ cần refactor code + chạy lại tests. Trong microservices, service A thay đổi response format → service B (consumer) *có thể* crash nếu không xử lý. Đây là bài toán **schema evolution** — và API versioning là giải pháp [7, Ch.4].

3.3.2. Ba chiến lược versioning

Bảng 3.6: Ba chiến lược API versioning

Chiến lược	Cách dùng	Ưu điểm	Nhược điểm
URL path	/api/v1/questions, /api/v2/questions	Đơn giản, rõ ràng	Nhiều URL, routing phức tạp
Query parameter	/questions?version=2	Ít thay đổi URL	Dễ bỏ sót
Header	Accept: application/vnd.kblab.v2+json	Clean URL	Khó debug, ít trực quan

Theo Sam Newman, **URL path versioning** phổ biến nhất vì đơn giản và developer-friendly [4a, Ch.4]. Đây cũng là cách mà hầu hết API public lớn (GitHub, Stripe, Twitter) sử dụng.

Ngoài ba chiến lược trên, còn có **Content-Type versioning** (Accept: application/vnd.kblab.question.v2+json) — phổ biến ở GitHub API: cho phép version từng resource riêng lẻ thay vì toàn bộ API. Tuy nhiên, phức tạp hơn cho cả provider và consumer.

Bảng 3.6b: Decision criteria — chọn versioning strategy nào?

Tiêu chí	URL path	Query param	Header	Content-Type
Dễ implement	★ ★ ★	★ ★ ★	★ ★	★
Cache-friendly	✓ (url khác nhau)	⚠ (cần Vary)	✗ (cần Vary header)	✗
API Gateway routing	✓ Dễ (path-based)	⚠ Trung bình	⚠ Trung bình	✗ Khó
Granularity	Toàn bộ API	Toàn bộ API	Toàn bộ API	Per resource
Dùng bởi	Stripe, Twitter	Ít phổ biến	Azure	GitHub

3.3.2.1. Versioning Policy — Nguyên tắc vận hành

Chọn strategy chưa đủ — cần **policy** rõ ràng cho cách version API:

1. **Khi nào bump version?** — Chỉ khi có **breaking change** (xóa field, đổi type, đổi behavior). Thêm field optional → KHÔNG cần version mới (backward compatible)
2. **Deprecation timeline** — Thông báo deprecation trước ít nhất **1 release cycle**: response header Deprecation: true, Sunset: 2026-06-01
3. **Chạy song song** — Version cũ và mới chạy đồng thời trong giai đoạn transition. Dùng Gateway routing để chuyển dần traffic (Strangler Fig pattern — xem Ch.10)

💡 Tip — KBLab Versioning Policy đề xuất

Với KBLab (LMS học thuật, team nhỏ), chiến lược đơn giản nhất: (1) **URL path versioning** (/api/v1/questions), (2) giữ **backward compatibility** bằng cách chỉ thêm fields, không xóa, (3) khi breaking change bắt buộc → tạo /api/v2/ → chạy song song 1 semester → sunset v1. Phức tạp hơn không cần thiết cho hệ thống giáo dục nội bộ.

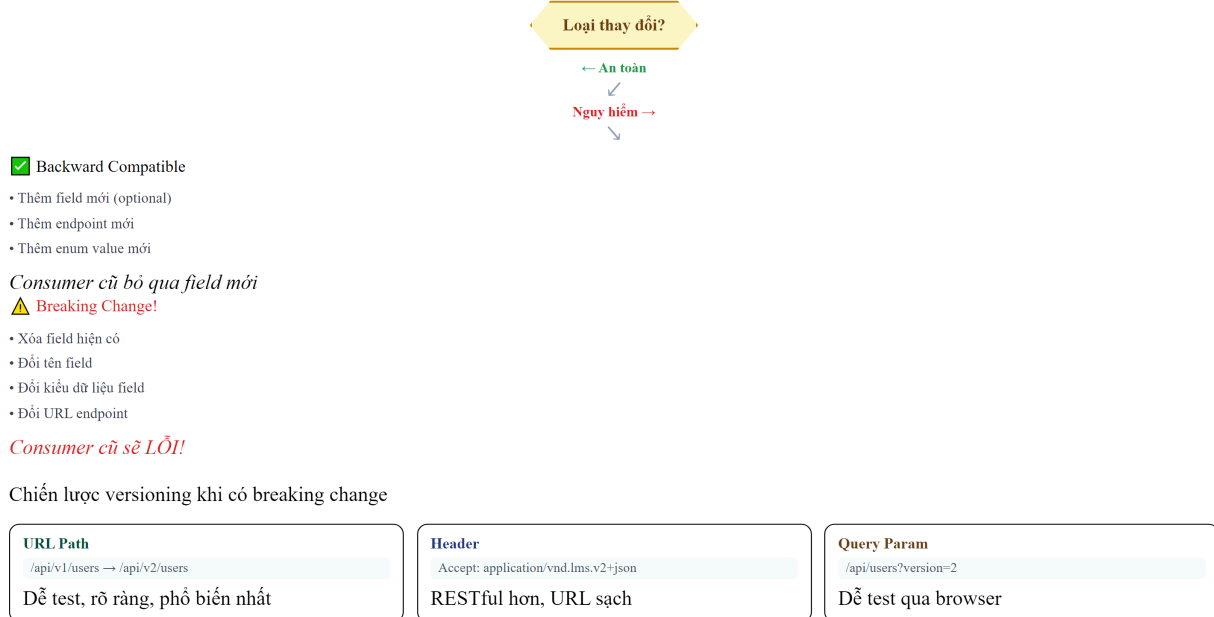
3.3.3. Schema Evolution — Thay đổi mà không breaking

Martin Kleppmann phân tích hai chiều compatibility [7, Ch.4]:

- **Backward compatible**: code mới đọc được data cũ (thêm field optional → OK)
- **Forward compatible**: code cũ đọc được data mới (bỏ qua field không biết → OK)

Quy tắc vàng khi thay đổi API:

Chiến lược versioning API



Hình 3.2: Quy tắc schema evolution — thay đổi nào là breaking change

① Phân tích — KBLab thiếu API versioning

KBLab hiện chưa chuẩn hóa API versioning — nhiều endpoints nằm tại `/api/*` không có version prefix. Đây là **thiếu sót cần khắc phục**: khi hệ thống thêm mobile app hoặc third-party integration, mỗi thay đổi API sẽ là breaking change cho consumer không kịp cập nhật. Stripe API — một trong những API được thiết kế tốt nhất — sử dụng date-based versioning (2024-12-18), cho phép consumer pin vào một version cụ thể. **Migration path**: thêm `/api/v1/` prefix cho tất cả route hiện tại, cấu hình gateway routing, và document versioning policy.

3.4. OpenAPI & API Documentation

3.4.1. Tại sao documentation quan trọng?

Chris Richardson nhấn mạnh rằng trong microservices, API documentation không phải “nice-to-have” mà là **hợp đồng** (*contract*) giữa provider và consumer [2a, Ch.3]. Nếu documentation sai hoặc thiếu, developer phải đọc source code để hiểu API — điều này phá vỡ nguyên lý *Service Abstraction* [1].

3.4.2. OpenAPI Specification (Swagger)

OpenAPI (trước đây gọi là Swagger) là chuẩn mô tả REST API dưới dạng YAML/JSON. Từ spec này, có thể tự động generate:

- Documentation UI (Swagger UI, ReDoc)
- Client SDK (Java, TypeScript, Python)
- Server stubs
- Tests

Trong Spring Boot (như LMS), sử dụng **SpringDoc OpenAPI** để tự động tạo spec từ code:

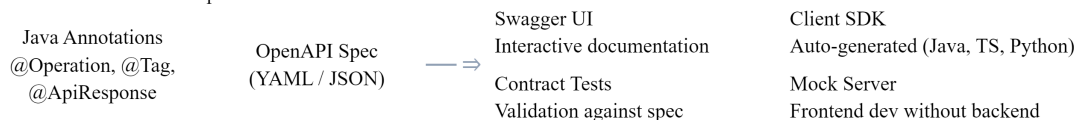
Listing 3.3: Spring Boot controller với OpenAPI annotations

```
// Annotations trên controller → auto-generate OpenAPI spec
@RestController
@RequestMapping("/api/questions")
@Tag(name = "Questions", description = "Quản lý câu hỏi SQL")
public class QuestionController {

    @GetMapping("/{id}")
    @Operation(summary = "Lấy chi tiết câu hỏi theo ID")
    @ApiResponse(responseCode = "200", description = "Thành công")
    @ApiResponse(responseCode = "404", description = "Không tìm thấy")
    public QuestionResponse getById(@PathVariable UUID id) {
        // ...
    }
}
```

Kết quả: truy cập `/swagger-ui.html` → có UI tương tác để test API trực tiếp.

API-First Workflow với OpenAPI



Nguyên tắc: Design API trước (contract-first), implement sau. Mọi thay đổi API phải cập nhật spec.

LMS: Hiện đang code-first (annotations). Migration: export spec → review → contract tests.

Hình 3.3: Pipeline từ code annotations đến Swagger UI và client SDK

Tip — Documentation-as-Code

Coi API documentation như code: version control, review, automate. Nếu documentation nằm ngoài code (wiki, Confluence), nó sẽ nhanh chóng lỗi thời. SpringDoc/OpenAPI giải quyết vấn đề này bằng cách generate docs trực tiếp từ code.

3.5. DTO Pattern — Tách Internal Model và External Contract

3.5.1. Vấn đề: Entity ≠ API Response

Sai lầm phổ biến trong microservices (và monolith) là trả entity trực tiếp qua API:

Listing 3.4: Anti-pattern — trả entity trực tiếp qua API

```
// ❌ Trả entity trực tiếp – expose internal structure
@GetMapping("/questions/{id}")
public Question getById(@PathVariable UUID id) {
    return questionRepository.findById(id).orElseThrow();
}
```

Vấn đề:

- **Coupling:** thay đổi database schema = thay đổi API response
- **Security:** có thể expose field nhạy cảm (password hash, internal IDs)
- **Performance:** entity có thể chứa quan hệ lazy-loaded → N+1 queries hoặc serialization errors

3.5.2. DTO (Data Transfer Object)

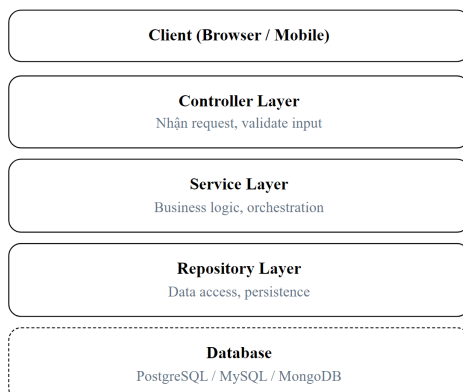
DTO pattern tạo lớp đệm giữa internal model và external contract [2a]:

Kiến trúc phân lớp truyền thống (Layered)
Vấn đề

⚠ Dependency 1 chiều (top-down)

⚠ Domain logic bị phụ thuộc vào framework (Spring, JPA, HTTP...)

⚠ Khó test business logic mà không cần database hoặc web server



Luồng request

- ① HTTP Request vào Controller
- ② Controller gọi Service
- ③ Service gọi Repository
- ④ Repository truy vấn DB
- ⑤ Kết quả trả ngược lên

Hình 3.4: DTO pattern — tách internal model và external contract

Listing 3.5: Request DTO, Response DTO và Mapper

```
// ✅ Request DTO – chỉ chứa field client cần gửi
public record CreateQuestionRequest(
    String title,
    String description,
    String solution,
    String difficulty
) {}

// ✅ Response DTO – chỉ chứa field client cần nhận
public record QuestionResponse(
    UUID id,
    String title,
    String difficulty,
    int submitCount,
    double acceptRate,
    LocalDateTime createdAt
) {}

// Mapper chuyển đổi Entity ↔ DTO
@Mapper(componentModel = "spring")
public interface QuestionMapper {
    QuestionResponse toResponse(Question entity);
    Question toEntity(CreateQuestionRequest request);
}
```

3.5.3. Lợi ích thực tế

Bảng 3.7: Lợi ích của DTO pattern

Lợi ích	Giải thích
Decoupling	Thay đổi entity (thêm cột) không tự động thay đổi API
Security	Không vô tình expose password, internal fields
Performance	Chỉ trả field cần thiết, không lazy-load cả graph
Versioning	Có thể tạo <code>QuestionResponseV2</code> mà không sửa entity
Validation	@Valid trên DTO, không trên entity

Trong KBLab, pattern này được áp dụng qua `dto/request/` và `dto/response/` packages, với MapStruct (`BaseMapper`) để tự động chuyển đổi. Các base classes `BaseRequest` và `BaseResponse` trong shared library cung cấp format phản hồi chuẩn (`status`, `message`, `data wrapper`).

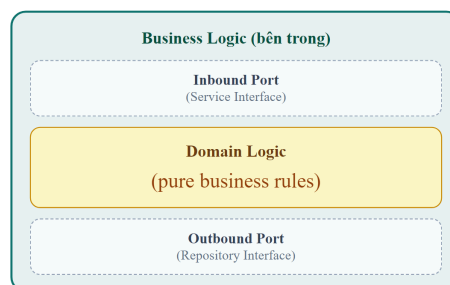
3.5.4. Hexagonal Architecture — Service như hệ thống cổng và adapter

DTO pattern ở trên là một phần của kiến trúc rộng hơn mà Richardson mô tả chi tiết trong [2b, Ch.3 & Ch.13]: **Hexagonal Architecture** (còn gọi là Ports & Adapters, do Alistair Cockburn đề xuất). Đây là kiến trúc được đề xuất cho mỗi service trong microservices:

Hexagonal Architecture (Ports & Adapters)

Adapters (bên ngoài) — Inbound

REST Controller
(inbound adapter)
Kafka Consumer
(inbound adapter)



Adapters (bên ngoài) — Outbound

JPA Repository
(outbound adapter)
Feign Client
(outbound adapter)



Core principle: Domain Logic không biết gì về HTTP, JPA, Kafka.

Lợi ích: Test business rules mà không cần infrastructure. Thay đổi adapter không ảnh hưởng domain.

Hình 3.6b: Hexagonal Architecture cho Core Service trong KBLab

Nguyên tắc cốt lõi: Business logic **không phụ thuộc** vào adapters — ngược lại, adapters phụ thuộc vào business logic. Điều này có nghĩa:

- Đổi từ REST sang gRPC? → chỉ thay inbound adapter, business logic không đổi
- Đổi từ MySQL sang PostgreSQL? → chỉ thay outbound adapter
- Test business logic? → mock outbound ports, gọi inbound ports trực tiếp

Thành phần	Trong LMS	Vai trò
Inbound Adapter	@RestController, @KafkaListener	Nhận request từ bên ngoài
Inbound Port	Service interface (ví dụ: API của business logic QuestionService)	
Business Logic	Domain entities, service implementations	Xử lý nghiệp vụ
Outbound Port	Repository interface (JpaRepository)	Cách business logic gọi external
Outbound Adapter	JPA impl, Feign client, Kafka producer	Kết nối bên ngoài

💡 Tip — Iceberg Principle trong API Design

Hexagonal Architecture là hiện thực hóa của **Iceberg Principle** (Richardson [2b, Ch.4]): Inbound Ports (API) là phần nổi — nhỏ, ổn định. Business Logic + Outbound Adapters là phần chìm — lớn, tự do thay đổi. DTO pattern là cơ chế đảm bảo entity (phần chìm) không bị expose qua API (phần nổi).

3.6. Case Study: API Design trong KBLab — Bài học từ thực tế

3.6.1. Hiện trạng

Phân tích source code KBLab cho thấy một số quyết định API design thực tế — một số tốt, một số là *anti-pattern* do phát triển hữu cơ (organic growth):

Điểm mạnh:

- DTO pattern được áp dụng nhất quán (request/response separation)
- Swagger/OpenAPI tích hợp sẵn
- JSON response format chuẩn (`BaseResponse<T>` wrapper)
- KBLab giữ REST/OpenAPI làm contract chính cho LMS core, đồng thời mở rộng Network Lab bằng SOAP/REST/gRPC/JNP và DevOps Lab bằng Go/Chi router. Vì vậy, case study hiện tại là **multi-protocol có chủ đích**, không còn là một hệ thống REST thuần.

Điểm cần cải thiện (từ source-registry.md §3.1):

Bảng 3.8: Phân tích API design trong KBLab — các điểm cần cải thiện

#	Vấn đề	Hiện trạng	Cách cải thiện
1	Inconsistent naming	Mix (singular) với (plural), /submit-history (kebab)	Chuẩn hóa: tất cả plural + kebab-case
2	Không có API versioning	Tất cả endpoint tại /api/*	Thêm prefix /api/v1/*
3	Error response không nhất quán	ServerError.description vs JwtUtil.message	Chuẩn hóa error format qua GlobalExceptionHandler
4	Hard delete only	Xóa vĩnh viễn, không soft-delete	Thêm soft-delete cho audit trail
5	SQL Judge worker contract chưa chuẩn hóa	judge điều phối tới judge-mysql, judge-sqlserver, ... nhưng JudgeRequest/ JudgeResult cần contract rõ	Chuẩn hóa DTO/ OpenAPI chung cho judge-*, kèm versioning và idempotency key

3.6.2. Bài học rút ra

Những inconsistencies này không phải “lỗi” — chúng là kết quả tự nhiên khi hệ thống phát triển nhanh với team nhỏ, qua nhiều iteration. Bài học quan trọng:

1. **Naming convention nên được thống nhất sớm** — chi phí sửa sau này tăng theo số lượng consumer. Khi API chỉ có 1 consumer (frontend), đổi tên dễ. Khi có 3+ consumers (mobile, third-party, internal services), đổi tên = breaking change.
2. **Error format chuẩn hóa từ đầu** — GlobalExceptionHandler với @ControllerAdvice giải quyết 90% vấn đề. Trong KBLab, handler tồn tại nhưng Gateway xử lý JWT errors riêng (vì được phát triển độc lập) → hai format khác nhau.
3. **API versioning chỉ cần khi có multiple consumers** — với team sở hữu cả provider và consumer, implicit versioning (deploy cùng nhau) là đủ. Đừng over-engineer.

! Nguyên tắc — Tolerant Reader

Martin Fowler đề xuất pattern “Tolerant Reader”: consumer nên bỏ qua fields không biết và chỉ đọc fields cần thiết. Điều này cho phép provider thêm fields mới mà không breaking consumers — **backward compatibility tự nhiên** [W2].

3.6.3. GraphQL — Khi REST không đủ linh hoạt

REST là chuẩn phổ biến nhất cho microservices APIs, nhưng không phải lựa chọn duy nhất. **GraphQL** (Facebook, 2012; open-sourced 2015) giải quyết một số hạn chế của REST bằng cách cho phép client **chọn chính xác data cần lấy**.

3.6.3.1. Vấn đề mà GraphQL giải quyết

Với REST, mỗi endpoint trả về *fixed structure*. Client lấy nhiều hơn hoặc ít hơn data cần thiết:

Bảng 3.9: Over-fetching và Under-fetching trong REST — GraphQL giải quyết

Vấn đề REST	Ví dụ KBLab	GraphQL giải quyết
Over-fetching	GET /questions/1 trả về 20 fields, mobile chỉ cần title + difficulty	Client chỉ request fields cần: { question(id:1) { title difficulty } }
Under-fetching	Dashboard cần question + submissions + user → 3 API calls	1 GraphQL query lấy tất cả: { question { title submissions { score } user { name } } }
Multiple round trips	Mobile: gọi 3 endpoints → 3 network requests → latency	1 request, 1 response — critical cho mobile networks

3.6.3.2. REST vs GraphQL — So sánh**Bảng 3.10:** REST vs GraphQL — so sánh chi tiết

Tiêu chí	REST	GraphQL
Endpoint	Nhiều endpoints (/questions, /submissions, /users)	Một endpoint (/graphql)
Data shape	Server quyết định structure trả về	Client quyết định fields cần lấy
Versioning	URL versioning (/v1/, /v2/) hoặc headers	Schema evolution — thêm fields mới, deprecate fields cũ
Caching	HTTP caching tự nhiên (GET, ETag, Cache-Control)	Phức tạp hơn (POST requests, cần Apollo Cache hoặc tương tự)
Tooling	Swagger/OpenAPI mature, ubiquitous	GraphQL Playground, Apollo DevTools — powerful nhưng hệ sinh thái nhỏ hơn
Learning curve	Thấp (HTTP verbs, status codes)	Trung bình (schema language, resolvers, N+1 problem)
Error handling	HTTP status codes (400, 404, 500)	Luôn trả về 200 OK + errors array — mất semantic clarity

3.6.3.3. N+1 Problem — Thách thức chính của GraphQL

Query: lấy tất cả questions và submissions cho mỗi question
 { questions { title submissions { score } } }

Server thực thi:

1. SELECT * FROM questions (N questions)

2. Với MỖI question: SELECT * FROM submissions WHERE question_id = ?

→ N+1 queries! (1 cho questions + N cho submissions)

Giải pháp: DataLoader pattern — batch N queries thành 1: SELECT * FROM submissions WHERE question_id IN (1, 2, ..., N). Facebook open-sourced DataLoader library cho mục đích này. Nhưng đây là responsibility *server-side* mà REST không gặp (vì server kiểm soát query structure).

3.6.3.4. Schema-First Approach

GraphQL yêu cầu **schema definition** trước khi implementation — tương tự contract-first cho REST (OpenAPI spec trước code):

Listing 3.6: GraphQL schema definition cho KBLab

```
type Question {  
  id: ID!  
  title: String!  
  difficulty: Difficulty!  
  submissions: [Submission!]!  
  createdBy: User!  
}  
  
type Query {  
  question(id: ID!): Question  
  questions(difficulty: Difficulty, limit: Int = 20): [Question!]!  
}  
  
enum Difficulty { EASY MEDIUM HARD }
```

Schema này vừa là documentation, vừa là contract, vừa là type system — GraphQL introspection cho phép tools tự động generate client code và documentation.

3.6.3.5. Khi nào chọn REST, khi nào chọn GraphQL?

Bảng 3.11: Khi nào chọn REST, khi nào chọn GraphQL

Scenario	Khuyến nghị	Lý do
Service-to-service (backend)	REST hoặc gRPC	Simple, cacheable, teams kiểm soát cả hai phía
BFF cho mobile	GraphQL	Mobile cần flexible queries, giảm round trips
Public API	REST	HTTP standards, caching, documentation ecosystem
Dashboard aggregation	GraphQL	1 query thay vì 5-10 REST calls
CRUD đơn giản	REST	GraphQL overhead không justify
Real-time (subscriptions)	GraphQL + WebSocket	Native support cho subscriptions

KBLab hiện tại: REST vẫn là lựa chọn phù hợp cho LMS core vì đơn giản, dễ debug và được OpenAPI hỗ trợ tốt. Tuy nhiên, KBLab v2.0 đã có bức tranh API rộng hơn: Network Lab dùng nhiều protocol để phục vụ bài học tích hợp hệ thống; DevOps Lab dùng Go/Chi cho router và API vận hành; các luồng chấm bài có thể đi qua Kafka thay vì HTTP đồng bộ. GraphQL sẽ có giá trị nếu KBLab thêm: (1) **mobile app** — cần flexible queries cho different screen sizes, (2) **teacher dashboard** — cần aggregate data từ nhiều services trong 1 request, (3) **third-party API** — consumers bên ngoài cần kiểm soát data họ nhận.

⚠ Nguyên tắc — GraphQL ≠ Thay thế REST

GraphQL và REST giải quyết vấn đề khác nhau. REST tối ưu cho service-to-service communication (simple, cacheable, standard). GraphQL tối ưu cho client-facing APIs (flexible, single endpoint, typed). Nhiều hệ thống dùng cả hai: REST giữa services, GraphQL cho BFF layer. Richardson trong [2b, Ch.8] thảo luận GraphQL như một API Gateway pattern choice.

⚠ Lưu ý —

1. **Trả entity trực tiếp qua API** — Không dùng DTO, expose toàn bộ internal structure (password hash, internal IDs, lazy-loaded relations). Hậu quả: thay đổi schema = breaking change cho mọi consumer, rủi ro bảo mật. *Phòng tránh*: luôn dùng DTO pattern (§3.4) — tách internal model khỏi external contract.
2. **Không thống nhất naming convention từ đầu** — Mix /question (singular) với /users (plural), mix kebab-case với camelCase. Hậu quả: khi có 3+ consumers (mobile, third-party, internal), sửa naming = breaking change hàng loạt. *Phòng tránh*: chọn convention (plural + kebab-case) và enforce từ PR đầu tiên.
3. **Bỏ qua error format chuẩn** — Mỗi service trả lỗi theo format riêng, mỗi developer viết error handling riêng. Hậu quả: consumer phải handle N format khác nhau, debugging khó khăn. *Phòng tránh*: dùng `GlobalExceptionHandler` với `@ControllerAdvice` và error response format thống nhất cho toàn hệ thống (§3.5).

🌐 Trực quan hóa tương tác (Interactive Demo)

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/rest-api-explorer.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Thiết kế REST API**.

3.7. Tổng kết

Chương này đã trình bày nền tảng thiết kế API cho microservices — từ nguyên tắc REST (resources, HTTP verbs, status codes) đến các vấn đề thực tế mà developer thường gặp.

API versioning và schema evolution là bài toán không thể tránh khỏi khi hệ thống phát triển. Ba chiến lược versioning (URL path, query param, header) mỗi cách có trade-off riêng — URL path phổ biến nhất vì developer-friendly. Quy tắc backward compatibility giúp thay đổi API mà không crash consumer.

OpenAPI/Swagger biến documentation thành code — tự động, luôn cập nhật, và có thể test trực tiếp. DTO pattern tách biệt internal model và external contract — điều kiện cần để API evolution không gắn chặt vào database schema.

Phân tích KBLab cho thấy: API design “đủ tốt” hoàn toàn khả thi cho team nhỏ, nhưng cần ý thức chuẩn hóa sớm (naming, error format, versioning policy) để tránh technical debt tích lũy.

Ở Chương 4, chúng ta sẽ đi vào **giao tiếp đồng bộ** giữa các service: OpenFeign, error handling, resilience patterns — khi API đã được thiết kế, câu hỏi là *service gọi service khác như thế nào, và xử lý lỗi ra sao?*

3.8. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *SOA: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.7: Service API Design, Service Contract
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, REST API Design
3. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.3: Architecture Fundamentals, Hexagonal Architecture; Ch.13: Service Design with Ports & Adapters
4. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, REST best practices
5. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution, Schema Evolution

Nguồn trực tuyến:

- [W2] Martin Fowler, “TolerantReader” — martinfowler.com/bliki/TolerantReader.html
- Leonard Richardson, “Richardson Maturity Model” — martinfowler.com/articles/richardsonMaturityModel.html
- OpenAPI Specification 3.1 — spec.openapis.org

PHẦN II

Giao tiếp & Dữ liệu

Giao tiếp và dữ liệu — Trái tim của hệ thống phân tán

Khi các dịch vụ đã được tách riêng, câu hỏi lớn nhất là: *chúng nói chuyện với nhau như thế nào?* Phần II đi sâu vào các pattern giao tiếp đồng bộ, bất đồng bộ qua messaging, xử lý giao dịch phân tán bằng Saga, và truy vấn dữ liệu chéo service bằng CQRS. Đây là phần khó nhất nhưng cũng quan trọng nhất — nơi mà sự khác biệt giữa “biết Microservices” và “làm được Microservices” thể hiện rõ nhất.

Chương 4 Giao tiếp Đồng bộ — REST, gRPC & Resilience

Chương 5 Giao tiếp Bất đồng bộ — Kafka, Events & Messaging

Chương 6 Giao dịch Phân tán — Saga Pattern

Chương 7 Quản lý Dữ liệu trong Microservices

4. Giao tiếp Đồng bộ — REST, gRPC & Resilience

“Microservices favor smart endpoints and dumb pipes. The communication infrastructure should be simple; intelligence belongs in the services.” — Sam Newman, Building Microservices [4a]

4.1. Bạn sẽ học được gì

- Hiểu các kiểu tương tác (interaction styles) trong microservices
- So sánh REST và gRPC — khi nào dùng cái nào
- Sử dụng OpenFeign để gọi service-to-service một cách declarative
- Nắm vững Service Discovery và Load Balancing với Eureka
- Áp dụng các resilience patterns: Circuit Breaker, Retry, Timeout, Bulkhead
- Phân tích bài toán dispatch SQL và tích hợp ngoài trong KBLab

4.2. Giao tiếp đồng bộ — Khi nào nên, khi nào tránh?

4.2.1. Request-Response: mô hình quen thuộc

Giao tiếp đồng bộ (*synchronous communication*) là mô hình quen thuộc nhất với hầu hết developer: service A gửi request đến service B, **chờ** response, rồi xử lý tiếp. HTTP/REST là protocol phổ biến nhất cho mô hình này.

Giao tiếp đồng bộ — Core Service blocked
Client
API Gateway
Core Service
Judge Service
POST /submissions
Forward request
POST /judge/execute [sync call]

BLOCKED
(chờ Judge)

Result: score=85
200 OK + score
Response to client

⚠ Vấn đề: Core bị block toàn bộ thời gian Judge xử lý (có thể vài giây). Thread bị giữ, không phục vụ request khác.

Hình 4.1: Giao tiếp đồng bộ — Core Service blocked trong khi chờ Judge response

4.2.2. Ưu và nhược điểm

Bảng 4.1: Ưu và nhược điểm của giao tiếp đồng bộ

	Ưu điểm	Nhược điểm
Đơn giản	Model request-response quen thuộc, dễ debug	Caller bị block cho đến khi nhận response
Nhất quán	Biết ngay kết quả (thành công/thất bại)	Temporal coupling: cả hai service phải online cùng lúc
Tooling	HTTP tooling phong phú (Postman, curl, browser)	Cascading failures: service B down → A cũng down
Tracing	Request ID để trace qua chuỗi calls	Latency accumulation: A→B→C = tổng latency

Richardson phân tích rõ trong [2a, Ch.3]: giao tiếp đồng bộ tạo **temporal coupling** (cả hai service phải sẵn sàng cùng lúc) và **runtime dependency** (service gọi không thể hoạt động nếu service được gọi down). Đây là lý do microservices thường ưu tiên async cho các flow không cần response ngay lập tức.

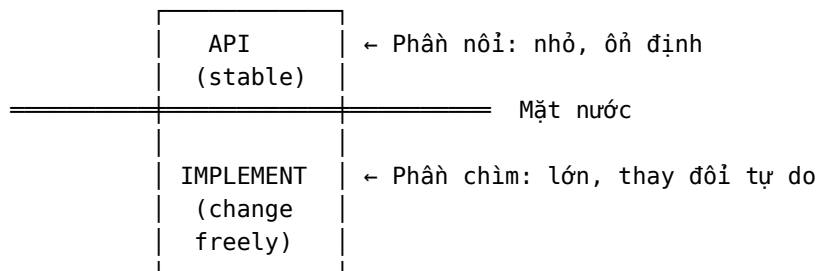
4.2.3. Design-time Coupling vs Runtime Coupling

Richardson trong phiên bản thứ hai [2b, Ch.4] phân biệt rõ hai loại coupling cơ bản — hiểu sai sự khác biệt này là nguyên nhân phổ biến nhất dẫn đến thiết kế microservices kém:

Bảng 4.1b: Hai loại coupling trong microservices

	Design-time Coupling	Runtime Coupling
Định nghĩa	Thay đổi service A → phải thay đổi service B	Service A cần B đang chạy để xử lý request
Khi nào xảy ra	Khi phát triển, compile, deploy	Khi hệ thống đang chạy (runtime)
Ảnh hưởng	Giảm team autonomy, tăng coordination	Giảm availability, tăng latency
Ví dụ KBLab	Core Service đổi DTO format → Auth Service phải cập nhật parser	Core Service gọi sync Judge → Judge down → Core trả lỗi
Giải pháp	API versioning (§3.2), backward compatible changes	Async messaging (Ch.5), Circuit Breaker (§4.4)

Iceberg Principle — Richardson gọi đây là nguyên tắc nền tảng cho loose coupling [2b, Ch.4]:



API (phần nổi) phải nhỏ nhất có thể — chỉ expose những gì consumer thực sự cần. Implementation (phần chìm) có thể lớn và thay đổi tự do mà không ảnh hưởng consumer. **Đây cũng là lý do dùng DTO pattern (§3.4)** thay vì trả entity trực tiếp — entity là “phần chìm” không nên lộ ra.

💡 Tip — DRY và Coupling: Nghịch lý trong Distributed Systems

Trong monolith, DRY (Don't Repeat Yourself) gần như luôn đúng. Nhưng trong microservices, DRY có thể tạo coupling không mong muốn. Ví dụ: `OrderTotalCalculator` dùng chung qua shared library → mỗi lần update business rule → tất cả services dùng library phải upgrade đồng loạt (lock-step deployment). Đôi khi **duplication có chủ đích** tốt hơn coupling — đặc biệt khi logic có thể diverge hợp lệ giữa các contexts. Richardson trong [2b, Ch.4] khuyên: chỉ DRY khi divergent implementations gây **bugs**, không phải mọi trường hợp.

4.2.4. Khi nào dùng sync?

Giao tiếp đồng bộ phù hợp khi:

- **Cần response ngay** — query data (GET), validation, authentication
- **Flow đơn giản** — chuỗi call ngắn ($A \rightarrow B$), không phải chuỗi dài ($A \rightarrow B \rightarrow C \rightarrow D$)
- **Read-heavy operations** — lấy thông tin, không phải thay đổi trạng thái phức tạp

Không phù hợp khi:

- **Long-running operations** — chấm bài SQL mất 5–30 giây
- **Fire-and-forget** — gửi notification, log event
- **Cross-service transactions** — cần saga pattern (Chương 6)

💡 Tip — Quy tắc ngón tay cái

Nếu caller *phải biết* kết quả để xử lý tiếp → sync. Nếu caller chỉ cần *trigger* hành động và không cần kết quả ngay → async (Chương 5). Richardson trong [2a, Ch.3] minh họa rõ: gọi sync để *validate* (cần kết quả ngay), nhưng gửi async để *process* (không cần chờ). Trong KBLab, query Judge health dùng sync, nhưng submit bài chấm dùng async (Kafka) — hai nhu cầu khác nhau.

4.2.5. Interaction Styles — Phân loại kiểu tương tác

Richardson phân loại interaction styles theo hai chiều [2a, Ch.3]:

Bảng 4.2: Interaction styles trong microservices theo Richardson [2a, Ch.3]

	One-to-one	One-to-many
Synchronous	Request/Response	—
Asynchronous	Async request/response, One-way notification	Publish/Subscribe, Publish/ Async responses

Hầu hết service dùng kết hợp nhiều kiểu. Ví dụ: KBLab Core Service dùng *request/response* (Feign) để query Judge, *publish/subscribe* (Kafka) để gửi submissions, và *one-way notification* (WebSocket) để push kết quả.

4.2.6. REST vs gRPC — Hai lựa chọn cho sync

REST (HTTP/JSON) là lựa chọn phổ biến nhất, nhưng không phải duy nhất. **gRPC** — framework RPC mã nguồn mở của Google — là lựa chọn phổ biến thứ hai cho giao tiếp đồng bộ giữa microservices [2a, Ch.3].

Bảng 4.3: REST vs gRPC — so sánh chi tiết

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Format	JSON (text, human-readable)	Protocol Buffers (binary, compact)
Performance	Chậm hơn (text parsing)	Nhanh hơn 2-10x (binary + HTTP/2 multiplexing)
Contract	OpenAPI spec (optional)	.proto file (bắt buộc)
Code gen	Optional (OpenAPI generators)	Built-in (protoc generates client/server)
Browser support	Native	Cần gRPC-Web proxy
Streaming	Không native (phải dùng SSE/WebSocket)	Bi-directional streaming native
Debugging	Dễ (curl, Postman, browser)	Khó (cần gRPC tools)

gRPC Architecture — Tại sao nhanh hơn? gRPC nhanh hơn REST không chỉ nhờ binary encoding. Ba yếu tố kiến trúc quan trọng:

1. **HTTP/2 multiplexing** — Nhiều requests cùng dùng một TCP connection (không cần mở connection mới mỗi request). Quan trọng khi service A gọi service B hàng trăm lần/giây.
2. **Protocol Buffers** — Schema-first, binary encoding. Payload nhỏ hơn JSON 3-5x. Schema evolution built-in (backward/forward compatible) — .proto file là contract.
3. **Four communication patterns:** Unary (request-response, như REST), Server streaming (server gửi stream), Client streaming (client gửi stream), **Bi-directional streaming** (cả hai gửi stream đồng thời). Bi-directional streaming rất phù hợp cho real-time use cases — ví dụ: Judge Service stream kết quả từng test case cho Frontend trong khi sinh viên vẫn đang xem.

Khi nào dùng gRPC thay REST?

- **Internal service-to-service:** performance quan trọng, không cần browser → gRPC
- **Public API / Frontend:** cần browser support, human debugging → REST
- **Streaming:** cần real-time bi-directional → gRPC (hoặc WebSocket)
- **Polyglot:** team dùng nhiều ngôn ngữ → gRPC (code gen đa ngôn ngữ)

❗ Phân tích — KBLab không còn là hệ thống REST thuần

LMS core của KBLab vẫn ưu tiên REST/JSON vì browser, dashboard và OpenAPI tooling cần tính dễ debug. Nhưng Network Lab chủ động dùng SOAP/REST/gRPC/JNP để sinh viên thấy khác biệt protocol trong cùng một domain bài tập. Vì vậy, migration sang gRPC không phải mục tiêu “đổi toàn bộ REST”; cách đúng hơn là **chọn protocol theo boundary**: REST cho client-facing API, gRPC cho internal high-throughput/streaming nếu có nhu cầu, và batch/job queue cho các tác vụ judge dài hoặc stateful.

4.2.7. Resilience Metrics — Đo lường độ tin cậy

Trước khi áp dụng resilience patterns (§4.4), cần hiểu cách **đo lường** resilience. Bốn metrics cốt lõi:

Bảng 4.4: Resilience metrics cốt lõi

Metric	Định nghĩa	Ví dụ KBLab
MTBF (Mean Time Between Failures)	Thời gian trung bình giữa hai lần sự cố	Judge Service crash trung bình 1 lần/tuần → MTBF = 168h
MTTR (Mean Time To Recovery)	Thời gian trung bình để khôi phục	Trung bình 15 phút để phát hiện + restart → MTTR = 15min
MTTF (Mean Time To Failure)	Thời gian trung bình từ recovery đến failure tiếp theo	MTTF = MTBF - MTTR
Availability	% thời gian system hoạt động	$= \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$

Availability “nines”:

Bảng 4.5: Availability “nines” — downtime cho phép theo target

Target	Downtime/năm	Downtime/tháng	Phù hợp cho
99% (two nines)	3.65 ngày	7.3 giờ	Internal tools, dev environments
99.9% (three nines)	8.76 giờ	43.8 phút	KBLab production (phù hợp)
99.99% (four nines)	52.6 phút	4.38 phút	E-commerce, banking
99.999% (five nines)	5.26 phút	26.3 giây	Critical infrastructure

❗ Nguyên tắc — Error Budgets

Google SRE (Site Reliability Engineering) đề xuất: nếu target availability = 99.9%, hệ thống có **error budget** = **0.1%** downtime/tháng (43.8 phút). Dùng error budget cho: deployments, experimentation, controlled maintenance. Khi hết budget → freeze deployments, focus stability. Đây là cách biến “availability target” từ khái niệm mơ hồ thành **nguyên tắc vận hành cụ thể** (xem thêm SLI/SLO/SLA ở Ch.11).

4.3. OpenFeign — Declarative REST Client

4.3.1. Vấn đề: gọi service khác không đơn giản như gọi hàm

Trong monolith, gọi module khác là một dòng code: `judgeService.execute(request)`. Trong microservices, cùng logic đó trở thành: xây dựng HTTP request, serialize payload, gắn headers (JWT token), gửi qua network, xử lý response codes, deserialize result, handle timeout/error.

4.3.2. OpenFeign: gọi REST như gọi hàm

OpenFeign (Spring Cloud OpenFeign) cho phép khai báo API call như một Java interface — framework tự động generate implementation:

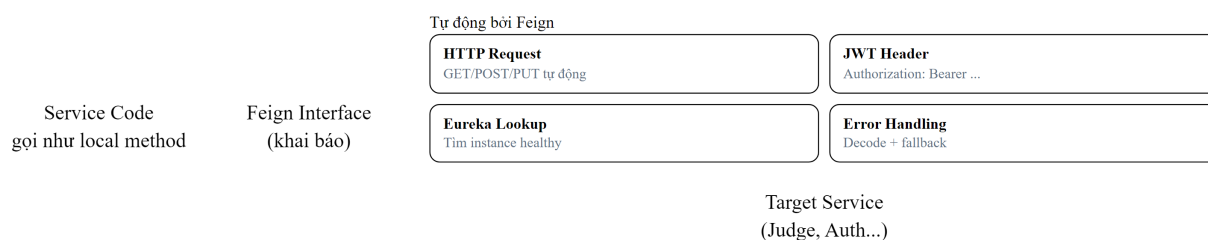
Listing 4.1: So sánh `RestTemplate` (verbose) và `OpenFeign` (declarative)

```
// ❌ Manual REST call – verbose, boilerplate
RestTemplate restTemplate = new RestTemplate();
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(jwtToken);
HttpEntity<JudgeRequest> entity = new HttpEntity<>(request, headers);
ResponseEntity<JudgeResult> response = restTemplate.exchange(url, HttpMethod.POST,
entity, JudgeResult.class);

// ✅ Declarative Feign client – clean, type-safe
@FeignClient(name = "judge", url = "${feign.judge.url}")
public interface JudgeClient {
    @PostMapping("/api/judge/sql")
    JudgeResult submitSql(@RequestBody JudgeRequest request); // Gọi như hàm bình
thường!
}
```

4.3.3. Cách hoạt động

Feign Proxy — Tự động hóa HTTP request



```
@FeignClient(name = "judge-service")
public interface JudgeClient {
    @PostMapping("/api/judge/execute")
    JudgeResult execute(@RequestBody SubmitRequest req);
}
```

Hình 4.2: Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing

Feign tự động xử lý: serialization/deserialization (JSON ↔ Java), header injection (JWT token qua `RequestInterceptor`), service discovery (Eureka lookup thay vì URL hardcoded), và error decoding (response code → exception).

4.4. Service Discovery & Load Balancing

4.4.1. Vấn đề: URL cứng trong distributed system

Trong monolith, gọi module khác là gọi hàm — không cần biết “module ở đâu”. Trong microservices, mỗi service là một process riêng, chạy trên host:port khác nhau. Câu hỏi: **service A tìm service B ở đâu?**

Cách đơn giản nhất: hardcode URL trong config. Nhưng khi service scale (3 instances của Judge Service), URL nào? Khi service di chuyển (deploy lên container mới), IP thay đổi — ai cập nhật?

4.4.2. Service Discovery patterns

Có hai pattern chính [2a, Ch.3]:

Client-side discovery — Client (service gọi) tự query service registry để tìm danh sách instances, rồi tự load balance:

Client-side Discovery — Service tự query Eureka
Core Service
Eureka Registry
Judge #1
:8081
Judge #2
:8082
1. Query: judge-service instances?
2. [judge:8081, judge:8082]
3. Client-side LB
Round-Robin chọn instance
4. POST /judge/execute
5. Result (score=85)

Client-side LB: Core Service chọn instance (Round-Robin). Spring Cloud LoadBalancer / Ribbon thực hiện điều này. Ưu điểm: Không có hop trung gian. Nhược điểm: Client cần tích hợp thư viện LB.

Hình 4.3: Client-side discovery — service tự query Eureka và load balance

Server-side discovery — Load balancer (hoặc API Gateway) đứng giữa, client chỉ cần biết URL của load balancer:

Server-side Discovery — Load Balancer điều hướng
Core Service
Load Balancer
AWS ALB / Nginx / k8s
Judge #1
Judge #2
1. POST /judge/execute
2. Chọn instance
query Service Registry
3. Forward to healthy instance
4. Result
5. Forward response

LB là trung gian: AWS ALB, Nginx, Kubernetes Service. Client không cần biết instance nào. Trade-off: Thêm 1 hop mạng, nhưng client đơn giản hơn — không cần tích hợp thư viện LB.

Hình 4.4: Server-side discovery — load balancer điều hướng request

4.4.3. Netflix Eureka trong KBLab

KBLab sử dụng **Netflix Eureka** cho cụm Java/Spring services — client-side discovery pattern:

Bảng 4.6: Thành phần Eureka trong KBLab

Thành phần	Vai trò	Port
eureka-registry	Service registry server	9000
Mỗi service	Eureka client (tự đăng ký)	—
Spring Cloud LoadBalancer	Client-side load balancing	—

Khi microservice khởi động, nó đăng ký tại Eureka server với tên (`spring.application.name`) và địa chỉ. Gateway và các service khác query Eureka để tìm instances. Trong config Gateway: `uri: lb://core-service` nghĩa là tra cứu Eureka tìm tất cả instances có tên `core-service`, rồi load balance giữa chúng.

❗ Phân tích — Service naming không nhất quán

Trong snapshot cũ của KBLab, `core-service` và `assignment-service` từng dùng `spring.application.name = "app"` — vi phạm nguyên tắc unique service identity [2a]. Khi Eureka nhận hai services cùng tên, nó coi chúng là instances của *cùng một service* → routing sai. Đây là hậu quả của việc Assignment tách ra từ Core nhưng không đổi service name. **Migration path:** đổi thành `kblab-core` và `kblab-assignment`, cập nhật Eureka config và Gateway routes.

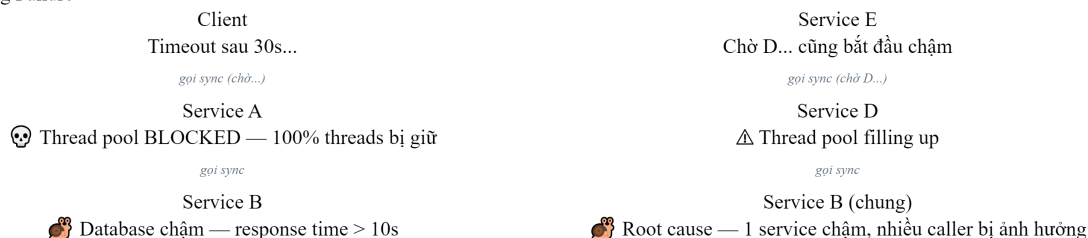
4.5. Resilience Patterns — Sống sót trong Distributed System

4.5.1. Tại sao cần resilience?

Trong monolith, nếu database chậm, *toàn bộ* ứng dụng chậm — nhưng ít nhất lỗi rõ ràng. Trong microservices, khi service B chậm, service A *vẫn chạy* nhưng threads bị block chờ B → requests đến A cũng bị chậm → service C gọi A cũng bị chậm → **cascading failure** lan khắp hệ thống [5, Ch.7].

Hugo Rocha trong [5] nhấn mạnh: trong distributed system, *lỗi không phải ngoại lệ* — *lỗi là trạng thái bình thường*. Network sẽ timeout, services sẽ crash, databases sẽ chậm. Câu hỏi không phải “nếu lỗi xảy ra” mà là “khi lỗi xảy ra”.

Cascading Failure



B chậm → A bị block → Client timeout
B chậm → D bị ảnh hưởng → E cũng bị ảnh hưởng

Giải pháp: **Circuit Breaker** (fig 4.6) — fail fast thay vì chờ vô hạn · **Bulkhead** (fig 4.7) — cách ly failure, không lan truyền · **Timeout** — giới hạn thời gian chờ

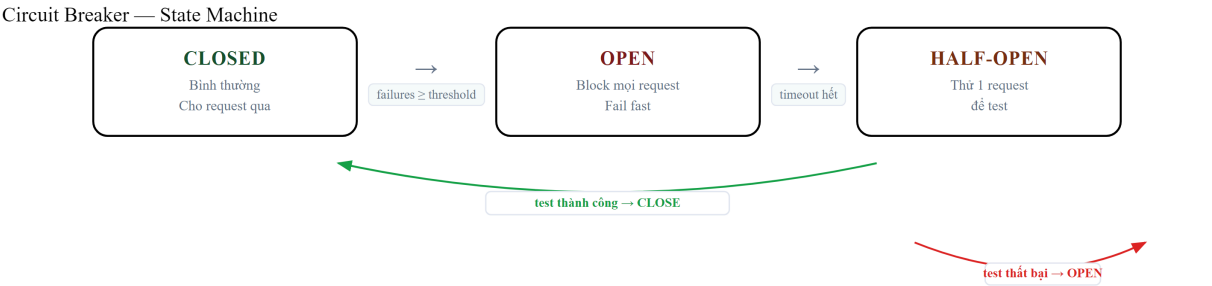
Hình 4.5: Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng

4.5.2. Bốn resilience patterns cốt lõi

Bảng 4.7: Bốn resilience patterns cốt lõi

Pattern	Nguyên lý	Tương tự	Ví dụ KBLab
Timeout	Đặt giới hạn thời gian chờ. Không bao giờ chờ vô hạn	Hẹn giờ nấu ăn — không chờ vô hạn	Feign call timeout 3s cho Judge
Retry	Tự động thử lại khi gặp lỗi tạm thời	Gọi điện lại khi tín hiệu kém	Retry 3 lần với exponential backoff
Circuit Breaker	Ngắt mạch khi service downstream liên tục lỗi	Cầu dao điện — ngắt khi quá tải	Judge down → trả fallback message
Bulkhead	Cách ly resources theo service/function	Khoang tàu thủy — 1 khoang ngập, tàu không chìm	Thread pool riêng cho Judge vs Question

4.5.2.1. Circuit Breaker — Chi tiết state machine



Cấu hình Circuit Breaker (Resilience4j)

```
failureRateThreshold = 50% | slidingWindowSize = 10 | waitDurationInOpenState = 30s
```

LMS: @CircuitBreaker annotation trên JudgeClient.execute()

Hình 4.6: Circuit Breaker state machine — Closed, Open, Half-Open

Khi circuit ở trạng thái **Open**, mọi request được reject *ngay lập tức* — không gửi tới service downstream. Điều này bảo vệ cả caller (không block threads) và callee (không bị overwhelm khi đang recovery).

4.5.2.2. Bulkhead — Cách ly resources

Bulkhead Pattern — Thread Pool cách ly

Core Service

→ Judge requests

→ Question requests

→ Contest requests



Judge pool đầy → chỉ Judge bị ảnh hưởng. Question và Contest vẫn hoạt động bình thường = Cách ly failure, không lan truyền.

❌ Không có Bulkhead

1 pool chung 45 threads. Judge chậm → tất cả block.

✅ Có Bulkhead

Pool riêng. Judge chậm → chỉ Judge bị ảnh hưởng, Question/Contest vẫn OK.

```
@Bulkhead(name = "judge", type = THREADPOOL, maxConcurrentCalls = 10)
```

Hình 4.7: Bulkhead pattern — thread pool cách ly theo function

Tên “Bulkhead” lấy từ thiết kế tàu thủy: khoang tàu bị nước tràn vào, các khoang khác vẫn khô — tàu không chìm. Michael Nygard giới thiệu pattern này trong *Release It!* (2007) [5, Ch.7].

4.5.2.3. Lưu ý quan trọng cho Retry

Chỉ retry idempotent operations (GET, PUT, DELETE). Retry POST có thể tạo duplicate data. Richardson trong [2a, Ch.3] nhấn mạnh: retry token (idempotency key) là bắt buộc khi retry non-idempotent operations.

4.5.2.4. Kết hợp các patterns

Trong thực tế, các patterns thường kết hợp theo thứ tự:

Request → Timeout (3s) → Retry (3 lần, backoff) → Circuit Breaker → Bulkhead → Actual Call

Spring Cloud Circuit Breaker + Resilience4j là stack phổ biến nhất cho Java microservices. Chỉ cần thêm dependency và annotation — không cần viết logic retry/circuit breaker thủ công.

4.5.2.5. Resilience4j Implementation — Code ví dụ cho LMS

Listing 4.2: Resilience4j annotations cho Judge Feign Client

```
// Core Service – gọi Judge với đầy đủ resilience patterns
@Service
public class JudgeCallService {
    private final MySQLClient mysqlClient;

    @CircuitBreaker(name = "judge", fallbackMethod = "judgeFallback")
    @Retry(name = "judge")
    @TimeLimiter(name = "judge")
    @Bulkhead(name = "judge")
    public CompletableFuture<JudgeResult> executeSQL(JudgeRequest request) {
        return CompletableFuture.supplyAsync(() ->
            mysqlClient.execute(request)
        );
    }

    // Fallback khi circuit OPEN hoặc tất cả retries thất bại
    private CompletableFuture<JudgeResult> judgeFallback(
        JudgeRequest request, Throwable t) {
```

```

        log.warn("Judge unavailable: {}, submissionId={}",
            t.getMessage(), request.getSubmissionId());
        return CompletableFuture.completedFuture(
            JudgeResult.pending("Sandbox đang bận, bài sẽ được chấm khi sẵn sàng")
        );
    }
}

```

Listing 4.3: Resilience4j YAML configuration cho LMS

```

# application.yml – Resilience4j configuration
resilience4j:
  circuitbreaker:
    instances:
      judge:
        slidingWindowSize: 10           # Đánh giá trên 10 requests gần nhất
        failureRateThreshold: 50        # Mở circuit khi 50% requests lỗi
        waitDurationInOpenState: 30s    # Chờ 30s trước khi thử lại (half-open)
        permittedNumberOfCallsInHalfOpenState: 3 # Thử 3 calls khi half-open
        recordExceptions:               # Chỉ count những exceptions này
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - feign.FeignException.ServiceUnavailable

  retry:
    instances:
      judge:
        maxAttempts: 3                 # Tối đa 3 lần thử
        waitDuration: 1s               # Chờ 1s giữa các lần
        exponentialBackoffMultiplier: 2 # 1s → 2s → 4s (exponential)
        retryExceptions:
          - java.io.IOException        # Chỉ retry lỗi tạm thời
        ignoreExceptions:
          - com.lms.exception.BusinessException # KHÔNG retry lỗi logic

  timelimiter:
    instances:
      judge:
        timeoutDuration: 5s           # Timeout 5s cho mỗi call

  bulkhead:
    instances:
      judge:
        maxConcurrentCalls: 10        # Tối đa 10 calls đồng thời đến Judge
        maxWaitDuration: 500ms        # Chờ tối đa 500ms nếu bulkhead đầy

```

Bảng 4.8: Giá trị recommend cho từng environment

Parameter	Development	Production (LMS)	High-traffic
slidingWindowSize	5	10	20
failureRateThreshold	50%	50%	30%
waitDurationInOpenState	10s	30s	60s
retry.maxAttempts	1	3	3
timelimiter.timeoutDuration	10s	5s	3s
bulkhead.maxConcurrentCalls	5	10	25

💡 Tip — Thứ tự áp dụng annotations

Resilience4j xử lý annotations theo thứ tự **ngoài vào trong**: Bulkhead → TimeLimiter → CircuitBreaker → Retry → Actual Call. Nghĩa là: request phải qua bulkhead trước (kiểm tra concurrent limit), rồi timeout, rồi circuit breaker check, rồi retry nếu lỗi. Thứ tự này quan trọng — nếu Retry bao ngoài CircuitBreaker, retry sẽ thử lại ngay cả khi circuit đã OPEN (vô nghĩa). Cấu hình mặc định của Resilience4j đã xử lý đúng thứ tự [5, Ch.7].

🚨 Nguyên tắc — Design for Failure

“In distributed systems, failure is not an exception — it’s a normal state. Design every inter-service call assuming it will fail. Timeout, Retry, Circuit Breaker, Bulkhead — these are not optional, they are prerequisites.”

— Tổng hợp từ Hugo Rocha [5, Ch.7] và Michael Nygard, *Release It!*

4.6. Case Study: KBLab SQL Judge — Coordinator và DBMS Workers

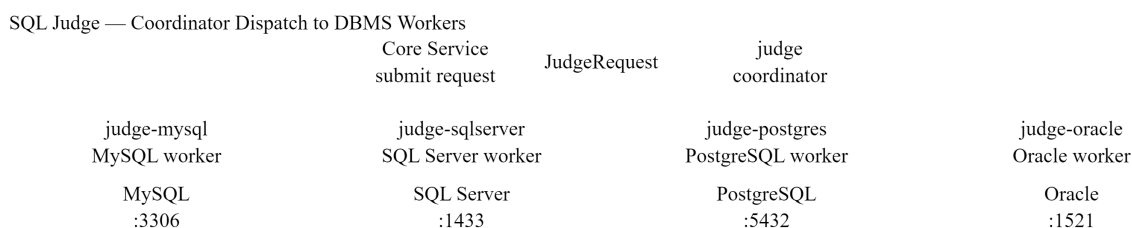
4.6.1. Bài toán

KBLab hỗ trợ sinh viên thực hành SQL trên nhiều DBMS. Mỗi DBMS chạy trong sandbox container riêng biệt. Khi sinh viên nộp bài, hệ thống cần:

1. Xác định DBMS nào (dựa vào câu hỏi)
2. Gửi SQL đến đúng sandbox service
3. So sánh kết quả với đáp án (SHA-256 hash)
4. Trả về kết quả

4.6.2. Hiện trạng: Strategy Pattern + OpenFeign

KBLab implement bài toán này bằng một service điều phối **judge** kết hợp các worker **judge-*** theo từng DBMS:



Ownership: Core gọi judge; judge sở hữu routing DBMS và worker contract.

Scale: Thêm DBMS mới = thêm một worker judge-* + registry config.

Resilience: Circuit Breaker riêng theo worker. MySQL down không ảnh hưởng SQL Server.

Migration: Phase 2: Async queue cho job dài — không block Core khi worker chậm.

Hình 4.8: SQL Judge coordinator — dispatch SQL đến các worker judge-* theo DBMS

Core Service không nên biết chi tiết MySQL hay SQL Server được xử lý ở worker nào. Nó gửi yêu cầu chấm đến **judge**; **judge** đóng vai trò coordinator: nhận database type, chọn worker tương ứng (**judge-mysql**, **judge-sqlserver**, ...), gọi worker qua HTTP/Feign hoặc queue, rồi chuẩn hóa kết quả trả về Core. Đây là pattern phổ biến: một coordinator giữ contract nghiệp vụ ổn định, các processor phía sau chuyên môn hóa theo runtime/DBMS.

4.6.3. Phân tích các vấn đề

Bảng 4.9: Phân tích vấn đề SQL Judge coordinator/worker

#	Vấn đề	Mô tả	Best Practice
1	Routing ownership chưa rõ	Nếu Core cũng chứa logic chọn DBMS, routing bị rò rỉ ra ngoài Judge boundary	Single ownership: judge là coordinator duy nhất
2	Magic IDs / hardcoded routing	Database type xác định bằng hardcoded UUID/string	Enum hoặc configuration-driven registry
3	Resilience theo worker chưa rõ	judge-mysql down không nên kéo hỏng toàn bộ judge hoặc worker khác	Circuit Breaker/Timeout riêng cho từng worker
4	Fallback chưa nhất quán	Worker DBMS lỗi → user nhận lỗi kỹ thuật hoặc submission stuck	Fallback message, retry/queue, status rõ ràng
5	Worker contract chưa chuẩn hóa	Mỗi judge-* dễ trả format/result khác nhau	Chuẩn hóa JudgeRequest/JudgeResult contract

4.6.4. Đề xuất migration



Khuyến nghị: Bắt đầu từ Phase 1 — ROI cao nhất, không cần thay đổi kiến trúc.
Phase 3: Chỉ thực hiện khi submission volume tăng và blocking trở thành bottleneck thực sự.

Hình 4.9: Lộ trình migration cho SQL Judge — từ resilience đến async

- **Phase 1 — Làm rõ ownership** (ưu tiên cao, effort thấp): Core chỉ gọi judge; mọi routing DBMS nằm trong judge coordinator. Không để routing DBMS rò sang Core hoặc các service không sở hữu nghiệp vụ chấm SQL.
- **Phase 2 — Thêm resilience theo worker** (ưu tiên cao): @CircuitBreaker, @Retry, @TimeLimiter riêng cho từng judge-*. Khi judge-mysql down → trả fallback: “Sandbox MySQL đang bận, bài sẽ được chấm khi sẵn sàng”, không ảnh hưởng judge-sqlserver.
- **Phase 3 — Chuẩn hóa worker contract** (effort trung bình): Tất cả worker nhận cùng JudgeRequest, trả cùng JudgeResult, routing config-driven thay vì hardcoded IDs.
- **Phase 4 — Chuyển flow dài sang async** (effort cao, giá trị lớn): Submit → Kafka/queue → judge coordinator → judge-* worker → Result event → Core cập nhật. User không

chờ đồng bộ — nhận notification khi có kết quả. Đây chính là pattern mà KBLab đã bắt đầu implement (Chương 5).

4.6.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice

Không phải mọi tích hợp trong KBLab đều nên đi qua cùng một protocol. Với QLĐT hoặc GitHub Classroom, KBLab nên đặt **Anti-Corruption Layer** ở boundary: adapter dịch contract bên ngoài thành command/event nội bộ, che giấu format, lỗi và rate limit của hệ thống đối tác. Với Network Practice, điểm đáng học là ngược lại: context này được thiết kế độc lập để đánh giá giao tiếp mạng, không phụ thuộc vào service chấm SQL. Vì vậy, nó là ví dụ tốt cho bounded context theo protocol, còn vấn đề coordinator/worker cần phân tích nằm ở SQL Judge.

📌 Nguyên tắc — Sync → Async là hành trình phổ biến

Nhiều hệ thống bắt đầu với sync (đơn giản, dễ debug) rồi chuyển sang async khi cần scale. Richardson trong [2a] minh họa hành trình này: bắt đầu với REST calls giữa services, sau đó chuyển sang Saga khi cần distributed transaction. KBLab đang ở giữa hành trình: một số flow đã async (Kafka cho submission), một số vẫn sync (Feign cho query), và SQL Judge cần tách rõ coordinator/worker trước khi scale ngang an toàn. Chương 5 sẽ tiếp tục phần async.

⚠️ Lưu ý —

1. **Không đặt timeout cho inter-service calls** — Mặc định nhiều HTTP client chờ vô hạn (hoặc 30-60 giây). Hậu quả: khi service downstream chậm, threads caller bị block → cascading failure lan khắp hệ thống. *Phòng tránh*: luôn cấu hình timeout rõ ràng (Netflix chuẩn: 1-3 giây cho internal calls).
2. **Tin tưởng rằng network luôn reliable** — Viết code như thể mọi HTTP call đều thành công. Hậu quả: lỗi đầu tiên ở production mới phát hiện không có retry, không fallback. *Phòng tránh*: áp dụng resilience patterns từ đầu (Circuit Breaker + Retry + Timeout) — ngay cả khi chưa cần scale (§4.4).
3. **Retry mà không kiểm tra idempotency** — Retry POST requests tạo duplicate data (hai submissions cho cùng một lần nộp). Hậu quả: dữ liệu sai, user bị tính điểm trùng. *Phòng tránh*: chỉ retry idempotent operations (GET, PUT, DELETE), hoặc dùng idempotency key cho POST.

4.7. Tổng kết

Giao tiếp đồng bộ là mô hình đơn giản nhất nhưng tiềm ẩn rủi ro lớn nhất trong distributed system. Temporal coupling, cascading failures, và latency accumulation là ba vấn đề cốt lõi mà developer phải đối mặt.

OpenFeign đơn giản hóa service-to-service calls thành Java interface declarations, loại bỏ boilerplate HTTP code. Service Discovery (Eureka) giải quyết bài toán “tìm service ở đâu” — một vấn đề không tồn tại trong monolith nhưng quan trọng sống còn trong microservices.

Resilience patterns — Timeout, Retry, Circuit Breaker, Bulkhead — không phải optional. Chúng là điều kiện tiên quyết để hệ thống microservices hoạt động ổn định trong production.

Phân tích KBLab cho thấy thiếu resilience patterns là gap nghiêm trọng nhất trong giao tiếp đồng bộ.

Ở Chương 5, chúng ta sẽ khám phá **giao tiếp bất đồng bộ** — giải pháp cho những giới hạn của sync: Apache Kafka, event-driven architecture, và cách KBLab sử dụng messaging cho submission pipeline.

4.8. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, Service Discovery
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.7: Resilience & Reliability

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Coupling Taxonomy (design-time vs runtime), Iceberg Principle, DRY in distributed systems; Ch.9: IPC patterns
5. Michael Nygard, *Release It!*, 2nd Ed. (2018) — Circuit Breaker, Bulkhead, Timeout patterns

Nguồn trực tuyến:

- [W3] Netflix Technology Blog — “Making the Netflix API More Resilient” (2011)
- Resilience4j documentation — resilience4j.readme.io
- Spring Cloud OpenFeign reference — docs.spring.io

5. Giao tiếp Bất đồng bộ — Kafka, Events & Messaging

“Event streams become the heart of data sharing throughout the company. Data no longer sits solely on a database accessible only through synchronous interfaces.” — Hugo Rocha, Practical Event-Driven Microservices Architecture [5]

5.1. Bạn sẽ học được gì

- Hiểu tại sao async messaging giải quyết các giới hạn của giao tiếp đồng bộ
- Phân biệt hai loại message broker: durable (Kafka) vs ephemeral (RabbitMQ)
- Nắm vững kiến trúc Apache Kafka: topics, partitions, consumer groups
- Hiểu kiến trúc RabbitMQ: exchanges, queues, bindings
- Thiết kế event schema: 4 loại message (Command, Event, Document, Query)
- So sánh delivery guarantees: at-most-once, at-least-once, exactly-once
- Sử dụng WebSocket cho real-time notifications
- Phân tích Kafka pipeline và DB queue trong KBLab

5.2. Tại sao cần Async? — Giới hạn của giao tiếp đồng bộ

5.2.1. Vấn đề mà sync không giải quyết được

Chương 4 đã phân tích ba vấn đề cốt lõi của giao tiếp đồng bộ: temporal coupling, cascading failures, và latency accumulation. Giao tiếp bất đồng bộ (*asynchronous messaging*) giải quyết cả ba bằng cách **tách rời** (*decouple*) producer và consumer qua một trung gian — message broker.

So sánh Giao tiếp Đồng bộ vs Bất đồng bộ
Đồng bộ (Sync) — Tight Coupling



- ➔ A bị **blocked** khi chờ B phản hồi
- ➔ B down → A down (cascading failure)
- ➔ Throughput giới hạn bởi B

Bất đồng bộ (Async) — Loose Coupling



- ✅ A gửi message và **tiếp tục** ngay lập tức
- ✅ B xử lý khi sẵn sàng — không phụ thuộc A
- ✅ Broker buffer → throughput cao hơn

Bảng so sánh

TIÊU CHÍ	SYNC (REST / GRPC)	ASYNC (KAFKA / RABBITMQ)
Latency	Thấp — phản hồi trực tiếp	Cao hơn — qua broker
Coupling	Cao — A phụ thuộc B trực tiếp	Thấp — A không biết B
Failure Mode	Cascading — lỗi lan rộng	Isolated — lỗi cô lập tại B
Throughput	Giới hạn bởi tốc độ của B	Buffer bởi broker — scale độc lập
Complexity	Đơn giản — request/response rõ ràng	Phức tạp hơn (idempotency, ordering)

Hình 5.1: So sánh giao tiếp đồng bộ (coupled) và bất đồng bộ (decoupled)

Bảng 5.1: Giao tiếp đồng bộ vs bất đồng bộ — giải pháp cho ba vấn đề

Vấn đề Sync	Giải pháp Async
Temporal coupling — cả hai service phải online	Producer gửi message xong tiếp tục, consumer xử lý khi sẵn sàng
Cascading failure — B down → A down	A vẫn gửi được vào broker, B xử lý khi recovery
Latency — A chờ B xử lý xong	A không chờ, user nhận kết quả qua notification/polling

Richardson trong [2a, Ch.3] phân tích: async messaging cải thiện **availability** vì services không phụ thuộc lẫn nhau tại runtime. Tuy nhiên, đổi lại bằng **complexity** — eventual consistency, message ordering, duplicate handling — đây là chi phí không thể tránh.

5.2.2. Messaging patterns

Có ba messaging patterns cơ bản [2a, Ch.3]:

1. Point-to-point (Queue) — Một message, một consumer. Phù hợp cho command: “hãy chấm bài này”.

2. Publish/Subscribe (Topic) — Một message, nhiều consumers. Phù hợp cho event: “bài đã được chấm xong”.

3. Request/Async Response — Gửi request qua messaging, nhận response qua message khác (có correlation ID). Kết hợp lợi ích của cả hai.

Hai mô hình: Publish/Subscribe vs Point-to-Point
Publish / Subscribe

Producer
Core Service
Topic
submission-topic
Consumer 1
Judge
Consumer 2
Leaderboard
Consumer 3
Notification

Mỗi consumer nhận **TẤT CẢ** messages từ topic.
Kafka: mỗi consumer group đọc offset riêng độc lập.
Point-to-Point

Producer
Core Service
Queue
dead-letter-queue
Consumer
DLQ Handler

Mỗi message chỉ **MỘT** consumer xử lý.
RabbitMQ: default behavior — round-robin delivery.

LMS sử dụng cả hai mô hình

Pub/Sub: submission-topic → Judge Consumer + Leaderboard Consumer + Notification Consumer
Point-to-Point: dead-letter-queue → DLQ Consumer (xử lý message lỗi, retry có kiểm soát)
Kafka note: Consumer Group = point-to-point trong cùng group. Khác group = pub/sub riêng biệt.

Hình 5.2: Point-to-Point (Queue) vs Publish/Subscribe (Topic)

💡 Tip — Command vs Event

Phân biệt rõ **command** (yêu cầu hành động: “ChấmBài”, “GửiEmail”) và **event** (thông báo sự kiện đã xảy ra: “BàiĐãĐượcChấm”, “ĐiểmĐãCậpNhật”). Commands thường point-to-point, events thường pub/sub. Sự phân biệt này ảnh hưởng đến thiết kế schema và error handling [5, Ch.8].

5.3. Message Broker — Hai trường phái thiết kế

5.3.1. Durable vs Ephemeral Brokers

Rocha trong [5, §3.1.3] phân loại message brokers thành hai trường phái cơ bản dựa trên vòng đời của message:

Ephemeral (tạm thời) — Message bị xóa sau khi consumer acknowledge. Đại diện: RabbitMQ, ActiveMQ, Amazon SQS.

Durable (bền vững) — Message được lưu trữ trên disk và vẫn available sau khi consume. Đại diện: **Apache Kafka**, Apache Pulsar, Amazon Kinesis.

Durable Broker (Kafka) vs Ephemeral Broker (RabbitMQ)
Durable Broker — Kafka

Producer
Core Service
Event Log
append-only, persistent
✅ Retained — Replay bất kỳ lúc nào

Consumer A
Judge Svc
Consumer B
Leaderboard

Message **lưu lại** trong log.
Consumer mới có thể đọc lại từ offset đầu tiên.
Ephemeral Broker — RabbitMQ

Producer
Core Service
Queue
in-memory / durable
❌ Deleted sau khi ACK

Consumer
Notification Svc

Message **bị xóa** sau khi consumer ACK.
Không replay được — phù hợp task queue đơn giản.

LMS chọn Kafka vì

1. **Replay**: Khi Judge bị lỗi, có thể replay messages từ offset cũ mà không mất dữ liệu
2. **Multi-consumer**: Judge + Leaderboard + Notification cùng đọc một topic với offset độc lập
3. **Event log**: Audit trail đầy đủ cho mọi submission trong contest
4. **Throughput**: Kafka xử lý 100K+ msg/s — phù hợp với contest mode đồng thời

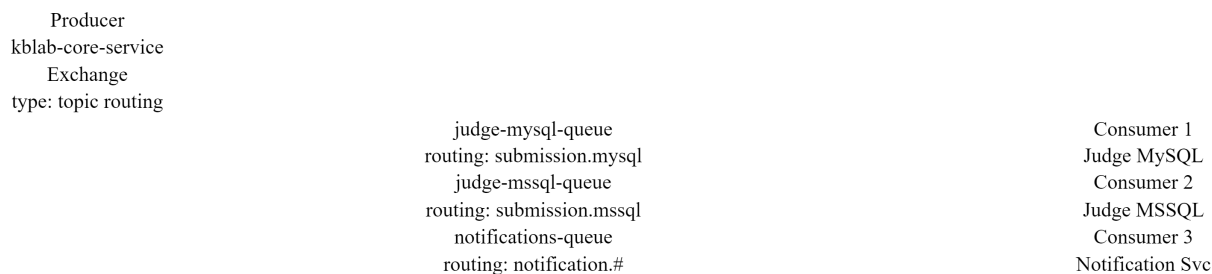
Hình 5.3: Ephemeral broker (message xóa sau ack) vs Durable broker (message vẫn còn)

Sự khác biệt này **không chỉ là kỹ thuật** — nó ảnh hưởng đến cách thiết kế hệ thống. Durable brokers cho phép replay, event sourcing, và multiple consumers đọc cùng data. Ephemeral brokers tập trung vào smart routing, priority queues, và point-to-point communication.

5.3.2. RabbitMQ — Smart Routing & Flexible Messaging

RabbitMQ dựa trên **AMQP** (Advanced Message Queuing Protocol), với kiến trúc phong phú hơn Kafka về routing:

RabbitMQ Architecture trong KBLab



Routing rules — Exchange type: topic

```

submission.mysql → queue judge-mysql-queue → Consumer 1 xử lý SQL trên MySQL sandbox
submission.mssql → queue judge-mssql-queue → Consumer 2 xử lý SQL trên MSSQL sandbox
notification.# → queue notifications-queue → Consumer 3 gửi kết quả cho student
  
```

Mỗi queue chỉ nhận message phù hợp routing key — Consumer xử lý hoàn toàn độc lập.

Hình 5.4: Kiến trúc RabbitMQ — Exchange routing messages đến queues

Bảng 5.2: Các concepts cốt lõi của RabbitMQ

Concept RabbitMQ	Mô tả
Exchange	Nhận message và route đến queues theo rules
Queue	Lưu message chờ consumer (FIFO)
Binding	Rule kết nối exchange → queue (routing key, headers)
Ack/Nack	Consumer xác nhận đã xử lý (ack) hoặc từ chối (nack → requeue/dead letter)
Exchange types	Direct, Fanout, Topic, Headers — mỗi loại routing logic khác nhau

RabbitMQ mạnh ở **smart routing**: một message có thể đến đúng queue dựa trên routing key, header matching, hoặc fanout tới tất cả queues. Kafka thì routing đơn giản (topic-based, key-based partitioning).

5.3.3. So sánh toàn diện

Bảng 5.3: Apache Kafka vs RabbitMQ — so sánh toàn diện

Tiêu chí	Apache Kafka	RabbitMQ
Loại	Durable — event log	Ephemeral — message queue
Model	Append-only log, consumers theo dõi offset	Queue FIFO, message xóa sau ack
Throughput	Rất cao (millions msg/sec)	Trung bình (tens of thousands/sec)
Ordering	Đảm bảo trong partition	Đảm bảo trong queue
Replay	✅ Replay từ offset bất kỳ	❌ Không thể replay
Routing	Đơn giản (topic + partition key)	Linh hoạt (4 loại exchange + routing keys)
Priority	❌ Không hỗ trợ	✅ Priority queues
Delayed messages	❌ Không native	✅ Delayed message exchange
Protocol	Custom binary protocol	AMQP, STOMP, MQTT
Use case	Event sourcing, stream processing, audit log, CDC	Task queues, delayed jobs, complex routing, RPC-over-messaging

ⓘ Nguyên tắc — Bài học thực tế — RabbitMQ trong production high-throughput

Rocha chia sẻ kinh nghiệm từ một e-commerce platform lớn [5, §3.1.3]: team đã dùng RabbitMQ nhiều năm trong production high-throughput. Vấn đề: khi message load peaks tích tụ, **toàn bộ cluster bị ảnh hưởng** — các service không liên quan cũng bị chậm. Nhìn lại, use case của họ (event streaming, high throughput) phù hợp với durable broker hơn. Bài học: **chọn broker phải dựa trên use case**, không phải quen thuộc.

5.3.4. Khi nào dùng gì?

Bảng 5.4: Khi nào chọn Kafka, khi nào chọn RabbitMQ

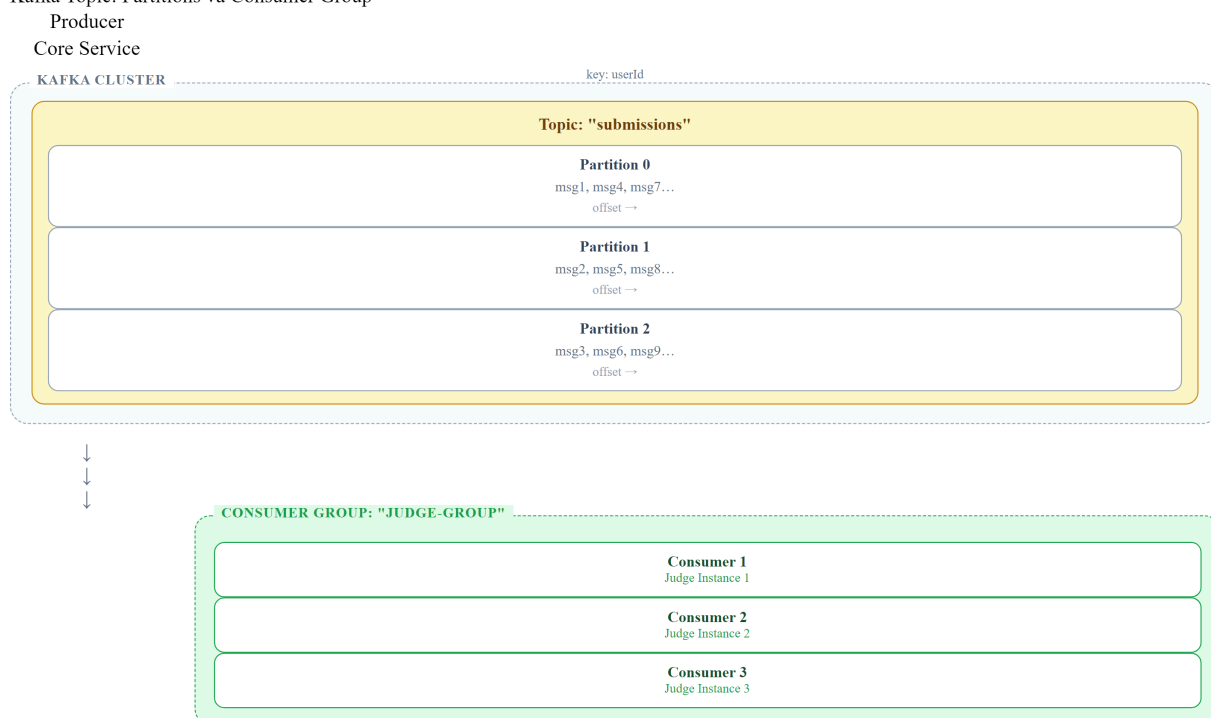
Nhu cầu	Chọn
Event sourcing, audit trail, replay	Kafka
High throughput (100K+ msg/sec)	Kafka
Stream processing (KStreams, ksqlDB)	Kafka
Task/job queues, background workers	RabbitMQ
Complex routing (headers, topic patterns)	RabbitMQ
Priority queues, delayed messages	RabbitMQ
Cả hai	Kafka cho event backbone + RabbitMQ cho task queues

KBLab chọn Kafka cho submission pipeline vì cần **replay** (chăm lại bài khi Judge đổi logic), ordering theo người dùng và khả năng mở rộng khi contest có nhiều submissions cùng lúc [5,

Ch.3]. Tuy nhiên, Kafka không phải câu trả lời mặc định cho mọi queue: DevOps Lab dùng DB queue cho job vận hành vì cần transactional enqueue đơn giản, dễ quan sát cùng dữ liệu domain và không cần replay event log toàn hệ thống.

5.4. Apache Kafka — Kiến trúc chi tiết

Kafka Topic: Partitions và Consumer Group



Mỗi partition được gán cho một consumer trong group — đảm bảo xử lý song song và ordering trong partition.

Hình 5.5: Kiến trúc Kafka — Topic, Partitions và Consumer Group

Bảng 5.5: Các concepts cốt lõi của Apache Kafka

Concept	Mô tả	Ví dụ KBLab
Topic	Luồng message theo category	submissions, judge-results, notifications
Partition	Chia topic thành segments song song	3 partitions cho submissions
Consumer Group	Nhóm consumers chia nhau partitions	judge-group — 3 Judge instances
Offset	Vị trí đọc của consumer trong partition	Consumer 1 đã đọc đến offset 42
Retention	Thời gian giữ message	7 ngày (default) hoặc vĩnh viễn

Partition key quyết định message vào partition nào. Messages cùng key → cùng partition → **đảm bảo thứ tự**. Trong KBLab, partition key = `userId` hoặc key theo bài/contest giúp đảm bảo ordering ở đúng phạm vi nghiệp vụ.

Kleppmann trong [7, Ch.11] giải thích: Kafka kết hợp ưu điểm của database (durable storage, replay) với ưu điểm của messaging (real-time, decoupling) — đây là mô hình **log-based messaging** khác biệt cơ bản với queue-based messaging.

5.5. Producer/Consumer Pattern trong KBLab

5.5.1. Kafka Producer

Trong KBLab, submissions được gửi qua Kafka khi sinh viên nộp bài:

Listing 5.1: Kafka Producer — Core Service gửi submission

```
// Producer: Core Service gửi submission vào Kafka
@Service
public class SubmitProducer extends BaseProducerService<SubmitMessage> {

    private static final String TOPIC = "submissions";

    public void sendSubmission(Submission submission) {
        SubmitMessage message = SubmitMessage.builder()
            .submissionId(submission.getId())
            .questionId(submission.getQuestionId())
            .userId(submission.getUserId())
            .sqlContent(submission.getSqlContent())
            .databaseType(submission.getDatabaseType())
            .build();

        // key = userId → đảm bảo ordering per user
        kafkaTemplate.send(TOPIC, submission.getUserId().toString(), message);
    }
}
```

5.5.2. Kafka Consumer

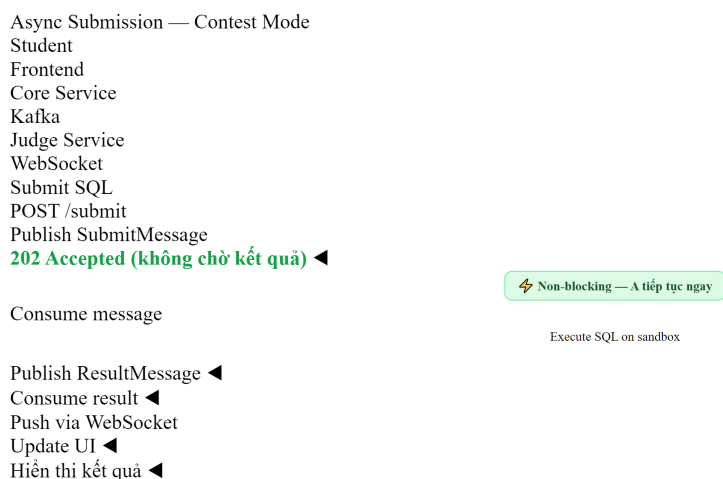
Judge Service consume submissions và trả kết quả:

Listing 5.2: Kafka Consumer — Judge Service nhận và xử lý submission

```
// Consumer: Judge Service nhận và xử lý
@KafkaListener(
    topics = "submissions",
    groupId = "judge-group",
    containerFactory = "kafkaListenerContainerFactory"
)
public void processSubmission(SubmitMessage message) {
    JudgeResult result = sqlExecutorService.execute(message);

    // Gửi kết quả ngược lại qua topic khác
    resultProducer.send("judge-results", message.getSubmissionId(), result);
}
```

5.5.3. Flow hoàn chỉnh



Hình 5.6: Luồng hoàn chỉnh — từ submit qua Kafka đến kết quả qua WebSocket

Lưu ý: response là **202 Accepted** (không phải 200 OK) — nghĩa là “request đã nhận, đang xử lý”. User nhận kết quả qua WebSocket notification, không phải HTTP response.

❗ Phân tích — Thiếu error handling trong Kafka pipeline

Một số pipeline KBLab vẫn cần hardening cho message failures: nếu Judge Service crash giữa chừng, message có thể bị mất hoặc bị xử lý lặp tùy cách commit offset. **Best practice** theo [2a, Ch.3]: dùng manual offset commit + dead letter topic cho messages thất bại. **Migration path**: (1) chuyển sang manual commit, (2) thêm `@RetryableTopic` với dead letter queue, (3) implement monitoring cho consumer lag.

5.6. Delivery Guarantees & Idempotency

5.6.1. Ba cấp độ đảm bảo

Đây là một trong những khái niệm quan trọng nhất trong messaging — và cũng dễ bị hiểu sai nhất [7, Ch.11]:

Bảng 5.6: Ba cấp độ delivery guarantee

Guarantee	Mô tả	Hậu quả
At-most-once	Message được gửi tối đa 1 lần. Có thể mất.	Nhanh nhưng unreliable
At-least-once	Message được gửi ít nhất 1 lần. Có thể trùng.	Reliable nhưng cần idempotency
Exactly-once	Message được xử lý đúng 1 lần.	Lý tưởng nhưng rất khó/đắt

Kleppmann trong [7, Ch.11] phân tích: **exactly-once semantics** trong thực tế là *effectively-once* — đạt được bằng cách kết hợp at-least-once delivery + idempotent processing. Kafka transactions (since 0.11) hỗ trợ exactly-once **trong Kafka** nhưng không across systems (Kafka → DB).

5.6.2. Idempotency — Thiết kế to chịu được duplicate

Khi dùng at-least-once (phổ biến nhất), consumer *có thể* nhận cùng message nhiều lần. Code phải **idempotent** — xử lý nhiều lần cho cùng kết quả [5, Ch.8]:

Listing 5.3: Idempotent consumer — kiểm tra trước khi xử lý

```
// ❌ Không idempotent – chấm trùng = điểm sai
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
    // Nếu message trùng → score bị cộng dồn hoặc ghi đè không đúng state
}

// ✅ Idempotent – check trước khi xử lý
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    if (submissionRepository.isAlreadyJudged(msg.getSubmissionId())) {
        log.info("Submission {} already judged, skipping", msg.getSubmissionId());
        return; // Skip duplicate
    }
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
}
```

Rocha trong [5, Ch.8] nhấn mạnh quy tắc: **event consumption phải associative, commutative, và idempotent** — xử lý nhiều lần, theo thứ tự khác nhau, vẫn cho cùng kết quả.

Trong KBLab, idempotency đặc biệt quan trọng ở SQL Judge: submission có thể được retry qua Kafka/queue, và judge coordinator có thể gọi lại cùng một judge-* worker sau timeout. Mỗi command nên có submissionId hoặc judgeRunId ổn định; consumer/worker kiểm tra trạng thái trước khi ghi kết quả. Nếu đã chấm xong, hệ thống trả về kết quả hiện có hoặc bỏ qua message, thay vì tạo thêm một lần chấm mới.

5.7. Event Schema Design

5.7.1. Cấu trúc một event

Rocha đề xuất mỗi event nên có ba phần [5, Ch.8]:

Listing 5.4: Cấu trúc event với metadata và payload

```
{
  "metadata": {
    "eventId": "e7d3f2a1-...",
    "eventType": "SubmissionJudged",
    "version": "1.0",
    "timestamp": "2025-03-10T14:30:00Z",
    "source": "judge-service",
    "correlationId": "req-abc-123"
  },
  "payload": {
    "submissionId": "sub-456",
    "questionId": "q-789",
    "userId": "user-001",
    "result": "CORRECT",
    "executionTimeMs": 245
  }
}
```

Bảng 5.7: Cấu trúc event — metadata và payload

Phần	Mục đích	Fields quan trọng
metadata	Tracing, deduplication, versioning	eventId (cho idempotency), correlationId (cho tracing), version
payload	Business data	Domain-specific data

5.7.2. Bốn loại message

Rocha phân loại message thành 4 loại, mỗi loại có mục đích và naming convention riêng [5, §3.1.4]:

Bảng 5.8: Bốn loại message trong event-driven architecture

Loại	Mô tả	Naming	Ví dụ KBLab	Delivery
Command	Yêu cầu thực hiện hành động — có thể bị từ chối	Verb imperative	JudgeSubmission, SendNotification	Point-to-point
Event	Thông báo sự kiện đã xảy ra — sự thật, không thể reject	Past participle	SubmissionJudged, ContestStarted	Pub/sub
Document	Snapshot toàn bộ entity khi thay đổi — <i>full state, không chỉ delta</i>	Noun	SubmissionDocument, UserDocument	Pub/sub
Query	Yêu cầu thông tin, không thay đổi state	Noun + request	Thường qua HTTP, không qua broker	Sync

5.7.3. Schema evolution

Tương tự API versioning (Ch.3), event schema cũng cần evolution strategy [7, Ch.4]:

- **Thêm field optional** → backward compatible 
- **Bỏ field** → breaking  (consumer cũ vẫn expect)
- **Đổi field type** → breaking 

Best practice: sử dụng **Schema Registry** (Confluent Schema Registry, AWS Glue) để quản lý compatibility tự động. Mỗi schema change được validate trước khi publish.

Phân tích — KBLab thiếu event schema chuẩn

Messages trong KBLab là plain Java objects (POJOs) serialize bằng JSON — không có metadata chuẩn (eventId, timestamp, correlationId), không có schema registry, không có version. Khi Judge Service đổi format message, Core Service có thể crash nếu chưa update.

Migration path: (1) thêm metadata wrapper cho tất cả Kafka messages, (2) sử dụng eventId cho idempotency checks, (3) cân nhắc Avro + Schema Registry khi team mở rộng.

5.7.4. Kafka vs DB Queue trong KBLab

Bảng 5.8b: Chọn Kafka hay DB queue theo use case

Use case	Lựa chọn	Lý do
SQL submission / contest	Kafka	Cần throughput, ordering theo key, replay và consumer group
SQL Judge worker dispatch (judge → judge-*)	Kafka hoặc DB queue + worker	Mỗi job có submissionId/judgeRunId, để retry/idempotency; worker DBMS xử lý độc lập
DevOps Lab operations	DB queue	Job gắn chặt với state trong database, cần transaction enqueue cùng thay đổi domain
Audit/learning analytics	Kafka	Nhiều consumers có thể đọc cùng event stream

DB queue không “kém microservice” hơn Kafka. Nó là lựa chọn hợp lý khi volume vừa phải, consumer ít, và yêu cầu quan trọng nhất là atomicity giữa business state và job record. Kafka mạnh hơn khi cần fan-out, replay, partitioning và stream processing.

5.8. WebSocket — Real-time Notifications

5.8.1. Bài toán

Sau khi Judge Service chấm xong, kết quả cần đến tay sinh viên. Có ba cách:

Bảng 5.9: Ba cách đưa kết quả đến client

Cách	Mô tả	Nhược điểm
Polling	Client hỏi server mỗi X giây	Lãng phí bandwidth, delay
Long polling	Client giữ connection, server trả khi có data	Phức tạp, scaling khó
WebSocket	Full-duplex connection, server push real-time	Cần maintain connection state

KBLab sử dụng **STOMP over SockJS** — WebSocket protocol dựa trên Spring WebSocket:

Listing 5.5: WebSocket server — push kết quả chấm bài qua STOMP

```
// Server-side: push kết quả chấm bài
@Service
public class NotificationService {
    private final SimpMessagingTemplate messagingTemplate;

    public void notifyJudgeResult(UUID userId, JudgeResult result) {
        messagingTemplate.convertAndSendToUser(
            userId.toString(),
            "/queue/notifications",
            new JudgeNotification(result)
        );
    }
}
```

```

    );
  }
}

```

Listing 5.6: WebSocket client — subscribe nhận kết quả real-time

```

// Client-side: subscribe nhận kết quả
const stompClient = new StompJs.Client({
  brokerURL: 'ws://localhost:8080/ws'
});

stompClient.onConnect = () => {
  stompClient.subscribe('/user/queue/notifications', (message) => {
    const result = JSON.parse(message.body);
    showNotification(`Kết quả: ${result.status} ✅`);
  });
};

```

WebSocket phù hợp cho **notification cuối pipeline** (kết quả chấm bài, cập nhật leaderboard trong contest) nhưng không thay thế Kafka cho **inter-service messaging** — hai vai trò khác nhau.

5.9. Case Study: Kafka Pipeline trong Contest Mode

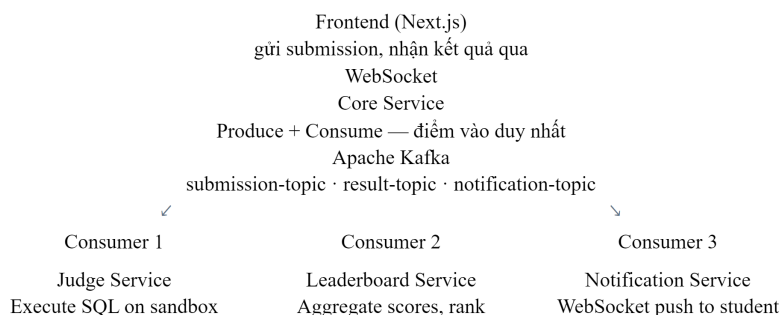
5.9.1. Bài toán Contest

Trong Contest mode, 100+ sinh viên đồng thời nộp bài trong thời gian giới hạn (60–120 phút). Yêu cầu:

- High throughput: xử lý hàng trăm submissions/phút
- Fair ordering: submission nộp trước được chấm trước (per user)
- Real-time feedback: sinh viên thấy kết quả ngay
- Leaderboard update: bảng xếp hạng cập nhật liên tục

5.9.2. Kiến trúc 4-topic pipeline

Kiến trúc Async đầy đủ — LMS Contest Mode



Lợi ích của kiến trúc Async này

1. **Non-blocking:** Core Service không bị block khi Judge xử lý chậm
2. **Extensible:** Thêm consumer mới (Analytics, Replay) không cần sửa code Core
3. **Resilient:** Judge down → message chờ trong Kafka, không mất, xử lý lại khi recover
4. **Independent scaling:** Scale Judge pods độc lập dựa theo contest load

Hình 5.7: Kiến trúc 4-topic pipeline chấm bài trong Contest mode

Bảng 5.10: Chi tiết các Kafka topics trong pipeline

Topic	Producer	Consumer	Message	Key
submissions	Core Service	Judge Service	SQL + metadata	userId
judge-results	Judge Service	Core Service	Result + timing	submissionId
score-updates	Core Service	Leaderboard Aggregator	Score change	contestId
(WebSocket)	Core Service	Frontend	Notification	userId

5.9.3. Phân tích các vấn đề

Bảng 5.11: Phân tích vấn đề Kafka pipeline trong KBLab

#	Vấn đề	Hiện trạng	Best Practice [2a]
1	Auto-commit offset	Commit trước khi xử lý xong → message loss	Manual commit sau khi xử lý thành công
2	Không có DLT	Failed messages bị retry vô hạn hoặc mất	Dead Letter Topic cho messages lỗi
3	Không idempotent	Duplicate có thể tạo kết quả sai	Check submissionId trước khi judge
4	Thiếu monitoring	Không biết consumer lag	Kafka consumer lag metrics + alerting
5	Single partition	Tất cả submissions vào 1 partition → bottleneck	Tăng partitions, key = userId

5.9.4. Đề xuất migration

Phase 1 — Reliability (ưu tiên cao):

- Manual offset commit + @RetryableTopic (3 retries) + Dead Letter Topic
- Idempotency check bằng submissionId

Phase 2 — Observability (ưu tiên cao):

- Kafka consumer lag monitoring (Kafka Exporter + Prometheus)
- correlationId xuyên suốt pipeline (submission → judge → result → notification)

Phase 3 — Performance (khi cần scale):

- Tăng partitions cho submissions topic (từ 1 → 3+)
- Thêm Judge instances vào consumer group

⚠ Lưu ý —

1. **Auto-commit offset trước khi xử lý xong** — Consumer acknowledge message trước khi processing hoàn thành. Hậu quả: nếu consumer crash giữa chừng, message bị mất vĩnh viễn (đã commit nhưng chưa xử lý). *Phòng tránh*: dùng manual offset commit — chỉ commit *sau* khi processing thành công.
2. **Không có Dead Letter Topic** — Message lỗi bị retry vô hạn hoặc block toàn bộ pipeline. Hậu quả: một message poison pill chặn mọi message phía sau. *Phòng tránh*: cấu hình DLT (@RetryableTopic trong Spring Kafka) — messages lỗi sau N retries được chuyển sang DLT để xử lý riêng.
3. **Không thiết kế idempotent consumer** — Giả định mỗi message chỉ đến đúng một lần. Hậu quả: khi Kafka rebalance hoặc retry, message trùng → dữ liệu sai (chấm bài trùng, tính điểm sai). *Phòng tránh*: kiểm tra trạng thái trước khi xử lý — nếu đã xử lý rồi thì skip (§5.5).
4. **Chọn broker vì quen thuộc, không vì use case** — Dùng RabbitMQ cho event streaming chỉ vì team đã biết RabbitMQ. Hậu quả: khi load tăng, broker không phù hợp → phải migrate đau đớn. *Phòng tránh*: đánh giá use case trước (§5.2) — durable log (Kafka) cho event streaming, smart routing (RabbitMQ) cho task queues.

🌐 **Trực quan hóa tương tác (Interactive Demo)**

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/message-broker.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Cơ chế hoạt động của Message Broker**.

5.10. Tổng kết

Giao tiếp bất đồng bộ giải quyết ba giới hạn cốt lõi của sync: temporal coupling, cascading failures, và latency. Hai trường phái message broker — durable (Kafka) và ephemeral (RabbitMQ) — phục vụ use case khác nhau, và nhiều hệ thống dùng cả hai.

Kafka với mô hình event log (durable, replayable, high-throughput) trở thành nền tảng phổ biến nhất cho event-driven architecture. RabbitMQ với smart routing và flexible messaging phù hợp cho task queues, delayed jobs, và complex routing patterns. Partitions, consumer groups, và exchange types cho phép mỗi broker scale theo cách riêng.

Delivery guarantees (at-most/at-least/exactly-once) và idempotency là hai khái niệm không thể tách rời. Thiết kế event schema chuẩn — với metadata, versioning, và correlation — là đầu tư cần thiết cho hệ thống dễ debug và dễ evolve.

Phân tích KBLab cho thấy pipeline Kafka hoạt động nhưng thiếu nhiều best practice quan trọng: error handling, idempotency, monitoring. Đồng thời, DevOps Lab nhắc lại một bài học thực tế: có những job queue nên nằm gần database thay vì ép qua Kafka. Đây là technical debt và trade-off phổ biến khi team nhỏ ưu tiên tính năng trước reliability.

Ở Chương 6, chúng ta sẽ đối mặt với bài toán khó nhất của distributed systems: **distributed transactions**. Khi data nằm ở nhiều service khác nhau, làm thế nào để đảm bảo consistency? Saga pattern sẽ là câu trả lời.

5.11. Đọc thêm

Sách tham khảo chính:

1. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.1: Why EDA; Ch.3: Kafka; Ch.8: Event Schema Design
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Async Messaging, Transactional Outbox
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.11: Stream Processing, Event Logs

Sách bổ trợ:

4. [3] Mitra & Nadareishvili, *Microservices: Up and Running* — Ch.4: Event-Driven Communication

Nguồn trực tuyến:

- Apache Kafka documentation — kafka.apache.org/documentation
- Confluent Schema Registry — docs.confluent.io/platform/current/schema-registry
- Spring Kafka reference — docs.spring.io/spring-kafka

6. Giao dịch Phân tán — Saga Pattern

“A saga is a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga.” — Chris Richardson, *Microservices Patterns* [2a]

6.1. Bạn sẽ học được gì

- Hiểu tại sao distributed transactions (2PC) không phù hợp với microservices
- Nắm vững Saga pattern: định nghĩa, cấu trúc, và compensating transactions
- So sánh hai coordination mechanisms: Choreography vs Orchestration
- Áp dụng countermeasures để xử lý thiếu isolation giữa sagas
- Hiểu eventual consistency và cách quản lý từ góc nhìn business
- Phân tích submit flow trong KBLab — implicit saga và migration path

6.2. Vấn đề: Distributed Transactions

6.2.1. Khi một transaction span nhiều services

Trong KBLab monolith ban đầu, khi sinh viên nộp bài SQL, toàn bộ flow nằm trong một database transaction:

Listing 6.1: Monolith transaction — ACID đảm bảo tất cả thành công hoặc rollback

```
BEGIN TRANSACTION;  
INSERT INTO submissions (id, user_id, sql_content) VALUES (...);  
INSERT INTO judge_queue (submission_id, status) VALUES (... , 'PENDING');
```

```
UPDATE user_stats SET total_submissions = total_submissions + 1 WHERE user_id
= ...;
COMMIT;
-- Tất cả thành công hoặc tất cả rollback – ACID đảm bảo
```

Khi KBLab chuyển sang microservices, data nằm ở nhiều service/database khác nhau: Core Service quản lý submissions (database A), Judge Service quản lý execution (database B), Notification Service quản lý alerts. Không có “shared transaction” giữa chúng — database A không biết database B có commit thành công hay không.

6.2.2. Tại sao 2PC không phù hợp?

Two-Phase Commit (2PC) là cơ chế truyền thống để distributed transactions [7, Ch.9]:

Two-Phase Commit (2PC)

Coordinator

Core Service (DB1)

Judge Service (DB2)

Phase 1 — Prepare

PREPARE?

PREPARE?

YES (ready) ◀

YES (ready) ◀

Phase 2 — Commit

COMMIT!

COMMIT!

ACK ◀

ACK ◀

⚠ Anti-pattern trong Microservices

2PC **không phù hợp** cho microservices: single point of failure (Coordinator), high latency (2 round-trips), lock resources xuyên service.

Giải pháp thay thế: Saga pattern (Hình 6.2 – 6.6)

Hình 6.1: Two-Phase Commit — Coordinator điều phối prepare và commit

Richardson trong [2a, Ch.4] liệt kê lý do 2PC (XA transactions) không phù hợp cho microservices:

Bảng 6.1: Tại sao 2PC không phù hợp với microservices

Vấn đề	Mô tả	Ảnh hưởng đến KBLab
Synchronous blocking	Tất cả participants bị lock cho đến khi commit	Judge Service xử lý SQL 5-30s → Core Service lock toàn bộ thời gian đó
Single point of failure	Coordinator crash → tất cả participants stuck	Nếu coordinator down khi Judge đang chạy → submission stuck
Reduced availability	Tất cả services phải online cùng lúc	Judge MySQL down → không submit được bất kỳ DBMS nào
NoSQL incompatible	Nhiều database không hỗ trợ XA	Kafka (backbone messaging của KBLab) không hỗ trợ XA
Performance	Lock giữ lâu → contention cao	500+ submissions/phút trong contest mode → bottleneck nghiêm trọng

Kleppmann trong [7, Ch.9] bổ sung: 2PC là “blocking protocol” — nếu coordinator crash sau phase 1, participants phải chờ vô hạn. Trong production với contest mode real-time, đây là rủi ro không chấp nhận được.

💡 Tip — Tư duy compensation thay vì locking

Nghĩ về quy trình nộp bài thực tế: sinh viên nộp SQL → hệ thống nhận bài → Judge chấm → trả kết quả. Nếu Judge gặp lỗi, chúng ta không “undo” việc nhận bài — ta *cập nhật trạng thái* thành ERROR và thông báo sinh viên thử lại. Đây chính là tư duy compensation — hành động nghiệp vụ ngược thay vì database rollback. Richardson ghi nhận: “Not even Starbucks uses two-phase commit” [2a, Ch.4].

6.3. Saga Pattern — Chuỗi Local Transactions + Compensation

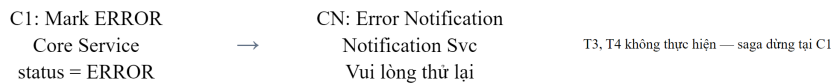
6.3.1. Định nghĩa

Saga là chuỗi **local transactions**, mỗi transaction cập nhật database của một service và publish event/message để trigger transaction tiếp theo. Nếu một transaction thất bại, saga thực hiện **compensating transactions** để undo các thay đổi trước đó [2a, Ch.4].

6.3.2. KBLab Submit Saga

Áp dụng cho flow nộp bài SQL trong KBLab:

Choreography Saga — Submission Flow

✅ **HAPPY PATH**❌ **COMPENSATION — KHI T2 THẤT BẠI**

Cơ chế Choreography

1. Core gửi `SubmissionCreated` event lên Kafka
2. Judge lắng nghe, execute SQL, gửi `SubmissionJudged` event
3. Core lắng nghe `SubmissionJudged`, update score, gửi `ScoreUpdated`
4. Notification lắng nghe `ScoreUpdated`, gửi push notification cho student

Ưu điểm: đơn giản, không cần orchestrator trung tâm | Nhược điểm: khó theo dõi toàn bộ flow, khó debug khi có lỗi

Hình 6.2: KBLab Submit Saga — chuỗi local transactions và compensating transactions

Richardson trong [2a, Ch.4] minh họa saga pattern với 4 participants và credit card authorization làm pivot transaction. Trong KBLab, saga submit đơn giản hơn (2-3 participants) nhưng phức tạp ở chỗ Judge execution là **long-running** — thách thức đặc thù của bài toán chấm bài tự động.

6.3.3. Ba loại transactions trong saga

Richardson phân loại mỗi step trong saga thành ba loại [2a, Ch.4]. Áp dụng cho KBLab Submit Saga:

Bảng 6.2: Ba loại transactions trong saga — áp dụng cho KBLab Submit Saga

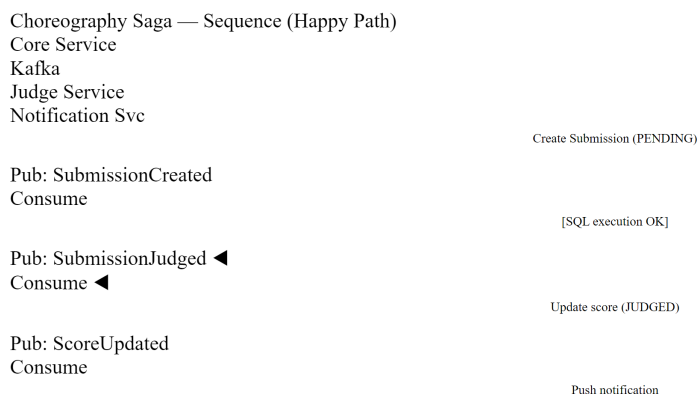
Loại	Mô tả	Có compensation?	Ví dụ trong KBLab
Compensatable	Có thể bị undo bởi compensating transaction	✅ Có	T1: Create Submission (→ C1: Mark ERROR)
Pivot	Điểm quyết định go/no-go — sau đây saga cam kết hoàn thành	❌ Không cần	T2: Execute SQL (pass/fail)
Retriable	Bước sau pivot — phải thành công (retry cho đến khi xong)	❌ Không cần	T3: Update Score, T4: Notify Student

Cấu trúc saga luôn là: **Compensatable** → **Pivot** → **Retriable**. Sau khi T2 (Execute SQL) thành công, saga *cam kết hoàn thành* — T3 và T4 sẽ được retry nếu thất bại, không cần compensation.

6.4. Choreography vs Orchestration

6.4.1. Choreography — Phi tập trung, event-driven

Trong choreography, **không có coordinator**. Mỗi service lắng nghe events và tự quyết định hành động tiếp theo [5, §4.2]. Đây là cách KBLab hiện đang hoạt động:



Hình 6.3: Choreography — các service tự phối hợp qua events

Bảng 6.3: Ưu và nhược điểm của Choreography

Ưu điểm	Nhược điểm
Đơn giản: không cần thêm service orchestrator	Khó theo dõi: logic phân tán, không ai biết “saga ở bước nào”
Loosely coupled: services không biết nhau, chỉ biết events	Cyclic dependencies: rủi ro event loops (A → B → A)
Dễ thêm participants: subscribe event mới là xong	Testing khó: test flow đầy đủ cần tất cả services chạy

6.4.2. Orchestration — Tập trung, commanding

Trong orchestration, **saga orchestrator** điều phối toàn bộ flow, gửi commands và nhận replies [2a, Ch.4]. Nếu KBLab dùng orchestration cho Submit Saga:

Orchestration Saga — Submit Flow

Submit Saga
Orchestrator
Core Service
Judge Service
Notification Svc

Saga bắt đầu

1. createSubmission()
OK (PENDING) ◀
2. executeSQL()
Result: score=85 ◀
3. updateScore(85)
OK (JUDGED) ◀
4. notifyStudent()

✅ [Success] Saga hoàn thành

So sánh với Choreography

Orchestrator **biết rõ từng bước** và gọi trực tiếp — dễ debug, dễ thêm compensation logic.

Nhược điểm: Orchestrator là *single point of complexity* — cần test kỹ và xử lý lỗi từng bước.

Hình 6.4: Orchestration — saga orchestrator điều phối toàn bộ flow

Bảng 6.4: Ưu và nhược điểm của Orchestration

Ưu điểm	Nhược điểm
Dễ hiểu: logic tập trung, biết saga đang ở bước nào	Centralization risk: orchestrator là component thêm cần maintain
Không cyclic: dependencies luôn orchestrator → service	Thêm complexity: cần build orchestrator service
Dễ test: mock orchestrator, test từng step riêng lẻ	Smart orchestrator risk: logic business có thể “leak” vào orchestrator

6.4.3. Khi nào dùng gì?

Rocha trong [5, §4.4] đề xuất kết hợp cả hai — và đây là cách tiếp cận thực tế nhất:

Bảng 6.5: Choreography vs Orchestration — khi nào dùng gì

Scenario	Recommendation	KBLab context
Saga đơn giản (2-3 steps)	Choreography	Submit flow hiện tại (3 steps)
Saga phức tạp (4+ steps, branching)	Orchestration	Nếu thêm plagiarism check + rubric grading
Cross-bounded context	Choreography giữa contexts	Core ↔ Judge (khác bounded context)
Team chưa quen EDA	Orchestration — dễ debug hơn	Phù hợp nếu team mở rộng

❗ Nguyên tắc — KBLab nên giữ choreography hay chuyển orchestration?

Với submit flow hiện tại (3 steps, 2 services), **choreography là đủ** — thêm orchestrator sẽ over-engineering. Tuy nhiên, nếu tương lai submit flow mở rộng (thêm plagiarism detection service, thêm grading rubric service, thêm analytics service), orchestration trở nên cần thiết. Đây là quyết định kiến trúc mở — ghi nhận để revisit khi requirements thay đổi.

6.5. Compensating Transactions & Isolation

6.5.1. Thiết kế compensation trong KBLab

Compensation không phải “undo” — nó là **hành động nghiệp vụ ngược** [2a, Ch.4]. Trong ngữ cảnh KBLab:

Bảng 6.6: Thiết kế compensation trong KBLab

Forward Transaction	Compensation	Lưu ý
T1: Create Submission (PENDING)	C1: Mark ERROR	Không DELETE submission — đổi status
T2: Execute SQL	(Pivot — không cần compensation)	Judge tự cleanup sandbox
T3: Update Score	C3: Revert Score	Trừ lại score nếu đã cộng
T4: Send Notification	C4: Send Correction Notification	Không thể “un-send” — gửi correction

Nguyên tắc: compensation phải **idempotent** — gọi nhiều lần cho cùng kết quả. Trong KBLab: gọi `markError(submissionId)` nhiều lần chỉ set `status=ERROR` một lần — không side effect.

6.5.2. Vấn đề isolation — Anomalies

Saga không có isolation (chữ I trong ACID). Trong database truyền thống, transactions chạy đồng thời nhưng cách ly bởi isolation levels (READ COMMITTED, SERIALIZABLE). Saga không có cơ chế tương đương — kết quả trung gian của saga *có thể nhìn thấy* bởi sagas khác. Richardson trong [2a, Ch.4] gọi đây là **lack of isolation** — vấn đề nghiêm trọng nhất của Saga pattern.

Ba anomalies chính:

1. Lost Updates — Saga A ghi đè kết quả mà Saga B đã viết, mà không biết Saga B đã thay đổi data.

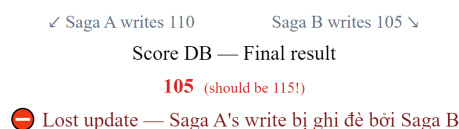
Race Condition — Thiếu Idempotency

Saga A — Submit #1

1. Read score: **100**
2. Add 10 points (correct answer)
3. Write score: **110** → DB

Saga B — Submit #2 (concurrent)

1. Read score: **100** ⚡ **cùng lúc!**
2. Add 5 points (partial answer)
3. Write score: **105** → DB



Giải pháp

1. **Idempotency key**: mỗi submission có unique ID, DB UNIQUE constraint ngăn xử lý trùng
 2. **Optimistic locking**: version field — `UPDATE score SET value=? WHERE id=? AND version=N`
 3. **Database constraint**: `UNIQUE(submission_id)` trong score table — lỗi nếu duplicate
- ⚠ *LMS hiện tại: Chưa có idempotency key → contest mode có thể bị duplicate submission khi network retry.*

Hình 6.5: Lost Updates anomaly — hai saga ghi đè kết quả của nhau

Ví dụ KBLab: hai submissions của cùng user — submission A (score +10) và B (score +5) xử lý đồng thời. Cả hai đọc score=100, A set 110, B set 105. Score đúng phải là 115 nhưng chỉ là 105 — update của A bị mất.

2. Dirty Reads — Saga A đọc data mà Saga B đã ghi nhưng chưa hoàn thành (có thể sẽ rollback).

Ví dụ KBLab: Saga B bắt đầu judging submission → update score tạm. Leaderboard (Saga A) đọc score mới. Saga B gặp lỗi → compensate, revert score. Nhưng leaderboard đã hiển thị score sai → user confused.

3. Non-repeatable/Fuzzy Reads — Data thay đổi giữa hai lần read trong cùng saga.

Ví dụ KBLab: Contest ranking thay đổi giữa lúc user mở bảng xếp hạng và lúc submit — user thấy mình đứng hạng 3, nhưng khi kết quả trả về, rankings đã thay đổi vì saga khác hoàn thành.

📌 Nguyên tắc — ACD thay vì ACID

Richardson trong [2a, Ch.4] chỉ ra: Sagas chỉ đảm bảo **ACD** (Atomicity, Consistency, Durability) — *không có Isolation*. Atomicity đạt được nhờ compensating transactions (rollback logic). Consistency bảo toàn bởi business rules trong mỗi local transaction. Durability do mỗi database đảm bảo. Nhưng Isolation phải được xử lý riêng — bằng **countermeasures**.

6.5.3. Countermeasures

Richardson đề xuất các countermeasures [2a, Ch.4], áp dụng cho KBLab:

1. Semantic Lock — Đặt cờ “đang xử lý” trên record, ngăn saga khác đọc/ghi data chưa final:

Listing 6.2: Semantic Lock — đặt cờ “JUDGING” ngăn thao tác trên data chưa final

```
// Khi saga bắt đầu – không cho phép re-submit cùng câu hỏi
submission.setStatus(SubmissionStatus.JUDGING); // semantic lock
submissionRepository.save(submission);

// Service khác kiểm tra trước khi đọc score
if (submission.getStatus() == SubmissionStatus.JUDGING) {
    // Score chưa final – hiện "Đang chấm..." trên leaderboard
}

// Khi saga hoàn thành – release lock
submission.setStatus(SubmissionStatus.JUDGED); // release
submissionRepository.save(submission);
```

2. Commutative Updates — Thiết kế updates không phụ thuộc thứ tự — giải quyết **lost updates**:

Listing 6.3: Commutative Updates — delta thay vì absolute value

```
// ❌ Không commutative: set absolute value – thứ tự quan trọng
user.setTotalScore(150);

// ✅ Commutative: delta update – thứ tự không quan trọng
user.incrementScore(+10); // submission A correct
user.incrementScore(+5);  // submission B correct
// Kết quả giống nhau dù A trước B hay B trước A
```

3. Pessimistic View — Sắp xếp saga steps để giảm dirty reads: đặt retrievable steps (update score, send notification) *sau* pivot transaction (execute SQL). KBLab đã tự nhiên tuân thủ — score chỉ update sau khi Judge xong.

4. Re-read Value — Đọc lại data trước khi quyết định (tương tự optimistic locking) — giải quyết **non-repeatable reads**:

Listing 6.4: Re-read Value — kiểm tra lại trước khi quyết định

```
// Trước khi update score, kiểm tra submission chưa bị cancel
Submission fresh = submissionRepository.findById(submissionId);
if (fresh.getStatus() == SubmissionStatus.CANCELLED) {
    return; // User đã cancel – không update score
}
```

5. Version File — Ghi lại thứ tự operations, reorder nếu cần. Ví dụ: nếu Saga A và B đều update score, Version File ghi [A:+10, B:+5] — xử lý tuần tự thay vì đồng thời. Trong KBLab, Kafka topic `score-updates` tự nhiên là Version File: messages xử lý theo thứ tự partition.

6.5.4. Tổng hợp: Anomaly → Countermeasure

Bảng 6.7: Anomaly → Countermeasure — áp dụng cho KBLab

Anomaly	Countermeasure phù hợp	Áp dụng KBLab
Lost updates	Commutative updates, Version File	incrementScore(+delta) thay vì setScore(value)
Dirty reads	Semantic lock, Pessimistic view	JUDGING status flag ngăn đọc score chưa final
Non-repeatable reads	Re-read value	Check status != CANCELLED trước khi commit

6.6. Eventual Consistency — Chấp nhận và quản lý

6.6.1. Từ ACID đến BASE

Microservices chuyển từ ACID sang **BASE** [7, Ch.9]:

Bảng 6.8: ACID (Monolith) vs BASE (Microservices)

	ACID (KBLab monolith)	BASE (KBLab microservices)
A	Atomic — submit + judge + score = all or nothing	Basically Available — system luôn nhận submission
C	Consistent — score luôn đúng tại mọi thời điểm	Soft state — score có thể tạm thời chưa cập nhật
I	Isolated — hai submissions không thấy nhau	Eventual consistency — leaderboard cuối cùng sẽ đúng
D	Durable — committed = persistent	(Same)

6.6.2. Consistency window trong KBLab

Nguyên tắc — Consistency Window

Rocha trong [5, Ch.5] định nghĩa **consistency window**: khoảng thời gian từ khi event xảy ra đến khi tất cả services phản ánh trạng thái mới nhất. Mục tiêu: giữ consistency window **đủ nhỏ** để user không nhận thấy.

Trong KBLab: sau khi sinh viên nộp bài, có consistency window trước khi kết quả xuất hiện (tùy complexity của bài và tải của Judge). Trong thời gian đó:

- `submission.status = JUDGING` (semantic lock)
- Leaderboard hiện score cũ (chưa cập nhật)
- UI hiện “Đang chấm...” — user chấp nhận được

6.6.3. Strategies cho UI và business

Bảng 6.9: Strategies cho UI khi chấp nhận eventual consistency

Strategy	Mô tả	Cách KBLab implement
Optimistic UI	Hiện trạng thái “sẽ thành công” trước khi xong	“Bài đã gửi thành công” (dù chưa chấm xong)
Push notification	Cập nhật UI khi có kết quả	WebSocket push: “Kết quả: Correct 
Compensation notification	Thông báo nếu phải rollback	“Bài không chấm được, vui lòng thử lại”
Progress indicator	Hiện trạng thái từng bước	“Đang chấm...” → “Đang so sánh kết quả...” → “Hoàn thành”

6.7. Case Study: Submit Flow trong KBLab — Implicit Saga

6.7.1. Hiện trạng: Implicit Saga (Choreography)

KBLab hiện có một implicit saga — choreography qua Kafka events, nhưng KHÔNG được định nghĩa rõ ràng như saga:



Hình 6.6: Implicit Saga trong KBLab — choreography qua Kafka events

Richardson trong [2a, Ch.4] mô tả saga pattern với explicit orchestrator, state machine rõ ràng, và compensation cho từng step. Submit flow của KBLab tương đối ngắn, nhưng vẫn cần safety mechanisms: explicit saga definition, compensation và timeout.

6.7.2. Phân tích các vấn đề

Bảng 6.10: Phân tích vấn đề Submit Flow trong KBLab

#	Vấn đề	Hiện trạng KBLab	Best Practice [2a, Ch.4]
1	Implicit saga	Flow phân tán trong code, không có saga definition	Explicit saga class với state machine
2	Không compensation	Judge crash → submission PENDING vĩnh viễn	Timeout-based compensation: PENDING > 5 min → ERROR
3	Không semantic lock	User có thể submit lại khi đang judging	Status JUDGING block re-submission cùng câu hỏi
4	Không saga tracking	Không biết saga ở bước nào	Status tracking: PENDING → JUDGING → JUDGED/ERROR/ TIMEOUT
5	Không timeout	Judge không response → PENDING mãi	Scheduled job check + timeout compensation

6.7.3. Đề xuất migration

Phase 1 — Semantic Lock + Timeout (ưu tiên cao, effort thấp): **Listing 6.5:** Semantic Lock + Timeout compensation cho submissions

```
// Khi nhận submission – semantic lock
submission.setStatus(SubmissionStatus.JUDGING);
submission.setJudgeStartedAt(Instant.now());
submissionRepository.save(submission);

// Scheduled job kiểm tra timeout
@Scheduled(fixedRate = 60000)
public void checkSubmissionTimeouts() {
    List<Submission> stuck = submissionRepository
        .findByStatusAndJudgeStartedAtBefore(
            SubmissionStatus.JUDGING,
            Instant.now().minus(5, ChronoUnit.MINUTES)
        );
    stuck.forEach(s -> {
        s.setStatus(SubmissionStatus.TIMEOUT); // compensation
        notificationService.notify(s.getUserId(),
            "Judge timeout – bài sẽ được chấm lại tự động");
        // Re-publish to Kafka for retry
    });
}
```

```

        submitProducer.sendSubmission(s);
    });
}

```

Phase 2 — Explicit Saga Definition (effort trung bình):

- Định nghĩa `SubmitSaga` class với state machine: PENDING → JUDGING → JUDGED / ERROR / TIMEOUT
- Mỗi transition kèm compensation action rõ ràng
- Log saga state changes cho auditing và debugging

Phase 3 — Saga Orchestrator (effort cao, khi cần scale):

- Cần nhắc khi submit flow mở rộng (thêm plagiarism check, grading rubric)
- Hiện tại choreography vẫn đủ — flow chỉ 2-3 steps
- Threshold: khi flow > 4 steps hoặc có complex branching → chuyển sang orchestration

6.7.4. Workflow học vụ cũng là saga

KBLab không chỉ có submit flow. Các luồng như duyệt bài tập, phê duyệt đề tài/đồ án, công bố điểm hoặc cập nhật CLO cũng có bản chất saga: nhiều bước, nhiều người tham gia, có trạng thái trung gian và có hành động bù trừ khi bị từ chối hoặc quá hạn.

Bảng 6.11: Ví dụ workflow học vụ dưới góc nhìn Saga

Workflow	Compensatable	Pivot	Retriable
Assignment approval	Draft assignment	Lecturer/Admin approve	Publish to course, notify students
Thesis/project review	Student submit proposal	Committee decision	Update status, notify student/advisor
Grade publication	Compute provisional grade	Lecturer confirms	Publish grade, sync reporting read model

Các workflow này thường nên bắt đầu bằng **explicit state machine** hơn là orchestrator riêng. Khi số bước và nhánh tăng, orchestration mới đáng cân nhắc.

⚠ Lưu ý —

1. **Cố dùng distributed transaction (2PC) trong microservices** — Quen với ACID từ monolith, cố tìm cách giữ transaction xuyên service. Hậu quả: blocking, single point of failure, giảm availability — mất hết lợi ích microservices. *Phòng tránh*: chấp nhận eventual consistency, dùng Saga pattern (§6.2).
2. **Không định nghĩa compensation** — Chỉ nghĩ đến happy path, bỏ qua “nếu step 3 thất bại thì step 1 và 2 undo thế nào?” Hậu quả: data inconsistent, records stuck ở trạng thái trung gian mãi mãi. *Phòng tránh*: mỗi compensatable transaction phải có compensating action rõ ràng trước khi implementation (§6.4).
3. **Không có timeout cho saga** — Saga bắt đầu nhưng không ai kiểm tra “bao lâu rồi chưa xong?” Hậu quả: submissions stuck ở PENDING vĩnh viễn, user không biết bài có được chấm hay không. *Phòng tránh*: scheduled job kiểm tra timeout + compensation tự động (§6.6).
4. **Không dùng semantic lock** — Cho phép thao tác trên data đang trong saga (ví dụ: user re-submit khi bài đang JUDGING). Hậu quả: race conditions, dirty reads, kết quả sai. *Phòng tránh*: set status = JUDGING ngay khi saga bắt đầu, block operations trên record cho đến khi saga hoàn thành (§6.4).

🌐 **Trực quan hóa tương tác (Interactive Demo)**

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/saga-orchestration.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Saga Pattern (Orchestration)**.

6.8. Tổng kết

Distributed transactions là bài toán khó nhất khi chuyển từ monolith sang microservices. Two-Phase đoạn Commit — giải pháp truyền thống — không phù hợp vì blocking, single point of failure, và giảm availability.

Saga pattern giải quyết bằng cách thay một ACID transaction bằng chuỗi local transactions + compensating transactions. Ba loại transactions (compensatable, pivot, retrievable) tạo cấu trúc rõ ràng cho mỗi saga. Choreography (phi tập trung) phù hợp cho sagas đơn giản, orchestration (tập trung) cho sagas phức tạp.

Thiếu isolation là thách thức lớn nhất. Semantic lock, commutative updates, pessimistic view, và re-read value là bốn countermeasures giảm thiểu anomalies — tất cả đều áp dụng được cho KBLab.

Eventual consistency là trade-off có chủ đích cho availability và scalability. Trong KBLab, consistency window ngắn là chấp nhận được — sinh viên quen với việc chờ kết quả chấm bài. Quản lý consistency window bằng semantic lock và real-time notification là đủ cho use case hiện tại.

Phân tích KBLab cho thấy hệ thống đang dùng implicit saga (choreography) và cần làm rõ compensation, semantic lock, timeout handling. Submission hoặc workflow học vụ có thể stuck ở trạng thái trung gian — đây là technical debt nghiêm trọng cần migration ưu tiên.

Ở Chương 7, chúng ta sẽ giải quyết bài toán **data management** — database-per-service, CQRS, Event Sourcing — và phân tích sâu hơn vấn đề shared database trong KBLab.

6.9. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.4: Managing Transactions with Sagas
2. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.4: Sagas (Choreography, Orchestration); Ch.5: Eventual Consistency
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.7: Transactions; Ch.9: Consistency and Consensus

Nguồn trực tuyến:

- Gregor Hohpe, “Starbucks Does Not Use Two-Giai đoạn Commit” (2004) — enterpriseintegrationpatterns.com
- Caitie McCaffrey, “Applying the Saga Pattern” (Strange Loop 2015) — [youtube.com](https://www.youtube.com/watch?v=8vXpYUwv8p0)
- Eventuate Tram Sagas framework — eventuate.io

7. Quản lý Dữ liệu trong Microservices

“Duplication is far better than coupling. Each service should own its data and make independent decisions about what to store.” — Sam Newman, *Monolith to Microservices* [4b]

7.1. Bạn sẽ học được gì

- Hiểu nguyên tắc **database-per-service** và tại sao data ownership là nền tảng của microservices
- Nắm được các chiến lược tách database từ monolith — từ shared database đến database-per-service
- Phân biệt khi nào data duplication là chấp nhận được và khi nào nó là vấn đề
- Hiểu CQRS (Command Query Responsibility Segregation) và khi nào nó cần thiết
- Nắm tổng quan Event Sourcing — ưu/nhược điểm và khi nào nên áp dụng
- Phân tích bài toán cross-service queries trong KBLab

7.2. Database-per-Service — Nguyên tắc Data Ownership

7.2.1. Vấn đề: shared database phá vỡ service independence

Trong monolith, toàn bộ ứng dụng truy cập một database duy nhất — đơn giản, nhất quán, và transactions ACID hoạt động tự nhiên. Khi chuyển sang microservices, nhiều team giữ nguyên shared database vì “tiện” — mọi service vẫn đọc/ghi cùng bảng. Thoạt nhìn, đây có vẻ là giải pháp tốt: không cần thay đổi data layer, không cần xử lý eventual consistency.

Tuy nhiên, bài toán data trong distributed systems bị ràng buộc bởi một giới hạn lý thuyết quan trọng — **CAP Theorem** (Consistency, Availability, Partition tolerance): trong hệ thống phân tán, khi network partition xảy ra, hệ thống chỉ có thể chọn *hoặc* consistency *hoặc* availability, không thể có cả hai [7, Ch.9]. Newman trong [4a, Ch.11] giải thích: đây là lý do tại sao microservices buộc phải chấp nhận eventual consistency — và tại sao thiết kế data management cần cân nhắc kỹ trade-offs giữa consistency và availability cho *từng use case cụ thể*.

Shared database **phá vỡ nguyên lý cốt lõi nhất** của microservices: independent deployability. Newman trong [4b, Ch.4] phân tích ba hậu quả nghiêm trọng:

Shared Database — Hậu quả

Shared Database — Vấn đề

Schema coupling

Thay đổi bảng X ảnh hưởng Service A, B, C — không thể deploy độc lập

Deploy coupling

Migrate schema = phải deploy tất cả services cùng lúc — zero-downtime impossible

Scale coupling

Không thể scale DB cho riêng một service — bottleneck ảnh hưởng toàn hệ thống

Service A

Service B

Service C

Shared Database

Tất cả cùng dùng chung — single point of coupling

Giải pháp: Database-per-Service (Hình 7.2)

Mỗi service own DB riêng. Giao tiếp qua API, không qua DB trực tiếp.

Hình 7.1: Ba hậu quả của shared database trong microservices

1. Schema coupling — Khi hai service đọc cùng bảng `users`, thay đổi schema (đổi tên cột, thêm constraint) ảnh hưởng cả hai. Mỗi database migration trở thành cross-team coordination exercise — chính xác vấn đề mà microservices muốn loại bỏ.

2. Deploy coupling — Database migration phải deploy cùng với tất cả services sử dụng bảng đó. Nếu Service A cần thêm cột vào `users`, Service B phải cập nhật code để handle cột mới *trước khi* migration chạy — dù Service B không cần cột đó.

3. Scale coupling — Không thể chọn database technology phù hợp nhất cho từng service. Service yêu cầu full-text search (Elasticsearch) và service yêu cầu ACID transactions (PostgreSQL) bị buộc dùng chung — hoặc phải dùng giải pháp hybrid phức tạp.

7.2.2. Database-per-service pattern

Nguyên tắc **database-per-service** yêu cầu: mỗi service sở hữu dữ liệu riêng, và **dữ liệu đó chỉ có thể truy cập qua API** của service sở hữu — không bao giờ truy cập trực tiếp vào database của service khác [2a, Ch.2].



Hình 7.2: Shared Database (đọc/ghi chung) vs Database-per-Service (data qua API)

“Database riêng” không nhất thiết là database server riêng. Có ba cấp độ tách:

Bảng 7.1: Ba cấp độ tách database

Cấp độ	Mô tả	Isolation	Chi phí
Separate schema	Cùng DB server, khác schema/namespace	Thấp (vẫn có thể cross-schema query)	Thấp
Separate database	Cùng DB server, khác database	Trung bình	Thấp
Separate server	Khác DB server hoàn toàn	Cao (polyglot possible)	Cao

Cấp độ nào phù hợp tùy thuộc vào yêu cầu isolation. Với KBLab hiện tại, **separate schema** là bước đầu tiên hợp lý — chi phí thấp nhưng ngăn được schema coupling ngầm.

⚠ Nguyên tắc — Data Should Be Owned, Not Shared

“If a service wants to access data held by another service, it should go and ask that service for the data it needs. And the service exposing the data should decide what to expose and what to hide.”

— Sam Newman, *Monolith to Microservices* [4b]

❗ Phân tích — KBLab shared database

KBLab sử dụng **shared database** (`app_db`) giữa Core Service và Assignment Service — cả hai service đọc/ghi cùng PostgreSQL instance, có khả năng truy cập chéo bảng. Đây là hậu quả trực tiếp của việc Assignment được tách ra từ Core (cả hai ban đầu là một monolith app). Hậu quả: thay đổi schema cho Assignment có thể ảnh hưởng Core, và ngược lại — phá vỡ independent deployability.

Migration path: (1) xác định bảng nào thuộc về Core, bảng nào thuộc Assignment (dựa vào bounded context analysis từ Ch.2), (2) tạo separate schema trong cùng PostgreSQL, (3) thay thế cross-table queries bằng API calls qua Feign, (4) dài hạn: tách database server khi cần polyglot hoặc independent scaling.

7.2.3. Industry Case Study: Uber — Domain-Oriented Microservice Architecture (DOMA)

Uber (2020) công bố bài học từ việc quản lý **4,000+ microservices** — trong đó data ownership là thách thức lớn nhất. Adam Gluck mô tả cách Uber chuyển từ “microservices spaghetti” sang **DOMA** (Domain-Oriented Microservice Architecture):

Bảng 7.2: Uber — hành trình từ microservices spaghetti đến DOMA

Giai đoạn	Vấn đề	Giải pháp
2012-2016 — Monolith → microservices	Services tăng nhanh, không ai hiểu dependency graph	Mỗi team tạo service riêng, thiếu chuẩn
2016-2018 — Microservices spaghetti	4,000+ services, mỗi service truy cập data của service khác trực tiếp	Data coupling giữa services phá vỡ autonomy
2018-2020 — DOMA	Nhóm services thành domains (tương tự bounded context), mỗi domain có <i>gateway service</i> duy nhất expose data cho bên ngoài	Giảm coupling: services bên ngoài domain chỉ giao tiếp qua domain gateway

Bài học cho từ Uber: (1) Database-per-service là cần thiết nhưng *chưa đủ* — cần thêm *domain-level data encapsulation*. (2) Khi hệ thống lớn, nhóm services thành domains giúp giảm exponential dependency growth. (3) Data duplication giữa domains là chấp nhận được — domain gateway kiểm soát data nào expose.

Nguồn: Adam Gluck, “Introducing Domain-Oriented Microservice Architecture,” Uber Engineering Blog, July 2020 (eng.uber.com/microservice-architecture/). Xem thêm: Uber Technology Day 2019 talks.

7.2.4. CAP Theorem — Hiểu đúng giới hạn

CAP Theorem được đề cập ngắn gọn ở trên, nhưng hiểu *đúng* CAP rất quan trọng khi thiết kế data management. Kleppmann trong [7, Ch.9] phân tích kỹ: CAP không phải “chọn 2 trong 3” như thường hiểu sai — mà là: **khi network partition xảy ra** (và nó sẽ xảy ra trong distributed systems), bạn phải chọn giữa consistency và availability.

CAP Theorem — Khi Network Partition xảy ra

⚡ Network Partition xảy ra — phải chọn 1 trong 2: Consistency hoặc Availability



CP
CP System — Chọn Consistency
Consistency + Partition Tolerance

✓ Trả lời **THẤT BẠI** (error/timeout) thay vì trả data sai
📦 Ví dụ: **PostgreSQL** (single leader replication)
🏠 LMS: **Auth Service** — user data phải chính xác

AP
AP System — Chọn Availability
Availability + Partition Tolerance

✓ Trả data **CÓ THỂ CŨ** nhưng luôn sẵn sàng (never fails)
📦 Ví dụ: **Cassandra, DynamoDB**
🏠 LMS: **Leaderboard** — eventual consistency chấp nhận được

Thực tế: Microservices mặc định là BASE

Basically Available · **S**oft-state · **E**ventually consistent

Không có CA trong distributed system

Network partition **LUÔN có thể xảy ra** — không thể bỏ qua P

Hình 7.3: CAP Theorem — CP (chọn consistency) vs AP (chọn availability) khi partition

CP Systems (Consistency when Partitioned): khi partition xảy ra, system từ chối request thay vì trả data không nhất quán. Ví dụ: PostgreSQL cluster đặt consistency lên trên — node không đồng bộ sẽ trả lỗi.

AP Systems (Availability when Partitioned): khi partition xảy ra, system vẫn trả response nhưng data có thể stale. Ví dụ: Cassandra, DynamoDB cho phép đọc/ghi ở bất kỳ node nào — data *eventually* consistent.

Bảng 7.3: CP vs AP cho từng service trong KBLab

Service	Data	Nên CP hay AP?	Lý do
Submission (chấm)	(tạo, Transactional	CP	Score phải chính xác, sai = tranh cãi. Chấp nhận lỗi “service unavailable” hơn là sai score
Leaderboard	Derived/read	AP	Eventual consistency OK — user chấp nhận ranking “chậm vài giây” hơn là “service unavailable”
User profile	Reference	AP	Tên sinh viên thay đổi rất hiếm, stale data vài phút không vấn đề
Contest timer	Real-time	CP	Thời gian thi phải chính xác — sai = ảnh hưởng công bằng

⚠ Nguyên tắc — Per-Service CAP Decision

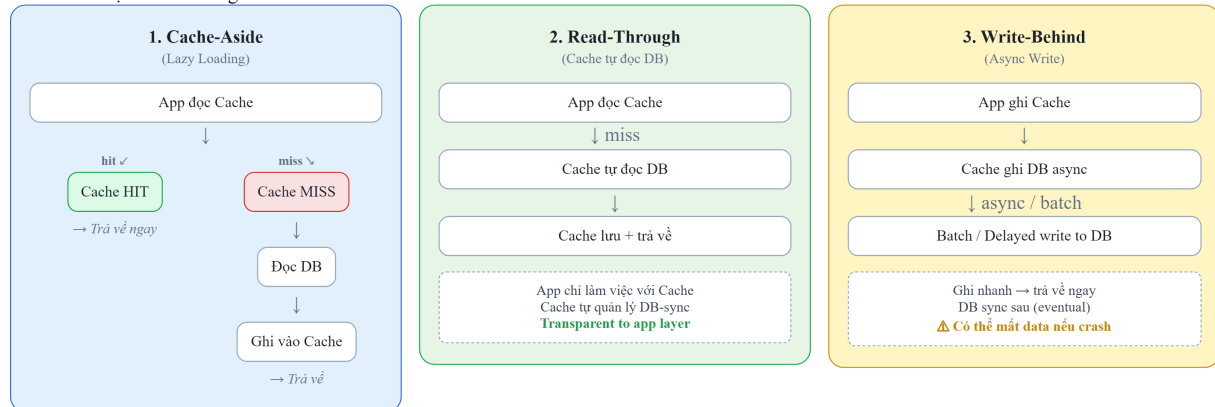
Không phải toàn bộ hệ thống “là CP” hay “là AP”. Mỗi service, thậm chí mỗi *use case*, chọn trade-off riêng. Kleppmann trong [7, Ch.9] gọi đây là “real-world application of CAP”: phân tích *từng data flow* — data nào cần strong consistency (ACID), data nào chấp nhận eventual consistency (BASE).

7.2.5. Caching Strategies — Giảm load và latency

Khi database đã tách, cross-service queries chậm hơn (network call). **Caching** là phương pháp hiệu quả nhất để giảm latency — nhưng cần chiến lược đúng để tránh stale data.

Ba patterns chính (Newman [4a, Ch.11]):

Các chiến lược Cache trong Microservices

**Cache-Aside**

App tự load khi cache miss. Dễ implement, phổ biến nhất (Redis + Spring Cache).

Read-Through

Cache layer tự đọc DB. App không biết DB tồn tại. Cần cache hỗ trợ.

Write-Behind

Ghi cache trước, sync DB sau async. Throughput cao nhưng rủi ro mất data.

Hình 7.4: Ba caching patterns — Cache-Aside, Read-Through, Write-Behind

Bảng 7.4: So sánh ba caching patterns

Pattern	Ưu điểm	Nhược điểm	Áp dụng KBLab
Cache-Aside	Đơn giản, app kiểm soát	Cache miss = 2 calls (cache + DB)	Question list, user profiles
Read-Through	Transparent cho app	Cache layer phức tạp hơn	—
Write-Behind	Write nhanh (async)	Risk mất data nếu cache crash	Leaderboard score updates

Cache Invalidation — “There are only two hard things in Computer Science: cache invalidation and naming things.” (Phil Karlton). Chiến lược:

Bảng 7.5: Chiến lược cache invalidation

Chiến lược	Mô tả	Khi nào dùng
TTL (Time-to-Live)	Cache hết hạn sau N giây	Reference data ít thay đổi (question list: TTL 5 phút)
Event-driven invalidation	Service publish event → cache bị xóa	Data thay đổi thường xuyên (score cập nhật → invalidate leaderboard cache)
Versioned keys	Cache key chứa version: questions:v3	Khi cần invalidate toàn bộ sau schema change

Trong KBLab, Redis phù hợp nhất cho caching: đã có trong stack (dùng cho rate limiting ở Gateway — Ch.8), hỗ trợ TTL tự động, pub/sub cho invalidation events.

7.3. Chiến lược tách Database từ Monolith

7.3.1. Vấn đề: tách service dễ, tách data khó

Nhiều team tách microservices ở tầng application (deploy riêng, repo riêng) nhưng vẫn **trở vào cùng database**. Newman trong [4b] gọi đây là “half-baked migration” — nửa vời. Service đã “tách” nhưng data vẫn coupled — lợi ích independent deployability gần như bằng 0.

Tách database là **bước khó nhất** trong hành trình từ monolith sang microservices. Kleppmann trong [7, Ch.5–6] phân tích: khi data nằm ở nhiều nơi, mọi vấn đề của distributed systems xuất hiện — replication lag, partition tolerance, consistency trade-offs. Đây là chi phí không thể tránh.

7.3.2. Năm chiến lược tách database

Newman đề xuất năm chiến lược, sắp xếp từ ít rủi ro đến nhiều rủi ro [4b, Ch.4]:

Lộ trình tách Database — 5 bước



Hình 7.5: Năm chiến lược tách database — từ ít rủi ro đến nhiều rủi ro

1. Database View — Tạo view read-only cho service mới, ẩn schema gốc. Ưu điểm: không thay đổi data, service cũ không bị ảnh hưởng. Nhược điểm: chỉ read-only, vẫn coupled vào schema.

2. Database Wrapping Service — Đặt một API đơn giản phía trước database, mọi access đi qua API này. Ưu điểm: bắt đầu tách coupling mà không di chuyển data. Nhược điểm: thêm một service cần maintain.

3. Separate Schema — Tách bảng vào schema/namespace riêng trong cùng database. Ưu điểm: isolation rõ ràng, không cần infra mới. Nhược điểm: cùng DB server → vẫn shared resources.

4. Data Transfer — Copy data sang database mới, giữ sync bằng events hoặc CDC (Change Data Capture). Ưu điểm: service mới có data riêng. Nhược điểm: cần cơ chế sync, eventual consistency.

5. Full Database Split — Mỗi service có database server hoàn toàn riêng. Ưu điểm: isolation tối đa, polyglot possible. Nhược điểm: effort lớn, phải xử lý mọi cross-service data access.

7.3.3. Áp dụng cho KBLab

Với bối cảnh KBLab (LMS học thuật, team nhỏ, cùng PostgreSQL), chiến lược phù hợp nhất là **tăng dần**:

Bảng 7.6: Migration path tách database cho KBLab

Giai đoạn	Chiến lược	Mô tả	Effort
1	Separate Schema	Tách <code>app_db</code> thành <code>schema core</code> và <code>schema assignment</code>	Thấp
2	API Wrapping	Thay <code>cross-schema queries</code> bằng <code>Feign calls</code>	Trung bình
3	Full Split (nếu cần)	Tách PostgreSQL server khi cần <code>independent scaling</code>	Cao

💡 Tip — Khi nào đủ?

Không phải hệ thống nào cũng cần đến Giai đoạn 3. Với KBLab, Giai đoạn 1 (separate schema) đã ngăn được schema coupling ngầm. Giai đoạn 2 (API wrapping) cần khi team mở rộng và cần deploy độc lập thực sự. Giai đoạn 3 chỉ cần khi có yêu cầu scale khác nhau hoặc polyglot persistence.

Chi tiết migration roadmap thực tế, Outbox Pattern (reliable messaging khi tách DB), và thứ tự triển khai → xem Ch.10.

7.4. Data Duplication vs Coupling — Khi nào chấp nhận duplicate?

7.4.1. Vấn đề: cần data từ service khác

Sau khi tách database, vấn đề tiếp theo lập tức xuất hiện: **service A cần data mà service B sở hữu**. Ví dụ trong KBLab: Assignment Service cần hiển thị tên sinh viên — nhưng `users` thuộc về Auth Service. Gọi Auth API mỗi lần cần tên? Lưu copy tên sinh viên tại Assignment?

Mitra trong [3, Ch.5] phân tích ba giải pháp, mỗi cách có trade-off riêng:

7.4.2. Ba chiến lược truy cập data xuyên service

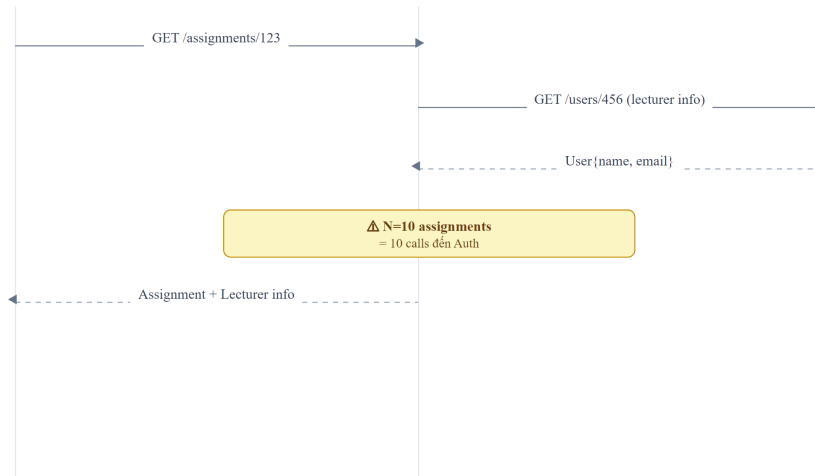
1. API Call (Runtime dependency)

API Composition — N+1 Problem

Client

Assignment Service

Auth Service



△ Vấn đề N+1 Queries

10 assignments → 10 calls riêng lẻ đến Auth Service. Độ trễ nhân lên tuyến tính. **Giải pháp:** batch API hoặc CQRS với data replication (Hình 7.8).

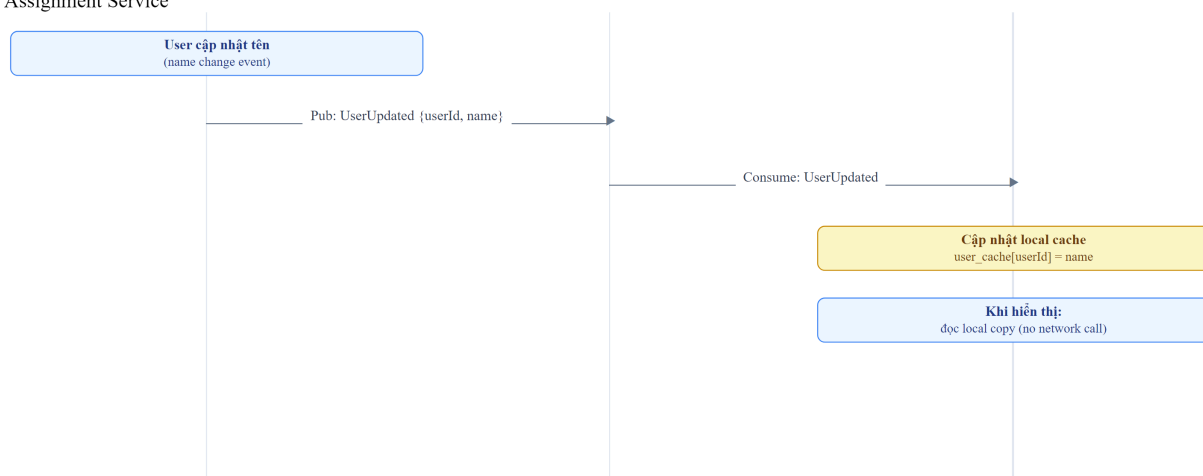
Hình 7.7: Truy cập dữ liệu xuyên service bằng API Call

Bảng 7.7: Ưu và nhược điểm của API Call pattern

Ưu điểm	Nhược điểm
Data luôn mới nhất	Temporal coupling: Auth down → Assignment không hiển thị được tên
Không duplicate	Latency tăng: mỗi request = thêm 1 network call
Đơn giản	N+1 problem: hiển thị 50 students = 50 API calls

2. Data Duplication (Local copy)

Data Replication qua Events
Auth Service
Kafka
Assignment Service



Eventual Consistency — chấp nhận được ở đây

Assignment có thể hiển thị tên cũ vài giây sau khi Auth cập nhật. Đánh đổi lấy: **zero network calls** khi hiển thị assignment list. Không cần gọi Auth mỗi lần render.

Hình 7.8: Truy cập dữ liệu xuyên service bằng Data Duplication qua Event Pipeline

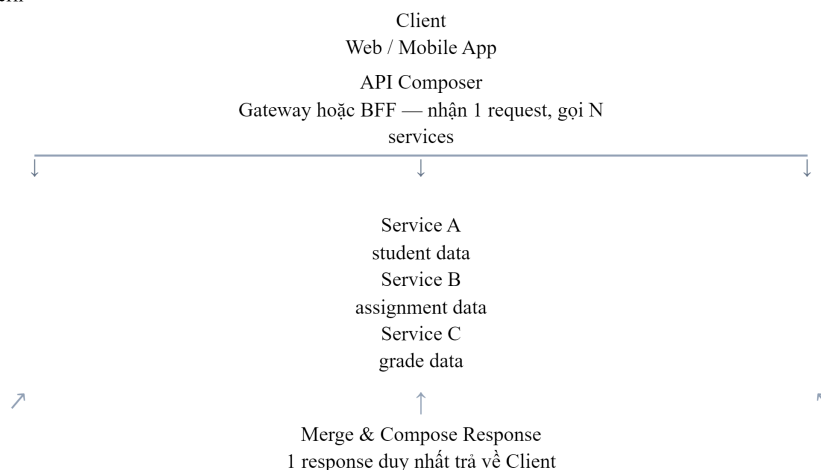
Bảng 7.8: Ưu và nhược điểm của Data Duplication pattern

Ưu điểm	Nhược điểm
Không runtime dependency	Data có thể stale (eventual consistency)
Query nhanh (local)	Cần event pipeline để sync
Dùng được khi Auth down	Storage tăng (duplicate data)

3. API Composition / Data Aggregation

Dùng khi cần join data từ nhiều services. Một service (hoặc API Gateway) gọi nhiều services rồi tổng hợp. Richardson trong [2a, Ch.7] gọi đây là **API Composition pattern**:

API Composition Pattern



Composer thực hiện

Gọi **parallel** 3 services, merge kết quả, trả về client 1 response duy nhất.

LMS Dashboard

Gateway làm composer cho dashboard page: student info + assignments + grades → 1 API call từ client.

Hình 7.9: API Composition pattern — Gateway hoặc BFF gọi nhiều services và tổng hợp

Ví dụ: hiển thị bảng xếp hạng contest cần data từ Core (submissions) và Auth (user names). API Composer gọi cả hai services, sau đó **in-memory join** — batch fetch user IDs để tránh N+1 problem, rồi `stream().map()` kết hợp thành `LeaderboardEntry`.

7.4.3. Khi nào chấp nhận duplication?

Newman trong [4b] đưa ra nguyên tắc rõ ràng: “**Duplication is far better than coupling.**” Tuy nhiên, đây không phải giấy phép duplicate mọi thứ. Quy tắc:

Bảng 7.9: Khi nào chấp nhận data duplication

Data	Nên duplicate?	Lý do
Reference data (tên user, tên khóa học)	✓ Có	Ít thay đổi, cần hiển thị ở nhiều nơi
Transactional data (điểm số, trạng thái submission)	✗ Không	Thay đổi liên tục, cần single source of truth
Configuration (loại database, danh mục)	✓ Có	Gần như static, cache tốt
Audit data (lịch sử thay đổi)	⚠ Tùy	Duplicate cho reporting OK, nhưng master ở source

⚠ Nguyên tắc — Single Source of Truth

Duplication is acceptable *only when* there is a clear **single source of truth**. Service A owns the data, Service B holds a copy. When data changes, the change flows from A → B (via events), never B → A. Nếu không rõ “ai sở hữu data này?”, đó là tín hiệu bounded context chưa rõ ràng — quay lại Ch.2.

① Phân tích — Cross-service queries trong KBLab

KBLab hiện dùng **Feign calls** để query dữ liệu xuyên service — ví dụ: Assignment Service gọi Core Service để lấy danh sách câu hỏi. Đây là API call pattern (chiến lược 1). Với quy mô học thuật vừa phải, cách này chấp nhận được. Tuy nhiên, trong contest mode hoặc dashboard nhiều dữ liệu, mỗi page load có thể trigger nhiều Feign calls → latency tăng đáng kể.

Migration path: (1) identify data nào Assignment cần *hiển thị* từ Core (chủ yếu reference data: tên câu hỏi, tên cuộc thi), (2) duplicate reference data qua Kafka events (UserUpdated, QuestionUpdated), (3) giữ Feign calls cho transactional queries (kiểm tra điểm real-time).

7.5. CQRS — Command Query Responsibility Segregation

7.5.1. Từ CQS đến CQRS

Trước khi nói về CQRS, cần hiểu nguồn gốc. Bertrand Meyer đề xuất **CQS** (Command Query Separation) như một nguyên tắc thiết kế ở *cấp độ method*: mỗi method hoặc trả về dữ liệu (query, không thay đổi state) hoặc thay đổi state (command, không trả về dữ liệu) — nhưng không làm cả hai. Rocha trong [5, §4.5.1] phân biệt rõ: **CQS là nguyên tắc ở cấp single-service/single-class**, còn **CQRS mở rộng nguyên tắc này lên cấp kiến trúc hệ thống** — tách hẳn *model* (thậm chí *database*) cho read và write. CQRS không bắt buộc phải có CQS, nhưng tinh thần tương tự: tách biệt trách nhiệm đọc và ghi.

7.5.2. Vấn đề: cùng model cho read và write không đủ

Trong monolith, cùng một entity **Submission** phục vụ cả hai mục đích:

- **Write:** nộp bài (INSERT), cập nhật kết quả (UPDATE)
- **Read:** hiển thị lịch sử (SELECT nhiều cột, JOIN nhiều bảng, pagination, filtering)

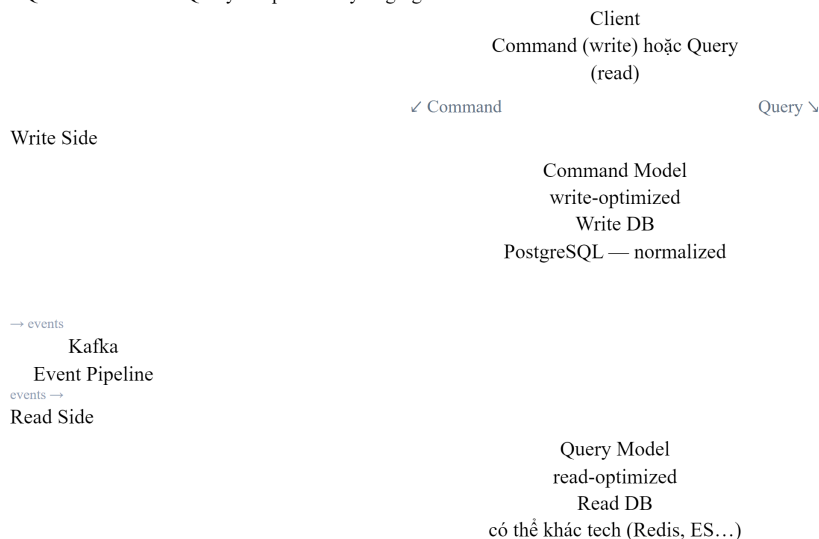
Yêu cầu read và write khác nhau cơ bản. Write cần **consistency**, validation, business rules. Read cần **performance**, denormalized data, flexible queries. Khi hệ thống phức tạp, một model duy nhất phải thỏa hiệp cả hai — và thường không tối ưu cho bên nào [2a, Ch.7].

Richardson trong [2a, Ch.7] mô tả vấn đề này đặc biệt nghiêm trọng khi cần **cross-service queries**: hiển thị bảng xếp hạng contest (data từ Core + Judge + Auth) bằng một query duy nhất — *không thể* khi data nằm ở nhiều databases.

7.5.3. CQRS pattern

CQRS (Command Query Responsibility Segregation) tách model thành hai phần riêng biệt [2a, Ch.7]:

CQRS — Command Query Responsibility Segregation



LMS ứng dụng CQRS

Write: Core Service ghi submissions vào PostgreSQL (normalized write model)**Read:** Leaderboard Aggregator đọc từ materialized view (pre-computed ranking)**Event:** Kafka đồng bộ write → read. Eventual consistency (vài giây delay).

Hình 7.10: CQRS pattern — tách command model (write) và query model (read)

Command side (write):

- Normalized data model (đúng 3NF)
- Strong validation, business rules
- Database: PostgreSQL (ACID transactions)

Query side (read):

- Denormalized, pre-joined views
- Tối ưu cho specific query patterns
- Database: có thể khác technology (Elasticsearch cho search, Redis cho leaderboard, PostgreSQL materialized view cho reports)

7.5.4. Khi nào cần CQRS?CQRS thêm complexity đáng kể — **đừng dùng khi không cần** [5, §4.5]:**Bảng 7.10: Khi nào cần CQRS**

Scenario	Cần CQRS?	Lý do
Read/write ratio cân bằng, cùng model	✗ Không	CRUD đơn giản là đủ
Read phức tạp (cross-service join)	✓ Có	Query model denormalized giải quyết join
Read volume >> Write volume	✓ Có	Scale read model độc lập
UI cần real-time views (leaderboard)	✓ Có	Pre-computed views nhanh hơn
Team nhỏ, MVP giai đoạn	✗ Không	Complexity tax quá cao cho lợi ích

7.5.5. Ví dụ: Leaderboard trong Contest Mode

Trong KBLab, bảng xếp hạng contest cần join data từ nhiều nguồn:

Bảng xếp hạng cần data từ nhiều services (Auth, Core) — SQL JOIN truyền thống không thể khi database đã tách.

Với CQRS, leaderboard có **read model riêng**: write side publish `ScoreUpdatedEvent` mỗi khi submission được chấm, read side consume event và cập nhật `LeaderboardEntry` denormalized (đã chứa sẵn `userName`, `totalScore`). Query leaderboard chỉ cần `findByContestIdOrderByTotalScoreDesc()` — đơn giản, nhanh, không cần JOIN.

CQRS Sequence — Leaderboard Update

Client

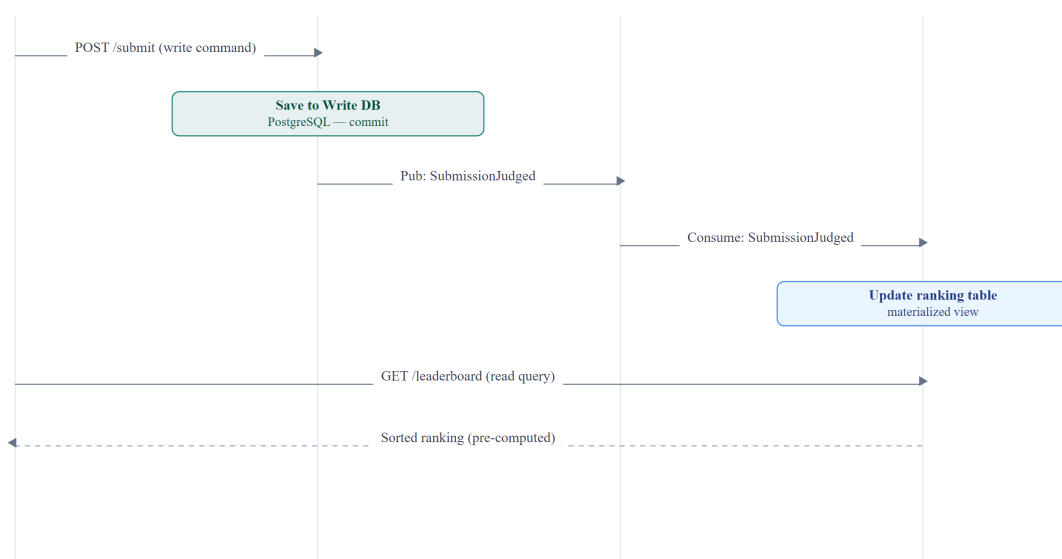
Write Side

(Core Service)

Kafka

Read Side

(Leaderboard)



Read và Write tách biệt — không block nhau

Write không block Read. Read có thể scale độc lập. Leaderboard read model cực nhanh vì pre-computed.

Hình 7.11: CQRS cho Leaderboard — write side publish events, read side denormalized

Nguyên tắc — CQRS ≠ Event Sourcing

CQRS và Event Sourcing thường được nhắc đến cùng nhau, nhưng chúng là **hai pattern độc lập**. CQRS tách read/write model — có thể dùng mà không cần Event Sourcing. Event Sourcing lưu events thay vì state — có thể dùng mà không cần CQRS. Kết hợp cả hai rất mạnh nhưng cũng *rất phức tạp*. Richardson trong [2a, Ch.6] khuyến nghị: bắt đầu với CQRS đơn giản trước, thêm Event Sourcing *khi có nhu cầu cụ thể* (audit trail, temporal queries).

7.6. Event Sourcing — Lưu Events thay vì State

7.6.1. Vấn đề: mất lịch sử khi chỉ lưu state

Với database truyền thống, chúng ta lưu **trạng thái hiện tại** (*current state*): `submission.status = JUDGED, score = 85`. Mọi thay đổi trước đó bị ghi đè — không biết `submission` đã qua những trạng thái nào, ai thay đổi, khi nào.

Trong nhiều domain (tài chính, audit, legal), lịch sử thay đổi có giá trị ngang bằng hoặc hơn trạng thái hiện tại. Ngay cả trong KBLab: “sinh viên nộp bài 3 lần, lần đầu sai, lần 2 đúng một phần, lần 3 đúng hoàn toàn” — thông tin này giá trị cho phân tích học tập, nhưng nếu chỉ lưu `status = CORRECT`, lịch sử bị mất.

7.6.2. Event Sourcing pattern

Event Sourcing thay đổi cách lưu trữ: thay vì lưu state, lưu **chuỗi events** (sự kiện) đã xảy ra. State hiện tại được **derive** bằng cách replay tất cả events [2a, Ch.6]:

Event Sourcing vs Truyền thống

Truyền thống: Lưu State

Bảng submissions

`id=1, status=JUDGED, score=85`

← Ghi đè (overwrite) mỗi lần cập nhật

Chỉ biết trạng thái HIỆN TẠI

Mất toàn bộ lịch sử thay đổi

❓ "Tại sao score = 85?" → Không biết
 ❓ "Đã retry mấy lần?" → Không biết
 ❓ "Khi nào status thay đổi?" → Không biết

Event Sourcing: Lưu Events

1. SubmissionCreated
user=A, sql='SELECT ...'
2. SQLExecuted
result=timeout
3. SubmissionRetried
user=A, attempt=2
4. SQLExecuted
result=correct, score=85
5. SubmissionJudged
status=CORRECT, finalScore=85

✓ FULL HISTORY — Replay / Audit bất kỳ lúc nào

 LMS ứng dụng

Mỗi submission có nhiều events (created, executed, retried, judged). Replay để debug tại sao student bị sai.

⚖ Trade-off

Event store lớn hơn nhiều. Cần Snapshot (Hình 7.12b) để tránh replay quá nhiều events khi query state hiện tại.

Hình 7.12: Event Sourcing — lưu chuỗi events thay vì chỉ state hiện tại

Kleppmann trong [7, Ch.11] giải thích: Event Sourcing coi event log là **nguồn sự thật** (*source of truth*), còn state views (database tables, materialized views) là **derived data** — có thể rebuild bất kỳ lúc nào bằng cách replay events.

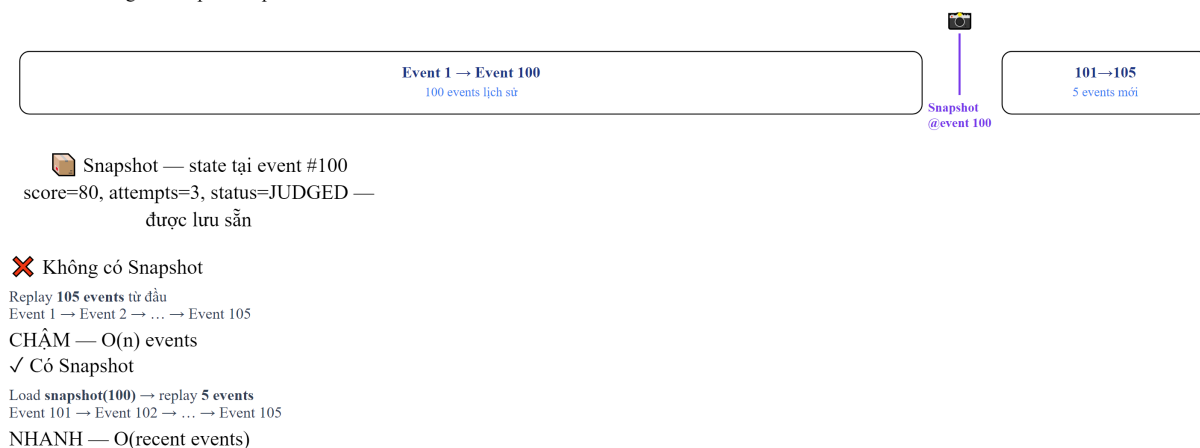
Bảng 7.11: Ưu và nhược điểm của Event Sourcing

Ưu điểm	Nhược điểm
Full audit trail — mọi thay đổi đều được ghi	Complexity — rebuild state cần replay, performance concern
Temporal queries — “state tại thời điểm T?”	Event schema evolution — events cũ format khác events mới
Debug dễ hơn — replay events để reproduce bugs	Eventual consistency — state views luôn “chậm hơn” events
Replay — rebuild state, tạo views mới	Storage — event store tăng vô hạn, cần snapshots
Kết hợp CQRS — events tự nhiên feed vào read models	Learning curve — tư duy khác hoàn toàn so với CRUD

7.6.3. Snapshot Pattern — Giải quyết Performance của Event Replay

Khi số lượng events tăng (1 submission có 5 events, 100,000 submissions = 500,000 events), replay toàn bộ để lấy state hiện tại trở nên **chậm không chấp nhận được**. **Snapshot pattern** giải quyết:

Event Sourcing — Snapshot Optimization



💡 Rule of thumb: tạo Snapshot mỗi N events (vd: N=100)

Snapshot không thay thế events — events vẫn được giữ nguyên cho audit/replay đầy đủ.

Hình 7.12b: Snapshot pattern — replay từ snapshot thay vì từ đầu

Bảng 7.12: Chiến lược snapshot

Chiến lược snapshot	Khi nào tạo	Trade-off
Mỗi N events	Sau mỗi 100 events	Đơn giản, predictable
Time-based	Mỗi 1 giờ	Phù hợp khi event rate đều
On-demand	Khi read request cần state	Lazy — chỉ tạo khi cần

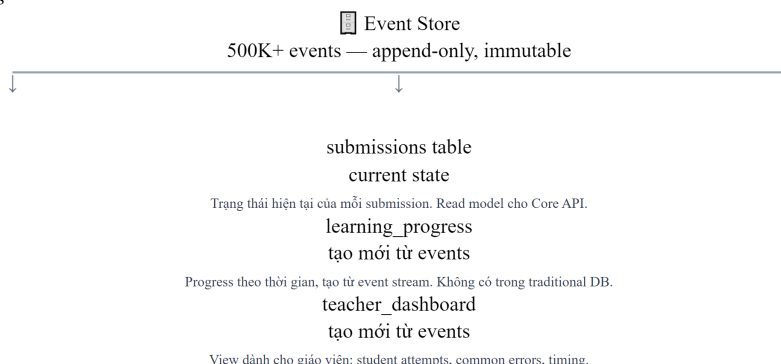
7.6.4. Projection Rebuilds — Tạo lại Views từ Events

Một trong những lợi ích mạnh nhất của Event Sourcing: **tạo views mới mà không cần migration**. Ví dụ trong KBLab:

- Tháng 1: chỉ cần bảng `submissions` (score per question)
- Tháng 6: cần thêm bảng `learning_progress` (trend score theo thời gian per student)

- Với CRUD: cần database migration + backfill data (khó/không thể nếu data bị ghi đè)
- Với Event Sourcing: **replay toàn bộ events** qua projection mới → `learning_progress` tự động được tạo với full history

Event Store + Projections



Projections

Đọc events và tạo read-model phù hợp cho từng use case. Projection có thể khác nhau hoàn toàn về schema.

Lợi ích chính

Thêm projection mới **không thay đổi event store**. Replay events để tạo projection bất kỳ lúc nào — kể cả retroactively.

Temporal Query

Có thể rebuild state *tại bất kỳ thời điểm nào* trong quá khứ bằng cách replay đến event đó.

Hình 7.13: Projection Rebuilds — tạo views mới từ events mà không cần migration

Bảng 7.13: Event Store — các công cụ phổ biến

Event Store	Mô tả	Phù hợp khi
EventStoreDB	Database chuyên dụng cho Event Sourcing (by Greg Young)	Cần full ES tính năng: subscriptions, projections, temporal queries
Axon Framework	Java framework cho CQRS + ES, tích hợp Spring	Team Java/Spring, cần opinionated framework
Kafka (with caveats)	Message broker với infinite retention	Đã dùng Kafka, chỉ cần event streaming (không cần query by aggregate)
PostgreSQL + custom	Relational DB với events table	Team nhỏ, muốn đơn giản, dùng DB đã quen

💡 Tip — Kafka ≈ Event Store?

Kafka với retention vĩnh viễn (infinite retention) có thể hoạt động như event store. KBLab đã dùng Kafka cho submission pipeline (Ch.5). Về lý thuyết, messages trên topic `submissions` và `judge-results` chính là chuỗi events. Tuy nhiên, Kafka được thiết kế là message broker, không phải event store chuyên dụng — querying events theo entity ID khó hơn so với EventStoreDB hoặc Axon. Kleppmann trong [7, Ch.11] phân tích: Kafka phù hợp cho **event streaming** (*process events real-time*), còn Event Sourcing cần **event store** (*query events by aggregate*). Hai use cases liên quan nhưng khác nhau.

7.6.5. Event Schema Evolution — Versioning Events

Events, giống API (Ch.3), cần **schema evolution** khi business thay đổi. Nhưng khác API: events đã lưu *không thể sửa* — event store là append-only. Hai chiến lược:

1. Upcasting — Khi load events cũ, transform sang format mới trước khi apply:

Listing 7.1: Upcasting — transform events cũ sang format mới

```
// Event v1 (2024): { type: "SubmissionCreated", userId: "123", sql: "SELECT ..." }
// Event v2 (2025): { type: "SubmissionCreated", userId: "123", sql: "SELECT ...",
language: "SQL" }
// Upcaster: nếu event thiếu "language" → set default "SQL"
```

2. Weak schema — Events chứa extra fields mà consumer bỏ qua nếu không hiểu (Tolerant Reader — Ch.3). Thêm fields mới → events cũ vẫn valid.

⚠ Nguyên tắc — Events là Facts, không phải Commands

Events mô tả *điều đã xảy ra* (past tense: `SubmissionJudged`), không phải *yêu cầu* (`JudgeSubmission`). Events immutable — không bao giờ sửa/xóa event đã lưu. Nếu cần “undo”, thêm event mới: `SubmissionResultCorrected`. Richardson trong [2a, Ch.6] nhấn mạnh: đây là *business decision*, không phải *technical decision* — domain experts nên đặt tên events.

7.6.6. Khi nào dùng Event Sourcing?

Bảng 7.14: Khi nào dùng Event Sourcing

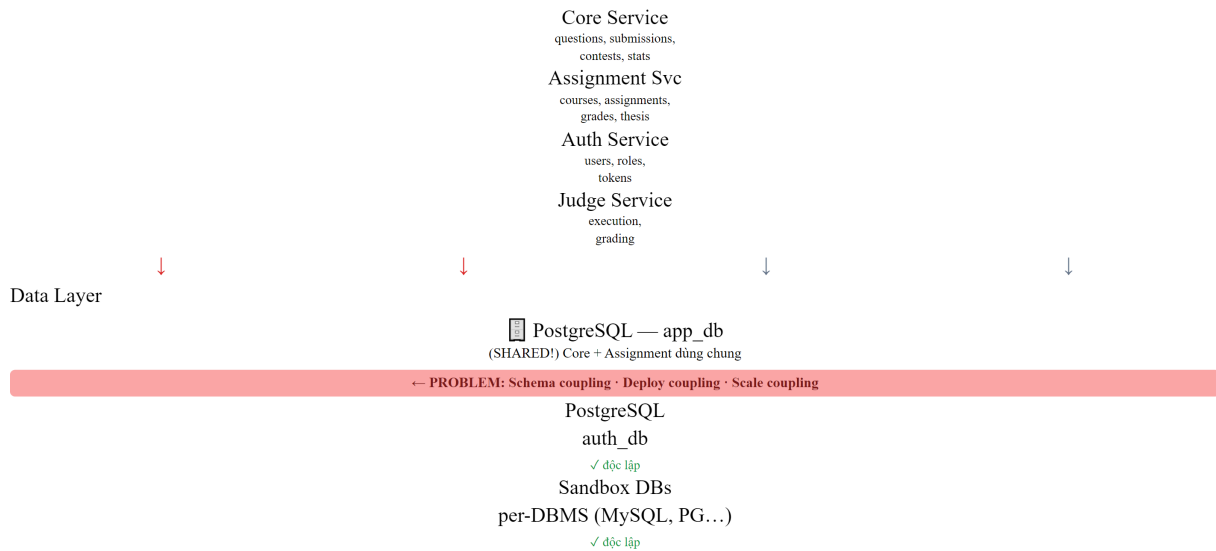
Scenario	Phù hợp?	Lý do
Audit requirements (tài chính, compliance)	✅ Rất phù hợp	Cần lịch sử đầy đủ, không thể xóa
Temporal analysis (phân tích xu hướng)	✅ Phù hợp	Query “trạng thái tại thời điểm X” tự nhiên
CRUD đơn giản (quản lý users, settings)	❌ Không	Overhead quá lớn cho lợi ích nhỏ
Domain phức tạp (đơn hàng, booking)	✅ Phù hợp	Business events map trực tiếp vào domain events
KBLab submission tracking	⚠ Tùy	Có giá trị cho learning analytics, nhưng hiện tại team nhỏ → không ưu tiên

7.7. Case Study: Quản lý dữ liệu trong KBLab

7.7.1. Hiện trạng

Phân tích source code KBLab cho thấy kiến trúc dữ liệu hiện tại:

KBLab — Kiến trúc dữ liệu hiện tại (vấn đề)
Application Layer



⚠ Vấn đề: Core và Assignment SHARED app_db
Cần tách theo lộ trình Hình 7.5 (5 bước). Không thể deploy Core mà không ảnh hưởng Assignment.

Hình 7.14: Kiến trúc dữ liệu hiện tại của KBLab — shared database là gap chính

Quan sát chính:

1. **Auth Service** — đã tách database riêng (auth_db) → ☒ đúng database-per-service
2. **Core + Assignment** — chia sẻ app_db → ☒ vi phạm database-per-service
3. **Judge Service** — không có database riêng, dữ liệu transient → ☒ stateless (OK)
4. **Cross-service data** — dùng Feign calls → ☒ runtime dependency

7.7.2. Phân tích cross-service query patterns

KBLab sử dụng hai pattern để truy vấn dữ liệu xuyên service: (1) **Interface Projections** (Spring Data JPA) — query trả về lightweight projection thay vì full entity, giảm data transfer, (2) **Feign calls** — gọi API service khác khi cần data ngoài boundary.

7.7.3. Từ hiện trạng đến best practice

Bảng 7.15: Tổng hợp vấn đề data management trong KBLab và chiến lược migration

#	Vấn đề	Hiện trạng KBLab	Chiến lược migration
1	Shared database	Core + Assignment cùng app_db	Separate schema → Full split
2	Cross-service joins	Cross-table query trực tiếp	Thay bằng Feign hoặc event-based copy
3	Leaderboard query	SQL JOIN trực tiếp	CQRS — pre-computed read model
4	Submission history	CRUD (chỉ lưu current state)	Event Sourcing (cho analytics)
5	User reference data	Feign call mỗi khi cần	Local copy via events
6	Grade Scheme / Attendance / MCQ Daily / CLO	Nhiều read model học vụ cần tổng hợp dữ liệu khóa học, lớp, bài làm, chuẩn đầu ra	Xác định owner rõ ràng, duplicate read-only qua events hoặc materialized views
7	Schema migration không đồng nhất	Java services thiên về JPA auto-DDL; Go services trong DevOps Lab phù hợp hơn với golang-migrate	Chuẩn hóa migration versioned scripts, không dựa vào auto-DDL ở production

7.7.4. Đề xuất migration path

Phase 1 — Schema Separation (ưu tiên cao, effort thấp): Tách app_db thành schema core_schema và assignment_schema. Mỗi service chỉ có quyền truy cập schema riêng.

Phase 2 — API-based Data Access (effort trung bình): Thay cross-schema queries bằng Feign calls — Assignment gọi Core API GET /api/questions/batch thay vì query trực tiếp core_schema.questions.

Phase 3 — Event-based Data Duplication (effort trung bình): Core publish QuestionUpdated events khi câu hỏi thay đổi. Assignment consume và lưu local copy (chỉ reference data: title, difficulty). Giảm runtime dependency cho read operations.

Phase 4 — CQRS cho Leaderboard (effort cao, giá trị lớn cho contest mode): Tách leaderboard thành read model riêng. Kafka events (ScoreUpdated) feed vào denormalized leaderboard table. Query trả kết quả trong <10ms thay vì complex JOIN.

Phase 5 — Read models cho learning analytics (khi mở rộng học vụ): Grade Scheme, Attendance, MCQ Daily và CLO nên được xem như read models học vụ. Owner của dữ liệu gốc vẫn nằm trong bounded context tương ứng; bảng tổng hợp chỉ phục vụ query nhanh cho giảng viên và báo cáo, không trở thành nguồn sự thật thứ hai.

⚠ Lưu ý —

1. **Chia database quá sớm** — Tách database trước khi hiểu rõ data access patterns. Hậu quả: phát hiện hai service cần JOIN data liên tục, phải build event pipeline phức tạp cho thứ trước đây chỉ cần SQL JOIN. *Phòng tránh*: bắt đầu với separate schema (cùng DB server), theo dõi cross-schema queries 2-4 tuần, rồi mới quyết định tách hoàn toàn.
2. **Dùng CQRS cho mọi thứ** — Áp dụng CQRS/Event Sourcing cho CRUD đơn giản (quản lý users, settings). Hậu quả: complexity tăng 3-5x, team mất thời gian maintain event pipeline cho data thay đổi 1 lần/tuần. *Phòng tránh*: CQRS chỉ khi read pattern phức tạp (cross-service join, high read volume, real-time views). CRUD là đủ cho 80% use cases.
3. **Duplicate data nhưng không xác định source of truth** — Copy data giữa services mà không rõ “ai sở hữu data này?” Hậu quả: hai services cùng sửa data → conflict, không biết bản nào đúng. *Phòng tránh*: mỗi data entity chỉ có **duy nhất một service sở hữu** — các bản copy khác là read-only replicas, chỉ update qua events từ owner.
4. **Quên xử lý replication lag** — Sau khi write vào service A, ngay lập tức read từ read model (service B) và thấy data cũ. Hậu quả: user nộp bài thành công nhưng leaderboard chưa cập nhật → confusion. *Phòng tránh*: UI pattern “optimistic update” — hiển thị kết quả dự kiến ngay, cập nhật khi event đến (đã thảo luận trong Ch.6 §6.5).

🌐 **Trực quan hóa tương tác (Interactive Demo)**

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/cqrs-event-sourcing.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Mô hình CQRS và Event Sourcing**.

7.8. Tổng kết

Quản lý dữ liệu là bài toán phức tạp nhất khi chuyển từ monolith sang microservices — phức tạp hơn cả việc tách application code. Database-per-service không chỉ là nguyên tắc kỹ thuật mà là **điều kiện tiên quyết** cho independent deployability — mục tiêu cốt lõi của microservices. CAP Theorem nhắc nhở: trong hệ thống phân tán, consistency và availability luôn phải đánh đổi khi có network partition — mọi quyết định data management đều mang theo trade-off này.

Năm chiến lược tách database (view → wrapping service → separate schema → data transfer → full split) cho phép migration **tăng dần** — không cần “big bang”. Với team nhỏ như KBLab, bắt đầu từ separate schema là bước đi thực tế nhất.

Data duplication không phải anti-pattern — đây là **trade-off có chủ đích** để giảm runtime coupling. Nguyên tắc: mỗi data chỉ có một source of truth, các bản copy là read-only replicas fed bằng events. Newman đã nói rõ: “Duplication is far better than coupling.”

CQRS (mở rộng từ nguyên tắc CQS của Meyer) tách read model và write model — giải quyết bài toán cross-service queries và high-volume reads. Event Sourcing bổ sung bằng cách lưu lịch sử đầy đủ thay vì chỉ state hiện tại. Cả hai đều mạnh mẽ nhưng phức tạp — chỉ áp dụng khi có nhu cầu cụ thể, không phải mặc định.

Một bài toán liên quan chặt chẽ là **distributed transactions** — khi một business operation cần thay đổi data ở nhiều services. Saga pattern (đã thảo luận chi tiết ở Ch.6) là giải pháp:

chuỗi local transactions + compensating actions thay vì distributed ACID transaction. Saga và data management bổ trợ nhau: database-per-service tạo ra nhu cầu Saga, còn CQRS/Event Sourcing cung cấp event pipeline mà Saga sử dụng.

Phân tích KBLab cho thấy gap nghiêm trọng nhất là shared database giữa Core và Assignment — vi phạm database-per-service và gây deploy coupling. Migration path rõ ràng: separate schema → API wrapping → event-based duplication → CQRS cho leaderboard và learning analytics. Mỗi giai đoạn độc lập, có thể dừng ở bất kỳ giai đoạn nào khi “đủ tốt” cho ngữ cảnh.

Ở Chương 8, chúng ta sẽ chuyển sang **API Gateway** — single entry point cho hệ thống microservices: routing, authentication, rate limiting, và cách KBLab sử dụng Spring Cloud Gateway.

7.9. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Ch.4: Decomposing the Database, Database-per-service patterns
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.6: Event Sourcing; Ch.7: Implementing Queries (API Composition, CQRS)
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.5: Replication; Ch.6: Partitioning; Ch.7,9: Transactions & CAP; Ch.11: Stream Processing, Event Sourcing

Sách bổ trợ:

4. [3] Ronnie Mitra, *Microservices: Up and Running* — Ch.5: Data Delegate Pattern, Event Sourcing in practice
5. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — §4.5: CQS vs CQRS, Command Sourcing; Ch.5: CAP in real world, eventual consistency
6. [6] Eric Evans, *Domain-Driven Design* — Aggregate design → data ownership per bounded context

Chương liên quan:

- **Ch.6: Saga Pattern** — Distributed transactions, orchestration vs choreography, compensating actions. Saga giải quyết bài toán “thay đổi data ở nhiều services trong một business transaction” — complement trực tiếp cho database-per-service.

Nguồn trực tuyến:

- Martin Fowler, “CQRS” — martinfowler.com/bliki/CQRS.html
- Greg Young, “CQRS Documents” — cqrs.files.wordpress.com
- Confluent, “Event Sourcing with Kafka” — developer.confluent.io

PHẦN III

Hạ tầng & Vận hành

Hạ tầng và vận hành — Đưa hệ thống vào thế giới thực

Một kiến trúc Microservices tốt không chỉ nằm ở thiết kế dịch vụ, mà còn ở khả năng vận hành trong môi trường production. Phần III hoàn thiện bức tranh với API Gateway, bảo mật, chuyển đổi thực tế từ monolith, observability, và triển khai tự động. Các chương trong phần này có thể đọc tương đối độc lập, tùy theo nhu cầu của bạn.

Chương 8 API Gateway

Chương 9 Bảo mật Microservices

Chương 10 Chuyển đổi Thực tế — Từ Monolith đến Microservices

Chương 11 Observability

Chương 12 Triển khai & DevOps

8. API Gateway

“The API Gateway is the single entry point for all clients. It handles cross-cutting concerns so that individual services don’t have to.” — Chris Richardson, *Microservices Patterns* [2a]

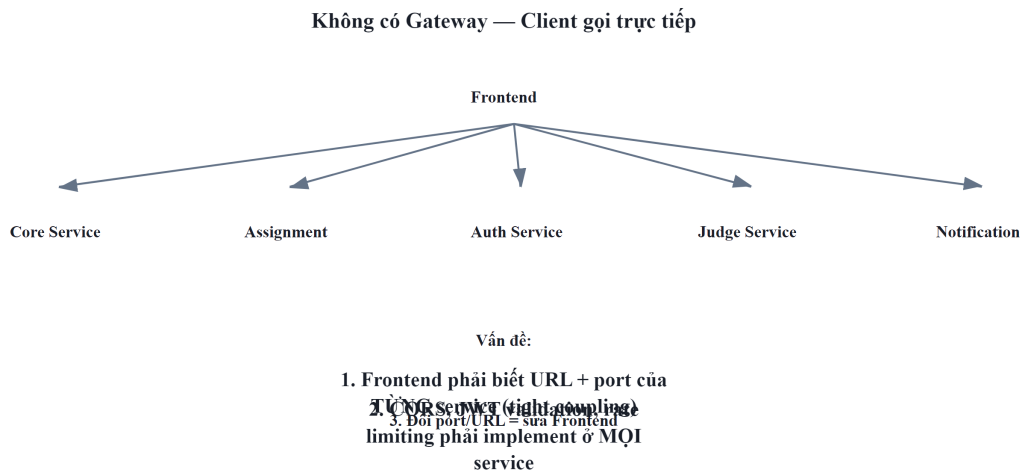
8.1. Bạn sẽ học được gì

- Hiểu tại sao microservices cần API Gateway và các vấn đề nó giải quyết
- Nắm vững API Gateway pattern và Backend for Frontend (BFF) pattern
- Sử dụng Spring Cloud Gateway (WebFlux) để triển khai gateway
- Cấu hình route với Eureka service discovery (load-balanced URIs)
- Thiết kế cross-cutting concerns tại gateway: authentication, CORS, rate limiting
- Phân tích kiến trúc gateway trong KBLab

8.2. API Gateway Pattern — Tại sao cần?

8.2.1. Vấn đề: client giao tiếp trực tiếp với nhiều services

Khi không có gateway, client (web, mobile) phải biết địa chỉ của *từng* microservice và gọi trực tiếp. Với KBLab gồm nhiều services và module lab, mỗi trang web có thể cần gọi nhiều API khác nhau:



Hình 8.1: Không có Gateway — client phải biết địa chỉ của từng service

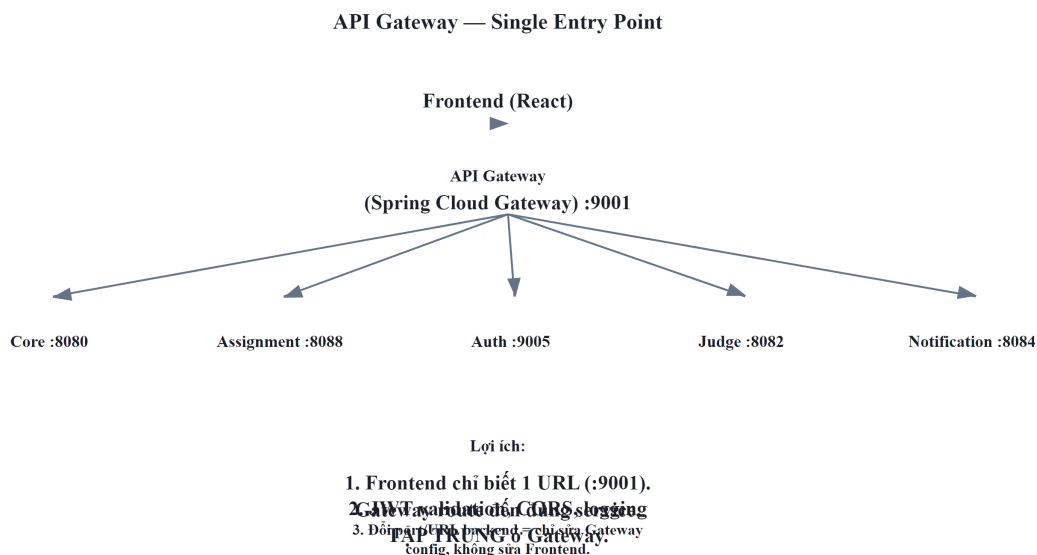
Richardson trong [2a, Ch.8] liệt kê năm vấn đề khi client gọi trực tiếp:

Bảng 8.1: Vấn đề khi client gọi trực tiếp microservices

#	Vấn đề	Hậu quả
1	Nhiều endpoints	Client phải biết URL của mọi service — coupling chặt
2	Giao thức khác nhau	Một số service dùng REST, một số dùng gRPC, WebSocket — client phải handle tất cả
3	Cross-cutting concerns phân tán	Mỗi service tự implement authentication, CORS, rate limiting — duplicate, inconsistent
4	Network không an toàn	Internal services bị expose ra Internet — attack surface lớn
5	API không phù hợp	Internal API thiết kế cho service-to-service, không tối ưu cho mobile (quá nhiều calls, payload lớn)

8.2.2. API Gateway pattern

API Gateway là **single entry point** — tất cả requests từ client đi qua gateway, gateway route đến đúng service:



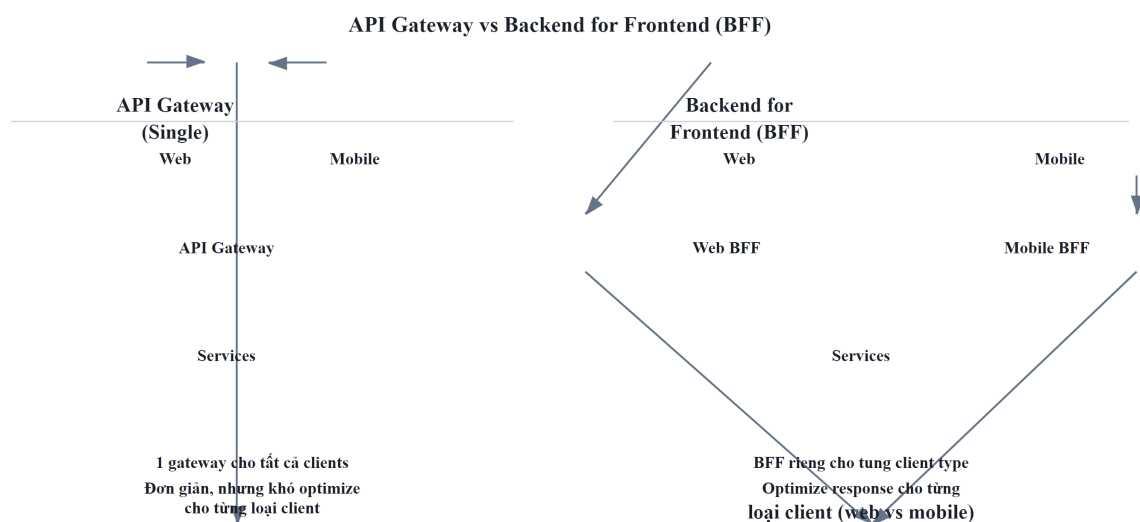
Hình 8.2: API Gateway — single entry point route đến từng service

Gateway xử lý **cross-cutting concerns tập trung**: authentication, CORS, rate limiting, logging, SSL termination. Services phía sau chỉ tập trung vào business logic — không cần biết CORS là gì.

Newman trong [4a, Ch.4] mô tả gateway là “smart pipe” duy nhất được phép trong microservices: các pipes giữa services nên “dumb” (simple routing), nhưng gateway — điểm tiếp xúc với client — cần xử lý cross-cutting concerns.

8.2.3. API Gateway vs BFF (Backend for Frontend)

Richardson trong [2a, Ch.8] phân biệt hai biến thể:



Hình 8.3: API Gateway (một gateway chung) vs BFF (gateway riêng cho từng client)

Bảng 8.2: API Gateway vs BFF — khi nào dùng gì

Pattern	Mô tả	Khi nào dùng
API Gateway	Một gateway cho tất cả clients	Team nhỏ, clients cần API tương tự
BFF	Gateway riêng cho mỗi loại client	Mobile cần API khác web (ít data, batched calls)

KBLab sử dụng **single API Gateway** cho LMS core — phù hợp vì các web frontends chính có API requirements tương tự. Với DevOps Lab, KBLab bổ sung một lớp router riêng theo hostname để điều hướng workspace/lab runtime; đây là gateway chuyên biệt cho một bounded context, không thay thế Spring Cloud Gateway của LMS core.

Khi nào cần BFF? BFF trở nên cần thiết khi different clients có **fundamentally different API needs** — không chỉ “filter bớt fields”:

Bảng 8.3: Kịch bản chọn Single Gateway vs BFF

Scenario	Single Gateway	BFF
Web + Web admin (API tương tự)	✓ Đủ	Over-engineering
Web + Mobile (API khác nhau)	✗ Mobile phải nhiều calls	✓ Mobile BFF aggregate
Web + IoT + Partner API	✗ Gateway quá phức tạp	✓ BFF per client type

Ví dụ: nếu KBLab thêm **mobile app** cho sinh viên, mobile cần: (1) **batched API** — màn hình “Dashboard” cần gọi 1 API trả về cả profile, recent submissions, leaderboard rank (thay vì 3 calls — mobile network chậm hơn), (2) **reduced payload** — mobile không cần HTML-ready data, chỉ cần raw data nhẹ, (3) **push notification integration** — gateway riêng cho mobile xử lý device tokens.

Newman trong [4a, Ch.4] khuyến nghị: BFF nên **owned by frontend team** — team mobile viết Mobile BFF, team web viết Web BFF. Mỗi BFF là thin layer: nhận request từ client → gọi downstream services → aggregate + transform → trả về format phù hợp cho client đó.

8.3. Spring Cloud Gateway — Reactive Gateway

8.3.1. Vấn đề: chọn gateway technology

Hai lựa chọn phổ biến nhất trong Spring ecosystem:

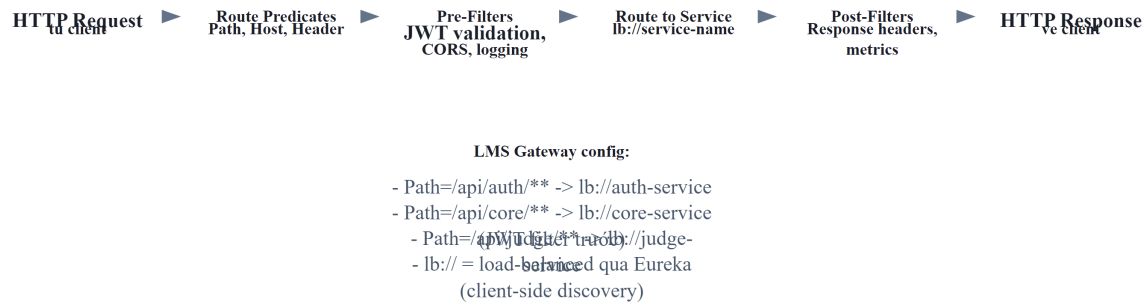
Bảng 8.4: Spring Cloud Gateway vs Netflix Zuul

	Spring Cloud Gateway	Netflix Zuul (1.x)
Model	Reactive (WebFlux, non-blocking)	Servlet (blocking, thread-per-request)
Performance	Cao (ít threads, nhiều connections)	Thấp hơn (thread pool giới hạn)
Status	Active, recommended	Deprecated (Netflix không maintain)
WebSocket	Native support	Không hỗ trợ
Ecosystem	Spring Cloud tích hợp sẵn	Legacy

LMS chọn **Spring Cloud Gateway** — lựa chọn đúng vì cần WebSocket support (cho notification push) và reactive performance.

8.3.2. Kiến trúc Spring Cloud Gateway

Spring Cloud Gateway — Request Pipeline



Hình 8.4: Kiến trúc Spring Cloud Gateway — Predicates, Filters, Routes

Bảng 8.5: Ba khái niệm cốt lõi của Spring Cloud Gateway

Concept	Mô tả	Ví dụ KBLab
Route	Mapping: predicate → URI đích	/api/core/** → lb://core-service
Predicate	Điều kiện match request	Path=/api/core/**, Method=GET, POST
Filter	Xử lý request/response trước/sau routing	JwtRequestFilter, AddRequestHeader

8.3.3. Dependency

Gateway sử dụng `spring-cloud-starter-gateway` (WebFlux) + `spring-cloud-starter-netflix-eureka-client`. **Lưu ý:** Gateway dựa trên WebFlux (reactive, non-blocking) — *không thể* dùng chung với `spring-boot-starter-web` (servlet, blocking). Thêm `spring-boot-starter-web` vào Gateway project → conflict, gateway không khởi động.

8.4. Route Configuration với Eureka

8.4.1. Vấn đề: routes hardcoded hay dynamic?

Cách đơn giản nhất: hardcoded URL cho mỗi service trong gateway config. Nhưng khi service scale (3 instances) hoặc di chuyển (deploy lên container mới), URL thay đổi — ai cập nhật gateway?

Giải pháp: kết hợp gateway routing với **Eureka service discovery** (đã học ở Ch.4). Gateway không cần biết IP:port cụ thể — chỉ cần tên service, Eureka giải quyết phần còn lại.

8.4.2. LMS Gateway Route Configuration

Listing 8.1: LMS Gateway route configuration (YAML + Eureka)

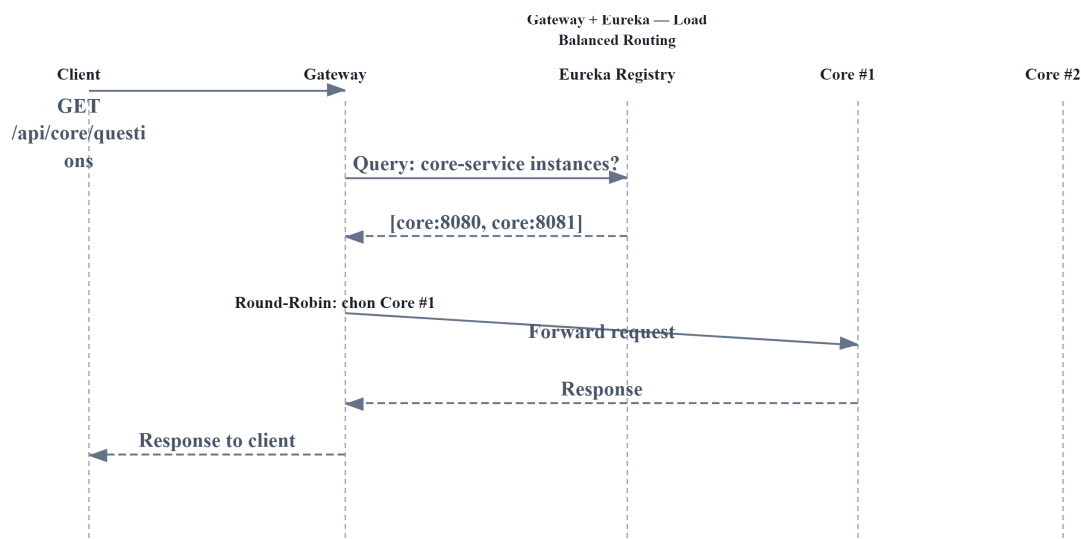
```
# application-lb.yml – LMS Gateway routing configuration
spring:
  cloud:
    gateway:
      routes:
        # Core Service – questions, submissions, contests
        - id: core-service
          uri: lb://core-service          # lb:// = Eureka lookup + load balance
          predicates:
            - Path=/api/core/**
          filters:
            - StripPrefix=2              # /api/core/questions → /questions

        # Assignment Service – courses, grades
        - id: assignment-service
          uri: lb://assignment-service
          predicates:
            - Path=/api/assignment/**
          filters:
            - StripPrefix=2

        # Auth Service – login, register
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2

        # Notification – WebSocket endpoint
        - id: notification-ws
          uri: lb://notification-service
          predicates:
            - Path=/ws/**
```

8.4.3. Cách lb:// hoạt động



Hình 8.5: Luồng lb:// — Eureka lookup + load balance + route

lb://core-service thực hiện ba bước tự động:

1. **Lookup:** query Eureka tìm tất cả instances có tên `core-service`
2. **Load balance:** chọn instance bằng Spring Cloud LoadBalancer (round-robin mặc định)
3. **Forward:** gửi request đến instance được chọn

StripPrefix=2 loại bỏ 2 phần đầu tiên của path: `/api/core/questions` → `/questions`. Nhờ đó, service không cần biết nó được mount ở `/api/core/` — service chỉ cần handle path của chính nó.

⚠ Nguyên tắc — Service không biết Gateway

Services phía sau gateway *không nên biết* gateway tồn tại. Mỗi service handle path riêng (`/questions`, `/users`), gateway thêm prefix và route. Điều này đảm bảo: (1) service test được standalone (không cần gateway), (2) đổi route structure ở gateway không ảnh hưởng service code, (3) services portable — có thể deploy sau gateway khác (NGINX, Kong) mà không đổi code.

8.5. Cross-Cutting Concerns tại Gateway

8.5.1. Vấn đề: mỗi service tự xử lý authentication, CORS, logging

Nếu mỗi service tự validate JWT token, tự configure CORS, tự implement rate limiting — code bị duplicate ở nhiều services, mỗi lần thay đổi policy phải update tất cả. Đây chính là vấn đề gateway giải quyết: **tập trung cross-cutting concerns tại một điểm duy nhất.**

8.5.2. Authentication — JWT Validation tại Gateway

KBLab implement JWT validation ở gateway thông qua custom `GatewayFilter`:

Listing 8.2: JWT validation filter tại Gateway

```
// Gateway JWT Filter – validate token trước khi route
@Component
public class JwtRequestFilter implements
GatewayFilterFactory<JwtRequestFilter.Config> {

    private final JwtUtil jwtUtil;

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            String path = exchange.getRequest().getURI().getPath();

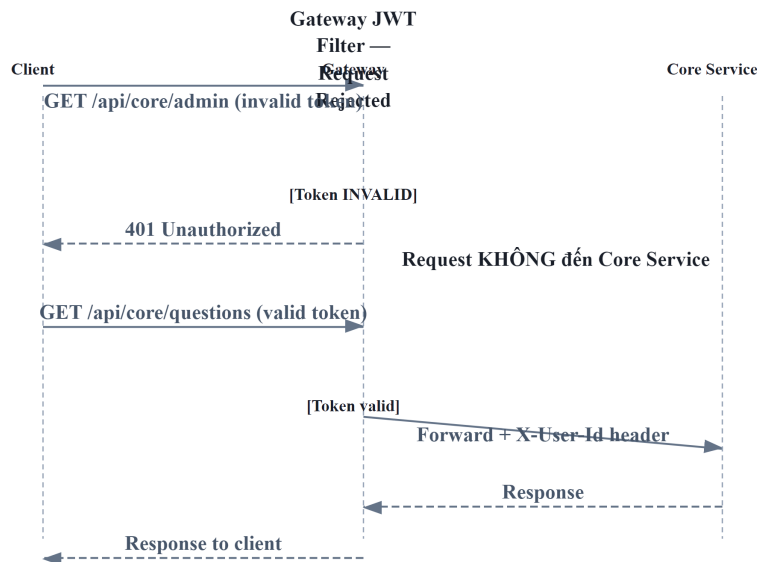
            // Skip auth cho public endpoints
            if (isPublicEndpoint(path)) {
                return chain.filter(exchange);
            }

            // Extract JWT từ Authorization header
            String token = extractToken(exchange.getRequest());
            if (token == null || !jwtUtil.validateToken(token)) {
                exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
                return exchange.getResponse().setComplete();
            }

            // Forward user info tới downstream services
            ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
                .header("X-User-Id", jwtUtil.getUserId(token))
                .header("X-User-Roles", jwtUtil.getRoles(token))
                .build();

            return chain.filter(exchange.mutate().request(mutatedRequest).build());
        };
    }
}
```

Luồng xử lý:



Hình 8.6: Luồng JWT validation tại Gateway — claims propagation qua trusted headers

Services phía sau nhận user info qua **custom headers** (X-User-Id, X-User-Roles) — không cần validate JWT lại. Đây là pattern **claims-based identity propagation**: gateway validate token, services tin tưởng gateway (vì traffic internal).

8.5.3. CORS — Cross-Origin Resource Sharing

CORS tại gateway = **một nơi duy nhất** quản lý origins, methods, headers. Cấu hình `globalcors` trong Spring Cloud Gateway cho phép khai báo `allowedOrigins` (domains hợp lệ), `allowedMethods`, `allowCredentials` — services phía sau không cần CORS config vì gateway đã xử lý.

❗ Phân tích — KBLab CORS

KBLab từng có cấu hình `allowedOrigins`: "*" — cho phép *mọi* origin gọi API. Trong development, đây là cách đơn giản để tránh CORS errors. Trong production, đây là **rủi ro bảo mật**: bất kỳ website nào có thể gọi API bằng credentials của user (CSRF attack). **Migration**: (1) liệt kê rõ các origins hợp lệ theo môi trường, không public domain cụ thể trong sách, (2) thêm `allowCredentials: true` cho cookie-based auth, (3) test kỹ với mobile app nếu có.

8.5.4. Rate Limiting

Rate limiting ngăn một client gửi quá nhiều requests — bảo vệ services khỏi abuse hoặc DDoS. Spring Cloud Gateway hỗ trợ sẵn `RequestRateLimiter` filter kết hợp Redis: cấu hình `replenishRate` (requests/giây), `burstCapacity` (burst tối đa), và `KeyResolver` (rate limit theo user, IP, hoặc route).

Bảng 8.6: Chiến lược rate limiting

Chiến lược rate limit	Mô tả	Use case
Per user	Mỗi user N requests/giây	Ngăn user abuse
Per IP	Mỗi IP address N requests/giây	Ngăn anonymous abuse
Per route	Mỗi endpoint N requests/giây	Bảo vệ heavy endpoints
Global	Tổng requests hệ thống	Bảo vệ infrastructure

8.5.4.1. Rate Limiting Implementation với Redis

Spring Cloud Gateway sử dụng **Token Bucket algorithm** (Redis-backed) — mỗi user có một “bucket” chứa tokens, mỗi request tiêu thụ 1 token, tokens được bổ sung theo `replenishRate`:

Listing 8.3: Rate limiting configuration cho submission endpoint

```
# application.yml – Rate limiting cho endpoint chấm bài
spring:
  cloud:
    gateway:
      routes:
        - id: core-service-rate-limited
          uri: lb://core-service
          predicates:
            - Path=/api/core/submissions/**
          filters:
            - StripPrefix=2
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5    # 5 requests/giây
                redis-rate-limiter.burstCapacity: 10  # Burst tối đa 10
                redis-rate-limiter.requestedTokens: 1  # 1 token/request
                key-resolver: "#{@userKeyResolver}"    # Rate limit theo user

  data:
    redis:
      host: localhost
      port: 6379
```

Listing 8.4: KeyResolver — xác định rate limit theo userId từ JWT claims

```
@Configuration
public class RateLimitConfig {

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> {
            // Lấy userId từ header đã được JWT filter inject
            String userId = exchange.getRequest().getHeaders()
                .getFirst("X-User-Id");
            if (userId != null) {
                return Mono.just(userId);          // Rate limit per user
            }
            // Fallback: rate limit per IP cho anonymous requests
            return Mono.just(
                exchange.getRequest().getRemoteAddress()
                    .getAddress().getHostAddress()
            );
        };
    }
}
```

```

    );
  };
}
}

```

Khi user vượt limit, Gateway tự động trả **HTTP 429 Too Many Requests** — client nhận thông báo rõ ràng, không cần services phía sau xử lý.

💡 Tip — Rate limit cho contest mode

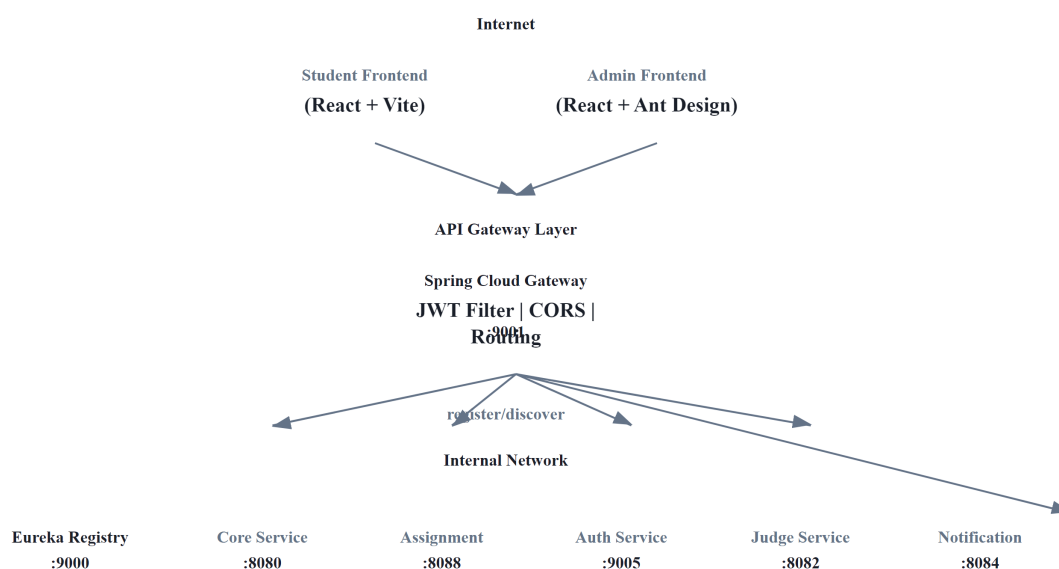
Trong contest mode, submission rate cao hơn bình thường (100+ students submit cùng lúc). Cân nhắc: (1) set rate limit cao hơn cho route `/submissions/**` trong contest, (2) hoặc dùng **per-route rate limit** riêng cho contest endpoints, (3) Redis atomic operations đảm bảo đếm chính xác ngay cả khi concurrent requests cao.

8.5.5. Logging & Tracing

Gateway là điểm lý tưởng để gắn **correlation ID** — unique ID theo dõi request xuyên suốt hệ thống. Pattern: **GlobalFilter** tại gateway kiểm tra header `X-Correlation-Id`, nếu chưa có thì generate UUID mới, gắn vào request → truyền qua mọi downstream service. Khi debug, grep logs bằng correlation ID để thấy *toàn bộ* journey của request (xem thêm Ch.11 Observability).

8.6. Case Study: Gateway trong KBLab

8.6.1. Kiến trúc tổng thể



Hình 8.7: Kiến trúc tổng thể KBLab — Gateway là single entry point

KBLab có hai kiểu gateway/router đáng phân biệt:

- **Spring Cloud Gateway** cho LMS core: route API theo path, validate JWT, CORS, rate limiting, correlation ID.

- **Go hostname-based router** cho DevOps Lab: route workspace/lab runtime theo hostname/subdomain nội bộ, tích hợp với k3s/Sysbox ở lớp hạ tầng.

8.6.2. Phân tích configuration

Bảng 8.7: Phân tích configuration Gateway trong KBLab

Aspect	Hiện trạng KBLab	Best Practice	Gap
Routing	lb:// URIs qua Eureka cho Java services; hostname routing cho DevOps Lab	✅ Đúng theo từng boundary	Cần tài liệu hóa route ownership
JWT Validation	Custom JwtRequestFilter	✅ Đúng (validate tại edge)	—
CORS	allowedOrigins: "*"	❌ Nên restrict	Liệt kê origins cụ thể
Rate Limiting	Không có	❌ Nên có	Redis + RequestRateLimiter
SSL/TLS	Không ở gateway level	⚠️ Tùy deployment	Thường terminate tại NGINX/LB
Correlation ID	Không có	⚠️ Nên thêm	GlobalFilter gắn UUID
Path Rewriting	StripPrefix filters	✅ Đúng	—
WebSocket	Route tới notification service	✅ Đúng	—
Health Check	Actuator endpoints	✅ Đúng	—

8.6.3. Vấn đề JWT Version Inconsistency

Một vấn đề đáng chú ý trong KBLab: gateway sử dụng **JJWT 0.11.5** (API mới: `parserBuilder()`) trong khi các services khác sử dụng **JJWT 0.9.1** (API cũ: `parser()`). Với HS256 đơn giản, hai versions tương thích ở happy path. Tuy nhiên, khi upgrade hoặc thêm RS256, version mismatch có thể gây inconsistent validation — gateway accept nhưng service reject, hoặc ngược lại.

📌 Phân tích — JWT library version inconsistency

Gateway dùng JJWT 0.11.5, services dùng 0.9.1. Với HS256 symmetric key đơn giản, hai versions tương thích ở happy path. Tuy nhiên, khi upgrade hoặc thêm tính năng (RS256, token refresh), version mismatch gây lỗi khó debug. **Migration path:** (1) thống nhất JJWT version trong parent POM, (2) extract JWT logic vào shared library đã thống nhất, (3) dài hạn: cân nhắc Spring Security OAuth2 Resource Server (built-in JWT support, không cần custom JwtUtil).

8.6.4. Đề xuất migration

Phase 1 — Security Hardening (ưu tiên cao, effort thấp):

- Restrict CORS origins (liệt kê frontend URLs cụ thể)
- Thống nhất JJWT version

- Tài liệu hóa boundary giữa Spring Cloud Gateway và Go hostname router

Phase 2 — Observability (ưu tiên trung bình):

- Thêm X-Correlation-Id GlobalFilter
- Request/response logging tại gateway

Phase 3 — Protection (ưu tiên trung bình):

- Rate limiting per user (Redis + RequestRateLimiter)
- Request size limits cho file upload endpoints

⚠ Lưu ý —

1. **Đưa business logic vào gateway** — Gateway xử lý data transformation, validation rules, hoặc orchestrate calls giữa services. Hậu quả: gateway trở thành monolith mới — mọi thay đổi business đều phải deploy gateway. *Phòng tránh*: gateway chỉ handle cross-cutting concerns (auth, CORS, routing, rate limiting). Business logic thuộc về services.
2. **Validate JWT ở cả gateway lẫn services** — Mỗi service vẫn tự validate JWT token dù gateway đã validate. Hậu quả: duplicate logic, latency tăng (mỗi request validate 2 lần), inconsistency khi JWT library version khác nhau. *Phòng tránh*: gateway validate JWT và truyền claims qua trusted headers (X-User-Id). Services tin tưởng gateway (traffic internal, không expose ra Internet).
3. **Không có fallback khi gateway down** — Gateway là single point of entry = single point of failure. Hậu quả: gateway crash → toàn bộ hệ thống unreachable. *Phòng tránh*: deploy gateway HA (nhiều instances + load balancer phía trước), cấu hình health checks, tự động restart.
4. **CORS allowAll trong production** — Cho phép mọi origin gọi API bằng user credentials. Hậu quả: rủi ro CSRF, bất kỳ website nào đều có thể thao tác API thay mặt user. *Phòng tránh*: liệt kê rõ origins hợp lệ, test kỹ interaction giữa allowed origins và credentials.

🌐 Trực quan hóa tương tác (Interactive Demo)

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/api-gateway-routing.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Cơ chế định tuyến của API Gateway**.

Ngoài ra, bạn cũng có thể xem minh họa về **Service Discovery & Registry** tại file `code/interactive/service-discovery.html`.

8.7. Tổng kết

API Gateway giải quyết một trong những thách thức cơ bản nhất khi client tương tác với hệ thống microservices: thay vì biết địa chỉ của từng service, client chỉ cần biết một endpoint duy nhất. Gateway đảm nhận routing, authentication, CORS, và các cross-cutting concerns — services phía sau tập trung vào business logic.

Spring Cloud Gateway (WebFlux) là lựa chọn hiện đại cho Java ecosystem — reactive, non-blocking, tích hợp sâu với Spring Cloud (Eureka, Circuit Breaker). Route configuration với `lb://` URIs kết hợp service discovery và load balancing tự động — services có thể scale mà không cần thay đổi gateway config.

Cross-cutting concerns tại gateway — JWT validation, CORS, rate limiting, correlation ID — là đầu tư giúp hệ thống bảo mật, dễ debug, và dễ vận hành. Nguyên tắc: gateway xử lý infrastructure concerns, services xử lý business logic — ranh giới rõ ràng, trách nhiệm rõ ràng.

Phân tích KBLab cho thấy gateway architecture cơ bản đúng (Spring Cloud Gateway + Eureka + JWT cho LMS core, Go hostname router cho DevOps Lab), nhưng thiếu rate limiting, CORS quá mở ở một số môi trường, và JWT library version inconsistency. Migration path rõ ràng: hardening trước (CORS, JWT version), observability sau (correlation ID, logging), protection cuối (rate limiting).

Ở Chương 9, chúng ta sẽ đi sâu vào **bảo mật microservices** — JWT structure, OAuth2 integration, dual validation strategy, và RBAC trong KBLab.

8.8. Đọc thêm

Sách tham khảo chính:

- [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.8: External API Patterns (API Gateway, BFF)
- [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
- [1] Thomas Erl, *SOA Analysis & Design* — API Gateway trong enterprise SOA context

Sách bổ trợ:

- [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.8: External API Patterns (updated)
- [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.9: UI in EDA, BFF pattern

Nguồn trực tuyến:

- Spring Cloud Gateway reference — docs.spring.io/spring-cloud-gateway
- Netflix Zuul → Spring Cloud Gateway migration guide
- OWASP API Security Top 10 — owasp.org/www-project-api-security

9. Bảo mật Microservices

“Security is not an afterthought. In a distributed system, every service is a potential attack surface.” — Sam Newman, *Building Microservices* [4a]

9.1. Bạn sẽ học được gì

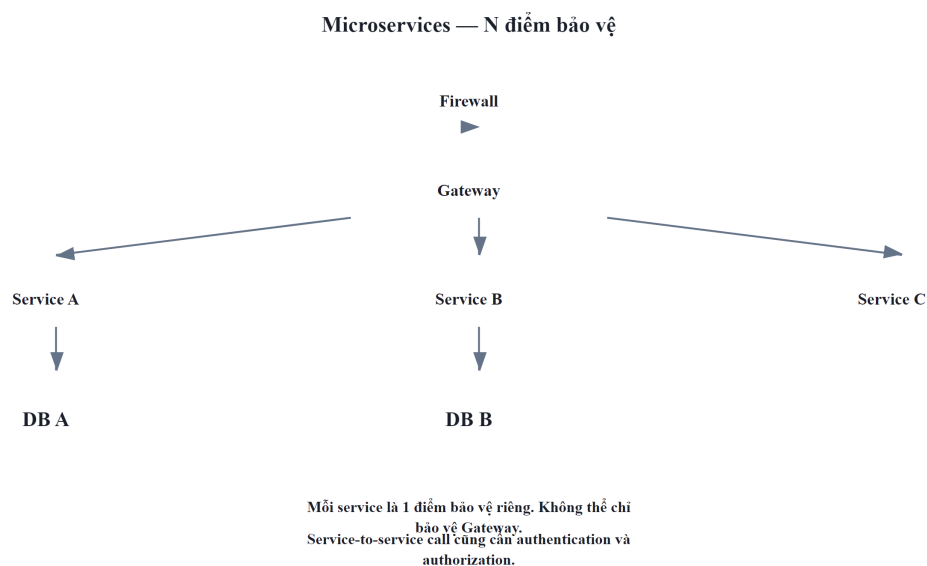
- Hiểu các thách thức bảo mật đặc thù của kiến trúc microservices
- Nắm vững cấu trúc JWT (JSON Web Token): header, payload, signature

- Phân biệt hai chiến lược validation: DB-based (full validation) vs claims-only (stateless)
- Hiểu OAuth2/OIDC flow và tích hợp external identity provider (Microsoft/Google)
- Thiết kế RBAC (Role-Based Access Control) cho hệ thống đa vai trò
- Phân tích kiến trúc bảo mật của KBLab

9.2. Security Challenges trong Microservices

9.2.1. Vấn đề: attack surface mở rộng

Trong monolith, bảo mật tập trung tại một điểm: một ứng dụng, một database, một authentication layer. Khi chuyển sang microservices, attack surface mở rộng theo số lượng services:



Hình 9.1: Monolith (1 điểm bảo vệ) vs Microservices (N điểm bảo vệ)

Newman trong [4a, Ch.9] liệt kê năm thách thức bảo mật đặc thù:

Bảng 9.1: Thách thức bảo mật trong microservices

#	Thách thức	Monolith	Microservices
1	Authentication	Một session store	Mỗi service cần verify identity — ai là user?
2	Authorization	Check in-process	Mỗi service cần check permissions — user có quyền gì?
3	Service-to-service trust	Không có (in-process)	Service A gọi B — B biết A có đáng tin?
4	Data in transit	Internal (in-process)	Network calls — cần encryption (TLS)
5	Secret management	Một config file	$N \text{ services} \times M \text{ secrets} = \text{quản lý phức tạp}$

9.2.2. Nguyên tắc “Defense in Depth”

Bảo mật microservices dựa trên nguyên tắc **defense in depth** — không dựa vào một lớp bảo vệ duy nhất:

Defense in Depth — 4 Layers

Layer 1: Network
TLS, firewall

Layer 2: Gateway
JWT validation, rate limiting

Layer 3: Service
authorization, input validation

Layer 4: Data
encryption at rest, access control

Mỗi layer là một barrier. Attacker phải vượt qua
TẤT CẢ layers.

LMS: TLS (HTTPS) -> Gateway
JWT filter -> Service @PreAuthorize
-> DB encryption

Hình 9.2: Defense in Depth — bảo vệ nhiều lớp từ network đến data

Trong thực tế, mức độ áp dụng tùy thuộc vào **trust boundary**. Nếu mọi service chạy trong private network (VPC) và chỉ gateway expose ra Internet, trust boundary nằm tại gateway — services bên trong có thể “trust” lẫn nhau ở mức độ nhất định.

⚠ Nguyên tắc — Zero Trust vs Pragmatic Trust

“Zero Trust” (mọi request đều verify, mọi service đều nghi ngờ) là lý tưởng. Trong thực tế, team nhỏ thường áp dụng **pragmatic trust**: gateway verify identity, services tin tưởng gateway. Newman trong [4a, Ch.9] ghi nhận: mức trust phù hợp phụ thuộc vào risk profile — hệ thống tài chính cần zero trust, hệ thống học thuật có thể pragmatic trust. KBLab là LMS học thuật → pragmatic trust là hợp lý nếu boundary nội bộ được kiểm soát rõ.

9.2.3. Advanced: mTLS, Secrets Management, OAuth2 Scopes

Khi hệ thống scale hoặc risk profile tăng (thêm payment, personal data), cần nâng cấp bảo mật:

mTLS (Mutual TLS) — Trong TLS thông thường, chỉ client verify server (browser verify HTTPS certificate). Trong mTLS, **cả hai bên verify nhau**: Service A gọi Service B → B verify certificate của A, A verify certificate của B. Ngăn service lạ gọi vào internal API.

Bảng 9.2: So sánh không có mTLS và có mTLS

Aspect	Không mTLS	Có mTLS
Internal calls	HTTP không mã hóa	TLS mã hóa + mutual authentication
Attacker vào network	Có thể gọi bất kỳ service	Bị từ chối vì thiếu certificate
Implementation	Đơn giản	Cần certificate management (cert-manager, Istio)

Trong KBLab, mTLS hiện chưa phải ưu tiên đầu tiên nếu các services chỉ giao tiếp trong boundary nội bộ được kiểm soát. Nhưng khi triển khai phân tán hơn hoặc có nhiều node/runtime, mTLS giúp ngăn lateral movement nếu một node bị compromise.

Secrets Management — Credentials (DB passwords, API keys, JWT secrets) không nên nằm trong code hoặc Docker images:

Bảng 9.3: Các cấp độ lưu trữ secrets

Level	Cách lưu secrets	Rủi ro
❌ Hardcode	password: "abc123" trong code	Lộ qua git history
⚠ Environment variables	.env file, Docker env	Lộ qua docker inspect, process listing
✅ Secrets manager	HashiCorp Vault, AWS Secrets Manager	Encrypted, audit log, rotation

KBLab hiện dùng environment variables ở một số môi trường (đủ cho scope academic) — nhưng JWT secret nên rotate định kỳ và không commit vào git.

OAuth2 Scopes — Hiện tại KBLab RBAC dùng roles (STUDENT, LECTURER, ADMIN). OAuth2 scopes mở rộng: thay vì “user có role ADMIN”, scope cho phép “client application X có quyền read:submissions nhưng không có write:grades”. Phù hợp khi KBLab expose API cho third-party (mobile app độc lập, integration với hệ thống khác).

9.3. JWT — Cấu trúc và Cơ chế

9.3.1. Vấn đề: sessions không scale trong microservices

Trong monolith, authentication thường dùng **server-side session**: user login → server tạo session → lưu trong memory/Redis → trả session ID (cookie). Mỗi request kèm cookie → server lookup session → biết user.

Trong microservices, session store tạo **centralized dependency**: mọi service phải query cùng session store → bottleneck, single point of failure. Nếu mỗi service có session store riêng, user phải login lại khi chuyển service.

9.3.2. JWT (JSON Web Token)

JWT giải quyết bằng cách mã hóa identity information vào *chính token* — stateless, không cần session store [4a, Ch.9]:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.      ← Header
eyJ1c2VySWQiOiJlc2VyLTZyMyIsInJvbGVzIjoiaU1R      ← Payload
VREVOVF9BRE1JTjkiImV4cCI6MTcxMDAwMDAwMH0.
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c      ← Signature
```

Ba phần của JWT:

JSON Web Token (JWT) — Cấu trúc

Header (algorithm, type)	eyJhbGciOiJIUzI1NiJ9
Payload (claims: userId, roles, exp)	eyJ1c2VySWQiOiJlc2VyLTZyMyIsInJvbGVzIjoiaU1R VREVOVF9BRE1JTjkiImV4cCI6MTcxMDAwMDAwMH0.
Signature (HMAC or RSA)	dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk

3 phần nối bởi dấu chấm: header.payload.signature

JWT chứa **user id, roles**
(ADMIN, EDITOR, USER, etc),
ENCIPHERED, không data sensitive
data trong payload.

Hình 9.3: Cấu trúc JWT — Header, Payload, Signature

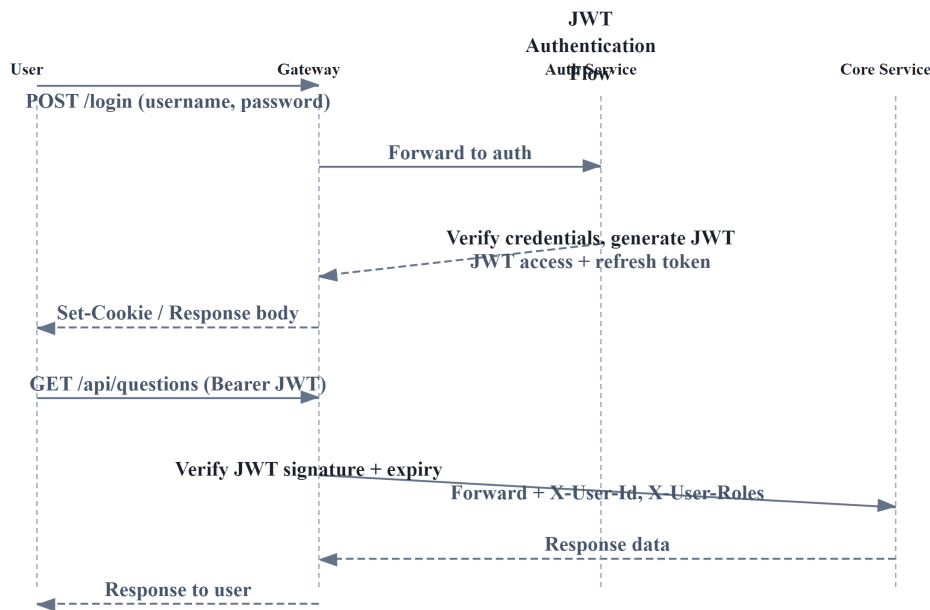
Bảng 9.4: Ba phần của JWT — nội dung và ví dụ

Phần	Nội dung	Ví dụ KBLab
Header	Algorithm + type	{"alg": "HS256", "typ": "JWT"}
Payload	Claims (dữ liệu)	{"userId": "user-123", "roles": "STUDENT", "exp": 1710000000}
Signature	Verify integrity	HMACSHA256(header + "." + payload, secretKey)

9.3.3. HS256 vs RS256**Bảng 9.5:** HS256 (symmetric) vs RS256 (asymmetric)

	HS256 (Symmetric)	RS256 (Asymmetric)
Key	Shared secret key	Private key (sign) + Public key (verify)
Who sign?	Auth Service (có secret)	Auth Service (có private key)
Who verify?	Bất kỳ service nào có secret	Bất kỳ service nào có public key
Security	Secret bị lộ → sign + verify đều bị	Private key bị lộ → sign bị, verify không ảnh hưởng
Key rotation	Phải update tất cả services	Chỉ update Auth Service (private key)
Use case	Internal services, trust boundary rõ	Public APIs, nhiều third-party verifiers

9.3.4. JWT Flow trong KBLab



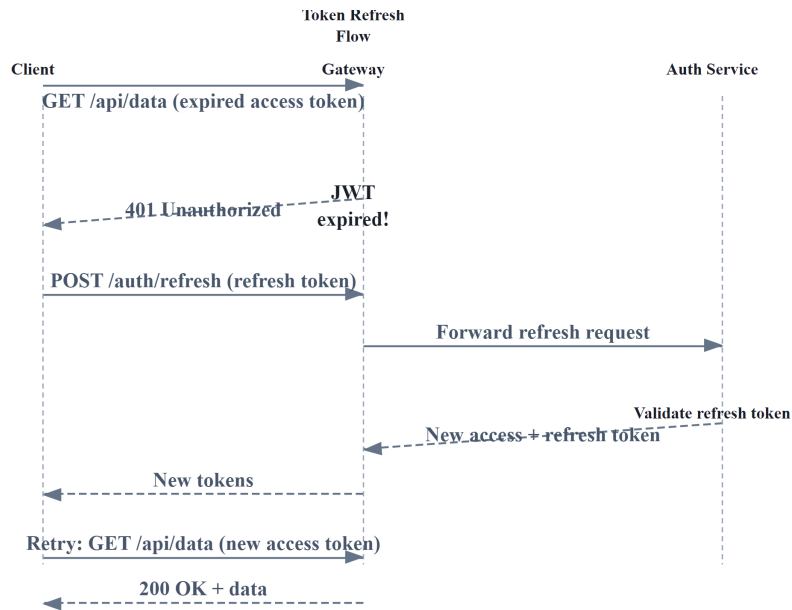
Hình 9.4: JWT Flow trong KBLab — login, token generation, và subsequent requests

❗ Phân tích — KBLab dùng HS256 với shared secret

KBLab sử dụng HS256 (symmetric) — các services chia sẻ cùng `jwt.secretKey`. Trong production, nếu bất kỳ service nào bị compromise, attacker có thể tạo JWT token giả mạo cho bất kỳ user nào. Với KBLab (academic, internal), rủi ro có thể chấp nhận ở giai đoạn đầu nếu trust boundary ở gateway rõ ràng. **Migration path** (khi cần nâng security): (1) chuyển sang RS256 — Auth Service giữ private key, các service khác chỉ có public key, (2) dùng Spring Security OAuth2 Resource Server (built-in JWT verification), (3) key rotation qua config server.

9.3.5. Token Refresh & Rotation — Vòng đời của JWT

JWT có thời hạn (`exp`). Khi access token hết hạn, user phải **re-authenticate** — kém trải nghiệm nếu session dài (giảng viên dùng suốt buổi giảng). **Refresh token** giải quyết: access token ngắn (15-60 phút), refresh token dài (7-30 ngày). Khi access token hết hạn, client dùng refresh token để nhận access token mới — không cần login lại.



Hình 9.5b: Token Refresh & Rotation flow — access token mới + refresh token mới

Token Rotation là chiến lược quan trọng: mỗi lần dùng refresh token, **cả refresh token cũng được thay mới** (rotate) và token cũ bị invalidate. Lợi ích: nếu refresh token bị đánh cắp và reuse, Auth Service phát hiện token đã dùng → **invalidate toàn bộ token family** → buộc user login lại [4a, Ch.9].

Listing 9.1: Token refresh endpoint với rotation

```
// Auth Service – Token refresh với rotation
@PostMapping("/refresh")
public TokenResponse refreshToken(@RequestBody RefreshRequest request) {
    RefreshToken stored = refreshTokenRepository
        .findByToken(request.getRefreshToken())
        .orElseThrow(() -> new AuthException("Invalid refresh token"));

    // Kiểm tra đã bị dùng chưa (reuse detection)
    if (stored.isUsed()) {
        // Token reuse detected! → Invalidate toàn bộ family
        refreshTokenRepository.invalidateFamily(stored.getFamily());
        throw new AuthException("Token reuse detected – please login again");
    }

    // Kiểm tra hết hạn
    if (stored.isExpired()) {
        throw new AuthException("Refresh token expired");
    }

    // Mark token cũ là đã dùng
    stored.setUsed(true);
    refreshTokenRepository.save(stored);

    // Generate tokens mới (cùng family)
    User user = stored.getUser();
    String newAccessToken = jwtUtil.generateAccessToken(user); // 30 phút
}
```

```
String newRefreshToken = createRefreshToken(user, stored.getFamily()); // 7 ngày
return new TokenResponse(newAccessToken, newRefreshToken);
}
```

Bảng 9.5b: Chiến lược token expiry

Token	Thời hạn	Lưu ở đâu	Lý do
Access token	15-60 phút	Client memory/ cookie	Ngắn → giảm window nếu bị leak
Refresh token	7-30 ngày	HttpOnly cookie + DB	Dài → trải nghiệm mượt, DB cho revocation
Token family	Theo refresh token	DB (family ID)	Phát hiện reuse → revoke toàn bộ

9.4. Dual Validation Strategy

9.4.1. Vấn đề: validate ở đâu? Mỗi service hay chỉ gateway?

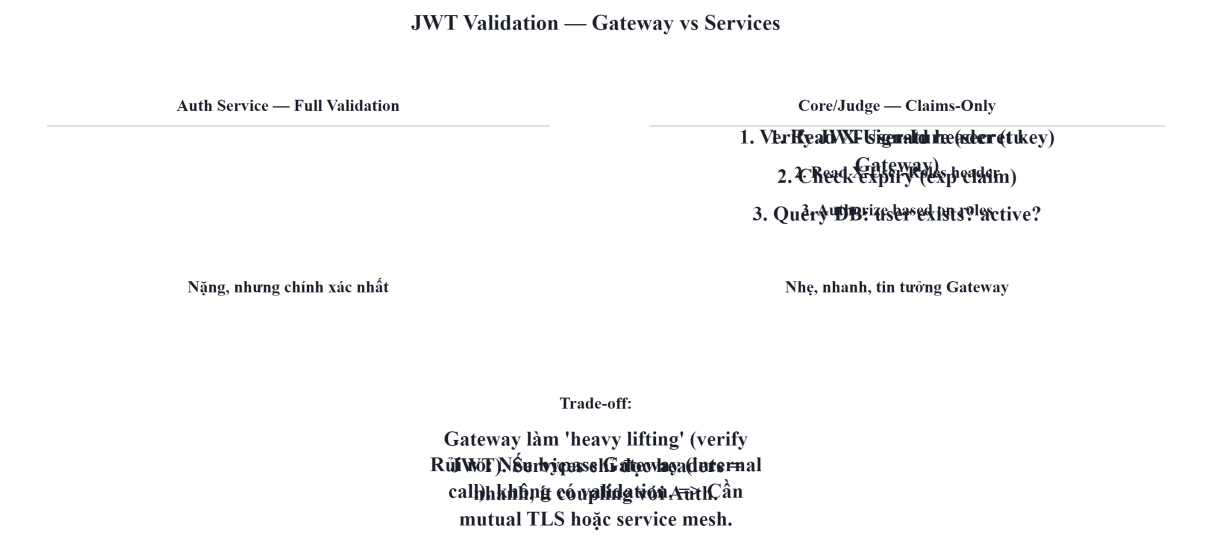
Hai cách tiếp cận trái ngược:

Full validation mỗi service — Mỗi service tự validate JWT, tự query database kiểm tra user còn active, token chưa bị revoke. Ưu: security cao (zero trust). Nhược: mỗi service cần JWT library + DB access + logic duplicate.

Gateway-only validation — Chỉ gateway validate, services tin tưởng headers từ gateway. Ưu: simple, không duplicate logic. Nhược: nếu bypass gateway (misconfigured network), services không có protection.

9.4.2. LMS approach: Dual Validation

LMS implement giải pháp **hybrid** — validation khác nhau tùy service:



Hình 9.5: Dual Validation — full validation (Auth) vs claims-only (internal services)

Bảng 9.6: Ba loại validation trong KBLab

Validation Type	Service	Khi nào	Chi tiết
Full (DB-based)	Auth Service	Login, token refresh, sensitive operations	Verify signature + check DB (user active, token valid)
Claims-only	Gateway	Mọi request	Verify JWT signature + expiry (stateless)
Header-trust	Core, Assignment	Judge, Mọi request (sau gateway)	Tin tưởng X-User-Id và X-User-Roles từ gateway

Auth Service (full validation): verify JWT signature + expiry, query DB (“user still exists and active?”), check token blacklist. Nặng nhưng đảm bảo chính xác — dùng cho login, token refresh, sensitive operations.

Core/Judge Service (claims-only): nhận X-User-Id và X-User-Roles từ headers (gateway đã inject). Không validate JWT lại — chỉ check authorization: `if (!roles.contains("STUDENT")) throw ForbiddenException`. Nhanh, stateless, tin tưởng gateway (vì traffic internal).

Pattern này gọi là **claims-based identity propagation**: gateway validate token, services phía sau tin tưởng gateway và chỉ focus vào authorization.

⚠️ Nguyên tắc — Validate at the Edge, Authorize at the Service

Gateway chịu trách nhiệm *authentication* (ai đang gọi?). Service chịu trách nhiệm *authorization* (người này có quyền làm hành động này?). Tách authentication và authorization cho phép mỗi lớp tập trung vào một nhiệm vụ — gateway không cần biết business rules, service không cần biết JWT format.

9.5. OAuth2 — External Identity Provider

9.5.1. Vấn đề: quản lý credentials phức tạp

Tự quản lý username/password đòi hỏi: hash passwords (bcrypt), xử lý forgot password, 2FA, brute force protection, compliance. Với team nhỏ, **delegate authentication** cho identity provider chuyên nghiệp (Google, Microsoft, GitHub) giảm đáng kể effort và rủi ro.

9.5.2. OAuth2 Authorization Code Flow

KBLab tích hợp OAuth2/OIDC cho đăng nhập qua identity provider của trường (ví dụ Microsoft Entra ID hoặc Google Workspace). Flow: User → redirect tới provider → login → callback với authorization code → Auth Service exchange code for tokens (server-to-server) → fetch userinfo → find/create user → generate KBLab JWT.

Điểm quan trọng: KBLab *không dùng* external token trực tiếp. Sau khi xác thực với provider, Auth Service tạo **JWT riêng của KBLab** — services phía sau không biết user login bằng Microsoft, Google, QLĐT hay username/password. Đây là pattern **token exchange**: external token → internal token.

9.5.3. Multiple Authentication Methods

KBLab hỗ trợ nhiều phương thức xác thực nhưng hội tụ về cùng một token nội bộ:

Bảng 9.7: Các phương thức xác thực trong KBLab

Method	Flow	Khi nào
Username/Password	Truyền thống, hash bcrypt	Default cho mọi user
Microsoft/Google OAuth2/OIDC	Authorization Code Flow + PKCE khi phù hợp	SSO bằng tài khoản trường
PTIT QLDT	Custom integration	Login bằng tài khoản quản lý đào tạo
Email verification	Verify email trước khi kích hoạt/khôi phục	Giảm tài khoản giả, hỗ trợ reset password
Turnstile	Challenge chống bot ở form nhảy cảm	Bảo vệ login/register/reset khỏi automation

Tất cả đều converge vào cùng output: **KBLab JWT token** (chứa userId, roles, expiry). Auth Service expose `generateTokens(user)` — nhận User entity từ bất kỳ login method nào, trả về `{accessToken, refreshToken}`. Downstream services không phân biệt. Đây là nền tảng cho SSO cross-platform: các module LMS core, Network Lab và DevOps Lab có thể chia sẻ identity nội bộ mà không buộc từng module hiểu mọi provider bên ngoài.

9.6. RBAC — Role-Based Access Control

9.6.1. Vấn đề: ai được làm gì?

Authentication trả lời “người dùng là ai?”. Authorization trả lời “người dùng được làm gì?”. Trong KBLab, ba vai trò chính cần permissions khác nhau:

Bảng 9.8: RBAC — vai trò và permissions trong KBLab

Role	Permissions	Ví dụ
STUDENT	Submit bài, xem kết quả, xem bài tập, xem contest	Sinh viên
LECTURER	Tất cả STUDENT + tạo bài, tạo contest, xem thống kê	Giảng viên
ADMIN	Tất cả LECTURER + quản lý users, quản lý hệ thống	Quản trị viên

9.6.2. RBAC Implementation trong KBLab

KBLab dùng Spring Security `@PreAuthorize` annotation với roles lưu trong JWT:

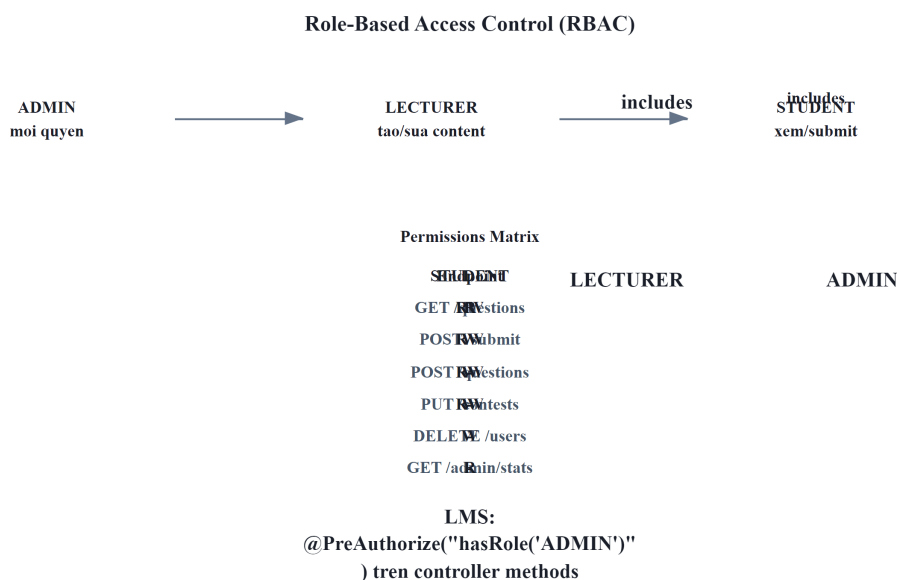
Listing 9.2: Spring Security `@PreAuthorize` — RBAC declarative

```
// Spring Security @PreAuthorize – declarative RBAC
@GetMapping("/questions")
public List<QuestionResponse> getQuestions() { ... } // Mọi user authenticated

@PreAuthorize("hasAnyRole('LECTURER', 'ADMIN')")
@PostMapping("/questions")
public QuestionResponse create(@RequestBody CreateQuestionRequest r) { ... }

@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/questions/{id}")
public void delete(@PathVariable UUID id) { ... }
```

9.6.3. Role Hierarchy



Hình 9.6: Role Hierarchy — ADMIN inherits LECTURER inherits STUDENT

KBLab lưu roles dạng **pipe-delimited** trong JWT: "ADMIN|LECTURER|STUDENT". Khi gateway extract roles, nó truyền qua header `X-User-Roles` — Core Service parse và check permissions.

9.6.4. API-driven Route Protection (Frontend)

KBLab frontend sử dụng pattern khác biệt: **route permissions được fetch từ API** (GET / api/auth/me/permissions) thay vì hardcoded — response chứa danh sách routes và actions mà user được phép. Frontend dynamic render routes dựa trên permissions này. Ưu điểm: thay đổi permissions ở backend → frontend tự cập nhật, không cần deploy lại.

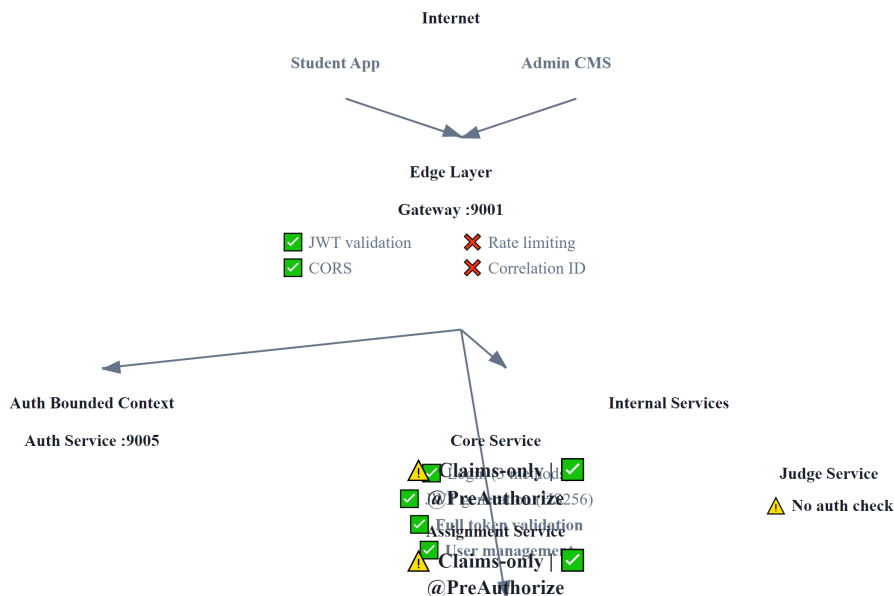
❗ Phân tích — RBAC trong KBLab

KBLab implementation hiện tại có vài vấn đề: (1) Roles lưu dạng pipe-delimited string thay vì array — phải parse thủ công, dễ lỗi nếu role name chứa pipe. (2) @PreAuthorize annotations phân tán ở mỗi controller — khó audit “user role X có thể access những endpoint nào?”. (3) Không có role hierarchy rõ ràng ở Spring Security level — ADMIN phải list cả LECTURER và STUDENT roles. (4) DevOps Lab cần policy bổ sung ở runtime layer: user nào được tạo workspace, network namespace, container hoặc job đặc quyền.

Migration path: (1) chuyển roles sang JSON array trong JWT payload, (2) cấu hình RoleHierarchy trong Spring Security, (3) cân nhắc tập trung authorization policies (Spring Security method security hoặc OPA).

9.7. Case Study: Kiến trúc bảo mật trong KBLab

9.7.1. Tổng quan



Hình 9.7: Kiến trúc bảo mật tổng thể của KBLab

9.7.2. Phân tích tổng hợp

Bảng 9.9: Phân tích bảo mật toàn diện KBLab

Aspect	Hiện trạng	Risk Level	Best Practice
JWT Algorithm	HS256 (symmetric, shared secret)	⚠️ Medium	RS256 cho production
Token Storage	Client-side (localStorage)	⚠️ Medium	HttpOnly cookie (XSS protection)
Secret Management	Hardcoded trong application.yml	🔴 High	Vault, AWS Secrets Manager
CORS	allowedOrigins: "*"	🔴 High	Restrict to specific origins
Service-to-service auth	Không có	⚠️ Medium	mTLS hoặc service token
Rate Limiting	Không có	⚠️ Medium	Redis-based rate limiter
Email verification / bot defense	Có/đang hoàn thiện theo module	⚠️ Medium	Verify email + Turnstile cho form nhạy cảm
DevOps Lab isolation	Runtime isolation riêng	🔴 High nếu cấu hình sai	Sysbox/k3s + NetworkPolicy/RBAC
Input Validation	Partial	⚠️ Medium	Bean Validation (@Valid)
HTTPS	Gateway level	✅ Low	—
Password Hashing	BCrypt	✅ Low	—

9.7.3. Đề xuất migration

Phase 1 — Quick Wins (effort thấp, impact cao):

- Restrict CORS origins
- Move secrets ra khỏi application.yml → environment variables (tối thiểu)
- Token vào HttpOnly cookie thay vì localStorage
- Bật email verification và Turnstile cho login/register/reset ở các flow public

Phase 2 — Authentication Hardening (effort trung bình):

- Chuyển HS256 → RS256
- Thống nhất JWT library version (đã đề cập ở Ch.8)
- Token refresh flow với rotation (mỗi lần refresh → invalidate token cũ)
- Chuẩn hóa SSO cross-platform: external token exchange → KBLab JWT → shared claims contract

Phase 3 — Service Mesh Security (effort cao, khi scale):

- Service-to-service authentication (mTLS hoặc internal JWT)
- Centralized secret management (HashiCorp Vault)
- API-level authorization policies (Open Policy Agent)

- DevOps Lab runtime guardrails: Kubernetes RBAC, NetworkPolicy, resource quotas và Sysbox isolation

9.7.4. Secret Management trong Production

Bảng 9.9 cho thấy “Secret Management” là risk level  High — `jwt.secretKey` hardcoded trong `application.yml` và commit vào Git. Đây là lỗi hỏng phổ biến nhất trong microservices và cần được xử lý ở mọi hệ thống production.

Ba cấp độ Secret Management:

Bảng 9.10: Chiến lược Secret Management — từ cơ bản đến enterprise

Cấp độ	Cách tiếp cận	Ưu điểm	Nhược điểm	Phù hợp
Level 1	Environment Variables	Đơn giản, tách secret khỏi code	Secrets trong plaintext trên server, không audit trail	Dev/Staging
Level 2	Encrypted Config (Spring Cloud Config + Jasypt)	Secrets mã hóa trong repo, decrypt lúc runtime	Vẫn cần quản lý encryption key	Small production
Level 3	Secret Manager (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager)	Auto-rotation, audit log, access policies, dynamic secrets	Infrastructure cost, operational complexity	Production at scale

Level 1 — Environment Variables (LMS nên áp dụng ngay):

Thay vì:

```
# ❌ application.yml – KHÔNG BAO GIỜ commit secrets
jwt:
  secretKey: "mySecretKey123"
  expiration: 86400000
```

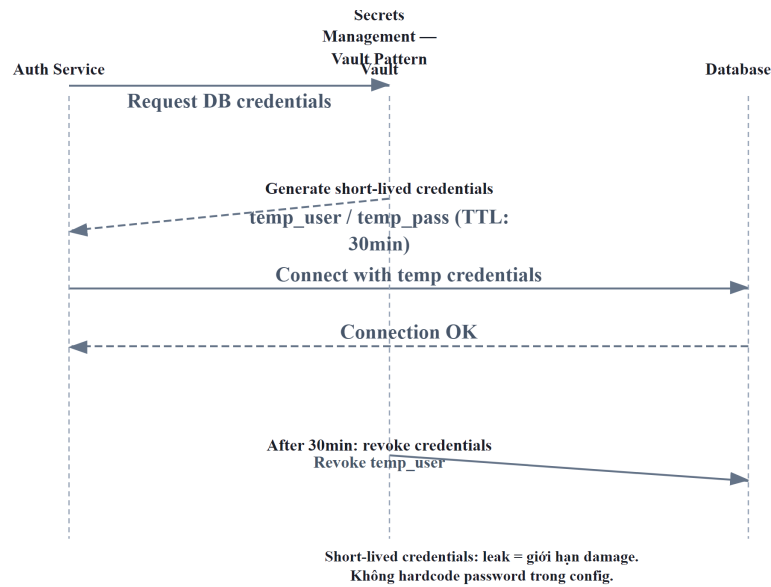
Sử dụng:

```
# ✅ application.yml – reference environment variable
jwt:
  secretKey: ${JWT_SECRET_KEY}
  expiration: ${JWT_EXPIRATION:86400000}

# Docker Compose – inject từ .env file (không commit .env)
services:
  auth-service:
    environment:
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
      - DB_PASSWORD=${DB_PASSWORD}
```

Level 3 — HashiCorp Vault (khi hệ thống production hoàn chỉnh):

Vault cung cấp **dynamic secrets** — database credentials được tạo tự động, có thời hạn, và auto-rotate. Không ai biết password database thực sự — kể cả developer.



Hình 9.8: Vault Dynamic Secrets — credentials tạm thời, tự động xoay vòng

💡 Tip — .env file và .gitignore

Bước đầu tiên và quan trọng nhất: thêm `.env` vào `.gitignore`. Tạo `.env.example` (không chứa giá trị thật) để team members biết cần set biến nào. Đây là “zero-cost security improvement” mà mọi project nên làm ngay.

⚠ Lưu ý —

1. **Lưu JWT trong localStorage** — Bất kỳ JavaScript nào chạy trên page đều đọc được (XSS attack). Hậu quả: nếu có XSS vulnerability, attacker steal token → impersonate user. *Phòng tránh*: lưu JWT trong HttpOnly cookie — JavaScript không thể đọc, trình duyệt tự gửi.
2. **Hardcode secrets trong source code** — `jwt.secretKey=mySecretKey123` trong `application.yml`, commit vào Git. Hậu quả: ai có access repo đều có thể tạo JWT giả mạo. *Phòng tránh*: dùng environment variables (tối thiểu), hoặc secret manager (Vault, AWS Secrets Manager).
3. **Không set expiry cho JWT** — Token không bao giờ hết hạn. Hậu quả: token bị leak = access vĩnh viễn, không cách nào revoke. *Phòng tránh*: access token ngắn (15-60 phút), refresh token dài (7-30 ngày) với rotation.
4. **Validate JWT ở mỗi service bằng cách khác nhau** — Mỗi service dùng JWT library version khác, parse token khác nhau. Hậu quả: gateway accept nhưng service reject (hoặc ngược lại), debugging rất khó. *Phòng tránh*: thống nhất validation ở gateway, services trust headers (§9.3).
5. **Confused Deputy Problem** — Service A gọi Service B thay mặt user, nhưng B không biết A đang “đại diện” cho user hay hành động với quyền riêng của A. Newman trong [4a, Ch.9] gọi đây là *confused deputy attack*. Hậu quả: Service A có thể vô tình escalate privileges — user chỉ có quyền STUDENT nhưng Service A gọi B với full service credentials. *Phòng tránh*: luôn truyền user identity (JWT hoặc headers `X-User-Id`, `X-User-Roles`) trong service-to-service calls — B authorize dựa trên *user* chứ không phải *service* gọi.

🌐 **Thực quan hóa tương tác (Interactive Demo)**

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/oauth2-jwt-flow.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Luồng xác thực OAuth2 & JWT**.

Ngoài ra, bạn cũng có thể xem minh họa về **Circuit Breaker Pattern** tại file `code/interactive/circuit-breaker.html`.

9.8. Tổng kết

Bảo mật microservices phức tạp hơn monolith vì attack surface lớn hơn — mỗi service, mỗi network call, mỗi database đều là điểm tiềm ẩn rủi ro. Defense in depth — bảo vệ nhiều lớp, không dựa vào một lớp duy nhất — là nguyên tắc nền tảng.

JWT giải quyết bài toán authentication trong distributed system: stateless, tự chứa identity information, không cần shared session store. HS256 (symmetric) đơn giản nhưng đòi hỏi shared secret — phù hợp cho internal services. RS256 (asymmetric) an toàn hơn cho production với nhiều verifiers.

Dual validation — full validation tại Auth Service, claims-only tại gateway, header-trust tại internal services — là cách tiếp cận thực tế: cân bằng giữa security và performance. OAuth2

cho phép delegate authentication cho identity provider chuyên nghiệp, giảm effort quản lý credentials.

RBAC là mô hình authorization phổ biến nhất — đủ tốt cho hầu hết hệ thống. KBLab implement RBAC qua `@PreAuthorize` annotations, nhưng thiếu role hierarchy rõ ràng và authorization policy tập trung.

Phân tích KBLab cho thấy kiến trúc bảo mật cơ bản đúng (JWT, gateway validation, RBAC, token exchange cho SSO) nhưng có nhiều quick wins cần xử lý: CORS restriction, secret management, token storage, email verification/Turnstile và runtime isolation cho DevOps Lab. Đây là technical debt bảo mật — không ảnh hưởng tính năng nhưng ảnh hưởng risk profile.

Ở Chương 10, chúng ta sẽ tổng hợp mọi kiến thức từ Chương 1-9 vào **bài toán thực tế nhất**: chuyển đổi từ monolith sang microservices — khi nào nên chuyển, Strangler Fig pattern, tách database, và migration roadmap cho KBLab.

9.9. Đọc thêm

Sách tham khảo chính:

1. [4a] Sam Newman, *Building Microservices* — Ch.9: Security
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.11: Developing secure services, Access Token pattern
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution (data formats, schema evolution — nền tảng cho hiểu binary/JSON encoding trong tokens và API messages)

Nguồn trực tuyến:

- JWT.io — jwt.io (interactive JWT decoder)
- OWASP API Security Top 10 — owasp.org/www-project-api-security
- Spring Security OAuth2 Resource Server — docs.spring.io/spring-security
- RFC 7519 — JSON Web Token (JWT) specification

10. Chuyển đổi Thực tế — Từ Monolith đến Microservices

“If you can’t build a monolith, what makes you think microservices are the answer?” — Simon Brown (trích dẫn bởi Sam Newman, [4b])

10.1. Bạn sẽ học được gì

- Hiểu khi nào nên và **khi nào KHÔNG nên** chuyển sang microservices
- Nắm vững **Strangler Fig Pattern** — chiến lược migration tăng dần phổ biến nhất
- Áp dụng **5 chiến lược tách database** từ góc nhìn migration thực tế
- Hiểu **Anti-Corruption Layer** và các migration patterns bảo vệ ranh giới hệ thống
- Thiết kế **Migration Roadmap** khả thi: phân chia phases, ưu tiên theo Impact × Effort

- Phân tích migration path cho KBLab — tổng hợp gap analyses xuyên suốt 12 chương

10.2. Khi nào (KHÔNG) nên chuyển sang Microservices

10.2.1. Vấn đề: microservices không phải đích đến mặc định

Sau 9 chương phân tích patterns, communication, data management, gateway, và security, đọc giả có thể có ấn tượng rằng microservices là kiến trúc mọi hệ thống nên hướng tới. **Đây là sai lầm nguy hiểm nhất** — và cũng là sai lầm phổ biến nhất.

Martin Fowler — người đồng tác giả bài viết gốc “Microservices” (2014) — cảnh báo rõ ràng [W1]:

“Almost all the cases where I’ve heard of a system that was built as a microservice system from scratch, it has ended up in serious trouble... You shouldn’t start a new project with microservices, even if you’re sure your application will be big enough to make it worthwhile.”

Newman trong [4b, Ch.1] bổ sung: microservices là **một lựa chọn kiến trúc**, không phải bước tiến hóa bắt buộc. Monolith không phải “kiến trúc sai” — monolith là kiến trúc đúng cho phần lớn hệ thống ở giai đoạn đầu.

10.2.2. Decision Matrix — Khi nào cân nhắc migration

Bảng 10.1: Decision Matrix — khi nào cân nhắc migration

Tiêu chí	Monolith đủ tốt	Cân nhắc Microservices
Team size	≤ 8 người	> 15 người, nhiều teams
Deploy frequency	Weekly/monthly	Cần deploy daily, per-team
Scale requirements	Scale toàn bộ đủ	Cần scale từng component riêng
Technology diversity	Một stack đủ	Cần polyglot (Java + Python + Go)
Domain complexity	Ranh giới chưa rõ	Bounded contexts đã xác định rõ (Ch.2)
Organizational maturity	Chưa có DevOps, CI/CD	Đã có CI/CD, containerization, monitoring

Richardson trong [2a, Ch.13] gọi đây là **Microservice Premium** — chi phí phải trả khi chuyển sang microservices: distributed system complexity, network failures, eventual consistency, operational overhead. Premium chỉ đáng trả khi benefits (independent deployment, scaling, team autonomy) **vượt qua** costs.

10.2.3. KBLab: Evolutionary Architecture, không phải Big Bang

KBLab không bắt đầu từ một bản thiết kế microservices hoàn hảo. Hệ thống đã đi qua nhiều thế hệ: các judge monolith riêng cho bài lập trình, Network Judge, SQL Judge mới, rồi dần hình thành các service cho auth, assignment, judge, notification và DevOps Lab. Vì vậy câu hỏi đúng không phải là “có nên microservices-first không?”, mà là: *mỗi bước tiến hóa có đang giải quyết đúng vấn đề của thời điểm đó không?*

❗ Phân tích — trong KBLab

KBLab chọn hướng microservices vì hai lý do cụ thể: (1) mục đích **giáo dục** — bản thân hệ thống là learning material cho sinh viên, (2) domain cốt lõi đã đủ rõ sau nhiều năm vận hành judge systems: Identity, SQL Practice, Network Practice, Academic Management, Evaluation/Infrastructure.

Tuy nhiên, trade-off hiện rõ: (1) shared database giữa Core và Assignment — coupling vẫn tồn tại dù services tách, (2) shared library chứa quá nhiều logic — “distributed monolith” risk, (3) polyglot Java+Go làm tăng chi phí build/test/deploy. Đây là ví dụ thực tế của Microservice Premium: **lợi ích giáo dục và nhu cầu cô lập judge/lab justify chi phí**, nhưng vẫn cần roadmap giảm coupling có kiểm soát.

Một bài học cụ thể nằm ở SQL Judge, không phải Network Practice. Network Practice được thiết kế độc lập để đánh giá giao tiếp qua mạng; nó minh họa tốt bounded context theo protocol. Với SQL Judge, bài toán là ranh giới giữa một judge coordinator và nhiều judge-* workers theo DBMS (judge-mysql, judge-sqlserver, ...). Migration đúng hướng là làm rõ ownership routing, chuẩn hóa contract job/result, thêm idempotency theo judgeRunId, rồi mới scale ngang từng worker theo workload DBMS.

💡 Nguyên tắc —

“Start with a monolith. Move to microservices only when the monolith becomes a problem — and you know which problems you’re solving. The worst microservices systems are those built by teams who started with microservices *before* understanding their domain boundaries.”

— Tổng hợp từ Martin Fowler [W1] và Sam Newman [4b, Ch.1]

10.2.4. Bài học thực tế — Khi Migration đi sai hướng (Anti-patterns)

Dù quyết định chuyển đổi là đúng đắn, *cách thức* chuyển đổi vẫn có thể dìm chết dự án. Richardson trong [2b, Ch.2] phân tích bốn anti-patterns phổ biến nhất khi chuyển từ monolith sang microservices — chúng ta minh họa qua các sai lầm *tiềm năng* nếu chia tách sai trong KBLab:

Bảng 10.1b: Anti-patterns khi chuyển đổi Microservices (minh họa với KBLab)

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBLab	Hậu quả
Data Services (<i>CRUD wrappers</i>)	Tách <code>SqlExecutorService</code> thành <code>QuestionDataService</code> riêng — Core phải gọi HTTP mỗi lần cần data câu hỏi	Tăng latency mạng, mất lợi ích transaction nội bộ
Fine-grained Services (<i>Chia quá nhỏ</i>)	Chia Auth thành 3 service: <code>UserProfile</code> , <code>UserCredential</code> , <code>UserSession</code>	Mỗi lần login phải orchestrate 3 services, khó debug
Microservices-first	Tách KBLab theo technical layer thay vì domain — mỗi thay đổi chấm điểm ảnh hưởng nhiều services cùng lúc	Phát sinh “Distributed Monolith” (xem thêm §10.6)
End-to-end QA Gate	Deploy độc lập nhưng QA phải chờ gộp tất cả để test	Triệt tiêu <i>Independent Deployability</i>

Chúng ta sẽ phân tích sâu hơn về các sai lầm migration bổ sung (Big Bang Rewrite, Lift-and-Shift, Over-engineering) tại §10.6.

10.3. Strangler Fig Pattern — Migration Tăng dần

10.3.1. Vấn đề: “Big Bang” vs Tăng dần migration

Hai cách migrate từ monolith sang microservices:

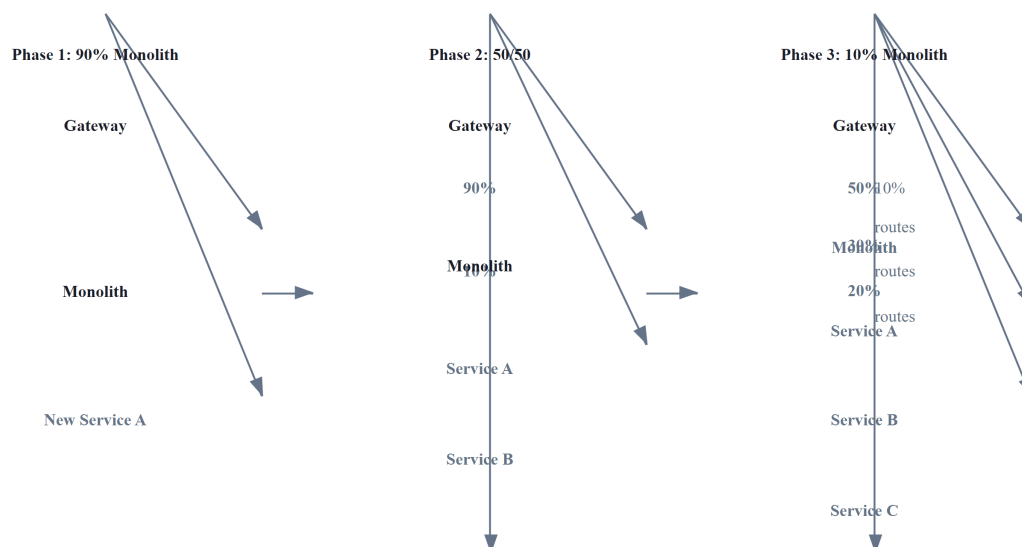
Bảng 10.2: Big Bang vs Tăng dần migration

Chiến lược	Mô tả	Rủi ro
Big Bang	Viết lại toàn bộ hệ thống mới, switch cùng lúc	 Rất cao — thường fail
Tăng dần (Strangler Fig)	Tách dần từng phần, chạy song song	 Thấp — rollback từng phần

Newman trong [4b, Ch.3] và Richardson trong [2a, Ch.13] đều khẳng định: **Big Bang rewrites gần như luôn thất bại** — “the second system effect” (Fred Brooks). Khi viết lại từ zero, team mất tất cả domain knowledge embedded trong code cũ, deadline liên tục trễ, và khách hàng phải chờ đợi *tất cả* tính năng trước khi nhận *bất kỳ* giá trị nào.

10.3.2. Strangler Fig Pattern

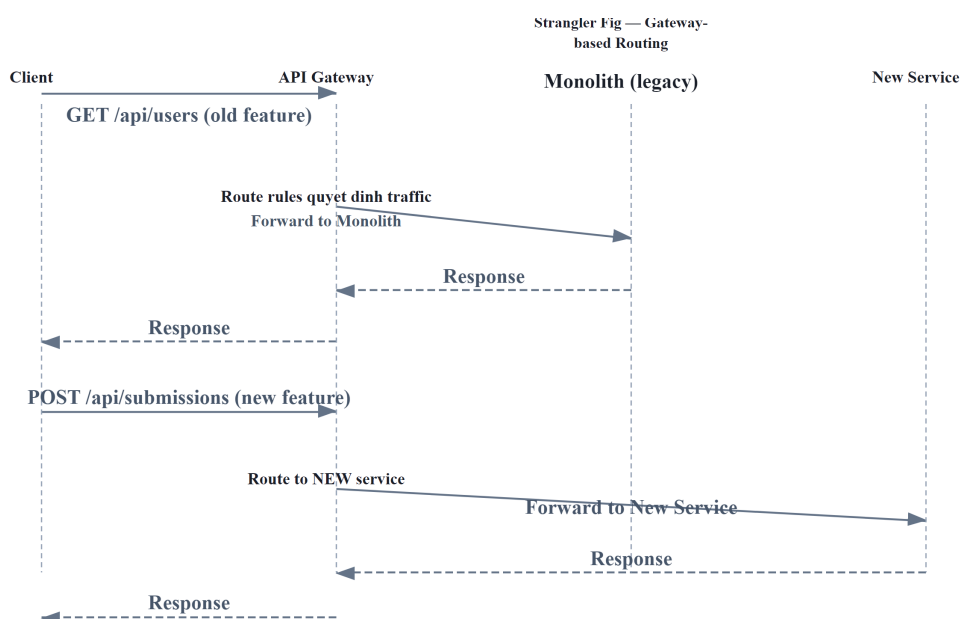
Strangler Fig — pattern do Martin Fowler đặt tên (2004), lấy cảm hứng từ cây đa bóp nghẹt (strangler fig) bao quanh cây chủ, dần thay thế cho đến khi cây chủ biến mất [W2].



Hình 10.1: Strangler Fig Pattern — dần thay thế monolith qua 3 giai đoạn

10.3.3. API Gateway — “Cây đa” trong microservices

API Gateway (đã học ở Ch.8) đóng vai trò tự nhiên làm **seam** cho Strangler Fig:



Hình 10.2: API Gateway làm seam cho Strangler Fig — client không biết route nào đến monolith, route nào đến service mới

Bảng 10.3: Ba implementation strategies cho Strangler Fig

Strategy	Mô tả	Khi nào dùng
Route-based	Gateway route paths khác nhau đến monolith/services	API-driven systems (REST, GraphQL)
Event-based	New service listen events từ monolith, dần thay thế logic	Event-driven systems (Kafka, RabbitMQ)
Asset-based	Tách static assets, UI components trước	Frontend-heavy applications

Với KBLab — nơi đã có API Gateway (Spring Cloud Gateway, Ch.8) — **route-based Strangler Fig** là lựa chọn tự nhiên nhất cho LMS chính. Với DevOps Lab, routing lại dựa trên hostname/wildcard DNS ở mức khái quát. Hai kiểu gateway này cùng tồn tại, cho thấy migration không nhất thiết dùng một pattern duy nhất cho mọi bounded context.

⚠ Nguyên tắc — Tăng dần Migration

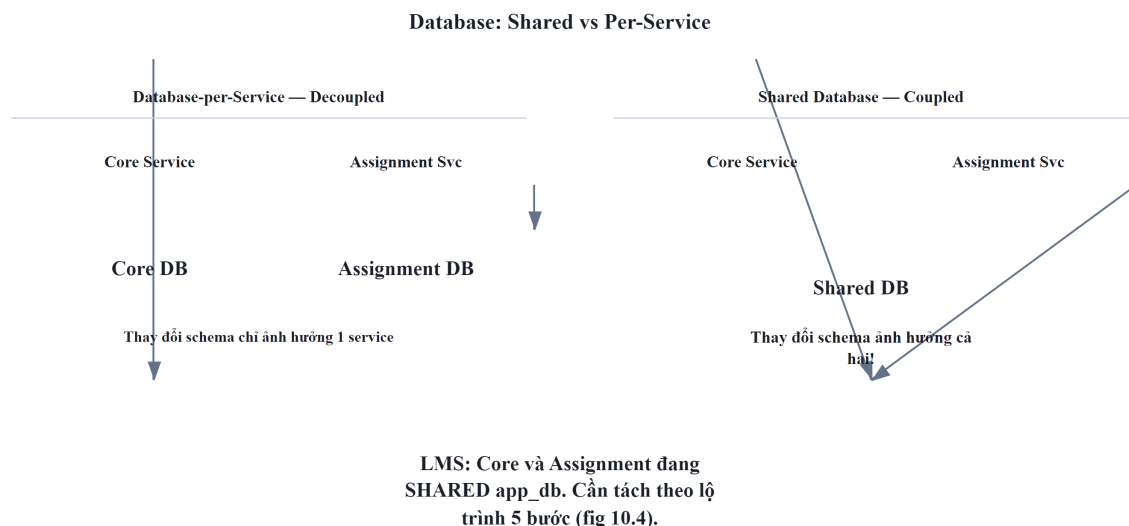
“Never do a big bang rewrite. Strangle the monolith incrementally — each step delivers value, each step is reversible, and the system is always running.”

— Sam Newman, *Monolith to Microservices* [4b, Ch.3]

10.4. Tách Database — Thách thức Lớn nhất

10.4.1. Vấn đề: database coupling nguy hiểm hơn code coupling

Tách code thành services không quá khó — tách API, deploy riêng. Nhưng khi hai services **chia sẻ cùng database**, chúng vẫn coupled ở tầng sâu nhất:



Hình 10.3: Shared Database (coupled) vs Database-per-Service (decoupled)

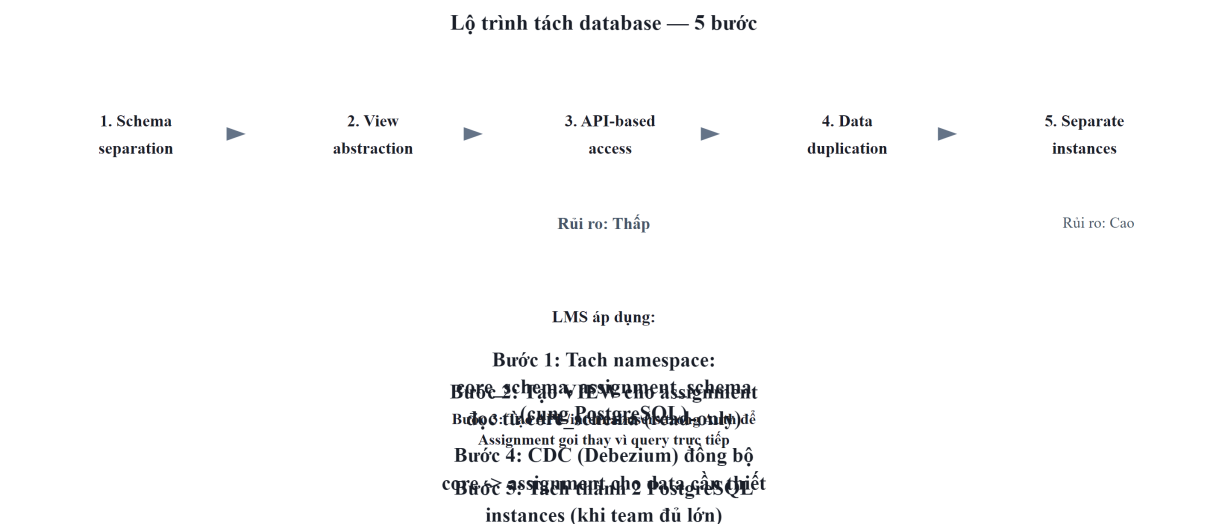
Newman trong [4b, Ch.4] gọi shared database là “**the hardest part of decomposition**” — và LMS là ví dụ trực tiếp: Core Service và Assignment Service chia sẻ cùng database, schema changes ảnh hưởng cả hai, và không thể deploy database migration độc lập.

10.4.2. chiến lược tách database

Từ Ch.7 (Database-per-Service), chúng ta đã biết nguyên tắc. Giờ nhìn từ góc migration — thứ tự thực hiện:

Bảng 10.4: 5 chiến lược tách database — từ góc migration

#	Strategy	Mô tả	Risk	Effort	Khi nào
1	Schema separation	Tách schema trong cùng DB instance	 Thấp	Thấp	Bước đầu tiên — luôn
2	View abstraction	Database views che giấu schema changes	 Thấp	Thấp	Khi service B cần data service A
3	API-based access	Service B gọi API service A thay vì query trực tiếp	 Trung bình	Trung bình	Khi đã tách schema
4	Data duplication	Copy data qua events (eventual consistency)	 Trung bình	Cao	Khi query cross-service phức tạp
5	Separate instances	Database instance riêng cho mỗi service	 Cao	Cao	Bước cuối — khi cần scale DB riêng



Hình 10.4: Thứ tự thực hiện 5 chiến lược tách database

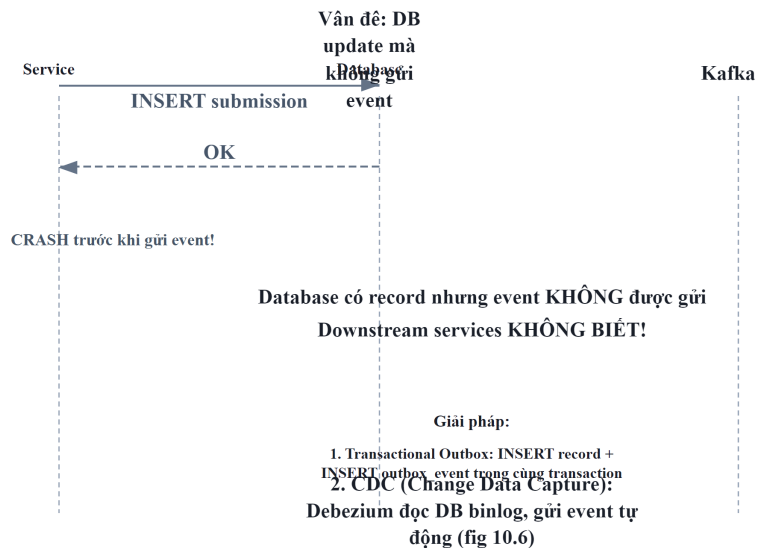
⚠ Nguyên tắc — Tăng dần Database Decomposition

“Don’t try to split the database and the code at the same time. Split the code first, then the database. Trying to do both simultaneously dramatically increases risk and complexity.”

— *Sam Newman, Monolith to Microservices [4b, Ch.4]*

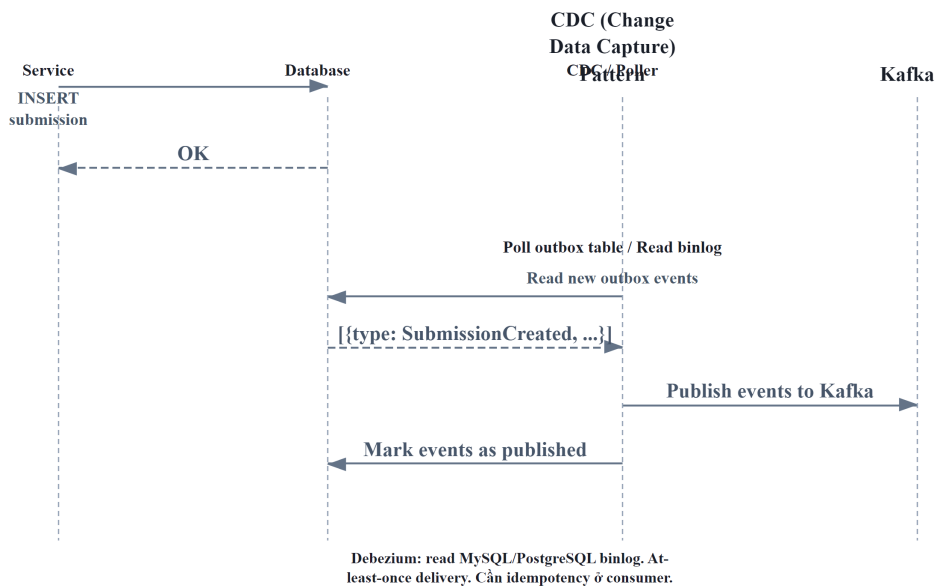
10.4.3. Dual-Write Pitfall và Outbox Pattern

Khi tách database, một anti-pattern phổ biến là **dual write**: service vừa ghi database vừa publish event — nếu một trong hai thất bại, data không nhất quán.



Hình 10.5: Dual-Write Pitfall — DB thành công nhưng event thất bại

Outbox Pattern giải quyết — ghi event vào database *cùng transaction* với business data, rồi một background process đọc outbox table và publish lên message broker:



Hình 10.6: Outbox Pattern — event ghi cùng transaction với business data

Bảng 10.5: Hai cách implement Outbox Pattern

Approach	Mô tả	Ưu điểm	Nhược điểm
Polling publisher	Background thread poll outbox table	Đơn giản, không cần thêm infra	Delay, DB load
CDC (Change Data Capture)	Debezium đọc DB transaction log	Real-time, không polling overhead	Thêm infra (Debezium + Kafka Connect)

① Phân tích — KBLab chưa có Outbox Pattern

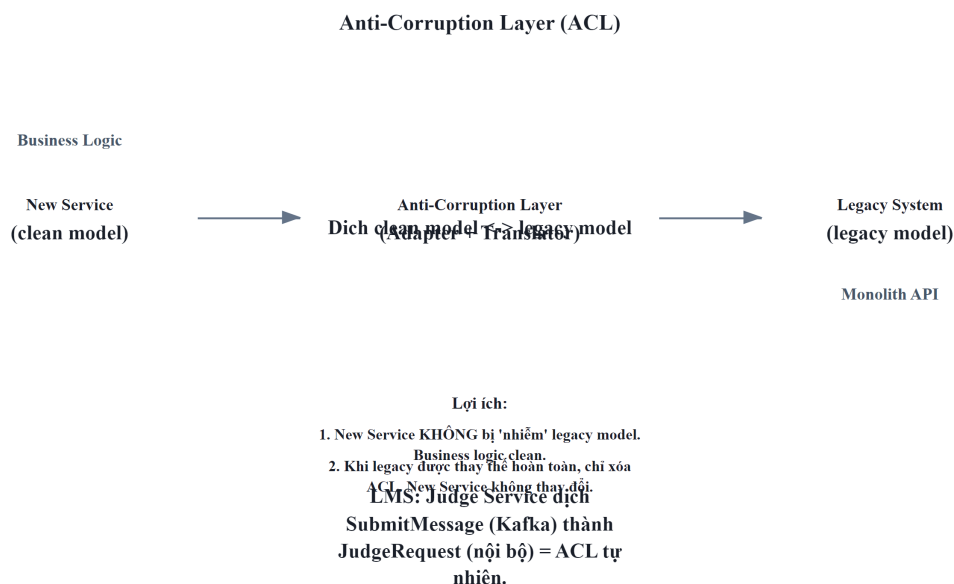
KBLab hiện dùng **dual write**: Core Service lưu submission vào DB rồi gọi `submitProducer.send()` đến Kafka. Nếu Kafka unavailable tại thời điểm đó, submission đã lưu nhưng Judge Service không nhận bài — sinh viên thấy “đã nộp” nhưng không nhận kết quả chấm.

Migration path: (1) Ngắn hạn — retry logic cho Kafka producer (đã có nhưng cần verify), (2) Trung hạn — Outbox table + polling publisher, (3) Dài hạn — Debezium CDC. Với traffic thấp của KBLab hiện tại, polling publisher (trung hạn) đủ tốt — Debezium chỉ cần khi scale lên production lớn.

10.5. Anti-Corruption Layer và Migration Patterns

10.5.1. Anti-Corruption Layer (ACL) — Bảo vệ ranh giới

Khi tách service mới ra khỏi monolith, service mới cần giao tiếp với code legacy — nhưng **không nên để model cũ “nhiễm” vào code mới**. Eric Evans trong [6] định nghĩa **Anti-Corruption Layer (ACL)**: một lớp dịch ngôn ngữ giữa hai bounded contexts có models khác nhau.

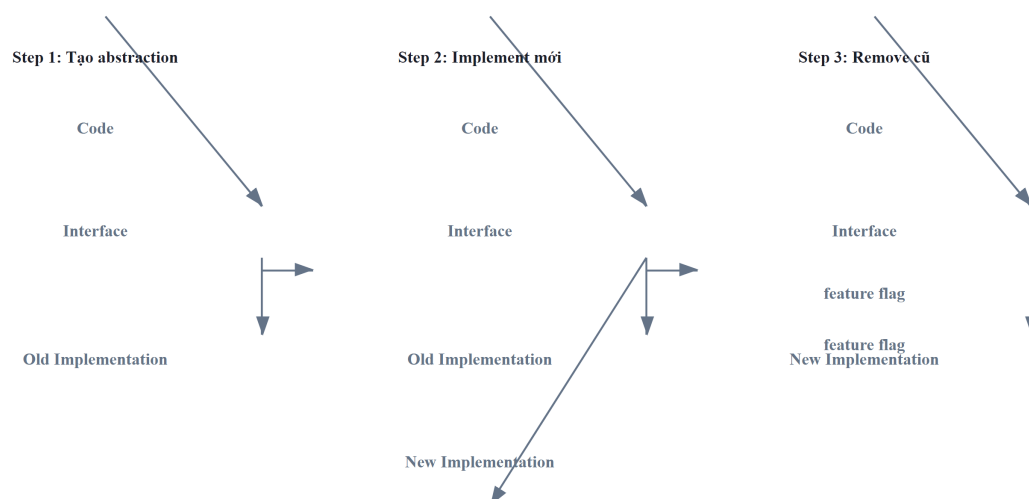


Hình 10.7: Anti-Corruption Layer — lớp dịch giữa service mới và legacy

Ví dụ KBLab: Judge Service nhận submissions từ Core Service qua Kafka. Core gửi format {problem_id, source, testcases} (legacy naming). Judge có thể: (1) ❌ dùng trực tiếp legacy field names trong code — coupling với legacy decisions, hoặc (2) ✅ tạo adapter dịch sang internal model JudgeRequest{questionId, sqlStatement, expectedResults} — clean boundary.

10.5.2. Branch by Abstraction

Newman trong [4b, Ch.3] đề xuất **Branch by Abstraction** cho migration an toàn:

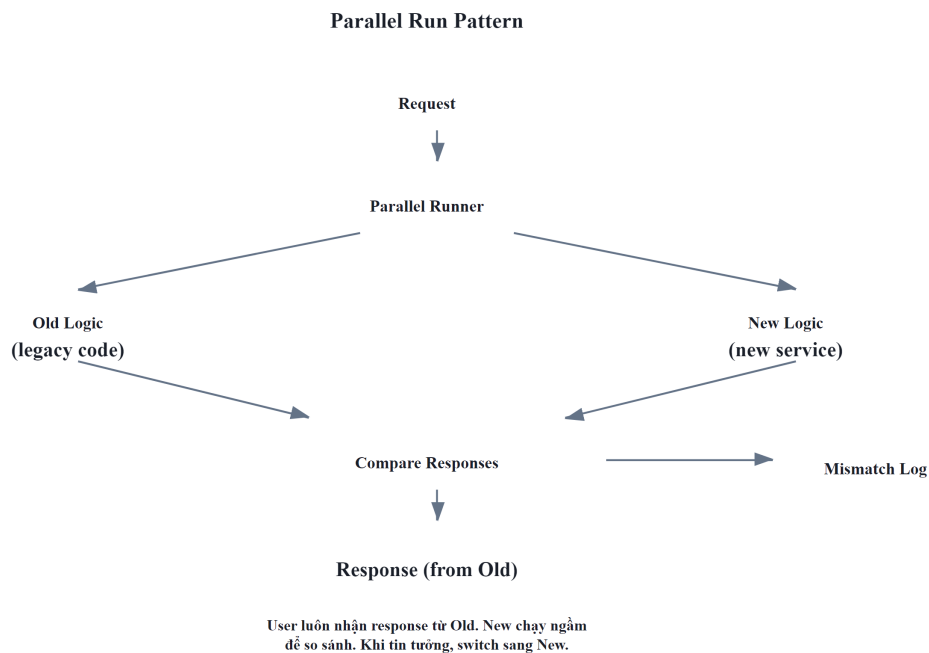


Hình 10.8: Branch by Abstraction — migration an toàn qua feature flag

1. **Tạo abstraction** (interface) trước old implementation
2. **Implement mới** đằng sau cùng interface, toggle bằng feature flag
3. **Switch traffic** dần sang implementation mới, verify
4. **Remove** old implementation khi confident

10.5.3. Parallel Run — Verify trước khi switch

Parallel Run — strategy cho phép **chạy đồng thời** old and new implementation, so sánh output:



Hình 10.9: Parallel Run — chạy song song old và new logic, so sánh output

- Old logic **trả response** cho user (production-safe)
- New logic chạy **song song** — response bị discard
- So sánh hai responses — log mismatches
- Khi mismatch rate $\approx 0\%$ → switch sang new logic

⚠ Lưu ý — Parallel Run chỉ phù hợp cho read operations

Write operations (INSERT, UPDATE) sẽ tạo side effects kép. Cho write operations, dùng **Canary Release** (Ch.12).

Áp dụng cho KBLab: nếu cần viết lại `CompareUtil` (logic so sánh kết quả SQL — critical nhất trong hệ thống), Parallel Run cho phép chạy old `CompareUtil` và new `CompareUtil` song song trên mọi submission, so sánh kết quả, đảm bảo new logic chấm điểm giống hệt trước khi switch.

❗ Nguyên tắc — Make Migration Reversible

“Every migration step should be reversible. If you can’t roll back a change easily, you’re taking on too much risk at once. Feature flags, parallel runs, and tăng dần routing all serve the same purpose: *making it safe to fail*.”

— Sam Newman, *Monolith to Microservices* [4b, Ch.3]

10.6. Migration Roadmap cho KBLab — Tổng hợp xuyên suốt

10.6.1. Tổng hợp Gap Analyses

Xuyên suốt 9 chương đầu, chúng ta đã phân tích gap giữa KBLab hiện tại và best practices. Bảng dưới tổng hợp toàn bộ — đây là “inventory” cho migration roadmap:

Bảng 10.6: Tổng hợp Gap Analyses xuyên suốt Ch.1–12

Chương	Gap	Mức độ	Ref
Ch.2	5 contexts rõ hơn nhưng shared library vẫn coupling	 Medium	§2.6
Ch.3	API naming/versioning chưa nhất quán; SQL Judge worker contract cần chuẩn hóa	 Medium	§3.5
Ch.4	SQL Judge coordinator/worker và external integration cần ACL/resilience rõ hơn	 Medium	§4.5
Ch.5	Kafka pipeline cần DLQ/idempotency; SQL Judge dispatch và DevOps Lab queue cần chọn broker theo workload	 Medium	§5.6
Ch.6	Implicit saga (không có orchestrator)	 Low	§6.4
Ch.7	Shared database (Core ↔ Assignment)	 High	§7.6
Ch.8	Spring Cloud Gateway + Go router cần thống nhất edge policies	 High	§8.5
Ch.9	HS256 shared secret, token storage, secrets, SSO cross-platform	 High	§9.6
Ch.11	Observability có Actuator/Prometheus một phần, chưa end-to-end tracing/log aggregation	 High	§11.7
Ch.12	LMS chính Docker Compose; DevOps Lab k3s/Sysbox; CI/CD và rollback chưa đồng đều	 High	§12.7

10.6.2. Migration Roadmap — 4 Phases

Nguyên tắc ưu tiên: **Quick Wins trước, High-Risk thay đổi sau**. Mỗi giai đoạn mang lại giá trị ngay lập tức — không cần đợi hoàn thành toàn bộ.



Hình 10.10: Migration Roadmap 4 phases — Quick Wins → Observability → Resilience → Decomposition

10.6.2.1. Giai đoạn 1 — Quick Wins (Effort thấp, Impact cao)

Bảng 10.7: Giai đoạn 1 — Quick Wins

#	Action	Gap	Effort	Impact
1	Restrict origins	CORS Ch.8	30 phút	Bịt lỗ hổng bảo mật
2	Unify version trong parent POM	JJWT Ch.8	1 giờ	Ngăn version mismatch bugs
3	Chuyển sang JSON logging (Logback encoder)	Ch.11	2 giờ	Searchable logs
4	Thêm Correlation ID GlobalFilter	Ch.8, Ch.11	4 giờ	Debug cross-service
5	Move secrets ra environment variables	Ch.9	2 giờ	Không leak qua Git

Triết lý Giai đoạn 1: Không thay đổi architecture, không thay đổi code logic — chỉ **cấu hình và hardening**. Team 1 developer có thể hoàn thành trong 1-2 sprint.

10.6.2.2. Giai đoạn 2 — Observability Foundation

Bảng 10.8: Giai đoạn 2 — Observability Foundation

#	Action	Gap	Effort	Impact
1	Deploy Loki + Grafana	Ch.11	1-2 ngày	Search logs một nơi
2	Thêm Micrometer Tracing	Ch.11	1 ngày	Biết bottleneck ở đâu
3	Custom metrics (submission rate, judge duration)	Ch.11	2 ngày	Proactive monitoring

Triết lý Giai đoạn 2: Trước khi thay đổi architecture (**Giai đoạn 3-4**), cần **nhìn thấy** hệ thống đang hoạt động thế nào. “You can’t improve what you can’t measure.”

10.6.2.3. Giai đoạn 3 — Resilience & CI/CD

Bảng 10.9: Giai đoạn 3 — Resilience & CI/CD

#	Action	Gap	Effort	Impact
1	Resilience4j Circuit Breaker cho Feign clients	Ch.4	2-3 ngày	Ngăn cascading failures
2	Kafka Dead Letter Queue + retry	Ch.5	2 ngày	Không mất messages
3	Rate limiting tại Gateway (Redis)	Ch.8	1-2 ngày	Bảo vệ khỏi abuse
4	Basic CI/CD pipeline (GitHub Actions)	Ch.12	3-5 ngày	Automated build + deploy

Triết lý Giai đoạn 3: Hệ thống phải **resilient trước khi decompose**. Tách database trong khi không có circuit breaker = cascade failures khi một DB down.

10.6.2.4. Giai đoạn 4 — Database Decomposition (Thận trọng nhất)

Bảng 10.10: Giai đoạn 4 — Database Decomposition

#	Action	Gap	Effort	Impact
1	Schema separation (Core vs Assignment tables)	Ch.7	1-2 tuần	Logical boundary
2	API-based access (Assignment gọi Core API)	Ch.7	2-3 tuần	Remove direct DB access
3	Refactor shared library (chỉ giữ DTOs, exceptions)	Ch.2	2-3 tuần	Reduce coupling
4	Outbox pattern cho submission pipeline	Ch.5, Ch.6	1-2 tuần	Reliable messaging

Triết lý Giai đoạn 4: Đây là thay đổi **riskiest** — phải có observability (Giai đoạn 2) và resilience (Giai đoạn 3) trước. Mỗi bước: implement → monitor → stabilize → bước tiếp.

10.6.3. Decision Matrix — Ưu tiên bằng Impact × Effort



Hình 10.11: Decision Matrix — ưu tiên theo Impact × Effort

 Tip — Migration là marathon, không phải sprint

Đừng cố fix mọi gap cùng lúc. Mỗi sprint chọn 1-2 items từ quadrant “High Impact, Low Effort” trước. Khi đã hết quick wins, mới chuyển sang “High Impact, High Effort” — và **luôn có observability** trước khi thay đổi lớn.

10.7. Sai lầm thường gặp khi Migration

 Lưu ý —

Ngoài 4 anti-patterns ở **Bảng 10.1b** (§10.1), những sai lầm sau cũng phổ biến trong quá trình migration:

1. **Distributed Monolith** — Tách services nhưng giữ shared database, deploy phải đồng bộ. Newman trong [4b, Ch.1] gọi đây là “the worst of both worlds”. *Phòng tránh*: database-per-service là tiêu chí quyết định.
2. **Big Bang Rewrite** — Viết lại toàn bộ từ zero, ngày “switch” không bao giờ đến. *Phòng tránh*: Strangler Fig (§10.2) — mỗi sprint tách một phần, system luôn deployable.
3. **Bỏ qua Conway’s Law** — Tách services nhưng team structure vẫn như cũ. *Phòng tránh*: tổ chức team theo bounded context (Ch.2) — mỗi team sở hữu 1-2 services.
4. **“Lift and Shift” không redesign** — Copy code nguyên xi vào containers, coi như “đã microservices”. *Phòng tránh*: Anti-Corruption Layer (§10.4) ngăn legacy model lan sang.
5. **Over-engineering infrastructure** — Deploy K8s + Istio cho 3 services với 100 req/min. *Phòng tránh*: bắt đầu đơn giản (Docker Compose, Ch.12), scale khi traffic thực sự đòi hỏi.

 Thực quan hóa tương tác (Interactive Demo)

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/strangler-fig-migration.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Chiến lược Strangler Fig Pattern**.

10.8. Tổng kết

Migration từ monolith sang microservices không phải quyết định kỹ thuật thuần túy — đây là sự kết hợp giữa **team organization** (Conway’s Law, Ch.2), **domain understanding** (Bounded Contexts, Ch.2), **communication patterns** (Ch.3-6), **data strategy** (Ch.7), **infrastructure readiness** (Ch.8-9), và **operational maturity** (Ch.11-12).

Bài học quan trọng nhất: **“Monolith First”** — bắt đầu đơn giản, migrate khi có lý do cụ thể, migrate tăng dần (Strangler Fig), và mỗi bước phải reversible. Big Bang rewrites gần như luôn thất bại; Strangler Fig gần như luôn thành công — vì nó cho phép fail nhỏ, learn nhanh, và deliver giá trị liên tục.

Tách database là thách thức lớn nhất — code coupling sửa được bằng refactoring, nhưng data coupling đòi hỏi chiến lược migration cẩn thận: schema separation → view abstraction → API-

based access → data duplication → separate instances. Outbox Pattern giải quyết bài toán reliable messaging trong quá trình migration — tránh dual-write pitfall.

Anti-Corruption Layer, Branch by Abstraction, và Parallel Run là toolkit cho migration an toàn — mỗi pattern phục vụ mục đích khác nhau nhưng chia sẻ nguyên tắc chung: **make migration reversible, make each step small, make the system always deployable**.

Migration Roadmap cho KBLab — tổng hợp gap analyses từ Ch.1-9 — cho thấy: Quick Wins (CORS, JWT, structured logging) có thể hoàn thành trong 1-2 tuần, mang lại giá trị ngay. Database decomposition — thay đổi lớn nhất — cần observability và resilience foundation trước khi bắt đầu. DevOps Lab cho thấy cùng một hệ thống có thể cần chiến lược khác nhau theo bounded context: Docker Compose đủ tốt cho LMS chính, trong khi lab isolation cần k3s/Sysbox. Mỗi decision là trade-off — không có “đúng tuyệt đối”, chỉ có “phù hợp với context tại thời điểm đó”.

Ở Chương 11, chúng ta sẽ xem cách **quan sát và giám sát** hệ thống sau migration — logging, metrics, tracing — ba trụ cột giúp phát hiện và xử lý vấn đề trước khi user bị ảnh hưởng.

10.9. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Toàn bộ sách: decomposition patterns, database splitting, migration strategies
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.13: Refactoring to Microservices — Strangler Fig, Anti-Corruption Layer
3. [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services (decomposition), Ch.5: Splitting the Monolith

Sách bổ trợ:

4. [6] Eric Evans, *Domain-Driven Design* — Ch.14: Anti-Corruption Layer, Context Mapping
5. [3] Ronnie Mitra, *Microservices: Up and Running* — Ch.3: Architecture Design, migration planning
6. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.2: Migration Anti-patterns; Ch.13: Refactoring to Microservices; Ch.21: Strangler Fig (updated)

Nguồn trực tuyến:

- [W1] Martin Fowler, “MonolithFirst” — martinfowler.com/bliki/MonolithFirst.html
- [W2] Martin Fowler, “StranglerFigApplication” — martinfowler.com/bliki/StranglerFigApplication.html
- Sam Newman, “Breaking Apart the Monolith” — samnewman.io
- Debezium (CDC) — debezium.io
- Martin Fowler, “BranchByAbstraction” — martinfowler.com/bliki/BranchByAbstraction.html

11. Observability

“Observability is not just monitoring — it’s the ability to ask new questions about your system’s behavior without deploying new code.” — Charity Majors (tổng hợp từ observability community)

11.1. Bạn sẽ học được gì

- Hiểu tại sao observability là **điều kiện tiên quyết** để vận hành microservices — không chỉ là “nice-to-have”
 - Phân biệt **observability vs monitoring** và khi nào cần gì
 - Nắm vững ba trụ cột: **Logs, Metrics, Traces** và cách chúng hỗ trợ nhau
 - Thiết kế chiến lược **centralized logging** với structured logs và correlation IDs
 - Hiểu **distributed tracing** và các khái niệm trace, span, context propagation
 - Nắm framework đo lường: **SLI/SLO/SLA** và chiến lược alerting
 - Xây dựng **error handling strategy** nhất quán xuyên services
 - Phân tích observability gaps trong KBLab và đề xuất migration path
-

11.2. Ba trụ cột của Observability

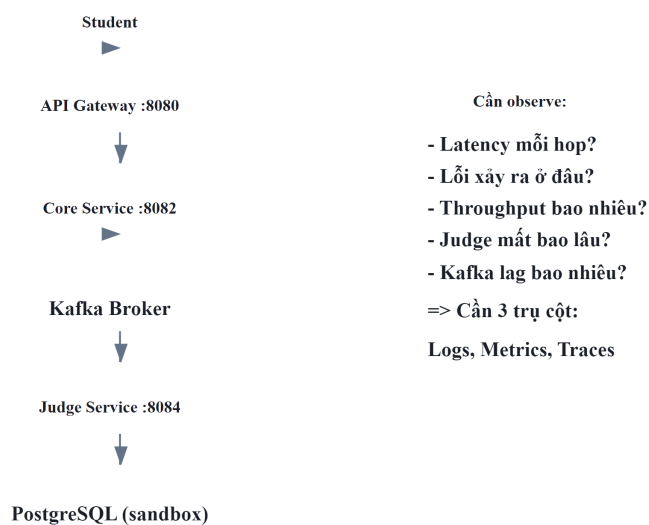
11.2.1. Vấn đề: “hệ thống lỗi nhưng không biết lỗi ở đâu”

Trong monolith, khi có bug: mở log file duy nhất, tìm exception, xem stack trace — xong. Toàn bộ request chạy trong cùng process, cùng log file, cùng database.

Trong microservices, một request từ user có thể đi qua nhiều services, mỗi service có log riêng, database riêng, deploy cycle riêng. Khi request fail:

- Service nào gây lỗi? Gateway? Core? Judge? Auth?
- Lỗi xảy ra ở bước nào trong chuỗi gọi?
- Latency 5 giây — do network, do database query, hay do Kafka consumer lag?

LMS Request Flow — Observability context



Hình 11.1: Request flow qua 5+ services — lỗi xảy ra ở đâu?

Newman trong [4a, Ch.8] gọi đây là thách thức lớn nhất khi vận hành microservices: **sự phức tạp không nằm ở code, mà nằm ở tương tác giữa các services**. Trong monolith, lỗi thường có stack trace rõ ràng — một file, một method. Trong microservices, lỗi có thể xuất hiện ở service B vì service A gửi data sai, hoặc vì Kafka message bị delay — không stack trace nào bắt được.

11.2.2. Ba trụ cột

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đều mô tả observability qua **ba trụ cột** (three pillars):

11.2.3. Observability vs Monitoring

Bảng 11.2: Observability vs Monitoring

	Monitoring	Observability
Mục đích	Phát hiện vấn đề đã biết	Khám phá vấn đề chưa biết
Cách tiếp cận	Đặt alerts cho metrics cụ thể	Explore data để hiểu hành vi hệ thống
Ví dụ	“CPU > 80% → alert”	“Tại sao latency tăng 3x ở 2PM mỗi thứ 3?”
Dữ liệu	Pre-defined dashboards	Logs + Metrics + Traces (kết hợp ad-hoc)
Khi nào đủ	Hệ thống đơn giản, failure modes biết trước	Hệ thống phân tán, failure modes không dự đoán được

Monitoring trả lời: “*Có vấn đề không?*” Observability trả lời: “*Tại sao có vấn đề này, và vấn đề này khác gì so với lần trước?*” Với monolith, monitoring thường đủ. Với microservices — nơi failure modes phức tạp và không dự đoán được — observability là yêu cầu bắt buộc.

⚠ Nguyên tắc — Observability ≠ Monitoring

“Monitoring tells you *that* something is wrong. Observability lets you ask *why*. Monitoring is a subset of observability — necessary but not sufficient.”

— Tổng hợp từ *Charity Majors* và cộng đồng *observability*

11.2.4. Observability Maturity Model

Không phải hệ thống nào cũng cần full observability stack ngay từ đầu. Maturity model giúp xác định *đang ở đâu* và *cần đến đâu*:

Bảng 11.3: Observability Maturity Model

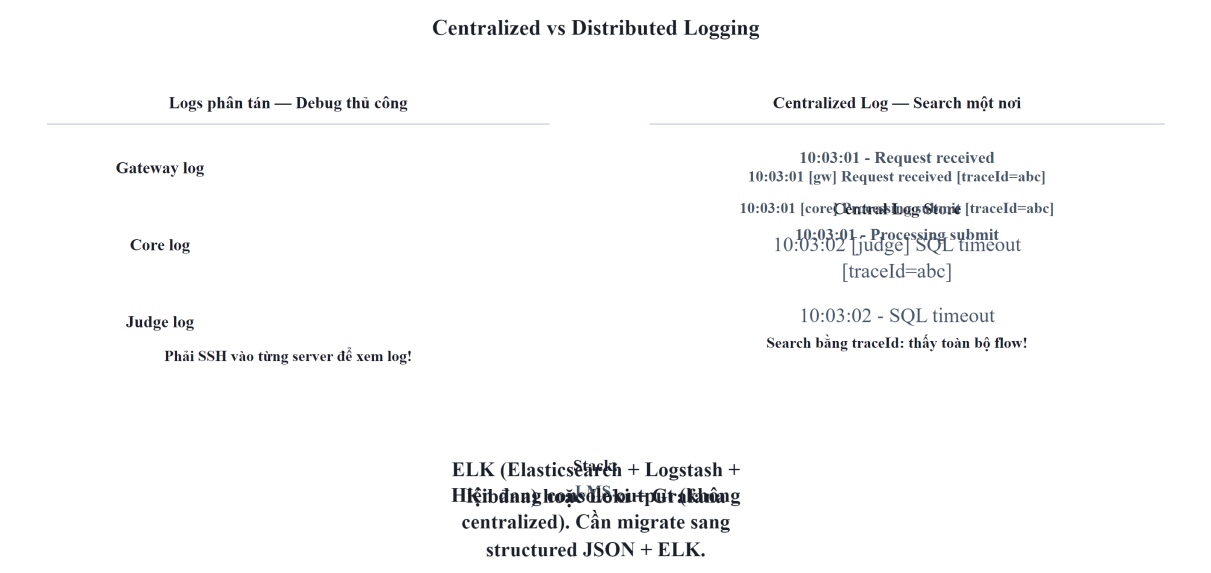
Level	Mô tả	Đặc điểm	Phù hợp với
1 — Reactive	Debug khi user báo lỗi	Console logs, không alert, manual investigation	Side project, prototype
2 — Basic monitoring	Dashboards cơ bản, alerts thô	Health checks, CPU/memory metrics, log tập trung	Team nhỏ, hệ thống ít services
3 — Proactive	Phát hiện vấn đề trước user	Structured logs, correlation IDs, custom metrics, alerting	Team trung bình, nhiều services
4 — Observability	Hỏi câu hỏi mới không cần code mới	Distributed tracing, SLI/SLO, high-cardinality data	Team lớn, hệ thống phức tạp
5 — Predictive	Dự đoán vấn đề trước khi xảy ra	ML-based anomaly detection, capacity planning	Enterprise, mission-critical

Với LMS — hệ thống giáo dục quy mô trung bình — mục tiêu hợp lý là **Level 3** (Proactive), với khả năng mở rộng lên Level 4 khi cần.

11.3. Logging — Từ Console đến Centralized Aggregation

11.3.1. Vấn đề: logs phân tán, không có context

Khi hệ thống có 7+ services chạy đồng thời, mỗi service ghi log vào container riêng. Để debug một request fail, developer phải SSH vào từng container, đọc logs, cố gắng ghép thời gian — rất khó với concurrent requests và clock skew giữa containers.



Hình 11.3: Logs phân tán (manual debug) vs Logs tập trung — Search một nơi

11.3.2. Structured Logging — Từ text sang JSON

Bước đầu tiên: chuyển từ **text logs** sang **structured logs** (JSON). Text logs dễ đọc cho người nhưng *rất khó phân tích tự động*:

Listing 11.1: Ví dụ Structured Log (JSON)

```
// ❌ Text log – khó parse, khó filter, khó aggregate
2024-01-15 10:03:02 ERROR [core-service] - SQL execution failed for submission 8a3f
    user 12ab, Timeout after 30s

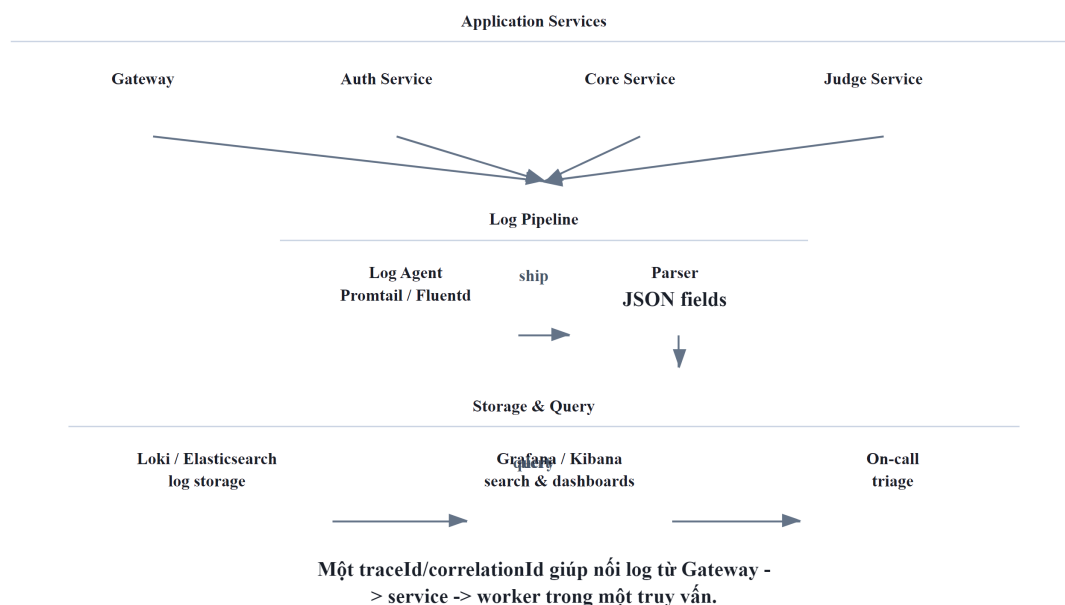
// ✅ Structured log (JSON) – dễ query, filter, aggregate
{"timestamp":"2024-01-15T10:03:02Z","level":"ERROR","service":"core-service",
  "message":"SQL execution failed","submissionId":"8a3f","userId":"12ab",
  "duration_ms":30000,"error":"QueryTimeoutException","traceId":"abc123"}
```

Với structured logs, có thể query: “*Tất cả errors trong 24h qua có error = QueryTimeoutException*” hoặc “*Average duration_ms của submissions theo giờ*” — không thể với text logs.

11.3.3. Centralized Log Aggregation

Sau khi có structured logs, bước tiếp theo là **centralized log aggregation** — gom logs từ tất cả services về một nơi. Newman trong [4a, Ch.8] nhấn mạnh: *centralized logging là yêu cầu bắt buộc* cho microservices, không phải optional.

Hình 11.4: Pipeline gom logs tập trung — từ service đến storage
Centralized Log Aggregation Pipeline



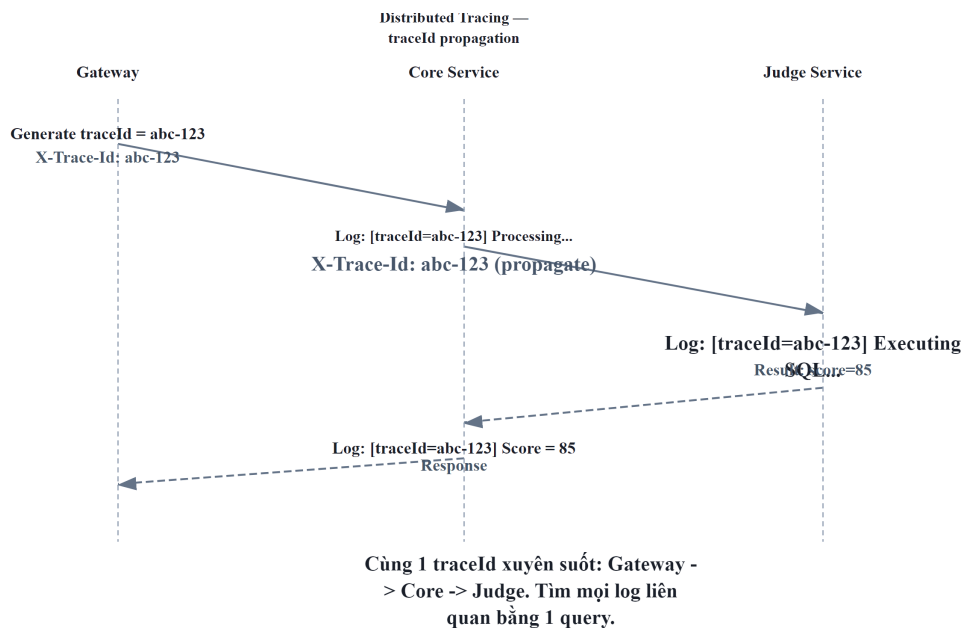
Hai stack phổ biến nhất:

Bảng 11.4: So sánh hai stack ELK và PLG

Stack	Components	Ưu điểm	Nhược điểm	Phù hợp
ELK	Elasticsearch + Logstash + Kibana	Mature, full-text search mạnh	Resource-heavy (~4GB RAM minimum)	Team lớn, nhiều logs
PLG	Promtail + Loki + Grafana	Lightweight, tích hợp metrics	Query kém linh hoạt hơn ELK	Team nhỏ-trung, KBLab

11.3.4. Correlation IDs — Kỹ thuật quan trọng nhất

Newman trong [4a, Ch.8] mô tả **Correlation ID** (hay Trace ID) là kỹ thuật quan trọng nhất cho debugging microservices: một identifier duy nhất gắn vào request từ entry point (Gateway), truyền qua tất cả services qua HTTP headers và Kafka message headers.



Hình 11.5: Truyền Correlation ID xuyên suốt request, kể cả qua Kafka

Cơ chế hoạt động:

1. **Gateway** generate UUID cho mỗi request incoming, gắn vào HTTP header `X-Correlation-Id`
2. **Downstream services** đọc header, ghi vào **MDC** (Mapped Diagnostic Context) — logging framework tự động gắn kèm vào mọi log entries
3. **Kafka messages** truyền correlation ID qua message headers — nối async flow
4. **Khi debug:** search `traceId = abc-123` trong centralized log → thấy toàn bộ hành trình

! Nguyên tắc — Every Request Gets a Trace ID

“Ensure that every request entering your system gets a unique identifier, and that this identifier is passed along to all downstream services. Without this, debugging distributed systems is virtually impossible.”

— Sam Newman, *Building Microservices* [4a, Ch.8]

11.4. Distributed Tracing — Theo dõi Request xuyên Services

11.4.1. Vấn đề: request chậm — chậm ở đâu?

Correlation IDs giúp nối logs lại — nhưng không cho biết **mỗi bước mất bao lâu**. Khi student submit bài và response mất 8 giây (bình thường 2 giây), developer biết request đi qua Gateway → Core → Judge → MySQL, nhưng không biết 6 giây thừa nằm ở bước nào mà không đọc timestamps thủ công.

Bảng 11.6: Sampling strategies cho Tracing

Strategy	Mô tả	Khi nào dùng	
Head-based	Quyết định sample tại entry point (Gateway)	Đơn giản, dễ implement	
Tail-based	Quyết định sau khi trace hoàn thành (giữ traces lỗi, bỏ traces thành công)	Capture 100% errors,	giảm storage
Rate-based	Sample N% requests	Production (1-10%)	high-traffic
Adaptive	Tự động điều chỉnh sample rate theo traffic volume	Enterprise, workloads	dynamic

Với LMS: contest mode (100+ concurrent users) nên dùng rate-based 10-20%; ngoài contest dùng 100%.

11.4.5. Tracing trong Spring Boot 3.x

Spring Boot 3.x tích hợp sẵn **Micrometer Tracing** — abstraction layer cho distributed tracing, hỗ trợ nhiều backend (Zipkin, Jaeger, OpenTelemetry). Chỉ cần thêm dependency và cấu hình endpoint — Spring Boot **tự động** truyền trace context qua HTTP headers (**traceparent**) và Kafka headers. Không cần viết code tracing thủ công cho phần lớn use cases.

! Nguyên tắc — Tracing Complements Logging

“Tracing cho biết request *đi đâu* và *mất bao lâu*. Logging cho biết *tại sao*. Dùng trace để identify bottleneck, dùng logs để understand root cause. Không cái nào thay thế cái nào.”

— Tổng hợp từ Richardson [2a, Ch.11] và Newman [4a, Ch.8]

11.5. Metrics, SLI/SLO, và Alerting Strategy

11.5.1. Metrics — Đo lường hành vi theo thời gian

Logs ghi lại *sự kiện đã xảy ra*. Metrics đo lường *xu hướng theo thời gian* — quan trọng cho **early detection**: phát hiện vấn đề trước khi user báo lỗi.

Richardson trong [2a, Ch.11] mô tả bốn loại metrics cần thu thập:

Bảng 11.7: Bốn loại metrics cần thu thập

Pattern	Mô tả	Ví dụ KBLab
Health Check API	Endpoint /health cho orchestrator biết service sống/chết	Eureka dùng health check để route traffic
Application metrics	Business-level metrics	Submission rate, judge duration, error rate
Infrastructure metrics	System-level metrics	CPU, memory, DB connections, Kafka consumer lag
Audit logging	Ghi lại hành vi user cho compliance	User login, role changes, data access

11.5.2. Hai framework đo lường: RED và USE**Bảng 11.8:** Framework RED và USE

Framework	Dùng cho	Metrics	Ví dụ KBLab
RED	Services (request-facing)	Rate , Duration	Errors , Submission rate: 50 req/min, Error rate: 2%, P99 latency: 3.2s
USE	Resources (infrastructure)	Utilization , Saturation , Errors	CPU: 45%, DB connection pool: 8/10 active, Disk I/O errors: 0

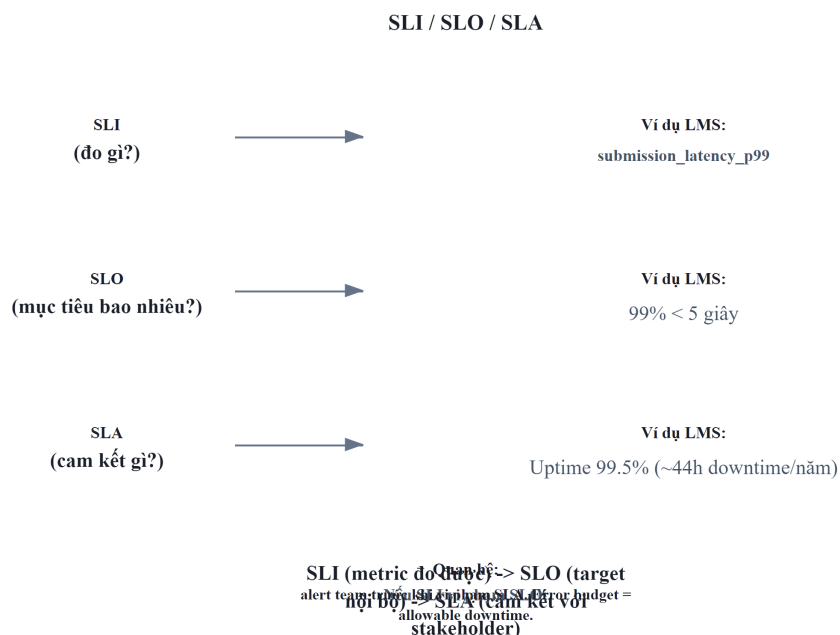
Kết hợp RED (cho services) + USE (cho resources) cho bức tranh đầy đủ. Metrics phát hiện “submission latency tăng” (RED), investigation cho thấy “DB connection pool saturated” (USE) → root cause: connection leak.

11.5.3. SLI/SLO/SLA — Từ metrics đến cam kết chất lượng

Metrics chỉ hữu ích khi biết **ngưỡng nào là chấp nhận được**. Framework SLI/SLO/SLA cung cấp ngôn ngữ chung:

Bảng 11.9: Sự khác biệt giữa SLI, SLO và SLA

Khái niệm	Mô tả	Ví dụ KBLab
SLI (Service Level Indicator)	<i>Metric cụ thể</i> đo chất lượng dịch vụ	Latency P99 của submission flow
SLO (Service Level Objective)	Mục tiêu nội bộ cho SLI	“99% submissions trả kết quả trong < 5 giây”
SLA (Service Level Agreement)	Cam kết với khách hàng (có hậu quả nếu vi phạm)	“Hệ thống available 99.5% thời gian trong semester”



Hình 11.7: Mối quan hệ giữa SLI, SLO và SLA

Với LMS — hệ thống giáo dục nội bộ — SLO phù hợp:

Bảng 11.10: SLO đề xuất cho KBLab

SLI	SLO	Lý giải
Submission latency P99	< 5 giây	Student chờ lâu hơn → frustration, bỏ cuộc
Availability trong contest	99.9% (~8.7h downtime/năm)	Contest có thời gian giới hạn — downtime ảnh hưởng trực tiếp điểm
Grading accuracy	100%	Chấm sai = ảnh hưởng điểm học thuật — không chấp nhận sai
API error rate	< 1%	Lỗi thường xuyên → mất niềm tin vào hệ thống

11.5.4. Alerting Strategy

Metrics + SLO tạo nền tảng cho alerting. Nhưng alert sai cách gây **alert fatigue** — team ignore mọi alerts vì quá nhiều false positives.

Bảng 11.11: Chiến lược Alerting theo mức độ nghiêm trọng

Level	Khi nào	Kênh	Hành động	Ví dụ
Critical	SLO bị vi phạm, impact user ngay	PagerDuty, phone call	On-call engineer xử lý ngay	Judge Service down, submission fails
Warning	Gần vi phạm SLO, cần attention	Slack, email	Review trong working hours	Latency P99 đạt 80% threshold
Info	Anomaly nhưng không impact user	Dashboard only	Review khi rảnh	Kafka consumer lag tăng nhẹ

 Nguyên tắc — Alert on SLOs, Not Symptoms

“Alert khi SLO có nguy cơ bị vi phạm, không phải khi một metric thay đổi. CPU 90% nhưng latency bình thường? Không cần alert. CPU 50% nhưng latency vượt SLO? Alert ngay.”

— Nguyên tắc SRE, dựa trên Google SRE book

11.6. Error Handling Strategy — Nhất quán xuyên Services

11.6.1. Vấn đề: mỗi service trả error format khác nhau

Khi hệ thống có nhiều services do nhiều developer phát triển, error responses dễ bị *inconsistent*. Service A trả error trong field `description`, Service B trả trong `message`, Gateway trả format hoàn toàn khác. Client (frontend) phải xử lý nhiều format — code rối, UX tệ.

11.6.2. Nguyên tắc thiết kế Error Handling

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đề xuất các nguyên tắc cho error handling trong microservices:

- 1. Centralized exception handler** — Mỗi service có một handler tập trung (trong Spring Boot: `@ControllerAdvice`) xử lý tất cả exceptions → chuyển thành format chuẩn. Đặt trong shared library để mọi service dùng chung.
- 2. Error code taxonomy** — Phân loại errors theo namespace để client và monitoring system hiểu nhanh:

Bảng 11.12: Taxonomy hệ thống mã lỗi (Error Code)

Range	Category	Ví dụ
1xxx	Authentication & Authorization	1001: Unauthorized, 1002: Token expired
2xxx	Resource not found	2001: Question not found, 2002: Contest not found
3xxx	Business logic violation	3001: Contest chưa bắt đầu, 3002: Hết lượt nộp
4xxx	Infrastructure / dependency	4001: Judge service unavailable, 4002: Kafka send failed
9xxx	Internal errors	9999: Unexpected error

3. Unified response format — Tất cả error responses dùng cùng cấu trúc:

Listing 11.2: Unified Error Response payload

```
//  Format nhất quán – client chỉ cần parse một structure
{
  "errorCode": 4001,
  "description": "Judge service unavailable",
  "timestamp": "2024-01-15T10:03:02Z",
  "traceId": "abc-123"
}
```

4. Never leak internals — Stack traces, SQL queries, internal paths không bao giờ xuất hiện trong error response. Log chi tiết ở server side (với traceId để correlation), trả thông báo generic cho client.

11.6.3. Error Handling và Observability — Mối liên hệ

Error codes tập trung không chỉ phục vụ client — chúng là **nguồn dữ liệu cho metrics và alerting**:

- Count errors by code → phát hiện patterns: JUDGE_UNAVAILABLE tăng đột biến → alert
- Group errors by category → dashboard: authentication issues vs business logic vs infrastructure
- Error rate per service → SLI cho reliability

 Phân tích — Error format inconsistency trong KBLab

KBLab sử dụng @ControllerAdvice với ErrorCode enum trong shared library — đúng pattern. Tuy nhiên, **Gateway** (Spring Cloud Gateway dùng WebFlux) xử lý JWT errors *bên ngoài* @ControllerAdvice (WebMVC pattern). Kết quả: Gateway trả error dạng { "message": "..."} trong khi các services trả { "description": "..."} — field name khác nhau. Frontend phải handle cả hai formats.

Migration path: (1) Tạo GatewayExceptionHandler riêng cho WebFlux, chuẩn hóa response format, (2) Đảm bảo tất cả error responses dùng cùng field names, (3) Thêm traceId vào error response để client có thể report cho support team.

11.7. Health Checks và Service Discovery

11.7.1. Health Checks — Nền tảng cho tự động hóa

Richardson trong [2a, Ch.11] mô tả **Health Check API** là observability pattern cơ bản nhất: mỗi service expose endpoint `/health` để orchestration tools biết service có hoạt động không.

Health checks phân loại theo mục đích:

Bảng 11.13: Ba loại Health Checks

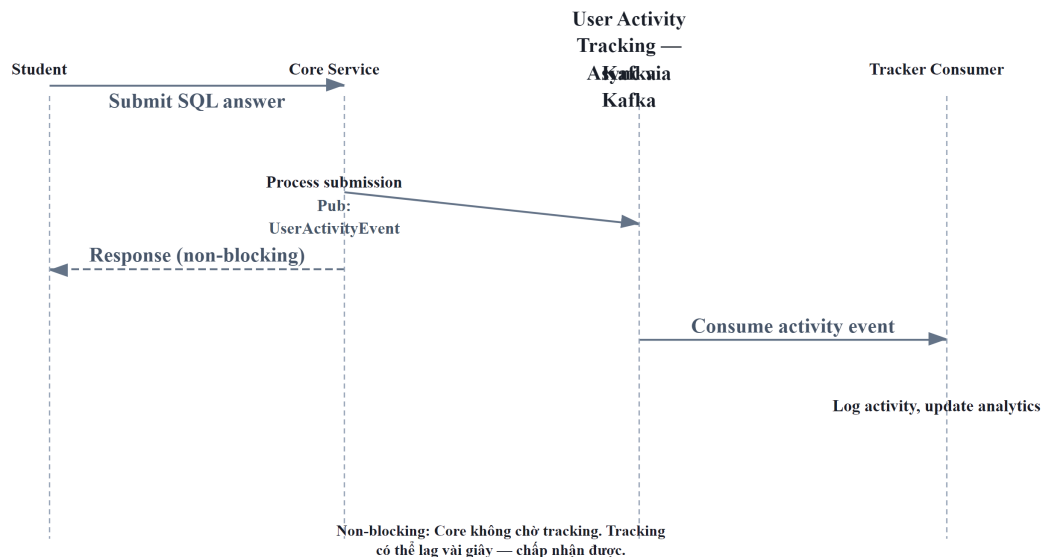
Loại	Câu hỏi trả lời	Ý nghĩa cho orchestrator	Ví dụ KBLab
Liveness	“Process còn chạy?”	Restart container nếu dead	JVM alive, không deadlock
Readiness	“Sẵn sàng nhận traffic?”	Ngừng route traffic nếu not ready	Database connected, Kafka connected
Startup	“Đã khởi tạo xong?”	Chờ startup xong mới check liveness	Schema migration done, cache warmed

Trong KBLab, **Eureka** (Service Discovery) sử dụng health checks để quyết định routing: nếu Core Service instance báo **DOWN**, Eureka ngừng route traffic đến instance đó — Gateway tự động chuyển sang instance khác. DevOps Lab bổ sung health/readiness ở runtime layer để router chỉ chuyển traffic đến workspace sẵn sàng. Đây là nền tảng cho **self-healing**: hệ thống tự phục hồi mà không cần can thiệp thủ công.

11.7.2. User Activity Tracking — Observability ở mức Business

Ba trụ cột (logs, metrics, traces) giúp quan sát **hệ thống kỹ thuật**. Nhưng với KBLab — một LMS — cần thêm: quan sát **hành vi học tập** (learning analytics). Sinh viên đang học gì? Bài nào khó nhất? Khi nào sinh viên hoạt động nhiều nhất?

KBLab đã triển khai **User Activity Tracking** qua Kafka pipeline: khi sinh viên xem câu hỏi, nộp bài, hoặc tham gia contest, Core Service publish tracking events lên Kafka topic. Consumer ghi vào database — hoàn toàn **async, non-blocking**, không ảnh hưởng response time.



Hình 11.8: User Activity Tracking pipeline qua Kafka

Bảng 11.14: Ứng dụng của User Activity Tracking

Data	Câu hỏi trả lời được	Ứng dụng
View patterns	“Câu hỏi nào được xem nhiều nhất?”	Đề xuất bài tập phù hợp
Submit patterns	“Sinh viên nào nộp bài nhiều lần?”	Phát hiện sinh viên cần hỗ trợ
Time patterns	“Peak hours là khi nào?”	Scale infrastructure phù hợp
Contest behavior	“Bao nhiêu % sinh viên hoàn thành contest?”	Đánh giá hiệu quả giảng dạy

Đây là điểm giao giữa technical observability và **domain-specific analytics** — một giá trị riêng mà observability mang lại cho LMS, vượt xa việc chỉ phục vụ ops team.

11.7.3. Chaos Engineering — Kiểm tra Resilience chủ động

Observability cho phép **phát hiện** sự cố. Nhưng làm sao biết hệ thống sẽ phản ứng thế nào *trước* khi sự cố thật xảy ra? **Chaos Engineering** — phương pháp được Netflix phổ biến qua Chaos Monkey (2011) — trả lời câu hỏi này bằng cách **chủ động inject failures** vào production/staging và quan sát phản ứng.

Nguyên tắc (theo *Principles of Chaos Engineering* — principlesofchaos.org):

1. **Xác định “steady state”** — hệ thống hoạt động bình thường trông như thế nào? (metrics: latency < 200ms, error rate < 0.1%, throughput > 100 req/s)
2. **Đặt giả thuyết** — “Nếu Judge Service chết, submissions vẫn được queue và xử lý khi Judge restart”
3. **Inject failure** — Kill Judge Service container
4. **Observe** — Steady state có bị phá vỡ không? System recover trong bao lâu?

5. **Fix weaknesses** — Nếu steady state bị phá vỡ → fix system, không phải fix test

Bảng 11.15: Phân loại Failure Injection trong Chaos Engineering

Loại failure injection	Ví dụ	Tool
Process failure	Kill container/service	Chaos Monkey, <code>docker stop</code>
Network failure	Delay, packet loss, partition	Toxiproxy, Gremlin
Resource exhaustion	CPU 100%, disk full, memory leak	stress-ng, Gremlin
Dependency failure	Database down, Kafka unavailable	Litmus, manual

Áp dụng cho KBLab — ba chaos experiments hữu ích:

Bảng 11.16: Chaos experiments cho KBLab

Experiment	Giả thuyết	Kết quả mong đợi
Kill Judge Service	Submissions queue trong Kafka, xử lý khi restart	Kafka retention giữ messages, Judge catch up
Network delay 500ms giữa Gateway↔Core	Users experience slower response nhưng không lỗi	Circuit breaker/timeout ngăn cascade
PostgreSQL restart	Core Service reconnect tự động	Connection pool (HikariCP) handle reconnection

💡 Tip — Bắt đầu Chaos Engineering từ staging

Không cần Chaos Monkey hoặc tools phức tạp. Bắt đầu đơn giản: `docker stop judge-service` trên staging → quan sát logs → xem system recover không. Đây đã là chaos engineering. Khi team quen, nâng lên: automated chaos experiments chạy trong CI/CD pipeline.

11.7.4. Industry Case Study: Netflix Simian Army

Netflix là pioneer của Chaos Engineering. Năm 2010, sau khi migrate từ datacenter sang AWS, Netflix tạo **Chaos Monkey** — tool tự động kill random EC2 instances trong production. Mục đích: nếu system survive random failures hàng ngày, nó sẽ survive failures thật.

Chaos Monkey phát triển thành **Simian Army** — tập hợp tools chuyên biệt:

Bảng 11.17: Netflix Simian Army

Tool	Chức năng	Bài học
Chaos Monkey	Kill random instances	Services phải stateless, auto-restart
Chaos Kong	Simulate entire region failure	Multi-region failover phải hoạt động
Latency Monkey	Inject network delay	Timeout + circuit breaker phải configured đúng
Conformity Monkey	Kiểm tra instance tuân thủ best practices	Automation replaces manual auditing

Kết quả: Netflix đạt **99.99% availability** (52 phút downtime/năm) — phục vụ 200+ triệu subscribers. Casey Rosenthal (Netflix Director of Engineering) sau đó founder Verica và co-author *Chaos Engineering: System Resiliency in Practice* (O'Reilly, 2020).

Bài học cho hệ thống nhỏ: LMS không cần Simian Army. Nhưng nguyên tắc cốt lõi áp dụng: nếu bạn sợ kill một service, đó là dấu hiệu hệ thống thiếu resilience. Bắt đầu manual (docker stop), tiến tới automated (CI/CD chaos stage).

Nguồn: Netflix Technology Blog, “The Netflix Simian Army,” 2011 (netflixtechblog.com). Casey Rosenthal et al., *Chaos Engineering: System Resiliency in Practice*, O'Reilly, 2020.

11.8. Case Study: Observability trong KBLab

11.8.1. Hiện trạng — Đánh giá theo Maturity Model

LMS Observability — Hiện trạng				
Logging	Metrics	Tracing	Error Handling	User Tracking
Spring Boot default Chưa cấu hình Chưa structured	Actuator basic (health, info) Chưa chuẩn Prometheus Thiếu business metrics	Chưa có Thiếu correlation ID Thiếu trace propagation	Centralized handler ErrorCode enum Gateway inconsistent	Kafka-based async Non-blocking Chỉ basic events
PARTIAL	BASIC	MISSING	PARTIAL	PARTIAL

Lộ trình cải thiện:

1. Structured logging (JSON) + ELK
2. Actuator + Prometheus + Grafana
3. Correlation ID + Microservice Tracing
4. Centralized error handling
5. OpenTelemetry (all-in-one)

Hình 11.9: Đánh giá hiện trạng Observability của KBLab

Đánh giá tổng thể: Level 2 đang tiến lên Level 3 — KBLab đã có Actuator/health ở một số services và nền tảng Prometheus/Grafana cho một phần hệ thống, nhưng vẫn cần chuẩn hóa correlation ID, log aggregation và tracing xuyên boundary. Khi submission chậm hoặc fail, developer không nên phải mở logs từng container và ghép timestamps thủ công.

Bảng 11.18: Mức độ trưởng thành observability phân theo component

Component	Hiện trạng	Maturity Level
Logging	Java services có log qua MDC ở một số nơi; DevOps Lab dùng zerolog; cần gom tập trung	🟡 Level 2
Metrics	Actuator/health + Prometheus/Grafana ở một phần hệ thống	🟡 Level 2
Tracing	Correlation ID có nền tảng nhưng chưa chuẩn hóa end-to-end	🟡 Level 2
Error handling	@ControllerAdvice + ErrorCode tốt, Gateway inconsistent	🟢 Level 3
User tracking	Kafka-based async — tốt	🟢 Level 3

11.8.2. Phân tích theo business context

KBLab phục vụ **sinh viên học SQL, mạng, DevOps và các hoạt động LMS** — có nhiều mode hoạt động với yêu cầu observability khác nhau:

Bảng 11.19: Hai chế độ hoạt động chính của LMS

Mode	Đặc điểm	SLO cần thiết	Observability cần
Practice mode	Sinh viên tự luyện tập, traffic thấp, không deadline	Latency < 10s OK, downtime chấp nhận được	Basic monitoring đủ
Contest mode	100+ sinh viên submit đồng thời, có deadline, leaderboard real-time	Latency < 5s, availability 99.9%	Metrics + alerts bắt buộc
DevOps runtime	Lab Workspace/container/network runtime biến động	Readiness chính xác, isolation lỗi	Promtail/Loki/Grafana + runtime metrics

Trong contest mode, **không có observability = bay mù**:

- Judge Service overloaded → submissions queue up → leaderboard outdated → sinh viên phàn nàn
- Không có custom metrics → không biết judge duration tăng cho đến khi student report
- Không có tracing → không biết bottleneck ở Judge hay ở Kafka consumer

① Phân tích — Observability chưa chuẩn hóa end-to-end

KBLab đã có một số mảnh observability đúng hướng: Actuator/Prometheus cho Java services, X-Request-ID/MDC ở shared layer, zerolog ở Go services, và PLG stack (Promtail/Loki/Grafana) cho DevOps Lab. Gap hiện tại là **chuẩn hóa end-to-end**: cùng một request/batch cần đi qua Gateway → service → Kafka/DB queue → Judge/runtime mà vẫn giữ được trace/correlation ID.

Migration path (incremental — mỗi phase đều mang lại giá trị ngay, target: Level 3):

Phase 1 — Structured Logging + Correlation ID (effort thấp, impact cao):

- Chuẩn hóa JSON logging cho Java services và zerolog field names cho Go services
- Thêm/chuẩn hóa X-Request-ID hoặc X-Correlation-Id tại Gateway/router, propagate qua HTTP/Kafka/DB queue headers
- Giá trị ngay: debug cross-service bằng cách search một traceId

Phase 2 — Centralized Log Aggregation (effort trung bình):

- Chuẩn hóa Promtail + Loki + Grafana cho cả LMS core và DevOps Lab
- Cấu hình Docker/k3s logging pipeline → push logs về Loki
- Giá trị ngay: search logs một nơi thay vì SSH vào từng container

Phase 3 — Distributed Tracing (effort trung bình):

- Thêm Micrometer Tracing + OpenTelemetry vào mỗi service (Spring Boot 3.x hỗ trợ sẵn)
- Deploy Jaeger hoặc Zipkin làm trace backend
- Giá trị ngay: biết chính xác bottleneck khi submission chậm — Judge? Kafka? DB?

Phase 4 — Business Metrics + Alerting (effort trung bình):

- Thêm Prometheus + Grafana cho metrics collection/visualization
- Tạo SLOs cho contest mode và DevOps Lab runtime: submission latency P99, judge availability, workspace readiness
- Set up alerts: Judge unavailable → critical, latency spike → warning

 Lưu ý

1. **Logging quá nhiều hoặc quá ít** — Log mọi request body (sensitive data leaked, storage explodes) hoặc chỉ log errors (mất context khi debug). Hậu quả: hoặc tốn hàng GB logs/ngày mà không dùng được, hoặc thiếu thông tin khi cần. *Phòng tránh*: log ở mức INFO cho business events (submit, login), DEBUG cho technical details. NEVER log passwords, tokens, hoặc PII.
2. **Không có correlation ID** — Mỗi service log độc lập, không thể nối events xuyên services. Hậu quả: ghép logs bằng timestamp — sai khi concurrent requests. *Phòng tránh*: generate correlation ID tại entry point (Gateway), propagate qua headers, ghi vào MDC. Effort gần như zero, giá trị rất lớn.
3. **Alert fatigue — quá nhiều alerts** — Set alert cho mọi metric → hàng chục alerts/ngày → team ignore tất cả. Hậu quả: alert thật bị bỏ qua. *Phòng tránh*: chỉ alert cho SLO violations — những vấn đề thật sự ảnh hưởng user. Chia thành: critical (pager), warning (Slack), info (dashboard only).
4. **Dùng tracing thay cho logging** — Tracing cho biết request đi đâu và mất bao lâu, nhưng KHÔNG cho biết tại sao fail. Hậu quả: biết Judge mất 5 giây nhưng không biết do query nào, missing index nào. *Phòng tránh*: tracing bổ trợ logging — trace = overview, log = details.
5. **Bỏ qua observability vì “team nhỏ”** — Team 2-3 người, “docker logs đủ rồi”. Hậu quả: khi scale lên hoặc có incident lúc 2AM, đọc logs thủ công từ 7 containers là nightmare. *Phòng tránh*: bắt đầu nhỏ — structured logging + correlation ID là bước đầu tiên, gần như zero effort.

 Trực quan hóa tương tác (Interactive Demo)

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/distributed-tracing.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Truy vết phân tán (Distributed Tracing)**.

Ngoài ra, bạn cũng có thể xem minh họa về **Tập trung hóa cấu hình (Config Server)** tại file `code/interactive/config-server.html`.

11.9. Tổng kết

Observability là **điều kiện tiên quyết** để vận hành microservices — không phải tính năng bổ sung. Trong monolith, một log file và một debugger đủ để tìm bug. Trong microservices, request đi qua nhiều services, failure modes không dự đoán được — cần công cụ cho phép *hỏi câu hỏi mới* về hành vi hệ thống mà không cần deploy code mới.

Ba trụ cột — Logs, Metrics, Traces — bổ trợ lẫn nhau: metrics phát hiện vấn đề (latency spike), traces xác định vị trí (Judge Service), logs giải thích nguyên nhân (SQL timeout). Observability Maturity Model giúp team xác định *mức độ cần thiết* — không phải hệ thống nào cũng cần Level 5. Với LMS, Level 3 (Proactive) là mục tiêu hợp lý.

Structured logging và Correlation ID là nền tảng — effort thấp nhất, impact cao nhất. SLI/SLO/SLA cung cấp framework để chuyển từ “monitoring metrics” sang “measuring service

quality” — đặc biệt quan trọng cho contest mode của LMS khi downtime ảnh hưởng trực tiếp đến trải nghiệm học tập.

Error handling nhất quán — centralized exception handler + error code taxonomy — phục vụ cả client (UX tốt hơn) và ops team (metrics theo error code category). User tracking qua Kafka pipeline mở rộng observability từ *hệ thống* sang *hành vi học tập* — giá trị riêng mà domain giáo dục khai thác từ observability.

Phân tích KBLab cho thấy observability đã có nhiều mảnh đúng hướng nhưng chưa thành một contract end-to-end. Migration path rõ ràng: structured logs → correlation IDs → centralized aggregation → tracing → metrics + SLOs. Mỗi giai đoạn mang lại giá trị ngay, không cần đợi “hoàn thiện toàn bộ” mới bắt đầu.

Ở Chương 12, chúng ta sẽ chuyển sang **Triển khai và DevOps** — containerization, Docker Compose, CI/CD pipelines, và cách deploy hệ thống microservices hoàn chỉnh.

11.10. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.11: Developing Production-Ready Services — Health check API, Log aggregation, Distributed tracing, Application metrics
2. [4a] Sam Newman, *Building Microservices* — Ch.8: Monitoring — Logs, Metric tracking, Synthetic monitoring, Correlation IDs

Sách bổ trợ:

3. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Distributed Tracing, Domain Segregation
4. [3] Ronnie Mitra, *Microservices: Up and Running* — Ch.9: Monitoring infrastructure
5. [4b] Sam Newman, *Monolith to Microservices* — Reporting, Monitoring and Troubleshooting during migration

Nguồn trực tuyến:

- OpenTelemetry official docs — opentelemetry.io
- Micrometer docs — micrometer.io (Spring Boot observability)
- Grafana Loki — grafana.com/oss/loki (lightweight log aggregation)
- Jaeger — jaegertracing.io (distributed tracing)
- Brendan Gregg, “USE Method” — brendangregg.com/usemethod.html
- Tom Wilkie, “RED Method” — grafana.com/blog/2018/08/02/the-red-method
- Google SRE Book, “Service Level Objectives” — sre.google/sre-book/service-level-objectives

12. Triển khai & DevOps

“If deploying is painful, deploy more frequently. Automation turns a dreaded chore into a non-event.” — Martin Fowler, *Continuous Delivery* (nguyên tắc DevOps)

12.1. Bạn sẽ học được gì

- Hiểu vì sao triển khai microservices phức tạp hơn monolith và cần **tự động hóa**
- Nắm vững **containerization** với Docker — từ Dockerfile đến image registry
- Sử dụng **Docker Compose** để orchestrate multi-service deployment trên một máy
- Thiết kế **CI/CD pipeline** cho microservices — build, test, deploy tự động
- So sánh các **deployment strategies**: Rolling, Blue/Green, Canary
- Hiểu **Infrastructure as Code** (IaC) và vai trò trong quản lý hạ tầng
- Phân tích deployment architecture của KBLab và đề xuất cải thiện

12.2. Thách thức Triển khai Microservices

12.2.1. Vấn đề: từ “deploy một file WAR” đến “deploy N services đồng bộ”

Trong monolith, triển khai nghĩa là: build một artifact (JAR/WAR), copy lên server, restart. Một file, một lần, xong.

Trong microservices, “deploy” có nghĩa khác hoàn toàn:

Bảng 12.1: Thách thức triển khai Monolith vs Microservices

Monolith	Microservices
1 artifact → 1 server	N artifacts → N servers/containers
1 database migration	N database migrations (mỗi service)
Restart 1 process	Restart N processes — thứ tự quan trọng
Rollback: quay về version cũ	Rollback: quay N services về versions tương thích
Test trước deploy: 1 environment	Test trước deploy: cần N services chạy cùng

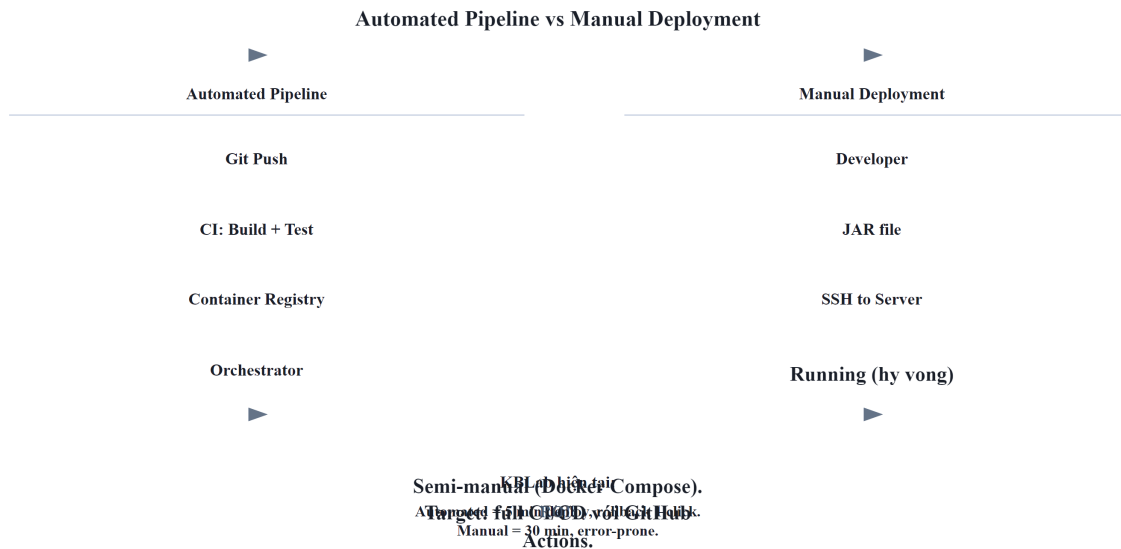
Newman trong [4a, Ch.6] nhấn mạnh: khả năng **independent deployment** là lợi ích quan trọng nhất của microservices — nhưng cũng là thách thức lớn nhất. Nếu không tự động hóa, deploy 7+ services thủ công trở thành “dreaded Friday afternoon deployment”.

12.2.2. Từ Manual đến Automation — DevOps Mindset

Mitra trong [3, Ch.6] mô tả ba nguyên tắc DevOps nền tảng cho microservices deployment:

Bảng 12.2: Ba nguyên tắc DevOps nền tảng

Nguyên tắc	Mô tả	Ý nghĩa
Immutable Infrastructure	Không patch server đang chạy — tạo mới, deploy mới, xóa cũ	Reproducible, no configuration drift
Infrastructure as Code	Mọi hạ tầng (server, network, database) định nghĩa trong code	Version control, review, rollback
Continuous Delivery	Code luôn ở trạng thái sẵn sàng deploy — chỉ cần ấn nút	Giảm risk per deployment, feedback nhanh



Hình 12.1: Triển khai thủ công (Manual) vs Tự động (Automated Pipeline)

Nguyên tắc — If It Hurts, Do It More Often

“If deploying is painful, deploy more frequently. The pain is information — it tells you what to automate.”

— Martin Fowler, *Continuous Delivery* (trích dẫn bởi Mitra [3, Ch.6])

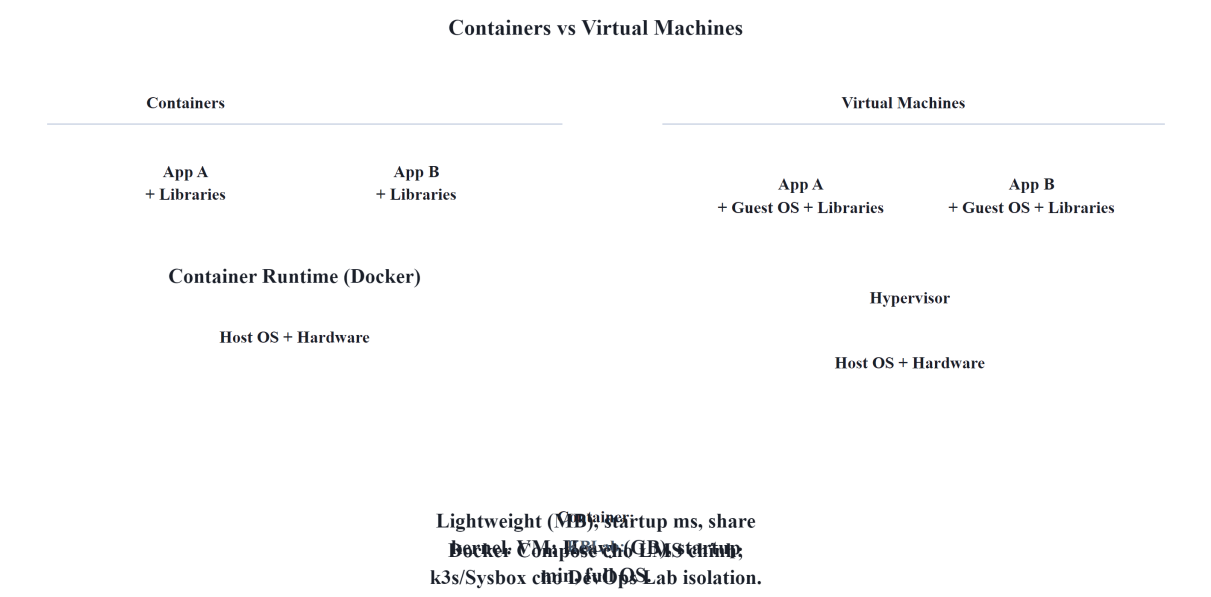
12.3. Containerization với Docker

12.3.1. Vấn đề: “Works on my machine”

Khi deploy microservices, mỗi service cần environment riêng: JDK version, dependencies, configuration. Truyền thống: cấu hình server thủ công → khác biệt giữa dev/staging/production → bugs chỉ xuất hiện trên production.

12.3.2. Container — Lightweight Isolation

Container (Docker) đóng gói ứng dụng + dependencies + runtime thành một **image** — chạy giống nhau trên mọi môi trường. Khác với Virtual Machine: container chia sẻ kernel với host OS → nhẹ hơn, khởi động trong giây thay vì phút.



Hình 12.2: So sánh kiến trúc Virtual Machines và Containers

Bảng 12.3: So sánh chi tiết VM và Container

So sánh	VM	Container
Khởi động	Phút	Giây
Kích thước	Hàng GB (Guest OS)	Hàng MB-GB (chỉ app + libs)
Isolation	Mạnh (hardware-level)	Trung bình (OS-level — share kernel)
Density	5-10 VMs/host	50-100+ containers/host
Phù hợp	Multi-tenant, security-critical	Microservices, CI/CD, dev environments

Richardson trong [2a, Ch.12] mô tả 5 deployment patterns, trong đó **Service as Container** là phổ biến nhất cho microservices — cân bằng giữa isolation, tốc độ, và resource efficiency.

12.3.3. Deployment Patterns — Từ Language-Specific đến Serverless

Richardson trong [2a, Ch.12] phân loại:

Bảng 12.4: Năm mô hình triển khai (Deployment Patterns)

Pattern	Mô tả	Ưu điểm	Nhược điểm
Language-specific	Deploy WAR/JAR trực tiếp lên server	Đơn giản	Không isolation, dependency conflicts
Service per VM	Mỗi service = 1 VM (AMI/OVA)	Isolation mạnh	Tốn resource, chậm provision
Service per Container	Mỗi service = 1 Docker container	Lightweight, fast, portable	Cần orchestration cho production
Serverless	Deploy functions, cloud quản lý infrastructure	Zero ops, auto-scale	Vendor lock-in, cold start, stateless only
Kubernetes	Container orchestration: scheduling, scaling, self-healing	Production-grade, declarative	Phức tạp, learning curve cao

12.3.4. Dockerfile — Định nghĩa Container Image

Dockerfile mô tả cách build container image cho một service. Mẫu phổ biến cho Spring Boot microservice:

Listing 12.1: Dockerfile sử dụng Multi-stage build

```
# Multi-stage build – tách build environment khỏi runtime
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw dependency:resolve          # Cache dependencies layer
COPY src src
RUN ./mvnw package -DskipTests        # Build JAR

FROM eclipse-temurin:21-jre           # Runtime image nhỏ hơn JDK
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Hai kỹ thuật quan trọng:

- **Multi-stage build:** stage 1 (JDK) build JAR, stage 2 (JRE) chỉ chứa runtime → image nhỏ hơn 40-60%
- **Layer caching:** COPY pom.xml trước COPY src → dependencies chỉ re-download khi pom.xml thay đổi, không phải mỗi lần code thay đổi

! Nguyên tắc — One Service, One Container

“Mỗi microservice chạy trong container riêng. Không nhồi nhiều services vào một container — mất isolation, khó scale, khó debug.”

— Nguyên tắc containerization (Richardson [2a, Ch.12], Newman [4a, Ch.6])

12.4. Docker Compose — Orchestration trên Một Máy

12.4.1. Vấn đề: chạy nhiều services + databases + Kafka bằng tay?

KBLab LMS chính có nhiều Java services, mỗi service cần cấu hình riêng; Kafka cần broker, Gateway cần service discovery, database sandbox cần môi trường cô lập. Khởi động thủ công:

Listing 12.2: Khởi động thủ công các services (Anti-pattern)

```
# ❌ Manual: 10+ lệnh docker run, đúng thứ tự, đúng network, đúng env vars
docker run -d postgres:15 ...
docker run -d zookeeper:3.8 ...
docker run -d kafka:3.5 ...
docker run -d eureka-server ...
docker run -d gateway --eureka-url=... ...
docker run -d core-service --db-url=... --kafka=... ...
# ... còn 5 services nữa
```

12.4.2. Docker Compose — Declarative Multi-Service

Docker Compose định nghĩa **toàn bộ stack** trong một file YAML — services, networks, volumes, dependencies. Một lệnh `docker compose up` khởi động toàn bộ hệ thống.

Cấu trúc Docker Compose cho KBLab LMS chính (đơn giản hóa):

Listing 12.3: Cấu hình Docker Compose cho KBLab LMS chính

```
# docker-compose.yml — KBLab LMS chính
services:
  # --- Infrastructure ---
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: lms_db
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on: [zookeeper]

  # --- Platform Services ---
  registry:
    image: registry.gitlab.com/lms-system/registry
    ports: ["9000:9000"]

  gateway:
    image: registry.gitlab.com/lms-system/gateway
    ports: ["9001:9001"]
    depends_on: [registry]

  # --- Application Services ---
  core-service:
    image: registry.gitlab.com/lms-system/core
```

```

    depends_on: [postgres, kafka, registry]
    environment:
        SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/lms_db

judge-service:
    image: registry.gitlab.com/lms-system/judge
    depends_on: [kafka, registry]

volumes:
    pgdata:

```

12.4.3. Docker Compose — Điểm mạnh và giới hạn

Bảng 12.5: Điểm mạnh và giới hạn của Docker Compose

Điểm mạnh	Giới hạn
Declarative: toàn bộ stack trong 1 file	Single host: chỉ chạy trên 1 máy
Reproducible: <code>docker compose up</code> = same result	Không có self-healing: container crash → không auto-restart
Dependencies: <code>depends_on</code> đảm bảo thứ tự khởi động	Không có load balancing: cần reverse proxy thêm
Isolated networking: services giao tiếp qua service name	Không production-grade: thiếu health checks, rolling updates

Docker Compose **phù hợp cho:** development environment, staging, CI/CD test environments, và **hệ thống nhỏ-trung production** (như KBLab LMS chính). Khi cần scale ra nhiều hosts, auto-healing, rolling updates → cần **Kubernetes** hoặc container orchestration platform. DevOps Lab là ngoại lệ có chủ đích: phần này cần k3s/Sysbox để cô lập lab environment, không phải vì toàn bộ hệ thống đã cần Kubernetes enterprise.

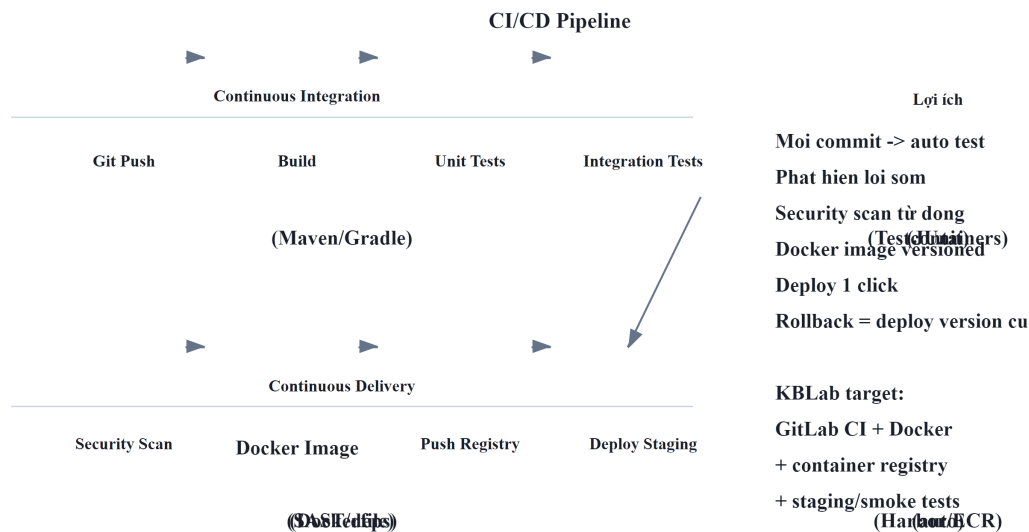
12.5. CI/CD Pipeline cho Microservices

12.5.1. Vấn đề: $N \text{ services} \times M \text{ stages} = \text{phức tạp}$

Trong monolith: 1 pipeline — build → test → deploy. Trong microservices: mỗi service có pipeline riêng, và cần đảm bảo:

- Service A deploy version mới → **không break** Service B (API versioning — Ch.3)
- Database migration chạy **trước** service deploy
- Rollback service → database migration cũng phải rollback

12.5.2. CI/CD Architecture cho Microservices



Hình 12.3: CI/CD Pipeline chuẩn cho Microservices

12.5.3. Mono-repo vs Poly-repo — Ảnh hưởng đến CI/CD

Bảng 12.6: Mono-repo vs Poly-repo

	Mono-repo	Poly-repo
Cấu trúc	Tất cả services trong 1 repository	Mỗi service 1 repository
CI/CD	1 pipeline, cần detect changed services	N pipelines, independent triggers
Shared code	Dễ — cùng repo, import trực tiếp	Cần publish shared library (Maven Central/internal)
Atomic changes	1 commit thay đổi nhiều services	Cần coordinate commits across repos
Ví dụ	Google, Meta (monorepo tools: Bazel, Buck)	Netflix, Amazon (mỗi team own repo)

Mitra trong [3, Ch.6] khuyến nghị: đối với team nhỏ-trung, **mono-repo đơn giản hơn** — tránh overhead quản lý N repos + N pipelines. Khi team lớn (>20 devs), poly-repo cho phép team autonomy tốt hơn.

12.5.4. Pipeline Best Practices

Bảng 12.7: Các best practices thiết kế CI/CD Pipeline

Practice	Mô tả	Lý do
Build once, deploy everywhere	Build image 1 lần → deploy staging → promote lên production	Cùng artifact, giảm risk “staging works, production doesn’t”
Externalized config	Config (DB URL, API keys) inject qua environment variables	Cùng image chạy dev/staging/production — chỉ khác config
Parallel pipelines	Mỗi service build/test/deploy độc lập	Deploy Service A không block Service B
Database migration as separate step	Chạy Flyway/Liquibase trước deploy service	Tách schema change khỏi code change — rollback dễ hơn
Contract test gate	Chỉ deploy khi contract tests pass	Ngăn breaking changes vào production

Nguyên tắc — Build Once, Deploy Everywhere

“Build artifact một lần duy nhất. Promote cùng artifact qua các environments (dev → staging → production). Nếu build khác nhau cho mỗi environment, bạn đang test artifact khác với production.”

— Nguyên tắc CI/CD (Newman [4a, Ch.6], Mitra [3, Ch.10])

12.6. Deployment Strategies

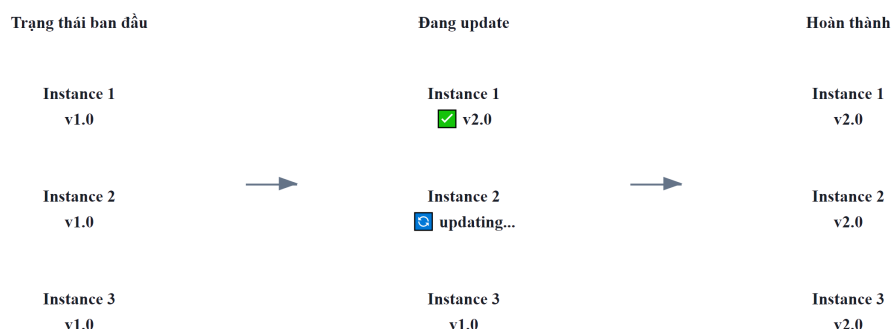
12.6.1. Vấn đề: deploy version mới mà không downtime

Khi deploy version mới của một service, làm sao đảm bảo users không bị ảnh hưởng? Nếu version mới có bug, làm sao rollback nhanh?

12.6.2. Ba strategy chính

12.6.2.1. Rolling Update

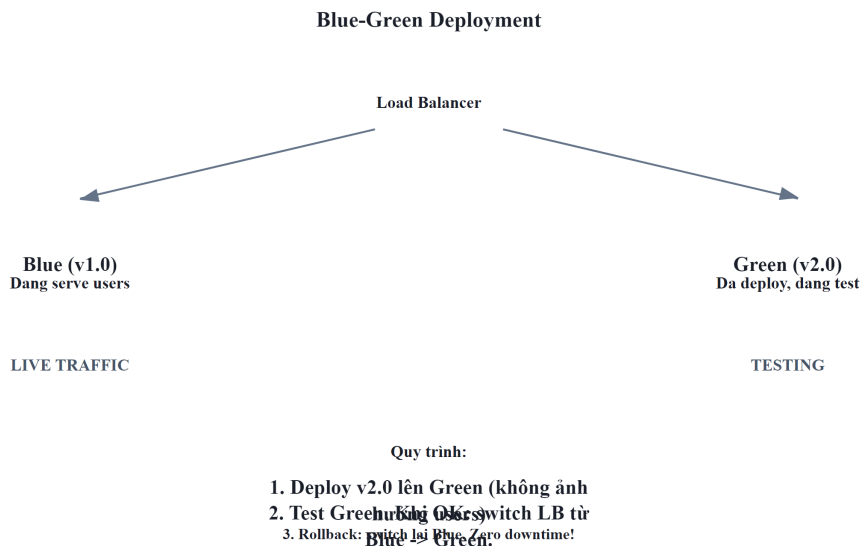
Thay thế **từng instance một** — dần dần chuyển từ version cũ sang version mới. Không cần gấp đôi infrastructure.



Hình 12.4: Quá trình diễn ra Rolling Update

12.6.2.2. Blue/Green Deployment

Chạy **hai bản hoàn chỉnh** song song — “Blue” (current) và “Green” (new). Router chuyển traffic một lần. Rollback = chuyển router ngược lại.



Hình 12.5: Kiến trúc Blue/Green Deployment

12.6.2.3. Canary Release

Deploy version mới cho **một phần nhỏ traffic** — monitor metrics — tăng dần nếu ổn.

12.6.3. So sánh

Bảng 12.8: So sánh các Deployment Strategies

Strategy	Downtime	Rollback	Resource cost	Phù hợp
Rolling	Zero (nếu ≥ 2 instances)	Chậm (phải roll ngược)	Thấp (N+1 instances)	Default cho Kubernetes
Blue/Green	Zero	Nhanh (switch router)	Cao (2× infrastructure)	Database migrations, major changes
Canary	Zero	Nhanh (route 100% về old)	Thấp-trung bình (N+1)	Risky changes, A/B testing

Newman trong [4a, Ch.6] khuyến nghị: chọn strategy dựa trên **mức độ rủi ro** của thay đổi. Bug fix nhỏ → rolling update. Thay đổi lớn ảnh hưởng nhiều services → blue/green. Tính năng mới chưa chắc chắn → canary.

12.7. Infrastructure as Code (IaC)

12.7.1. Vấn đề: “Ai cấu hình server? Config ở đâu?”

Khi hệ thống microservices chạy trên nhiều servers/containers, cấu hình thủ công dẫn đến:

- **Configuration drift:** server A cấu hình khác server B (ai đó patch A nhưng quên B)
- **Không reproducible:** không ai nhớ chính xác server được setup thế nào
- **Không rollback được:** thay đổi cấu hình sai → không biết quay về trạng thái trước

12.7.2. IaC — Hạ tầng như Code

Infrastructure as Code (IaC) nghĩa là mọi hạ tầng — servers, networks, databases, message brokers — được **định nghĩa trong code** (thường là declarative YAML/HCL), version controlled, và deploy tự động.

Mitra trong [3, Ch.6-7] mô tả IaC là nền tảng cho microservices deployment:

Bảng 12.9: Các công cụ Infrastructure as Code phổ biến

Thành phần	Công cụ phổ biến	Ví dụ KBLab
Infrastructure provisioning	Terraform, CloudFormation	Pulumi, Tạo VMs, networks, managed databases
Container orchestration	Kubernetes manifests, Helm charts	Deploy services, scaling rules
Configuration management	Ansible, Chef, Puppet	Cấu hình OS, install packages
Service deployment	Docker, Kubernetes, ArgoCD	Compose, Deploy microservices stack

12.7.3. Docker Compose as IaC

Docker Compose — dù đơn giản — đã là một dạng IaC: hạ tầng (databases, brokers) và services đều định nghĩa trong file YAML, version controlled, reproducible.

Bảng 12.10: Các mức độ trưởng thành của IaC

Level	Tool	Use case
Level 1	Docker Compose	Single host, development, small production
Level 2	Kubernetes + Helm	Multi-host, auto-scaling, self-healing
Level 3	Terraform + Kubernetes + ArgoCD	Full GitOps — infrastructure + services as code

Với KBLab LMS chính — hệ thống giáo dục quy mô trung bình — **Level 1 (Docker Compose)** hiện vẫn phù hợp. DevOps Lab dùng một cụm k3s nhỏ cho bài toán lab isolation riêng. Khi toàn hệ thống cần multi-host scaling, self-healing và rolling deployment đồng đều, mới chuyển dần lên Level 2 (Kubernetes).

Nguyên tắc — Infrastructure as Code

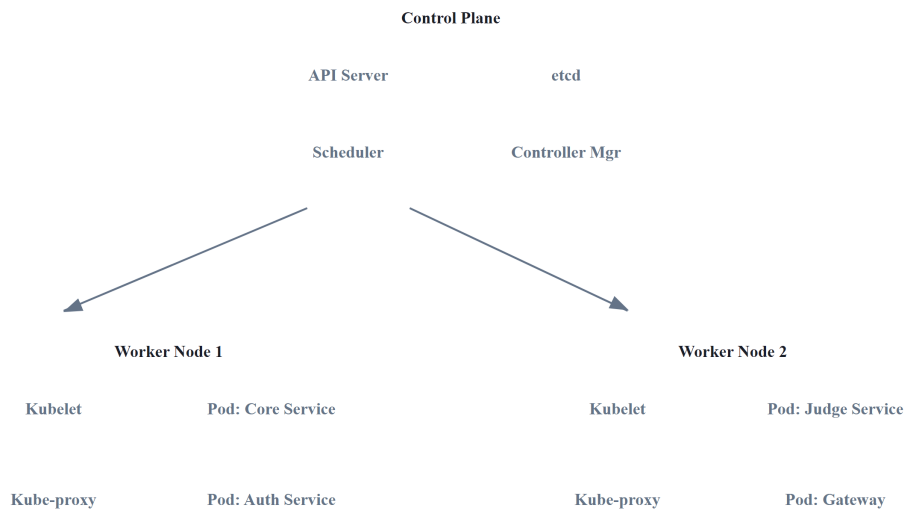
“If you can’t reproduce your infrastructure from scratch using code, you don’t truly own your infrastructure — you’re just renting it from whoever set it up last.”

— Mitra [3, Ch.6], nguyên tắc IaC

12.7.4. Kubernetes — Container Orchestration cho Production

Docker Compose phù hợp cho development và hệ thống nhỏ, ít yêu cầu điều phối. Khi cần **multi-node deployment, auto-scaling, self-healing**, Kubernetes (K8s) trở thành nền tảng tiêu chuẩn. Newman trong [4a, Ch.8] nhận xét: “Kubernetes has become the de facto platform for running microservices at scale.”

12.7.4.1. Kiến trúc Kubernetes



Hình 12.6: Kiến trúc Kubernetes — Control Plane và Worker Nodes

Control Plane quản lý cluster: API Server nhận requests (từ `kubectl` hoặc dashboard), etcd lưu toàn bộ cluster state, Scheduler quyết định pod chạy trên node nào, Controller Manager đảm bảo actual state = desired state.

Worker Nodes chạy workloads: Kubelet trên mỗi node nhận lệnh từ Control Plane → khởi động/dừng pods.

12.7.4.2. Concepts cốt lõi

Bảng 12.11: 6 Concepts cốt lõi trong Kubernetes

Concept	Mô tả	Tương đương Docker Compose
Pod	Đơn vị nhỏ nhất — 1+ containers cùng network/storage	Một service entry
Deployment	Quản lý N replicas của pod, rolling updates	Không có (manual)
Service	Stable network endpoint cho pods (load balancing)	Port mapping + links
Ingress	HTTP routing từ external → services (domain-based)	Nginx reverse proxy (manual)
ConfigMap / Secret	Externalized configuration	.env file
HPA (Horizontal Autoscaler)	Pod Tự động scale pods dựa trên CPU/memory/custom metrics	Không có

12.7.4.3. Docker Compose vs Kubernetes — So sánh chi tiết

Bảng 12.12: So sánh chi tiết Docker Compose và Kubernetes

Capability	Docker Compose	Kubernetes
Multi-host	✗ Single host only	✓ Cluster nhiều nodes
Auto-scaling	✗ Manual <code>--scale</code>	✓ HPA dựa trên metrics
Self-healing	⚠ <code>restart: always</code> (basic)	✓ Liveness/readiness probes, auto-replace
Rolling updates	✗ Downtime khi restart	✓ Zero-downtime rolling update
Service discovery	✓ DNS by service name	✓ DNS + load balancing
Secret management	⚠ <code>.env</code> files (plaintext)	✓ Secrets (base64, có thể encrypt)
Resource limits	⚠ Basic (<code>deploy.resources</code>)	✓ Requests + limits per container
Setup complexity	Thấp (1 YAML file)	Cao (nhiều YAML, cluster setup)
Learning curve	1-2 ngày	2-4 tuần
Team size cần thiết	1-3 người	3-5+ người (hoặc managed K8s)

12.7.4.4. KBLab Migration Scenario: Compose → Kubernetes

Nếu KBLab cần scale rộng hơn (nhiều đơn vị đào tạo, nhiều sinh viên đồng thời):

Listing 12.4: Scenario scale KBLab từ Compose lên Kubernetes

Phase 1 (Hiện tại): Docker Compose cho LMS chính

- └─ Phù hợp hạ tầng nhỏ và workload vừa phải
- └─ Deploy: `docker-compose up -d`
- └─ Chi phí vận hành thấp, thao tác đơn giản

Phase 2 (Scale): Managed Kubernetes (GKE/EKS/AKS)

- └─ Nhiều đơn vị đào tạo hoặc contest traffic lớn
- └─ Judge Service: HPA scale 1-10 pods khi contest
- └─ Core Service: 2-3 replicas cho high availability
- └─ PostgreSQL: managed service (Cloud SQL/RDS)
- └─ Chi phí và độ phức tạp tăng đáng kể

⚠ Nguyên tắc — Kubernetes khi nào?

Không chuyển lên K8s vì “Netflix dùng”. Chuyển khi có **ít nhất 2 tiêu chí**: (1) cần multi-node deployment hoặc workload isolation tốt hơn, (2) cần auto-scaling (load biến động: contest → bình thường), (3) cần zero-downtime deployment (SLA yêu cầu), (4) team đủ năng lực vận hành K8s. Managed Kubernetes (GKE, EKS, AKS) giảm đáng kể complexity — không cần tự setup/maintain control plane.

12.7.5. Serverless Deployment — Khi nào phù hợp?

Richardson trong [2a, Ch.12] liệt kê **Serverless** (AWS Lambda, Google Cloud Functions, Azure Functions) là một deployment pattern cho microservices. Thay vì quản lý containers, bạn deploy *functions* — cloud provider quản lý infrastructure, auto-scale, và billing per invocation.

Bảng 12.13: Container (Docker/K8s) vs Serverless

Aspect	Container (Docker/K8s)	Serverless
Quản lý server	Tự quản lý (hoặc managed K8s)	Cloud provider quản lý hoàn toàn
Scaling	Configure auto-scaling rules	Auto-scale tự động ($0 \rightarrow N$)
Cost model	Pay per server/hour (running or not)	Pay per invocation (không dùng = không trả)
Cold start	Không (container luôn chạy)	Có (100ms-3s khởi động lần đầu)
Stateful	Có thể (volumes, sessions)	Stateless only
Runtime limit	Không giới hạn	Thường 15 phút max per invocation
Vendor lock-in	Thấp (Docker portable)	Cao (API riêng mỗi cloud)

Khi nào serverless phù hợp cho microservices?

- **Event-driven, bursty workloads:** xử lý file upload, image resize, notification — không cần server chạy 24/7
- **Glue functions:** kết nối services, transform data, trigger workflows
- **Prototype/MVP:** deploy nhanh, không cần setup infrastructure

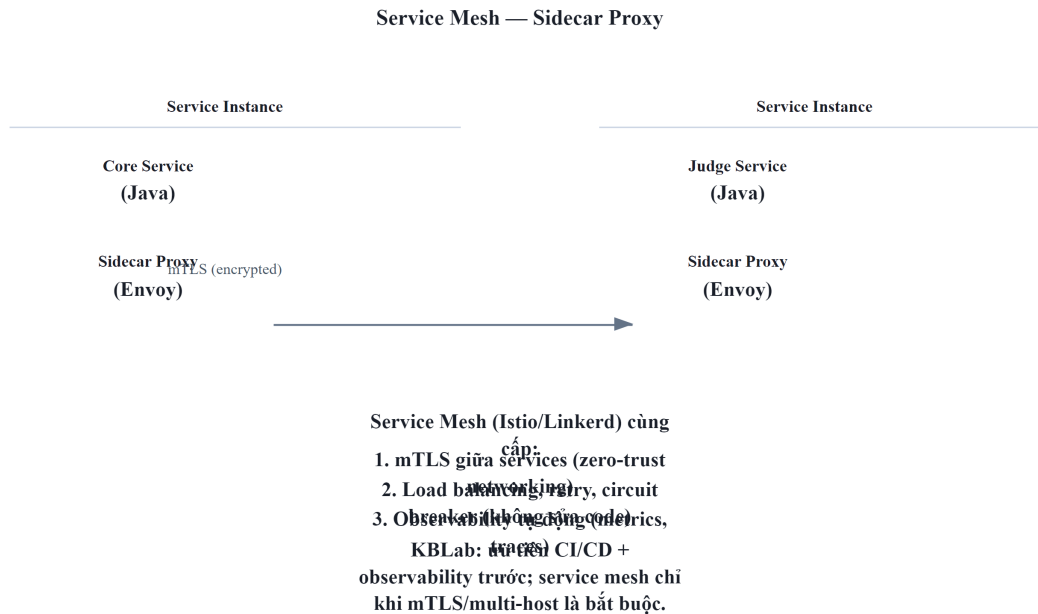
Khi nào KHÔNG phù hợp?

- **Low-latency critical paths:** cold start 100ms-3s không chấp nhận được cho synchronous API (LMS submit flow)
- **Long-running processes:** Judge Service chạy SQL queries có thể mất >15 phút → serverless timeout
- **Stateful services:** cần database connections, caching → serverless phải reconnect mỗi invocation

Trong LMS, **Notification Service** là candidate tốt nhất cho serverless: event-driven (nhận event từ Kafka → gửi email/push), bursty (contest → nhiều notifications, bình thường → ít), stateless. Các services khác (Core, Judge, Gateway) phù hợp với containers hơn.

12.7.6. Sidecar Pattern và Service Mesh

Khi hệ thống microservices lớn (20+ services), mỗi service cần implement cùng cross-cutting concerns: mTLS, logging, tracing, circuit breaker, rate limiting. **Sidecar pattern** giải quyết bằng cách đặt một **proxy process bên cạnh mỗi service instance** — proxy xử lý infrastructure concerns, service chỉ focus business logic.



Hình 12.7: Sidecar Pattern — Proxy xử lý infrastructure concerns (mTLS, tracing)

Service Mesh (Istio, Linkerd) = sidecar proxies trên mọi service + control plane quản lý tập trung. Tự động cung cấp: mTLS giữa services, distributed tracing, traffic management (canary routing), circuit breaking — **mà không cần thay đổi code**.

Bảng 12.14: So sánh không dùng và dùng Service Mesh

Aspect	Không Service Mesh	Có Service Mesh
mTLS	Tự implement trong code	Sidecar tự động
Tracing	Add library (OpenTelemetry)	Sidecar tự inject headers
Circuit breaker	Resilience4j trong code	Sidecar config (Envoy)
Canary routing	Manual load balancer config	Declarative traffic rules
Overhead	Không	~10-20ms latency per hop

Với KBLab hiện tại, service mesh vẫn **over-engineering** cho LMS chính. Dù DevOps Lab làm hệ thống trở thành polyglot Java+Go, nhu cầu trước mắt là CI/CD, observability và secret management. Service mesh phù hợp khi: số lượng service lớn hơn nhiều, multi-host deployment trở thành mặc định, hoặc yêu cầu security cao đến mức mTLS bắt buộc giữa mọi service.

12.8. Case Study: Deployment Architecture của KBLab

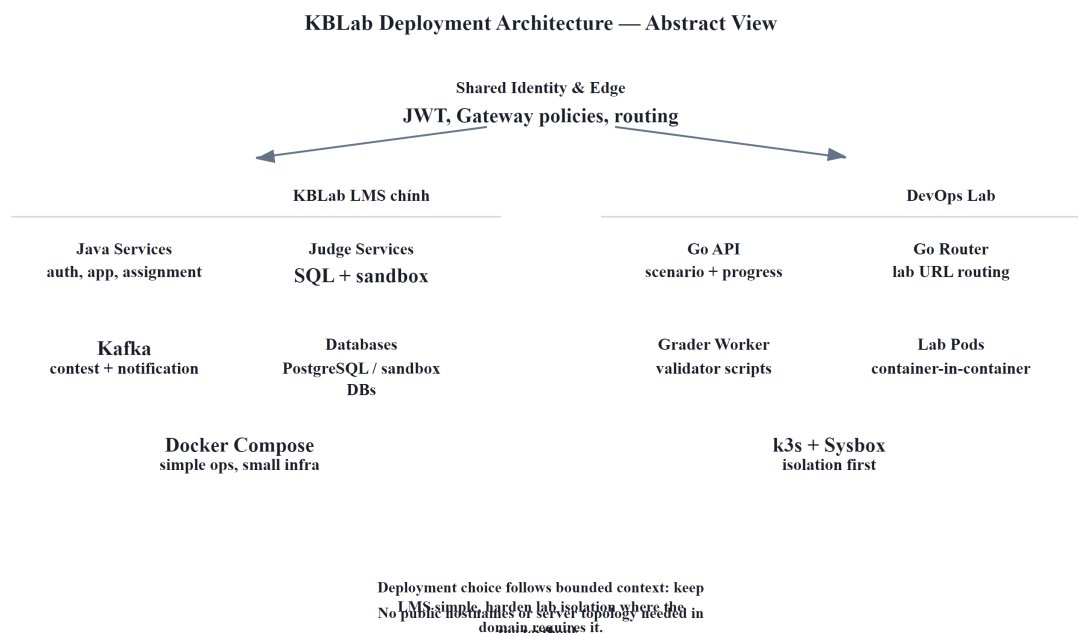
12.8.1. Hiện trạng

KBLab hiện có hai kiểu deployment song song ở mức khái quát:

- LMS chính:** các service Java/Spring Boot, database, Kafka, Gateway và frontend triển khai bằng Docker Compose trên hạ tầng nhỏ. Đây là lựa chọn thực dụng cho team nhỏ, dễ vận hành, không cần cluster phức tạp.

2. **DevOps Lab**: phần thực hành Docker/Kubernetes dùng Go, k3s và Sysbox để tạo lab environment cô lập. Đây là nhu cầu domain đặc thù: sinh viên cần chạy container/lab riêng trong môi trường an toàn.

Không cần public domain, IP hay sơ đồ máy chủ cụ thể; điều quan trọng với sách là **trade-off deployment**: cùng một hệ thống có thể dùng Docker Compose cho phần LMS chính và k3s/Sysbox cho bounded context cần isolation.



Hình 12.8: Deployment Architecture KBLab ở mức khái quát

12.8.2. Phân tích theo deployment maturity

Bảng 12.15: Phân tích mức độ trưởng thành triển khai của KBLab

Aspect	Hiện trạng	Maturity	Nhận xét
Containerization	Docker images cho tất cả services	 Tốt	Multi-stage builds, container registry (GitLab)
Orchestration	Docker Compose cho LMS chính; k3s/ Sysbox cho DevOps Lab	 Phù hợp theo context	Không ép một platform cho mọi phần
CI/CD	Manual build + push ở nhiều phần	 Gap lớn	Mỗi deploy thủ công làm tăng rủi ro version drift
Deployment strategy	All-at-once (stop → deploy → start)	 Có downtime	Sinh viên bị gián đoạn khi deploy
IaC	Docker Compose files version controlled	 Basic IaC	Có reproducibility nhưng thiếu automation
Config management	application.yml trong container	 Partially externalized	Một số config hardcoded, chưa fully externalized
Lab isolation	k3s + Sysbox cho lab pods	 Đúng bài toán	Cần resource quota, RBAC, network policy rõ

12.8.3. Phân tích business context

KBLab phục vụ sinh viên → **deployment windows** quan trọng:

Bảng 12.16: Deployment windows cho LMS

Thời điểm	Rủi ro deploy	Strategy phù hợp
Trước/sau contest	Cao — contest có deadline	Blue/Green — rollback ngay nếu lỗi
Giữa tuần (off-peak)	Trung bình	Rolling update
Summer break	Thấp	All-at-once OK
Emergency hotfix	Rất cao	Canary — test với ít users trước

❗ Phân tích — Manual deployment, no CI/CD pipeline

KBLab đã containerized, nhưng nhiều phần vẫn **deploy thủ công**: developer build image locally → push to registry → SSH vào hạ tầng → chạy lệnh deploy tương ứng. Không có automated testing trước deploy, chưa có rollback mechanism nhất quán, và CI/CD giữa LMS chính và DevOps Lab chưa đồng đều.

Khi có bug trên production: (1) developer phát hiện từ user report, (2) fix code → build → push → deploy thủ công, (3) nếu fix sai → lặp lại. MTTR (Mean Time To Resolve) cao, rủi ro deploy nhầm version.

Migration path (incremental):

Phase 1 — Basic CI/CD (effort thấp, impact cao):

- Tạo GitLab CI pipeline: push code → auto build → auto run tests → auto build Docker image → auto push to registry
- Giá trị ngay: không bao giờ deploy untested code, image version tracking tự động

Phase 2 — Automated Deployment (effort trung bình):

- Pipeline auto deploy to staging → smoke tests → manual approval → deploy production
- Thêm docker compose health checks: healthcheck directive cho mỗi service
- Giá trị ngay: one-click deploy, không cần SSH, audit trail

Phase 3 — Zero-Downtime Deployment (effort trung bình-cao):

- Chuyển sang rolling updates (chạy ≥2 instances mỗi critical service)
- Hoặc blue/green cho contest periods: deploy version mới song song, switch khi sẵn sàng
- Giá trị ngay: deploy bất kỳ lúc nào, không ảnh hưởng sinh viên

Phase 4 — Kubernetes/Cluster hardening (khi cần scale, effort cao):

- Chuyển dần LMS chính từ Docker Compose sang Kubernetes khi cần: multi-host, auto-scaling, self-healing
- Với DevOps Lab: ưu tiên RBAC, NetworkPolicy, ResourceQuota, lab TTL và observability cho k3s/Sysbox
- Hiện tại không cần ép toàn bộ hệ thống lên Kubernetes — **đừng over-engineer**

⚠ Lưu ý —

1. **Deploy Friday chiều** — Team deploy tính năng mới cuối tuần, bug phát hiện khi không ai online. Hậu quả: downtime kéo dài đến thứ hai. *Phòng tránh*: deploy sáng thứ hai-tư, khi team sẵn sàng xử lý vấn đề. Với KBLab: tránh deploy trước/trong contest hoặc buổi lab.
2. **Dùng Kubernetes cho hệ thống 3 services** — “Netflix dùng Kubernetes, chúng ta cũng nên”. Hậu quả: complexity overhead lớn (cluster management, RBAC, networking) cho team nhỏ. *Phòng tránh*: Docker Compose đủ cho hệ thống nhỏ, ít yêu cầu điều phối. Chuyển Kubernetes khi cần multi-node, isolation hoặc auto-scaling — không phải vì “industry trend”.
3. **Build image khác nhau cho mỗi environment** — Build riêng cho dev, staging, production — cùng code nhưng khác artifact. Hậu quả: “works on staging” nhưng fail production vì image khác. *Phòng tránh*: build once, deploy everywhere — externalize config qua environment variables.
4. **Không có rollback plan** — Deploy version mới, có bug, không biết rollback thế nào. Hậu quả: panic, manual fix trên production, thêm bug mới. *Phòng tránh*: mọi deployment phải có rollback procedure documented — biết chính xác “nếu có lỗi, chạy lệnh gì để quay lại”.
5. **Shared database giữa services** — Chương 7 đã phân tích. Nhưng khi deploy: nếu Service A và B chia sẻ database, database migration của A có thể break B. *Phòng tránh*: mỗi service own database schema riêng — migration independent.

🌐 **Thực quan hóa tương tác (Interactive Demo)**

Để hiểu rõ hơn về nội dung chương này, hãy mở file `code/interactive/deployment-strategies.html` trong mã nguồn đi kèm sách bằng trình duyệt web để trải nghiệm minh họa động về **Các chiến lược Deployment (Blue/Green, Canary)**.

12.9. Tổng kết

Triển khai microservices phức tạp hơn monolith vì N services cần build, test, deploy, và rollback độc lập nhưng tương thích với nhau. **Containerization** (Docker) giải quyết “works on my machine” — đóng gói service + dependencies thành image portable. **Docker Compose** cho phép orchestrate nhiều services trên hạ tầng nhỏ — phù hợp cho team nhỏ-trung và development/staging environments.

CI/CD pipeline là xương sống: code push → auto build → auto test → auto deploy. Không có CI/CD, microservices deployment trở thành “manual ceremony” — chậm, rủi ro, không reproducible. Three deployment strategies — Rolling, Blue/Green, Canary — lựa chọn theo mức rủi ro: bug fix nhỏ → rolling, thay đổi lớn → blue/green, tính năng mới → canary.

Infrastructure as Code đảm bảo hạ tầng reproducible và version controlled — Docker Compose files đã là basic IaC. Khi scale, Terraform + Kubernetes + ArgoCD tạo thành full GitOps pipeline — nhưng đừng over-engineer: hãy chọn mức orchestration theo từng bounded context.

Phân tích KBLab cho thấy containerization tốt (Docker images, registry) và DevOps Lab đã có nhu cầu k3s/Sysbox riêng, nhưng deployment manual vẫn là gap lớn nhất. Migration path rõ ràng: basic CI/CD pipeline → automated deployment → zero-downtime strategies → cluster hardening khi thật sự cần.

Qua 12 chương, chúng ta đã đi từ nền tảng SOA/Microservices (Ch.1-2), qua communication patterns (Ch.3-6), data management (Ch.7), infrastructure (Ch.8-9), migration thực tế (Ch.10), observability (Ch.11), đến deployment (Ch.12). Hành trình từ monolith đến microservices không phải “big bang migration” — mà là **chuỗi quyết định nhỏ, mỗi quyết định mang lại giá trị ngay lập tức**, với hiểu biết rằng mỗi pattern đều có trade-off.

12.10. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.12: Deploying Microservices — deployment patterns (language-specific, VM, container, Kubernetes, serverless)
2. [4a] Sam Newman, *Building Microservices* — Ch.6: Deployment — CI, build pipelines, CD, Docker, service-to-host mapping
3. [3] Ronnie Mitra, *Microservices: Up and Running* — Ch.6-7: Infrastructure Pipeline & Infrastructure — IaC, Terraform, Kubernetes; Ch.10: Releasing — Docker, Helm, ArgoCD; Ch.11: Managing Change — deployment patterns

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.18-19: Deploying Microservices & on Kubernetes (updated)
5. [4b] Sam Newman, *Monolith to Microservices* — Ch.3: Splitting the Monolith — deployment considerations during migration

Nguồn trực tuyến:

- Docker official docs — docs.docker.com
- Docker Compose documentation — docs.docker.com/compose
- Kubernetes official docs — kubernetes.io/docs
- Martin Fowler, “Continuous Delivery” — martinfowler.com/bliki/ContinuousDelivery.html
- Martin Fowler, “BlueGreenDeployment” — martinfowler.com/bliki/BlueGreenDeployment.html
- Terraform by HashiCorp — terraform.io
- ArgoCD — argo-cd.readthedocs.io (GitOps for Kubernetes)

PHỤ LỤC

Phụ lục A	Bảng thuật ngữ
Phụ lục B	Công cụ & Tài nguyên
Phụ lục C	Pattern Catalog
Phụ lục D	Anti-pattern Catalog
Bài tập	Bài tập & Case Studies
Tài liệu tham khảo	Bibliography

A. Phụ lục A: Bảng thuật ngữ

Thuật ngữ kỹ thuật sử dụng trong sách, sắp xếp theo bảng chữ cái. Sync với style-guide.md Section 2.

English	Cách dùng trong sách	Chương đầu
Aggregate	Aggregate	Ch.2
Anti-Corruption Layer	Anti-Corruption Layer (ACL)	Ch.4
API Gateway	API Gateway	Ch.8
Blue/Green Deployment	Blue/Green deployment	Ch.12
Bottleneck	bottleneck (điểm nghẽn)	Ch.1
Bounded Context	Bounded Context	Ch.2
Build	build (biên dịch/đóng gói)	Ch.12
Bulkhead	Bulkhead	Ch.4
Canary Deployment	Canary deployment	Ch.12
Circuit Breaker	Circuit Breaker	Ch.4
Choreography	choreography	Ch.6
CI/CD	CI/CD	Ch.12
Cluster	cluster (cụm)	Ch.12
Container	container	Ch.12
Correlation ID	correlation ID	Ch.8
CQRS	CQRS	Ch.7
Dead Letter Topic/Queue	Dead Letter Topic (DLT)	Ch.5
DB Queue	DB queue / database-backed queue	Ch.5
Debug	debug (gỡ lỗi)	Ch.1
Docker Compose	Docker Compose	Ch.12
Domain-Driven Design	DDD (Domain-Driven Design)	Ch.2
Downtime	downtime (thời gian ngừng hoạt động)	Ch.4
Event Sourcing	Event Sourcing	Ch.7
Event Storming	Event Storming	Ch.5
Event-driven	hướng sự kiện (event-driven)	Ch.5
Idempotency	idempotency / idempotent	Ch.4
Infrastructure as Code	IaC (Infrastructure as Code)	Ch.12
JWT	JWT (JSON Web Token)	Ch.9
Overhead	overhead (chi phí phụ)	Ch.1
Pipeline	pipeline (quy trình tự động)	Ch.5
Refactor	refactor (tái cấu trúc)	Ch.1
Scaling	scaling (mở rộng)	Ch.1
Team	team (nhóm/đội phát triển)	Ch.1
k3s	k3s	Ch.12
Load Balancing	load balancing	Ch.4
Message broker	message broker	Ch.5
Microservice	microservice / dịch vụ vi mô (lần đầu)	Ch.1
OAuth2	OAuth2	Ch.9
Observability	Observability	Ch.11
Role-Based Access Control	RBAC (Role-Based Access Control)	Ch.10

Lưu ý: Lần đầu xuất hiện trong text, thuật ngữ được in nghiêng kèm giải thích tiếng Việt. Sau đó dùng thuật ngữ tiếng Anh trực tiếp.

B. Phụ lục B: Công cụ & Tài nguyên

Tổng hợp các công cụ và tài nguyên được sử dụng hoặc đề cập trong sách.

B.1. Frameworks & Libraries

Tool	Mô tả	Chương	URL
Spring Boot	Java framework cho microservices	Xuyên suốt	spring.io
Spring Cloud	Distributed systems toolkit	Ch.4, 8	spring.io/cloud
Spring Cloud Gateway	API Gateway (WebFlux-based)	Ch.8	docs.spring.io
Netflix Eureka	Service Discovery & Registry	Ch.4, 8	github.com/Netflix/eureka
OpenFeign	Declarative REST client	Ch.4	github.com/OpenFeign
Resilience4j	Circuit Breaker, Retry, Bulkhead	Ch.4	resilience4j.readme.io
Spring Security	Authentication & Authorization	Ch.9	spring.io/security
JWT	JWT generation & validation	Ch.9	github.com/jwt/jwt
MapStruct	DTO mapping code generator	Ch.3	mapstruct.org
Chi	Go HTTP router cho API/router nhẹ	Ch.3, 8	github.com/go-chi/chi

B.2. Messaging & Data

Tool	Mô tả	Chương	URL
Apache Kafka	Distributed event streaming platform	Ch.5, 6	kafka.apache.org
PostgreSQL	Relational database	Ch.7	postgresql.org
Redis	In-memory cache & rate limiting	Ch.8	redis.io
golang-migrate	Versioned database migrations cho Go services	Ch.7, 12	github.com/golang-migrate/migrate

B.3. Infrastructure & DevOps

Tool	Mô tả	Chương	URL
Docker	Containerization platform	Ch.12	docker.com
Docker Compose	Multi-container orchestration	Ch.12	docs.docker.com/compose
Kubernetes	Container orchestration (overview)	Ch.12	kubernetes.io
k3s	Lightweight Kubernetes distribution	Ch.12	k3s.io
Sysbox	Container runtime hỗ trợ sandbox/container-in-container	Ch.9, 12	github.com/nestybox/sysbox
Jenkins / GitHub Actions	CI/CD automation	Ch.12	—

B.4. Migration & Release Management

Tool	Mô tả	Chương	URL
Debezium	Change Data Capture (CDC)	Ch.10	debezium.io
Flyway	Database migration versioning	Ch.10	flywaydb.org
LaunchDarkly / Unleash	Feature flags management	Ch.10	launchdarkly.com / unleash.io

B.5. Observability

Tool	Mô tả	Chương	URL
Micrometer	JVM metrics facade	Ch.11	micrometer.io
OpenTelemetry	Distributed tracing standard	Ch.11	opentelemetry.io
ELK Stack	Elasticsearch + Logstash + Kibana	Ch.11	elastic.co
Prometheus + Grafana	Metrics collection + visualization	Ch.11	prometheus.io, grafana.com
Promtail + Loki	Lightweight log aggregation stack	Ch.11	grafana.com/oss/loki
zerolog	Structured logging library cho Go	Ch.11	github.com/rs/zerolog

B.6. Security & Identity

Tool	Mô tả	Chương	URL
Microsoft Entra ID / OAuth2-OIDC	External Identity Provider cho SSO	Ch.9	microsoft.com/entra
Cloudflare Turnstile	Bot protection challenge	Ch.9	cloudflare.com/products/turnstile

B.7. Development Tools

Tool	Mô tả	URL
IntelliJ IDEA	Java IDE	jetbrains.com/idea
Postman	API testing & documentation	postman.com
Swagger UI	Interactive API documentation	swagger.io

Lưu ý: Phiên bản cụ thể phụ thuộc vào thời điểm sử dụng. Tham khảo trang chính thức của mỗi công cụ để có thông tin mới nhất.

C. Phụ lục C: Pattern Catalog

Bảng tổng hợp tất cả patterns được đề cập trong sách, sắp xếp theo chương. Sử dụng bảng này để tra cứu nhanh pattern cần thiết.

C.1. Communication Patterns

Pattern	Mô tả ngắn	Chương	Ref
Request/Response	Client gửi request, chờ response	Ch.4 §4.1	[2a] Ch.3
Publish/Subscribe	Producer publish event, consumers subscribe	Ch.5 §5.3	[2a] Ch.3
One-way Notification	Fire-and-forget message	Ch.5 §5.6	[2a] Ch.3
DB-backed Queue	Dùng database table làm hàng đợi job transactional	Ch.5 §5.6	[5] Ch.3
API Composition	Gọi nhiều services, tổng hợp response	Ch.7 §7.3	[2a] Ch.7
Backend for Frontend (BFF)	Gateway riêng cho từng loại client	Ch.8 §8.1	[2a] Ch.8

C.2. Resilience Patterns

Pattern	Mô tả ngắn	Chương	Ref
Circuit Breaker	Ngắt mạch khi service downstream liên tục lỗi	Ch.4 §4.4	[5] Ch.7
Retry	Tự động thử lại khi gặp lỗi tạm thời	Ch.4 §4.4	[5] Ch.7
Timeout	Giới hạn thời gian chờ response	Ch.4 §4.4	[5] Ch.7
Bulkhead	Cách ly resources theo function/service	Ch.4 §4.4	[5] Ch.7

C.3. Data Patterns

Pattern	Mô tả ngắn	Chương	Ref
Database-per-Service	Mỗi service sở hữu database riêng	Ch.7 §7.1	[4b] Ch.4
Saga (Orchestration)	Central orchestrator điều phối local transactions	Ch.6 §6.2	[2a] Ch.4
Saga (Choreography)	Services tự phối hợp qua events	Ch.6 §6.2	[2a] Ch.4
CQRS	Tách model read và write	Ch.7 §7.4	[2a] Ch.7
Event Sourcing	Lưu events thay vì current state	Ch.7 §7.5	[2a] Ch.6
Data Duplication	Copy reference data qua events	Ch.7 §7.3	[4b] Ch.4
Shared Kernel	Bounded Contexts chia sẻ common model	Ch.2 §2.6	[6]

C.4. Infrastructure Patterns

Pattern	Mô tả ngắn	Chương	Ref
API Gateway	Single entry point cho mọi clients	Ch.8 §8.1	[2a] Ch.8
Hostname-based Routing	Route request theo hostname/subdomain tới workspace/runtime	Ch.8 §8.5	[2a] Ch.8
Service Discovery	Registry để tìm service instances	Ch.4 §4.3	[2a] Ch.3
Client-side Discovery	Client query registry + tự load balance	Ch.4 §4.3	[2a] Ch.3
Server-side Discovery	Load balancer query registry	Ch.4 §4.3	[2a] Ch.3
Sidecar	Process phụ đi kèm mỗi service instance	Ch.12 §12.6	[3] Ch.8
Strangler Fig	Migration tăng dần từ monolith	Ch.10 §10.2	[4b] Ch.3

C.5. Security Patterns

Pattern	Mô tả ngắn	Chương	Ref
Token-based Auth (JWT)	Stateless authentication	Ch.9 §9.2	[4a] Ch.9
Token Exchange	External token → internal token	Ch.9 §9.4	[4a] Ch.9
Claims-based Identity Propagation	Gateway validate, services trust headers	Ch.9 §9.3	[2a] Ch.11
RBAC	Permissions based on user roles	Ch.9 §9.5	[4a] Ch.9
Runtime Isolation	Cách ly workspace/container bằng runtime, network policy, quota	Ch.9 §9.6	[4a] Ch.9

C.6. Observability Patterns

Pattern	Mô tả ngắn	Chương	Ref
Structured Logging	JSON logs với correlation fields	Ch.11 §11.2	[2a] Ch.11
Distributed Tracing	Trace request xuyên suốt services	Ch.11 §11.4	[2a] Ch.11
Health Check	Liveness + Readiness probes	Ch.11 §11.5	[2a] Ch.11
Correlation ID	Unique ID theo dõi request chain	Ch.8 §8.4, Ch.11 §11.2	[4a] Ch.8
SLI/SLO/SLA	Quantitative service quality targets	Ch.11 §11.3	[2a] Ch.11

C.7. Deployment Patterns

Pattern	Mô tả ngắn	Chương	Ref
Rolling Update	Thay từng instance, zero downtime	Ch.12 §12.5	[2a] Ch.12
Blue/Green	Hai environments, switch traffic	Ch.12 §12.5	[2a] Ch.12
Canary	Route % traffic tới version mới	Ch.12 §12.5	[2a] Ch.12
Infrastructure as Code	Declare infra bằng code (Docker Compose, K8s)	Ch.12 §12.6	[3] Ch.6
CI/CD Pipeline	Automate build → test → deploy	Ch.12 §12.4	[3] Ch.10

C.8. Migration Patterns

Pattern	Mô tả ngắn	Chương	Ref
Strangler Fig	Migration tăng dần từ monolith	Ch.10 §10.2	[4b] Ch.3
Anti-Corruption Layer	Dịch model giữa old/new systems	Ch.10 §10.4	[6] Ch.14
Branch by Abstraction	Tạo abstraction, switch implementation	Ch.10 §10.4	[4b] Ch.3
Outbox Pattern	Reliable messaging khi tách database	Ch.10 §10.3	[2a] Ch.4
Parallel Run	Chạy old + new, so sánh output	Ch.10 §10.4	[4b] Ch.3

C.9. DDD Patterns

Pattern	Mô tả ngắn	Chương	Ref
Bounded Context	Ranh giới ngôn ngữ và model	Ch.2 §2.3	[6] Ch.14
Context Map	Mối quan hệ giữa Bounded Contexts	Ch.2 §2.3	[6] Ch.14
Aggregate	Cluster of entities, consistency boundary	Ch.2 §2.2	[6] Ch.5
Ubiquitous Language	Ngôn ngữ thống nhất team-code-domain	Ch.2 §2.2	[6] Ch.2
Event Storming	Workshop khám phá domain qua events	Ch.5 §5.4	[3] Ch.4

Tổng cộng: 45+ patterns được đề cập xuyên suốt 12 chương.

Lưu ý: Bảng này liệt kê patterns ở mức overview — chi tiết, ví dụ, và trade-offs xem tại chương tương ứng.

D. Phụ lục D: Anti-pattern Catalog

Tổng hợp tất cả anti-patterns (⚠ Sai lầm thường gặp) từ 12 chương, sắp xếp theo chủ đề. Sử dụng bảng này để tra cứu nhanh khi phát hiện vấn đề trong hệ thống.

D.1. Architecture & Design

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
1	MS vì “trending” — Chọn microservices vì Netflix dùng, không vì vấn đề cụ thể	Thêm complexity mà không có lợi ích	Bắt đầu từ vấn đề (Decision Framework), không từ giải pháp	1
2	Big-bang migration — Nhảy thẳng từ monolith sang microservices, bỏ qua modular hóa	Distributed monolith — tệ hơn cả hai	Refactor ranh giới module rõ ràng <i>trước</i> , rồi mới tách service	1
3	Tách service trước khi tổ chức team — 10 services nhưng 1 team, deploy cùng nhau	Overhead vận hành, không có autonomy thực sự	Tổ chức team phù hợp <i>cùng lúc</i> hoặc <i>trước</i> khi tách service (Conway’s Law)	1
4	Entity-driven design — Tạo 1 entity = 1 bảng, 1 service = 1 nhóm bảng từ ERD	Ranh giới phản ánh schema, không phản ánh domain	Bắt đầu từ domain process (Event Storming), không từ schema	2
5	Nano-services — Tách mỗi entity thành 1 service riêng khi chúng luôn thay đổi cùng nhau	Latency tăng, debugging ác mộng (request cần 3-4 calls)	Tách theo bounded context, không theo entity	2
6	Shared lib lạm dụng — Dùng Shared Kernel vì tiện, trộn cross-cutting concern và domain logic	Deploy coupling ngầm, thay đổi shared lib ảnh hưởng tất cả	Phân biệt rõ: cross-cutting (nên share) vs domain logic (không share)	2

D.2. API Design

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
7	Trả entity qua API — Không dùng DTO, expose toàn bộ internal structure	Breaking change khi đổi schema, rủi ro bảo mật	Luôn dùng DTO pattern — tách internal model khỏi external contract	3
8	Naming convention không nhất quán — Mix singular/plural, kebab/camelCase	3+ consumers → sửa naming = breaking changes hàng loạt	Chọn convention (plural + kebab-case) và enforce từ PR đầu tiên	3
9	Bỏ qua error format chuẩn — Mỗi service trả lỗi format riêng	Consumer phải handle N formats, debugging khó	GlobalExceptionHandler + error response format thống nhất	3

D.3. Communication

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
10	Gọi sync chuỗi (service chain) — $A \rightarrow B \rightarrow C \rightarrow D$, mỗi service chờ cái tiếp theo	Latency cộng dồn, availability giảm theo phép nhân	Chuyển sang async cho non-critical paths, dùng API Composition	4
11	Không có circuit breaker — Gọi service lỗi liên tục, không dừng lại	Cascading failure, toàn bộ hệ thống chết	Resilience4j: circuit breaker + retry + timeout kết hợp	4
12	Retry không có backoff — Retry ngay lập tức, không exponential delay	Service đang quá tải bị thêm load, chết nhanh hơn	Exponential backoff + jitter: $1s \rightarrow 2s \rightarrow 4s$ (ngẫu nhiên $\pm$)	4
13	Auto-commit offset trước khi xử lý — Consumer acknowledge trước khi processing xong	Message mất vĩnh viễn nếu consumer crash	Manual offset commit — chỉ commit <i>sau khi</i> processing thành công	5
14	Không có Dead Letter Topic — Message lỗi retry vô hạn hoặc block pipeline	Poison pill chặn mọi message phía sau	@RetryableTopic — messages lỗi sau N retries $\rightarrow$ DLT	5
15	Consumer không idempotent — Giả định mỗi message chỉ đến 1 lần	Message trùng $\rightarrow$ dữ liệu sai khi Kafka rebalance/retry	Kiểm tra trạng thái trước khi xử lý — nếu đã xử lý thì skip	5
16	Chọn broker vì quen, không vì use case — RabbitMQ cho event streaming chỉ vì team biết	Broker không phù hợp khi scale $\rightarrow$ migrate đau đớn	Đánh giá use case: durable log (Kafka) vs smart routing (RabbitMQ)	5

D.4. Transactions & Data

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
17	2PC trong microservices — Cố giữ distributed transaction xuyên services	Blocking, SPOF, giảm availability	Chấp nhận eventual consistency, dùng Saga pattern	6
18	Không định nghĩa compensation — Chỉ nghĩ happy path, bỏ qua rollback scenario	Data inconsistent, records stuck vĩnh viễn	Mỗi transaction <i>phải</i> có compensating action trước khi code	6
19	Saga không có timeout — Không kiểm tra “bao lâu chưa xong?”	Submissions stuck PENDING vĩnh viễn	Scheduled job kiểm tra timeout + compensation tự động	6
20	Không dùng semantic lock — Cho phép thao tác trên data đang trong saga	Race conditions, dirty reads, kết quả sai	Set status flag khi saga bắt đầu, block cho đến khi hoàn thành	6
21	Chia database quá sớm — Tách DB trước khi hiểu data access patterns	Phát hiện cần JOIN liên tục → build event pipeline phức tạp	Bắt đầu separate schema, theo dõi 2-4 tuần, rồi mới quyết định	7
22	CQRS cho mọi thứ — Áp dụng cho CRUD đơn giản	Complexity tăng 3-5x cho data thay đổi 1 lần/tuần	CQRS chỉ khi read phức tạp, high volume, hoặc cross-service joins	7
23	Duplicate data không có source of truth — Copy data mà không rõ ai sở hữu	Conflict khi hai services cùng sửa, không biết bản nào đúng	Mỗi data entity chỉ có 1 owner — copies là read-only replicas	7
24	Quên replication lag — Write → read ngay → thấy data cũ	User confused: nộp bài OK nhưng leaderboard chưa cập nhật	UI “optimistic cập nhật” — hiển thị dự kiến ngay, cập nhật khi event đến	7

D.5. Security & Gateway

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
25	Mỗi service tự validate JWT — N services × N implementations = inconsistency	Logic xác thực phân tán, dễ sai sót	Gateway validate tập trung, services tin tưởng gateway headers	8
26	Không rate limiting — API Gateway không giới hạn requests	DoS (vô tình hoặc cố ý) — 1 user gây sập toàn bộ	Rate limiting ở Gateway (Redis-based sliding window)	8
27	Gateway quá thông minh — Gateway chứa business logic, transform data	Gateway trở thành bottleneck, thay đổi = deploy toàn bộ	Gateway chỉ xử lý cross-cutting: routing, auth, rate-limit, logging	8
28	JWT không hết hạn hoặc quá dài — Access token expiry 30 ngày	Token bị đánh cắp → attacker có quyền truy cập lâu dài	Short-lived access token (15-60 phút) + refresh token (30 ngày)	9
29	Lưu JWT trong localStorage — Frontend lưu token ở localStorage	XSS attack → đọc token dễ dàng	Lưu trong httpOnly cookie (không truy cập qua JS)	9
30	Hard-code roles trong code — if role == "ADMIN" khắp service	Thêm role mới → sửa code nhiều nơi, dễ sót	RBAC centralized — permissions mapping, không check role trực tiếp	9

D.6. Migration & Observability

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
31	Distributed Monolith — Tách services nhưng giữ shared DB, shared lib, deploy đồng bộ	Tất cả complexity của MS mà không có benefits	Database-per-service + thin shared libraries	10
32	Big Bang Rewrite — Viết lại toàn bộ hệ thống từ zero thay vì migrate dần	Cả hai hệ thống cần maintain, “switch day” không bao giờ đến	Strangler Fig — tách từng phần, mỗi phần mang giá trị ngay	10
33	Migrate quá nhanh — Tách 20 services trong 2 tháng vì “architecture decision”	Boundaries sai → refactor lại, effort gấp đôi	Migrate 1 service tại 1 thời điểm, bắt đầu từ service có giá trị cao nhất	10
34	Lift-and-shift không redesign — Copy code monolith nguyên xi vào containers	Network latency + failure modes, vẫn coupled	Anti-Corruption Layer, redesign domain model cho mỗi service	10
35	Over-engineering infra — K8s + Istio + full monitoring cho 3 services, 100 req/min	70% thời gian maintain infra thay vì phát triển tính năng	Docker Compose đủ cho ≤10 services, scale infra khi cần	10
36	Chỉ log khi lỗi — Không log request flow bình thường	Khi lỗi xảy ra, không đủ context để debug	Structured logging cho mọi request + correlation ID	11
37	Metric quá nhiều hoặc quá ít — 500 metrics hoặc chỉ CPU/RAM	Quá nhiều: noise, alert fatigue. Quá ít: blind spots	Focus SLI/SLO: latency, error rate, throughput	11
38	Alert mọi thứ — Mỗi exception → SNS → email/Slack	Alert fatigue: team ignore alerts, miss real incidents	Alert trên <i>symptoms</i> (SLO breach), không trên <i>causes</i>	11

D.7. Deployment & DevOps

#	Anti-pattern	Hậu quả	Phòng tránh	Ch.
39	Deploy Friday chiều — Deploy tính năng mới cuối tuần	Downtime khi không ai online → kéo dài đến thứ hai	Deploy sáng thứ hai-tư, khi team sẵn sàng xử lý	12
40	K8s cho 3 services — “Netflix dùng, ta cũng nên”	Complexity overhead lớn cho team 2-3 người	Docker Compose đủ cho ≤10 services. K8s khi cần multi-host	12
41	Build image khác per environment — Dev/staging/prod build riêng	“Works on staging” fail production vì image khác	Build once, deploy everywhere — externalize config qua env vars	12
42	Không có rollback plan — Deploy mới, có bug, không biết quay	Panic, fix production trực tiếp, thêm bug	Mọi deployment phải có rollback procedure documented	12

Tổng cộng: 42 anti-patterns xuyên suốt 6 chủ đề.

Cách sử dụng: Khi phát hiện vấn đề trong hệ thống, tra cứu bảng theo chủ đề → đọc phòng tránh → tham khảo chương tương ứng để hiểu chi tiết và context.

E. Bài tập & Case Studies

“The best engineers don’t just know patterns — they know when NOT to use them.” — Adapted from system design interview community

E.1. Hướng dẫn sử dụng

Mỗi bài tập/case study được phân loại:

Ký hiệu	Loại bài	Mô tả
	System Design	Thiết kế hệ thống từ requirements — dạng interview
	Trade-off Analysis	Phân tích trade-offs giữa các phương án — dạng ByteByteGo
	Case Study	Phân tích hệ thống thực tế (Netflix, Uber, Stripe...)
	Debugging Scenario	Tìm root cause từ symptoms — dạng on-call incident
	Architecture Decision Record	Viết ADR cho một quyết định thiết kế

Mức độ: ● Cơ bản | ●● Trung bình | ●●● Nâng cao (interview-level)

E.2. Chương 1: Monolith → Microservices

E.2.1. 1-1 Khi nào KHÔNG nên Microservices? ●●

Tình huống: Bạn là architect consultant. Ba công ty thuê bạn tư vấn “có nên chuyển sang microservices?”

Công ty	Mô tả	Team	Traffic	Pain point
A	Startup fintech, 8 tháng tuổi, product-market fit chưa rõ	4 devs	200 users/ngày	CEO muốn “scale như Grab”
B	E-commerce 5 năm, monolith 500K lines, deploy mất 4 giờ	25 devs, 4 teams	50K orders/ngày	Mỗi team deploy phải coordinate, release cycle 2 tuần
C	Hệ thống ERP nội bộ ngân hàng, Java 8, Oracle DB	10 devs, 2 teams	500 users nội bộ	Muốn “hiện đại hóa” vì vendor nói microservices tốt hơn

Câu hỏi phân tích:

- Áp dụng Decision Matrix (Bảng 10.1) cho từng công ty. Điểm số cụ thể cho mỗi tiêu chí?
- Tính **Microservice Premium** cho mỗi case: chi phí phải trả (distributed complexity, DevOps maturity cần thiết, learning curve) vs benefits thực tế
- Đưa ra recommendation cho mỗi công ty: **Yes** / **Not Yet** / **Never** — với reasoning cụ thể
- Cho công ty B (ứng viên mạnh nhất): service nào nên tách TRƯỚC? Vì sao? Áp dụng nguyên tắc “tách service mang lại nhiều value nhất trước” (Richardson [2a, Ch.13])
- Câu hỏi ByteByteGo:** Công ty A muốn bạn thiết kế microservices “để sẵn cho tương lai”. Bạn sẽ thuyết phục họ thế nào rằng đây là sai lầm? Dùng quote nào từ Martin Fowler?

E.2.2. 1-2 Case Study: Shopify — Monolith tỷ đô ●●●

Context: Shopify (2023) vận hành trên **Ruby on Rails monolith** phục vụ hàng triệu merchants, xử lý \$444 billion GMV. Thay vì chuyển sang microservices, Shopify chọn **Modular Monolith** — tách code thành components có boundaries rõ ràng nhưng deploy cùng nhau.

Tham khảo: Shopify Engineering Blog “Deconstructing the Monolith” (2019), “Under Deconstruction” (2020).

Câu hỏi phân tích:

1. Tại sao Shopify — với quy mô lớn hơn hầu hết companies — lại KHÔNG chuyển sang microservices? Đây có mâu thuẫn với lời khuyên của Newman không?
2. **Modular Monolith** khác gì **Microservices** về: (a) deployment boundary, (b) data isolation, (c) communication overhead, (d) team autonomy?
3. Shopify dùng concept **Component Boundaries** thay vì **Service Boundaries**. So sánh hai khái niệm này — khi nào component boundary đủ, khi nào cần service boundary?
4. Nếu KBLab trong sách áp dụng Modular Monolith thay vì Microservices, architecture sẽ trông thế nào? Vẽ diagram. Trade-offs so với kiến trúc hiện tại?
5. **Debate:** “Microservices là bắt buộc cho hệ thống lớn” — bạn đồng ý hay phản đối? Dùng Shopify làm evidence.

E.3. Chương 2: DDD & Bounded Contexts

E.3.1. 2-1 Event Storming: Thiết kế Hệ thống Đặt Xe ●●

Tình huống: Bạn đang thiết kế hệ thống ride-hailing (kiểu Grab/Uber đơn giản hóa). Stakeholders mô tả:

“Khách mở app, nhập điểm đón và điểm đến. Hệ thống tìm tài xế gần nhất, gửi request. Tài xế accept hoặc reject. Nếu accept, khách thấy vị trí tài xế real-time. Khi hoàn thành, tính tiền theo quãng đường. Khách đánh giá tài xế. Tiền được giữ trong ví, tài xế rút cuối tuần.”

Yêu cầu:

1. **Liệt kê Domain Events** (≥ 12 events) theo lịch trình: RideRequested → DriverFound → DriverAccepted → ...
2. **Xác định Aggregates:** nhóm events → xác định ≥ 4 aggregates (Ride? Driver? Payment? ...)
3. **Vẽ Bounded Contexts:** xác định ≥ 4 contexts với relationships
4. **Câu hỏi trade-off:** Nên tách “Rating” thành bounded context riêng hay gộp vào “Ride Management”? Phân tích ưu nhược của cả hai cách
5. **Conway’s Law:** Nếu team chỉ có 8 người, bạn tổ chức team thế nào để align với bounded contexts?

E.3.2. 2-2 Shared Library vs API — Khi nào nên gì? ●●

Tình huống: Hai microservices (Order Service và Inventory Service) cần dùng chung logic validate mã sản phẩm (product code format). Team tranh luận:

Option	Developer A đề xuất	Developer B đề xuất
Shared Library	Tạo common-validation JAR, cả hai services import	Để reuse, DRY principle
API Call	Tạo Validation Service riêng, gọi qua REST	Loose coupling, deploy độc lập
Duplicate Code	Copy validation logic vào mỗi service	Zero coupling, nhưng maintenance?

Câu hỏi phân tích:

1. Phân tích trade-off chi tiết cho từng option theo 5 tiêu chí: coupling, deploy independence, performance, maintenance, team autonomy
2. KBLab sử dụng shared library (kblab-shared) chứa DTOs, ErrorCode, utils. Đây có phải anti-pattern? Khi nào shared library là hợp lý?
3. Newman trong [4a] khuyên: “Chỉ share DTOs và cross-cutting concerns (logging, auth). KHÔNG share business logic.” Áp dụng nguyên tắc này: phân loại nội dung kblab-shared → cái nào nên giữ, cái nào nên di chuyển về service?
4. **Câu hỏi ByteByteGo:** Bạn join team mới, thấy shared library 50 classes được dùng bởi 8 services. Tất cả services phải deploy cùng lúc khi shared library thay đổi. Đây là symptom của vấn đề gì? Bạn propose giải pháp gì?

E.4. Chương 3: Thiết kế API

E.4.1. 3-1 🔍 Case Study: Stripe API — Tại sao được coi là “gold standard”? ●●

Context: Stripe API được cộng đồng developer đánh giá là **API design tốt nhất thế giới**. Phân tích tại sao.

Dữ liệu: Truy cập stripe.com/docs/api (hoặc dựa trên kiến thức).

Câu hỏi phân tích:

1. Stripe dùng **date-based versioning** (2024-12-18) thay vì v1/v2. So sánh với URL path versioning — ưu/nhược điểm gì? Khi nào date-based tốt hơn?
2. Stripe API trả response dạng **envelope pattern**: { "object": "list", "data": [...], "has_more": true }. So sánh với cách KBLab trả { "data": [...], "pagination": {...} } — cái nào tốt hơn? Tại sao?
3. Stripe sử dụng **Idempotency Key** cho mọi POST request (tránh charge customer 2 lần). Thiết kế cơ chế tương tự cho POST /api/submissions của LMS — sinh viên submit bài 2 lần do mạng lag → chỉ chấm 1 lần
4. Stripe luôn trả **error object nhất quán**: { "error": { "type": "card_error", "code": "expired_card", "message": "..." } }. So sánh với error format hiện tại của LMS — gap ở đâu?
5. **Thiết kế:** Nếu bạn phải thiết kế API cho tính năng mới: “Giảng viên tạo contest → sinh viên đăng ký → submit bài → leaderboard”, viết danh sách endpoints theo Stripe style

E.4.2. 3-2 🐞 Debugging: API Breaking Change Incident ●●●

Tình huống incident:

9:00 AM — Team deploy Core Service v2.3: đổi response field `questionTitle` → `title` (cleaner naming). 9:15 AM — Mobile app (v1.2, chưa cập nhật) crash khi load danh sách câu hỏi. 200 sinh viên đang luyện tập bị ảnh hưởng. 9:30 AM — Dashboard team phát hiện, rollback Core Service về v2.2. 10:00 AM — Postmortem bắt đầu.

Câu hỏi phân tích:

1. **Root cause:** Đây là loại breaking change nào? (rename, remove, type change?) Tại sao rename field tương đương “xóa cũ + thêm mới”?
2. **Prevention:** Liệt kê ≥3 mechanisms ngăn incident này tái diễn (contract testing, schema validation, versioning policy...)
3. **Thiết kế giải pháp:** Nếu PHẢI đổi tên field, làm thế nào mà KHÔNG breaking? Mô tả quy trình 3 bước
4. **Consumer-Driven Contract Testing:** Giải thích concept. Nếu mobile app có contract test, incident này có bị bắt trước production không?
5. **ADR:** Viết Architecture Decision Record cho quyết định: “Từ nay API response chỉ được THÊM field, không được XÓA hoặc ĐỔI TÊN field”

E.5. Chương 4: Giao tiếp Đồng bộ & Resilience

E.5.1. 4-1 🐞 Debugging: Cascading Failure — “Tại sao cả hệ thống chết?” ●●●

Tình huống incident (mô phỏng từ thực tế):

Lịch trình:

- 14:00 — Database server của Recommendation Service gặp sự cố hardware, tất cả queries timeout (30s)
- 14:02 — Product Service gọi Recommendation Service → timeout 30s → thread pool cạn kiệt
- 14:05 — Order Service gọi Product Service → cũng timeout → thread pool Order Service cạn kiệt
- 14:08 — API Gateway gọi Order Service → timeout → Gateway thread pool đầy
- 14:10 — **TOÀN BỘ HỆ THỐNG** không phản hồi. Health checks fail. PagerDuty alert.
- 14:25 — On-call engineer restart tất cả services. Hệ thống recover sau 5 phút.
- **Tổng downtime:** 25 phút. Ảnh hưởng: 15,000 users.

Câu hỏi phân tích:

1. Vẽ **lịch trình diagram** (kiểu sequence diagram) cho cascading failure. Service nào fail trước? Domino effect lan thế nào?
2. **Root cause vs Trigger:** Database hardware failure là trigger. Root cause thực sự là gì? (gợi ý: thiếu pattern nào?)
3. Nếu hệ thống có **Circuit Breaker** (Resilience4j) giữa mỗi service call, lịch trình sẽ thay đổi thế nào? Viết lại lịch trình
4. Nếu có Circuit Breaker VÀ **Bulkhead**, kết quả khác gì so với chỉ có Circuit Breaker?
5. Thiết kế **Resilience configuration** cho hệ thống 4 tầng này: timeout, retry, circuit breaker, bulkhead cho mỗi tầng. Giải thích tại sao timeout tầng ngoài phải > timeout tầng trong

6. **Câu hỏi ByteByteGo:** Netflix xử lý 2 tỷ requests/ngày. Thiếu circuit breaker ở MỘT service có thể crash entire platform. Netflix giải quyết bằng cách nào? (Hystrix → Resilience4j)

E.5.2. 4-2 REST vs gRPC vs GraphQL — Chọn gì cho scenario nào? ●●

Tình huống: Bạn đang thiết kế communication giữa microservices cho hệ thống e-commerce:

Giao tiếp	Đặc điểm
A: Mobile app ↔ BFF (Backend For Frontend)	Bandwidth hạn chế, cần flexible queries
B: Order Service ↔ Inventory Service (internal)	High throughput, low latency, strict schema
C: Admin Dashboard ↔ Product Service	Cần query nested data (product + reviews + inventory)
D: Public API cho third-party merchants	Stability, documentation, versioning

Câu hỏi phân tích:

- Với mỗi scenario (A-D), chọn REST / gRPC / GraphQL. Giải thích trade-off
- Có scenario nào mà 2 protocols phù hợp như nhau? Tiêu chí “tiebreaker” là gì?
- Netflix dùng gRPC cho inter-service communication, GraphQL cho mobile BFF. Spotify dùng gRPC internal + REST public API. Pinterest chuyển từ REST sang gRPC internal, giảm latency 50%. Phân tích: tại sao các công ty lớn đều hướng gRPC cho internal nhưng giữ REST/GraphQL cho external?
- KBLab hiện dùng REST cho LMS core, nhưng Network Lab đã có SOAP/REST/gRPC/JNP và DevOps Lab dùng Go/Chi cho một phần API. Nếu phải chọn protocol cho một boundary internal mới — chọn gì? Cost/benefit analysis?

E.6. Chương 5: Giao tiếp Bất đồng bộ

E.6.1. 5-1 Kafka vs RabbitMQ — Quyết định dựa trên gì? ●●

Tình huống: 4 scenarios khác nhau, mỗi cái cần message broker:

Scenario	Mô tả	Yêu cầu đặc biệt
A: Email notification	User action → send email	Đảm bảo gửi đúng 1 lần, order không quan trọng
B: Event sourcing cho banking	Mọi transaction = event, replay được	Ordering cực kỳ quan trọng, retention vĩnh viễn
C: IoT sensor data	100K sensors gửi data mỗi giây	Throughput cực cao, loss vài messages OK
D: Task queue	Image resize: upload → queue → process	Round-robin distribution, consumer acknowledgment

Câu hỏi phân tích:

- Cho mỗi scenario: chọn Kafka hay RabbitMQ? Giải thích bằng architectural differences (log-based vs queue-based)

2. Kleppmann trong [7, Ch.11] giải thích: Kafka = “distributed commit log” (append-only, consumer controls offset). RabbitMQ = “message queue” (broker pushes, message deleted after ack). Cách mental model này giúp quyết định thế nào?
3. LinkedIn (inventor Kafka) xử lý 7 triệu messages/giây. Họ CẦN Kafka. KBLab có nhiều loại queue khác nhau: submission pipeline, SQL Judge worker dispatch (judge → judge-*), DevOps Lab jobs. Flow nào thật sự cần Kafka, flow nào chỉ cần DB queue? Trade-off giữa “học tool đúng” vs “dùng tool phù hợp”?
4. **Exactly-once delivery**: Kafka đạt exactly-once transactional semantics từ 0.11. RabbitMQ không có native exactly-once. Giải thích tại sao exactly-once khó và tại sao “at-least-once + idempotent consumer” thường đủ
5. KBLab chọn Kafka cho submission pipeline. Nếu dùng RabbitMQ hoặc DB queue thay thế, architecture thay đổi thế nào? Feature nào mất?

E.6.2. 5-2 🐛 Debugging: “Messages biến mất” — Dead Letter & Poison Pills ●●●

Tình huống incident:

Symptoms: Kiểm tra cuối kỳ — 5 sinh viên report “nộp bài nhưng không nhận kết quả”. Giáo viên kiểm tra DB: submission records tồn tại, nhưng không có score.

Investigation log:

- Core Service logs: ☒ SubmissionCreated events published thành công
- Kafka topic submissions: ☒ Messages tồn tại (verified bằng kafka-console-consumer)
- Judge Service logs: 5 messages throw `JsonParseException` → consumer crash & restart → message được re-consume → crash lại → **infinite loop**
- Root cause: 5 submissions chứa SQL có ký tự Unicode đặc biệt → serialization fail

Câu hỏi phân tích:

1. Đây là example của “**Poison Pill**” message. Định nghĩa poison pill. Tại sao nó nguy hiểm hơn một message fail bình thường?
2. Không có DLT (Dead Letter Topic): consumer crash → restart → re-consume same message → crash → loop forever. **45 messages khác** (5 students không bị lỗi) cũng bị stuck sau poison pill. Giải thích mechanism (Kafka consumer offset không commit khi fail)
3. Thiết kế giải pháp **3 tầng**:
 - Tầng 1: Retry (bao nhiêu lần? backoff strategy?)
 - Tầng 2: Dead Letter Topic (message format? alert mechanism?)
 - Tầng 3: Manual review dashboard (admin xem DLT messages, reprocess hoặc discard)
4. Sau khi implement DLT, flow mới sẽ như thế nào? Vẽ diagram. 5 messages lỗi đi đâu? 45 messages còn lại có bị ảnh hưởng?
5. **Prevention**: Làm sao ngăn poison pill messages từ đầu? (Schema validation, contract testing ở producer)

E.7. Chương 6: Saga Pattern

E.7.1. 6-1 🏠 Design: Uber Ride Saga — Choreography hay Orchestration? ●●●

Tình huống: Một chuyến xe Uber trải qua flow:

1. Rider request ride
2. Match driver (tìm tài xế'gần nhất)
3. Driver accept
4. Trip starts (GPS tracking bắt đầu)
5. Trip ends (tính khoảng cách)
6. Calculate fare (surge pricing, discounts)
7. Charge rider's payment method
8. Pay driver (sau commission)
9. Send receipt

Nếu Step 7 fail (thẻ hết tiền):

Compensation: Cancel payment → Notify rider "add payment method" → Hold trip record (chờ retry)

NHƯNG: KHÔNG thể'undo trip (đã chạy rồi!) → Đây là "non-compensatable action"

Câu hỏi phân tích:

1. Xác định **compensable**, **pivot**, và **retrieable** transactions trong 9 steps. Step nào là point of no return?
2. Uber (thực tế) dùng **Orchestration** cho ride saga — tại sao? So sánh nếu dùng Choreography: bao nhiêu events cần? Ai track trạng thái tổng thể?
3. **Isolation problem**: Rider A request ride, Driver X được match → trong lúc Driver X đang đường đến → Rider B request ride, hệ thống cũng match Driver X. Đây là anomaly gì? (Lost update? Dirty read?) Countermeasure nào cho scenario này?
4. **Timeout design**: Driver không accept trong 30 giây → timeout → tìm driver khác. Nhưng network lag → Driver accept sau 31 giây (message arrive trễ). Thiết kế mechanism xử lý "late acceptance"
5. **KBLab comparison**: Submission saga trong KBLab (Submit → Judge → Score) đơn giản hơn nhiều. Nhưng nếu KBLab thêm feature "contest = phải trả phí" → saga phức tạp thêm thế nào? Vẽ diagram mới

E.7.2. 6-2 📊 Trade-off: Choreography vs Orchestration ●●

Bài tập: Cho 4 saga scenarios, phân tích nên dùng Choreography hay Orchestration:

Saga	Steps	Đặc điểm
A: User registration	Create account → Send verification email → Assign default role	3 steps, simple, low failure rate
B: E-commerce order	Reserve stock → Process payment → Ship → Update loyalty points	4 steps, payment critical, needs monitoring
C: Insurance claim	Submit claim → Fraud detection → Adjuster review → Payout → Notify	5 steps, human-in-the-loop, days-long process
D: Flight booking (multi-leg)	Book leg 1 → Book leg 2 → Book hotel → Book car → Payment	5 steps, external APIs, partial failure common

Câu hỏi phân tích:

1. Cho mỗi saga: Choreography hay Orchestration? Reasoning dựa trên complexity, number of steps, failure probability, human involvement
2. Richardson nói “ ≤ 4 steps → Choreography OK, > 4 steps → consider Orchestration”. Bạn đồng ý? Có exceptions?
3. **Saga D (Flight booking)** đặc biệt khó — external APIs (airline, hotel) có thể timeout, return ambiguous results. Thiết kế compensation cho: “Book leg 1 thành công, book leg 2 timeout — không biết thành công hay fail”. Đây là “uncertain state” — xử lý thế nào?
4. Netflix dùng **Conductor** (open-source orchestration engine). Uber dùng **Cadence/Temporal**. Tại sao các công ty lớn build orchestration engines riêng thay vì dùng choreography?

E.8. Chương 7: Quản lý Dữ liệu

E.8.1. 7-1 CAP Theorem trong Thực tế — “Pick Two” có đúng không? ●●●

Tình huống: Team tranh luận về database strategy:

“CAP theorem nói chỉ chọn được 2 trong 3: Consistency, Availability, Partition Tolerance. Vậy nếu partition xảy ra, mình phải chọn giữa C và A. Nhưng hệ thống mình cần CẢ HAI...”

Câu hỏi phân tích:

1. Martin Kleppmann phản bác “pick two” interpretation. Ông nói CAP thực ra là: “During a network partition, choose between consistency and availability. When there’s NO partition, you can have both.” Giải thích — khác biệt thực tế so với “pick two” là gì?
2. Phân loại databases theo CAP:

Database	Classification	Behavior khi partition
PostgreSQL (single master)	?	?
Cassandra	?	?
MongoDB (replica set)	?	?
Redis Cluster	?	?

3. KBLab dùng PostgreSQL cho nhiều dữ liệu giao dịch. Khi primary DB down một khoảng thời gian, hệ thống có thể unavailable cho các write path. Nếu dùng PostgreSQL với streaming replication, trade-off thay đổi thế nào?
4. **E-commerce scenario:** Shopping cart vs Payment — yêu cầu C/A khác nhau:
 - Shopping cart: availability > consistency (OK nếu số lượng hiển thị chậm 5s)
 - Payment: consistency > availability (KHÔNG OK nếu charge 2 lần)
 - Thiết kế: dùng database strategy nào cho từng feature?
5. **Câu hỏi ByteByteGo:** Amazon.com có SLA 99.99% availability nhưng shopping cart dùng eventually consistent storage (Dynamo). Jeff Bezos nói “Customers should always be able to add items to their cart, even during failures.” Bạn đồng ý trade-off này? Giải thích bằng CAP

E.8.2. 7-2 🏠 Design: Database Migration Strategy cho KBLab ●●

Tình huống: KBLab hiện có shared database — Core Service và Assignment Service dùng chung PostgreSQL. Bạn được giao nhiệm vụ tách database.

Constraints:

- Zero downtime (sinh viên đang dùng hệ thống)
- Team 2 developers
- Timeline: 3 tháng
- Không được mất data

Tables:

Core domain: questions, submissions, scores, contests

Assignment domain: assignments, assignment_questions, student_assignments

Shared: users (dùng bởi CẢ HAI)

Cross-reference: assignment_questions.question_id → questions.id

Câu hỏi phân tích:

1. Áp dụng 5 chiến lược tách database (Bảng 10.4): thiết kế timeline 3 tháng — tháng 1, 2, 3 làm gì?
2. Table users dùng bởi cả hai services. 3 options: (a) giữ ở Core, Assignment gọi API, (b) duplicate users data, (c) tạo User Service riêng. Phân tích trade-off mỗi option
3. assignment_questions reference questions.id — đây là cross-domain join. Sau khi tách DB, join này KHÔNG CÒN HOẠT ĐỘNG. Thiết kế giải pháp: API composition? Data duplication? CQRS?
4. **Dual write risk:** Trong quá trình migration, một giai đoạn cả code cũ (direct DB) và code mới (via API) cùng tồn tại. Làm sao tránh inconsistency? (Feature flag? Shadow writes?)
5. **Rollback plan:** Nếu tháng 2 phát hiện performance issues nghiêm trọng — cách nào rollback về shared database an toàn?

E.9. Chương 8: API Gateway

E.9.1. 8-1 Case Study: Netflix Zuul → Spring Cloud Gateway → Custom Gateway ●●

Context: Netflix đi qua 3 thế hệ API Gateway:

- **Zuul 1** (2013): Blocking I/O, servlet-based → bottleneck ở 10K concurrent connections
- **Zuul 2** (2016): Non-blocking, Netty-based → chưa bao giờ được Netflix dùng rộng rãi
- **Spring Cloud Gateway** (2019): Reactive (WebFlux), community-driven → nhiều companies adopt
- **Netflix Custom** (hiện tại): Build gateway riêng optimize cho use case cụ thể

Câu hỏi phân tích:

1. Tại sao Netflix KHÔNG dùng Zuul 2 mà tự build gateway mới? (Gợi ý: organizational reasons vs technical reasons)
2. **Blocking vs Non-blocking Gateway:** Zuul 1 dùng 1 thread/request (blocking). Spring Cloud Gateway dùng event loop (non-blocking). Với 10K concurrent connections, Zuul 1 cần bao nhiêu threads? Spring Cloud Gateway cần bao nhiêu? Implications cho memory?
3. KBLab dùng Spring Cloud Gateway cho LMS core và Go hostname-based router cho DevOps Lab. Với traffic học thuật vừa phải, blocking hay non-blocking có khác biệt? Khi nào KBLab CẦN non-blocking gateway/router?
4. **Gateway as Single Point of Failure:** Nếu Gateway crash, toàn bộ hệ thống down. Thiết kế: high availability cho Gateway? (Multiple instances? Load balancer phía trước?)
5. **BFF Pattern (Backend for Frontend):** Netflix có gateway riêng cho mobile app (ít bandwidth) và web app (cần nhiều data hơn). Nếu KBLab cần support mobile app, bạn sẽ thiết kế BFF thế nào?

E.9.2. 8-2 ADR: Rate Limiting Strategy ●●

Tình huống: Hệ thống LMS bị abuse:

- Một sinh viên viết script tự động submit 1000 lần/phút (brute-force đáp án SQL)
- Judge Service overloaded → ảnh hưởng contest của 200 sinh viên khác

Yêu cầu: Viết Architecture Decision Record cho Rate Limiting.

Template ADR:

1. **Title:** Rate Limiting Strategy cho LMS API Gateway
2. **Context:** Mô tả vấn đề (abuse, impact)
3. **Decision Drivers:** Security, Fair usage, Performance, Implementation cost
4. **Options Considered:**
 - Option 1: Rate limit per user (5 submissions/phút) — cấu hình như Listing 8.3
 - Option 2: Rate limit per IP
 - Option 3: Rate limit per route + time window (contest endpoints nghiêm ngặt hơn)
 - Option 4: Adaptive rate limiting (tự điều chỉnh theo total load)
5. **Decision:** Chọn option nào? Tại sao?
6. **Consequences:** Positive, Negative, Risks

7. **Follow-up:** Monitoring strategy? Alert khi user bị rate limited?

E.10. Chương 9: Bảo mật

E.10.1. 9-1 🦋 Security Incident: JWT Token Theft ●●●

Tình huống incident:

Report: Sinh viên A phát hiện điểm bài tập thay đổi — bài chưa nộp bỗng có điểm 100. Log cho thấy submissions được tạo bởi JWT token của sinh viên A, nhưng từ IP address khác.

Investigation: Sinh viên B (cùng phòng) copy JWT token từ browser DevTools của A khi A quên log out trên máy chung. Token dùng HS256, expiry 24 giờ, không có token refresh mechanism.

Câu hỏi phân tích:

1. **Tại sao JWT stateless lại nguy hiểm trong case này?** Token bị steal nhưng server không biết — không có cơ chế revoke. So sánh với session-based auth (session ID có thể revoke ngay lập tức)
 2. **Mitigation layers** — thiết kế 4 tầng bảo vệ:
 - Tầng 1: Token TTL (giảm expiry → bao nhiêu phút là hợp lý?)
 - Tầng 2: Token Refresh + Rotation (Ch.9, Listing 9.2) — giải thích tại sao rotation phát hiện được theft
 - Tầng 3: Device fingerprinting (bind token với device/browser)
 - Tầng 4: Anomaly detection (submit từ 2 IPs khác nhau trong 5 phút → flag)
 3. **HS256 vs RS256:** Nếu system dùng RS256, sinh viên B có token nhưng KHÔNG có private key. Có thể tạo token giả không? Khác biệt thực tế so với HS256?
 4. **Token Blacklist:** Implement token blacklist (Redis set) để revoke tokens bị steal. Trade-off: JWT “stateless” advantage bị mất. Khi nào blacklist đáng trade-off?
 5. **Zero Trust:** Google BeyondCorp model — “every request must be authenticated and authorized, regardless of network location”. Áp dụng vào KBLab: internal services có cần validate JWT không? (Hiện tại KBLab dùng “Pragmatic Trust” — trust service-to-service calls trong boundary nội bộ)
-

E.10.2. 9-2 🏠 OAuth2 + OIDC Flows — Chọn flow nào cho scenario nào? ●●

Bài tập: 4 applications cần integrate với Identity Provider (Keycloak/Auth0):

Application	Type	User interaction
A: Student web app (React SPA)	Single-page app, browser-based	User login, long sessions
B: CLI tool cho admin	No browser, terminal-based	Admin login
C: Judge Service (server-to-server)	No user, backend processing	Machine-to-machine
D: Mobile app (React Native)	Native app, custom login screen	User login, biometric

Câu hỏi phân tích:

1. Cho mỗi app (A-D): chọn OAuth2 flow nào? (Authorization Code + PKCE, Client Credentials, Device Authorization Grant)
2. **App A (SPA):** Tại sao Authorization Code flow cần PKCE cho SPAs? Không có PKCE → attack nào có thể xảy ra?
3. **App C (server-to-server):** Client Credentials flow không có user context. Judge Service nhận token này → biết service identity nhưng không biết user nào submit. Thiết kế: làm sao truyền user context qua service-to-service calls?
4. KBLab dùng token exchange: external provider/QLĐT/username-password → KBLab JWT. Nếu migrate sang Keycloak hoặc Microsoft Entra ID làm IdP trung tâm: những gì thay đổi? Lợi ích? Chi phí migration?

E.11. Chương 10: Chuyển đổi Thực tế

E.11.1. 10-1 🔍 Case Study: Amazon — Monolith → SOA → Microservices (2002–2020) ●●●

Context: Jeff Bezos' 2002 mandate nổi tiếng:

“All teams will henceforth expose their data and functionality through service interfaces. There will be no other form of inter-process communication allowed. Anyone who doesn't do this will be fired.”

Câu hỏi phân tích:

1. Bezos mandate 2002 ra đời TRƯỚC thuật ngữ “microservices” (2014). Amazon gọi kiến trúc của họ là “Service-Oriented Architecture”. Sự khác biệt giữa SOA của Amazon và microservices hiện đại là gì?
2. Amazon không làm “big bang” rewrite. Họ tách service dần dần qua nhiều năm. Strangler Fig Pattern (Ch.10) có thể áp dụng để mô tả quá trình này? Vẽ lịch trình 3 phases
3. **“Two-Pizza Team” rule** — mỗi team 6-8 người, sở hữu 1-2 services. Conway's Law in action? Liên hệ với Ch.2 (team topology ↔ service boundaries)
4. Amazon hiện có **hàng nghìn** microservices. Vấn đề mới: service sprawl, dependency management, ownership confusion. Giải pháp? (Service mesh, service catalog, platform teams)
5. **KBLab comparison:** KBLab có nhiều services/module cho team nhỏ. Áp dụng Two-Pizza Rule: bao nhiêu services là hợp lý cho team size này? KBLab có risk “too many services for small team” không?

E.11.2. 10-2 📊 Outbox Pattern vs Event Sourcing vs Change Data Capture ●●

Tình huống: Team cần giải quyết dual-write problem (save DB + publish event). 3 approaches:

Approach	Mechanism	Ví dụ tool
Outbox Pattern	Write event vào DB table (cùng transaction) → poll/ CDC publish	Custom code + cron/ Debezium
Event Sourcing	Events LÀ source of truth, derive state từ events	Axon Framework, EventStoreDB
CDC (Change Data Capture)	Đọc DB transaction log → auto-publish changes	Debezium + Kafka Connect

Câu hỏi phân tích:

- So sánh 3 approaches theo 6 tiêu chí: complexity, infrastructure cost, latency, reliability, learning curve, data model impact
- Event Sourcing** thay đổi fundamental data model (events first, state derived). Khi nào trade-off này đáng? Khi nào quá mức? Cho ví dụ domain phù hợp (banking) vs không phù hợp (CRUD admin panel)
- CDC chạy bên ngoài application code** — không cần thay đổi code. Ưu điểm rõ ràng. Nhưng nhược điểm gì? (DB-specific, operational complexity, schema change sensitivity)
- LMS chọn Outbox Pattern (Polling Publisher). Hợp lý? Nếu traffic tăng 100x, approach nào phù hợp hơn?
- Câu hỏi ByteByteGo:** Bạn đang interview. Interviewer hỏi: “Giải thích dual-write problem và 3 cách giải quyết, mỗi cách cho ví dụ real-world.” — Trả lời trong 3 phút

E.12. Chương 11: Observability

E.12.1. 11-1 🐞 Debugging: Latency Spike Mystery ●●●

Tình huống incident:

Alert: Grafana dashboard báo submission latency P99 tăng từ 2s → 12s. Xảy ra mỗi thứ Ba lúc 14:00.

Dữ liệu available:

- Metrics: CPU tất cả services < 30%. Memory OK. Kafka consumer lag tăng lúc 14:00.
- Logs: Không có errors. Judge Service log: “Executing SQL...” — bình thường.
- Tracing: Một trace sample cho thấy: Gateway (5ms) → Core (20ms) → Kafka publish (10ms) → Judge receive (delay 8000ms!) → Judge execute (800ms) → Core result (15ms)

Manh mối: Thứ Ba 14:00 = đầu giờ “Cơ sở dữ liệu” — 120 sinh viên access hệ thống cùng lúc.

Câu hỏi phân tích:

- Từ trace data, **bottleneck ở đâu?** Kafka consumer delay 8000ms — nguyên nhân có thể là gì? (Gợi ý: consumer lag, partition count, consumer threads)

2. **RED analysis**: tính Rate, Error rate, Duration tại thời điểm incident. Metric nào bất thường?
3. Judge Service xử lý từng message tuần tự (single consumer thread). 120 submissions cùng lúc → queue up. Thiết kế **scaling strategy**: (a) increase consumer threads/instances, (b) increase partitions, (c) cả hai? Trade-offs?
4. Thiết kế **SLO** cho submission latency. Hiện P99 = 2s normal, spike 12s. SLO hợp lý: “99% submissions < Xs” — X bao nhiêu? Alert threshold?
5. Nếu bạn là on-call engineer nhận alert lúc 14:05: mô tả **runbook** — 5 bước đầu tiên bạn kiểm tra (từ macro → micro)?

E.12.2. 11-2 Observability Stack: Build vs Buy vs Managed ●●

Tình huống: CTO yêu cầu bạn đề xuất observability stack cho hệ thống 15 microservices:

Option	Components	Cost (ước tính/tháng)	Effort setup
A: Self-hosted OSS	Prometheus + Grafana + Loki + Jaeger	Chi phí infra tự quản	2-3 tuần
B: Managed OSS	Grafana Cloud (free tier → paid)	Theo usage	2-3 ngày
C: Commercial	Datadog / New Relic / Dynatrace	Cao hơn, đổi lấy support	1 ngày (agent install)
D: Cloud-native	AWS CloudWatch + X-Ray / GCP Cloud Monitoring	Theo cloud provider	1-2 ngày

Câu hỏi phân tích:

1. Cho 3 team profiles, recommendation nào phù hợp nhất?
 - Startup 5 người, ngân sách \$0-100/tháng
 - Scale-up 20 người, ngân sách \$500/tháng, SLA 99.9%
 - Enterprise 100 người, regulated industry (banking)
2. KBLab hiện đã có một số mảnh Level 2 (Actuator/Prometheus/Grafana, zerolog/PLG ở DevOps Lab) nhưng chưa chuẩn hóa end-to-end. Migration path đề xuất đến Level 3: chọn option nào? Tại sao?
3. **Vendor lock-in**: Datadog proprietary query language vs Prometheus PromQL (open standard). Khi nào vendor lock-in chấp nhận được? Khi nào dangerous?
4. OpenTelemetry (OTel) đang trở thành standard — “instrument once, send to any backend”. Chiến lược: adopt OTel từ đầu → dễ switch backend sau. Trade-off so với adopt vendor SDK?

E.13. Chương 12: Triển khai & DevOps

E.13.1. 12-1 Docker Compose vs Kubernetes — At What Scale? ●●

Tình huống: 3 hệ thống ở các scale khác nhau:

Hệ thống	Services	Traffic	Team	Yêu cầu
A: Startup MVP	3 services	500 req/ngày	2 devs	Ship fast, ngân sách thấp
B: KBLab LMS + labs	Nhiều services/module	Traffic học thuật, burst khi contest/lab	3 devs	Reliability, easy ops
C: E-commerce platform	25 services	1M req/ngày	15 devs, 4 teams	Auto-scaling, zero downtime, multi-region

Câu hỏi phân tích:

1. Cho mỗi hệ thống: Docker Compose hay Kubernetes? Hoặc hybrid? Giải thích dựa trên tiêu chí: team capability, operational overhead, scaling needs
2. **Anti-pattern:** Hệ thống A (3 services, 2 devs) muốn dùng Kubernetes “để sẵn cho tương lai”. Tính **cost of Kubernetes** cho team 2 người: learning time, cluster maintenance, debugging complexity. Cost > Benefit?
3. **Migration path:** Hệ thống B (KBLab) hiện dùng Docker Compose cho LMS core và k3s/Sysbox cho DevOps Lab. Khi nào CẦN mở rộng Kubernetes cho nhiều phần hơn? Xác định 3 trigger signals (traffic, team size, SLA requirements)
4. **Managed vs Self-managed K8s:** GKE/EKS/AKS vs self-setup kubeadm. Cost comparison cho hệ thống C? Risk comparison?
5. **Câu hỏi ByteByteGo:** “How does Kubernetes ensure zero-downtime deployment?” — Trả lời: Rolling Update mechanism, readiness probes, PDB (Pod Disruption Ngân sách). Vẽ diagram

E.13.2. 12-2 🏠 Design: CI/CD Pipeline cho Microservices ●●●

Tình huống: Bạn cần thiết kế CI/CD pipeline cho KBLab (Java LMS core nhiều repo/service + DevOps Lab Go multi-binary):

Requirements:

- Push code → auto build → test → deploy staging → manual approval → deploy production
- Chỉ build/test services bị thay đổi (không build lại toàn bộ)
- Database migrations chạy TRƯỚC service deploy
- Rollback mechanism: quay lại version trước trong < 5 phút
- Secret management: không hardcode passwords trong CI config

Câu hỏi phân tích:

1. **Pipeline design:** vẽ diagram CI/CD pipeline với tất cả stages. Bao gồm: changed service detection, parallel builds, test gates, approval gate, deployment
2. **Changed service detection** trong mono-repo: Git diff → xác định service nào thay đổi → chỉ build service đó. Thiết kế algorithm. Edge case: shared library thay đổi → build TẤT CẢ services?
3. **Database migration ordering:** Core Service depends on shared tables. Assignment Service depends on Core’s tables. Migration order quan trọng? Thiết kế dependency graph cho migrations

4. **Rollback strategy:** Canary deployment cho production. Nếu canary instance báo error rate > 5% → auto rollback. Thiết kế mechanism (health check → metrics → decision → rollback). Diagram?
5. **GitOps vs Push-based:** Traditional CI/CD pushes deployments. GitOps (ArgoCD) pulls from Git. So sánh cho LMS context — nên dùng cái nào?

E.14. Capstone: System Design Interview — Thiết kế “Online Judge”

●●●

Prompt (dạng interview 45 phút):

“Thiết kế hệ thống Online Judge (như LeetCode/HackerRank) cho 10,000 concurrent users. Hệ thống cho phép: users submit code (Python, Java, SQL), hệ thống chạy code trong sandbox, so sánh output, trả kết quả. Có contest mode (1000 users submit cùng lúc) với real-time leaderboard.”

Gợi ý cấu trúc answer (theo framework system design interview):

1. **Requirements clarification** (5 phút): Functional requirements? Non-functional (latency, throughput, availability)? Scale?
2. **High-level design** (10 phút): Vẽ architecture diagram. Xác định services, databases, message queues
3. **Deep dive** (20 phút): Chọn 2-3 components quan trọng nhất:
 - **Sandbox execution:** Docker container per submission? Security? Resource limits?
 - **Queue management:** Kafka partition strategy cho fair scheduling (contest mode)?
 - **Leaderboard:** CQRS + Redis sorted set? Update frequency?
4. **Scale & Trade-offs** (10 phút):
 - 1000 concurrent submissions → Judge Service scaling strategy?
 - CAP trade-off cho leaderboard: real-time (eventual consistent) vs accurate (strong consistent)?
 - Cost estimation: mỗi submission = 1 Docker container × 10 giây = how much compute?

Tiêu chí đánh giá:

- ☐ Architecture có ≥4 services rõ ràng
- ☐ Communication patterns hợp lý (sync vs async, khi nào dùng gì)
- ☐ Data management strategy (database-per-service, CQRS cho leaderboard)
- ☐ Resilience patterns (circuit breaker cho Judge, DLT cho failed submissions)
- ☐ Security (sandbox isolation, JWT auth)
- ☐ Scalability analysis (horizontal scaling Judge, partition strategy)
- ☐ Trade-off discussions (≥3 trade-offs được phân tích rõ)

Ghi chú: Các bài tập case study dựa trên thông tin public từ engineering blogs (Netflix, Amazon, Uber, Shopify, Stripe). Facts được simplify cho mục đích giáo dục — nên tham khảo nguồn gốc để có bức tranh đầy đủ.

F. Tài liệu tham khảo

F.1. Sách tham khảo chính (theo chương)

F.1.1. Chương 1

- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*, 2nd Edition. O'Reilly Media.

F.1.2. Chương 10

- Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.

F.1.3. Chương 2

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

F.1.4. Chương 3, 4, 6, 8

- Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.

F.1.5. Chương 5

- Bellemare, A. (2020). *Building Event-Driven Microservices*. O'Reilly Media.

F.1.6. Chương 7

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

F.1.7. Chương 9

- Newman, S. (2021). *Building Microservices*, Chapter 11. O'Reilly Media.

F.1.8. Chương 11

- Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.

F.1.9. Chương 12

- Bastani, R. & Bryant, T. (2019). *Microservices: Up and Running*. O'Reilly Media.

F.2. Tài liệu bổ sung

- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
- Nygard, M. (2018). *Release It!*, 2nd Edition. Pragmatic Bookshelf.
- Burns, B. et al. (2019). *Kubernetes: Up and Running*, 2nd Edition. O'Reilly Media.
- Narkhede, N. et al. (2017). *Kafka: The Definitive Guide*. O'Reilly Media.
- Stopford, B. (2018). *Designing Event-Driven Systems*. O'Reilly Media.
- Sridharan, C. (2018). *Distributed Systems Observability*. O'Reilly Media.

F.3. Bài báo & Tài nguyên trực tuyến

- Fowler, M. (2014). *Microservices*. martinowler.com
- Richardson, C. *Microservices.io*. microservices.io
- Garcia-Molina, H. & Salem, K. (1987). *Sagas*. ACM SIGMOD Record.
- Clemson, T. (2014). *Testing Strategies in a Microservice Architecture*. martinowler.com.